

Elogios ao codificador limpo

"'Tio Bob' Martin definitivamente eleva a fasquia com seu último livro. Ele explica sua expectativa de um programador profissional em interações de gerenciamento, tempo gestão, pressão, colaboração e escolha das ferramentas a utilizar. Além TDD e ATDD, Martin explica o que todo programador que o considera- ou si mesmo um profissional não só precisa saber, mas também precisa seguir para poder fazer crescer a jovem profissão de desenvolvimento de software."

—Markus Gärtner

Desenvolvedor de Software Sênior

it-agile GmbH

www.it-agile.de

www.shino.de

"Alguns livros técnicos inspiram e ensinam; alguns deleitam e divertem. Raramente um livro técnico faz todas essas quatro coisas. Robert Martin sempre teve para mim e O Clean Coder não é exceção. Leia, aprenda e viva as lições deste livro e você pode se considerar um profissional de software com precisão."

—George Bullock

Gerente de Programa Sênior

Microsoft Corp.

"Se um diploma de ciência da computação tivesse 'leitura obrigatória para depois de se formar', isso seria isso. No mundo real, seu código incorreto não desaparece no final do semestre. acabou, você não recebe um A pela codificação da maratona na noite anterior ao vencimento de uma tarefa,

e, o pior de tudo, você tem que lidar com pessoas. Portanto, os gurus da codificação não são necessariamente

profissionais. O Clean Coder descreve a jornada para o profissionalismo. . . e isso faz um trabalho extraordinariamente divertido."

—Jeff Overbey

Universidade de Illinois em Urbana-Champaign

"O Clean Coder é muito mais do que um conjunto de regras ou diretrizes. Ele contém sabedoria e conhecimento adquiridos que normalmente são obtidos através de muitos anos de tentativa e erro ou trabalhando como aprendiz de um mestre artesão. Se você ligar você é um profissional de software, precisa deste livro."

—R. L. Bogetti

Designer Líder de sistema

Baxter Saúde

www.RLBogetti.com

Esta página foi intencionalmente deixada em branco

O codificador limpo

T

A série Robert C. Martin é dirigida a desenvolvedores de software, equipes líderes, analistas de negócios e gerentes que desejam aumentar seus habilidades e proficiência ao nível de um Mestre Artesão. A série contém livros que orientam os profissionais de software nos princípios, padrões e práticas de programação, gerenciamento de projetos de software, requisitos coleta, design, análise, testes e outros.

Visite informit.com/martinseries para obter uma lista completa das publicações disponíveis.

A série Robert C. Martin

O codificador limpo

UM CÓDIGO DE CONDUTA PARA
PROGRAMADORES PROFISSIONAIS

Roberto C. Martin

Upper Saddle River, NJ • Boston • Indianápolis • São Francisco

Nova York • Toronto • Montreal • Londres • Munique • Paris • Madri

Cidade do Cabo • Sydney • Tóquio • Singapura • Cidade do México

Muitas das designações utilizadas pelos fabricantes e vendedores para distinguir os seus produtos são reivindicadas como marcas registradas. Onde essas designações aparecem neste livro e o editor tinha conhecimento de uma marca registrada reivindicada, as designações foram impressas com iniciais maiúsculas ou todas em maiúsculas.

O autor e a editora tomaram cuidado na preparação deste livro, mas não fazem nenhuma declaração expressa ou garantia implícita de qualquer tipo e não assumimos qualquer responsabilidade por erros ou omissões. Nenhuma responsabilidade é assumida por danos incidentais ou consequenciais relacionados ou decorrentes do uso das informações ou programas aqui contidos.

A editora oferece excelentes descontos neste livro quando encomendado em grande quantidade para compras em grandes quantidades ou vendas especiais, que podem incluir versões eletrônicas e/ou capas personalizadas e conteúdo específico para o seu negócio, objetivos de treinamento, foco de marketing e interesses de marca. Para mais informações, entre em contato:

Vendas Corporativas e Governamentais dos EUA

(800) 382-3419

corpsales@pearsontechgroup.com

Para vendas fora dos Estados Unidos, entre em contato com:

Vendas Internacionais

internacional@pearson.com

Visite-nos na Web: www.informit.com/ph

Dados de catalogação na publicação da Biblioteca do Congresso

Martin, Roberto C.

O codificador limpo: um código de conduta para programadores profissionais / Robert Martin.

pág. cm.

Inclui referências bibliográficas e índice.

ISBN 0-13-708107-3 (pbk.: papel alk.)

1. Programação de computadores – Aspectos morais e éticos. 2. Computador programadores - Ética profissional. I. Título.

QA76.9.M65M367 2011

005.1092—dc22

2011005962

Direitos autorais © 2011 Pearson Education, Inc.

Ilustrações copyright 2011 de Jennifer Kohnke.

Todos os direitos reservados. Impresso nos Estados Unidos da América. Esta publicação é protegida por direitos autorais e

permissão deve ser obtida do editor antes de qualquer reprodução proibida, armazenamento em um arquivo de recuperação

sistema, ou transmissão em qualquer forma ou por qualquer meio, eletrônico, mecânico, fotocópia, gravação ou

da mesma forma. Para obter permissão para usar o material deste trabalho, envie uma solicitação por escrito à Pearson

Education, Inc., Departamento de Permissões, 200 Old Tappan Road, Old Tappan, New Jersey 07675, ou você

pode enviar sua solicitação por fax para (201) 236-3290.

ISBN-13: 978-0-13-708107-3

ISBN-10: 0-13-708107-3

Texto impresso nos Estados Unidos em papel reciclado na RR Donnelley em Crawfordsville, Indiana.

Sexta impressão, julho de 2015

Entre 1986 e 2000 trabalhei em estreita colaboração com Jim Newkirk, um colega da Teradina. Ele e eu compartilhamos uma paixão por programação e código limpo. Passaríamos noites, tardes e fins de semana juntos brincando com diferentes estilos de programação e técnicas de design. Estávamos continuamente planejando sobre ideias de negócios. Eventualmente, formamos juntos a Object Mentor, Inc.. Aprendi muitas coisas com Jim enquanto trabalhávamos juntos em nossos planos. Mas um dos o mais importante foi a sua atitude ética de trabalho; foi algo que eu me esforcei emular. Jim é um profissional. Tenho orgulho de ter trabalhado com ele e de ligar para ele meu amigo.

Esta página foi intencionalmente deixada em branco

ix	
Prefácio	
xiii	
Prefácio	
XIX	
Agradecimentos	
XXIII	
Sobre o autor	
xxxx	
Na capa	
xxxi	
Introdução aos pré-requisitos	
1	
Capítulo 1	
Profissionalismo	
7	
Tenha cuidado com o que você pede	
8	
Assumindo a responsabilidade	
8	
Primeiro, não faça mal	
11	
Ética de Trabalho	
16	
Bibliografia	
22	
Capítulo 2	
Dizendo não	
23	
Papéis adversários	
26	
Apostas altas	
29	
Ser um “jogador de equipe”	
30	
O custo de dizer sim	
36	
Código Impossível	
41	
CONTEÚDO	

CONTEÚDO

x

Capítulo 3

Dizendo sim

45

Uma linguagem de compromisso

47

Aprendendo a dizer “sim”

52

Conclusão

56

Capítulo 4

Codificação

57

Preparação

58

A zona de fluxo

62

Bloco do escritor

64

Depuração

66

Acompanhando seu próprio ritmo

69

Estar atrasado

71

Ajuda

73

Bibliografia

76

Capítulo 5

Desenvolvimento orientado a testes

77

O júri está presente

79

As três leis do TDD

79

O que TDD não é

83

Bibliografia

84

Capítulo 6

Praticando

85

Algumas informações básicas sobre a prática

86

O Dojo de Codificação

89

Ampliando sua experiência

93

Conclusão

94

Bibliografia

94

Capítulo 7

Teste de aceitação

95

Requisitos de comunicação

95

Testes de Aceitação

100

Conclusão

111

Capítulo 8

Estratégias de teste

113

O controle de qualidade não deve encontrar nada

114

CONTEÚDO

xii

A pirâmide de automação de testes

115

Conclusão

119

Bibliografia

119

Capítulo 9

Gerenciamento de tempo

121

Reuniões

122

Foco-Maná

127

Time Boxing e Tomates

130

Evitar

131

Becos sem saída

131

Pântanos, pântanos, pântanos e outras bagunças

132

Conclusão

133

Capítulo 10

Estimativa

135

O que é uma estimativa?

138

PERT

141

Estimando Tarefas

144

A Lei dos Grandes Números

147

Conclusão

147

Bibliografia

148

Capítulo 11

Pressão

149

Evitando pressão

151

Lidando com pressão

153

Conclusão

155

Capítulo 12

Colaboração

157

Programadores versus Pessoas

159

Cerebelos

164

Conclusão

166

Capítulo 13

Equipes e Projetos

167

Combina?

168

Conclusão

171

Bibliografia

171

CONTEÚDO

xii

Capítulo 14

Mentoria, aprendizagem e artesanato

173

Graus de falha

174

Mentoria

174

Aprendizagem

180

Artesanato

184

Conclusão

185

Apêndice A

Ferramentas

187

Ferramentas

189

Controle de código-fonte

189

IDE/Editor

194

Acompanhamento de problemas

196

Construção Contínua

197

Ferramentas de teste unitário

198

Ferramentas de teste de componentes

199

Ferramentas de teste de integração

200

UML/MDA

201

Conclusão

204

Índice

205

PREFÁCIO D

Você comprou este livro, então presumo que seja um profissional de software. Isso é bom; eu também. E já que tenho sua atenção, deixe-me dizer por que peguei este livro.

Tudo começou há pouco tempo, num lugar não muito distante. Cue a cortina, luzes e câmera, Charley....

Vários anos atrás, eu trabalhava em uma empresa de médio porte que vendia produtos altamente produtos regulamentados. Você conhece o tipo; nós nos sentamos em um cubículo em um prédio de três andares

prédio, diretores e superiores tinham escritórios particulares e recebiam todos que você precisava na mesma sala para uma reunião demorou cerca de uma semana.

Estávamos operando em um mercado muito competitivo quando o governo abriu criar um novo produto.

De repente, tínhamos um conjunto totalmente novo de clientes potenciais; tudo o que tínhamos que fazer era fazer com que eles comprassem nosso produto. Isso significava que tínhamos que arquivar por um certo

prazo com o governo federal, passar por uma auditoria de avaliação em outra data, e ir ao mercado em um terceiro encontro.

XIV

PREFÁCIO

Repetidamente, nossa administração enfatizou para nós a importância daqueles dados. Um único desliz e o governo nos manteria fora do mercado por um ano, e se os clientes não pudessem se inscrever no primeiro dia, todos eles se inscreveriam com outra pessoa e estaríamos fora do mercado.

Era o tipo de ambiente em que algumas pessoas reclamam e outras salientam que “a pressão produz diamantes”.

Fui gerente técnico de projetos, promovido desde o desenvolvimento. Minha responsabilidade era colocar o site no ar no dia do lançamento, para que clientes em potencial pudessem fazer download informações e, o mais importante, formulários de inscrição. Meu parceiro na empreitada era o gerente de projetos voltado para os negócios, a quem chamarei de Joe. O papel de Joe era trabalhar o outro lado, lidando com vendas, marketing e requisitos não técnicos.

Ele também era o cara que gostava do comentário “pressão faz diamantes”.

Se você trabalhou muito na América corporativa, provavelmente já viu o acusações, acusações e aversão ao trabalho que são completamente naturais.

Nossa empresa teve uma solução interessante para esse problema comigo e com Joe.

Um pouco como Batman e Robin, era nosso trabalho fazer as coisas. Eu me encontrei com a equipa técnica todos os dias num canto; reconstruiríamos a programação a cada dia, descubra o caminho crítico e, em seguida, remova todos os obstáculos possíveis desse caminho crítico. Se alguém precisasse de software; nós iríamos buscá-lo. Se eles “adorassem” configurar o firewall, mas “nossa, é hora da minha pausa para o almoço”, compraríamos eles almoçam. Se alguém quisesse trabalhar em nosso ticket de configuração, mas tivesse outras prioridades, Joe e eu iríamos conversar com o supervisor.

Depois o gerente.

Depois o diretor.

Nós fizemos as coisas.

É um pouco exagerado dizer que chutamos cadeiras, gritamos e gritou, mas usamos todas as técnicas que tínhamos para fazer as coisas, inventei alguns novos ao longo do caminho, e fizemos isso de uma forma ética que estou orgulho até hoje.

Eu me considerava um membro da equipe, não hesitando em começar a escrever um Instrução SQL ou fazer um pequeno emparelhamento para divulgar o código. Na época, Pensei em Joe da mesma forma, como um membro da equipe, não acima dela.

Por fim, percebi que Joe não compartilhava dessa opinião. Isso foi muito dia triste para mim.

Era sexta-feira às 13h; o site foi configurado para entrar no ar muito cedo no seguinte Segunda-feira.

Nós terminamos. *FEITO*. Cada sistema estava funcionando; estávamos prontos. eu tinha o inteiro equipe de tecnologia reunida para a reunião final do scrum e estávamos prontos para virar o jogo mudar. Mais do que “só” a equipe técnica, contamos com o pessoal de negócios da marketing, os proprietários do produto, conosco.

Estávamos orgulhosos. Foi um bom momento.

Então Joe apareceu.

Ele disse algo como: "Más notícias. O departamento jurídico não tem os formulários de inscrição pronto, então não podemos ir ao vivo ainda."

Isso não foi grande coisa; ficamos presos por uma coisa ou outra durante todo o tempo de todo o projeto e tinha a rotina do Batman/Robin bem definida. eu estava pronto, e minha resposta foi essencialmente: "Tudo bem, parceiro, vamos fazer isso mais uma vez.

O Departamento Jurídico fica no terceiro andar, certo?

Então as coisas ficaram estranhas.

Em vez de concordar comigo, Joe perguntou: "Do que você está falando, Matt?"

Eu disse: "Você sabe. Nossa música e dança habituais. Estamos falando de quatro PDFs arquivos, certo? Isso está feito; legal só precisa aprová-los? Vamos sair seus cubículos, dê-lhes mau-olhado e faça isso!

Joe não concordou com minha avaliação e respondeu: "Iremos ao ar mais tarde na próxima semana. Não é grande coisa.

PREFÁCIO

PREFÁCIO

Você provavelmente pode adivinhar o resto da troca; soou algo assim:

Matt: "Mas por quê? Eles poderiam fazer isso em algumas horas."

José:

"Pode ser necessário mais do que isso."

Matt: "Mas eles têm o fim de semana inteiro. Tempo de sobra. Vamos fazer isso!"

José:

"Matt, estes são profissionais. Não podemos simplesmente encará-los e insistir que eles sacrifiquem suas vidas pessoais pelo nosso pequeno projeto."

Matt: (pausa) "... Joe... o que você acha que estamos fazendo com o equipe de engenharia nos últimos quatro meses?"

José:

"Sim, mas estes são profissionais."

Pausa.

Respirar.

O que. Fez. José. Apenas. Dizer?

Na época eu achava que o corpo técnico era profissional, no melhor sentido a palavra.

Pensando nisso novamente, porém, não tenho tanta certeza.

Vejamos a técnica de Batman e Robin uma segunda vez, de um ponto de vista diferente. perspectiva. Achei que estava exortando a equipe ao seu melhor desempenho, mas suspeitar que Joe estava jogando, com a suposição implícita de que a técnica pessoal era seu oponente. Pense nisso: por que foi necessário correr, chutando cadeiras e apoiando-se nas pessoas?

Não deveríamos ter sido capazes de perguntar à equipe quando eles terminariam, conseguir um resposta firme, acreditar na resposta que nos foi dada e não se deixar queimar por essa crença? Certamente, para profissionais, deveríamos. . . e, ao mesmo tempo, não poderíamos.

Joe não confiou em nossas respostas e se sentiu confortável em microgerenciar a tecnologia

PREFÁCIO

equipe - e ao mesmo tempo, por algum motivo, ele confiava na equipe jurídica e não estava disposto a microgerenciá-los.

O que é isso?

De alguma forma, a equipe jurídica demonstrou profissionalismo de uma forma que a equipe técnica não.

De alguma forma, outro grupo convenceu Joe de que não precisava de uma babá, que eles não estavam jogando e que precisavam ser tratados como colegas que eram respeitados.

Não, não acho que tenha algo a ver com certificados sofisticados pendurados nas paredes ou alguns anos extras de faculdade, embora esses anos de faculdade possam ter incluído um pouco de treinamento social implícito sobre como se comportar.

Desde aquele dia, há muitos anos, me pergunto como a técnica a profissão teria que mudar para ser considerada profissional.

Ah, eu tenho algumas ideias. Blogei um pouco, li muito, consegui melhorar minha própria situação de vida profissional e ajudar alguns outros. No entanto, eu não conhecia nenhum livro que estabelecesse

elaborar um plano que tornasse tudo explícito.

Então, um dia, do nada, recebi uma oferta para revisar o primeiro rascunho de um livro; o livro que você está segurando em suas mãos agora.

Este livro lhe dirá passo a passo exatamente como se apresentar e interagir como um profissional. Não com clichês banais, não com apelos a pedaços de papel, mas com o que você pode fazer e como fazer.

Em alguns casos, os exemplos são palavra por palavra.

Alguns desses exemplos contêm respostas, contra-respostas, esclarecimentos e até conselhos para saber o que fazer se a outra pessoa tentar “simplesmente ignorar você”.

PREFÁCIO

Ei, olhe só, lá vem Joe de novo, desta vez à esquerda do palco:

Ah, aqui estamos, de volta à BigCo, com Joe e eu, mais uma vez no grande site projeto de conversão.

Só que desta vez, imagine um pouco diferente.

Em vez de se esquivar dos compromissos, a equipe técnica realmente os assume.

Em vez de fugir das estimativas ou deixar outra pessoa fazer o planejamento

(depois reclamando), a equipe técnica na verdade se auto-organiza e assume compromissos reais.

Agora imagine que a equipe está realmente trabalhando junta. Quando os programadores são bloqueados pelas operações, eles atendem o telefone e o administrador do sistema realmente começa o trabalho.

Quando Joe passa para acender uma fogueira para resolver o tíquete 14321, ele não precisa para; ele pode ver que o DBA está trabalhando diligentemente, não navegando na web. Da mesma forma, as estimativas que ele recebe da equipe parecem totalmente consistentes, e ele não entende a sensação de que o projeto é prioritário em algum lugar entre o almoço e verificando e-mail. Todos os truques e tentativas de manipular o cronograma não são se deparou com: “Vamos tentar”, mas em vez disso, “Esse é o nosso compromisso; se você quiser fazer definir seus próprios objetivos, fique à vontade.

Depois de um tempo, suspeito que Joe começaria a pensar na equipe técnica como, bem, profissionais. E ele estaria certo.

Esses passos para transformar seu comportamento de técnico em profissional? Você vai encontrá-los no resto do livro.

Bem-vindo ao próximo passo em sua carreira; Eu suspeito que você vai gostar.

—Matthew Heusser

Naturalista de Processos de Software

XIX

PR E FAC E

Às 11h39 EST do dia 28 de janeiro de 1986, apenas 73,124 segundos após o lançamento e a uma velocidade

altitude de 48.000 pés, o ônibus espacial Challenger foi feito em pedacinhos por

a falha do foguete propulsor sólido direito (SRB). Sete bravos astronautas,

incluindo a professora do ensino médio Christa McAuliffe, foram perdidos. A expressão em

o rosto da mãe de McAuliffe enquanto ela assistia ao falecimento de sua filha nove

milhas acima me assombram até hoje.

O Challenger quebrou porque os gases de escape quentes no SRB com defeito vazaram

saindo de entre os segmentos de seu casco, espalhando-se pelo corpo do

PREFÁCIO

tanque de combustível externo. O fundo do tanque principal de hidrogênio líquido estourou, inflamando o combustível e conduzindo o tanque para frente para colidir com o tanque de oxigênio líquido acima dele. Ao mesmo tempo, o SRB se separou do suporte traseiro e girou em torno de seu suporte dianteiro. Seu nariz perfurou o tanque de oxigênio líquido. Estes vetores de força aberrantes fizeram com que toda a nave, movendo-se bem acima de mach 1,5, gire contra a corrente de ar. As forças aerodinâmicas rapidamente destruíram tudo pedaços.

Entre os segmentos circulares do SRB havia dois sintéticos concêntricos anéis de borracha. Quando os segmentos foram aparafusados, os O-rings foram comprimido, formando uma vedação hermética de que os gases de escape não deveriam ter sido capaz de penetrar.

Mas na noite anterior ao lançamento, a temperatura na plataforma de lançamento subiu até 17°F, 23 graus abaixo da temperatura mínima especificada dos O-rings e 33 graus mais baixo do que qualquer lançamento anterior. Como resultado, os O-rings cresceram muito rígido para bloquear adequadamente os gases quentes. Após a ignição do SRB houve um pulso de pressão à medida que os gases quentes se acumulavam rapidamente. Os segmentos do o reforço inflou para fora e relaxou a compressão nos anéis de vedação. O a rigidez dos O-rings os impediu de manter a vedação estanque, então alguns dos gases quentes vazaram e vaporizaram os anéis de vedação em 70 graus de arco.

Os engenheiros da Morton Thiokol que projetaram o SRB sabiam que havia havia problemas com os O-rings, e eles relataram esses problemas para gerentes da Morton Thiokol e da NASA sete anos antes. Na verdade, os O-rings de lançamentos anteriores foram danificados de maneira semelhante, embora não o suficiente ser catastrófico. O lançamento mais frio sofreu os maiores danos. O engenheiros projetaram um reparo para o problema, mas a implementação desse o reparo foi adiado há muito tempo.

Os engenheiros suspeitaram que os anéis de vedação enrijeciam quando frios. Eles também sabiam que as temperaturas para o lançamento do Challenger foram mais frias do que qualquer anterior lançamento e bem abaixo da linha vermelha. Em suma, os engenheiros sabiam que o risco estava muito alto. Os engenheiros agiram com base nesse conhecimento. Eles escreveram memorandos

PREFÁCIO

levantando bandeiras vermelhas gigantes. Eles instaram fortemente Thiokol e os gerentes da NASA a não lançamento. Numa reunião de última hora, realizada poucas horas antes do lançamento, aqueles engenheiros apresentaram seus melhores dados. Eles se enfureceram, bajularam e protestaram. Mas no final, os gestores os ignoraram.

Quando chegou a hora do lançamento, alguns dos engenheiros recusaram-se a assistir ao transmitido porque temiam uma explosão na plataforma. Mas como o Desafiador subiram graciosamente para o céu e começaram a relaxar. Momentos antes do destruição, enquanto observavam o veículo passar por Mach 1, um deles disse que eles “se esquivaram de uma bala”.

Apesar de todos os protestos, memorandos e insistências dos engenheiros, os gerentes acreditavam que sabiam melhor. Eles pensaram que os engenheiros estavam exagerando. Eles não confiava nos dados dos engenheiros ou nas suas conclusões. Eles lançaram porque estavam sob imensa pressão financeira e política. Eles esperavam que tudo ficaria bem.

Esses gestores não eram apenas tolos, eram criminosos. A vida de sete bons homens e mulheres, e as esperanças de uma geração que olha para o espaço viagem, ficaram frustrados naquela manhã fria porque aqueles gerentes definiram seus próprios medos, esperanças e intuições acima das palavras de seus próprios especialistas. Eles fizeram um decisão que não tinham o direito de tomar. Eles usurparam a autoridade do povo quem realmente sabia: os engenheiros.

Mas e os engenheiros? Certamente os engenheiros fizeram o que estavam fazendo deveria fazer. Eles informaram seus gerentes e lutaram arduamente por sua posição. Eles passaram pelos canais apropriados e invocaram todos os direitos protocolos. Eles fizeram o que puderam, dentro do sistema – e ainda assim os gestores os substituiu. Portanto, parece que os engenheiros podem ir embora sem culpa.

Mas às vezes me pergunto se algum desses engenheiros ficava acordado à noite, assombrado por aquela imagem da mãe de Christa McAuliffe, e desejando que eles tivessem ligado Dan Em vez.

SOBRE ESTE LIVRO

Este livro é sobre profissionalismo em software. Ele contém muita pragmática aconselhamento na tentativa de responder perguntas, como

- O que é um profissional de software?
- Como se comporta um profissional?
- Como um profissional lida com conflitos, cronogramas apertados e gestores?
- Quando e como um profissional deve dizer “não”?
- Como um profissional lida com a pressão?

Mas, escondendo-se nos conselhos pragmáticos deste livro, você encontrará uma atitude lutando para romper. É uma atitude de honestidade, de honra, de auto-estima, respeito e de orgulho. É uma disposição para aceitar a terrível responsabilidade de ser um artesão e um engenheiro. Essa responsabilidade inclui trabalhar bem e trabalhando limpo. Inclui comunicar-se bem e estimar fielmente. Isso inclui gerenciar seu tempo e enfrentar decisões difíceis de risco-recompensa. Mas essa responsabilidade inclui outra coisa – uma coisa assustadora. Como um engenheiro, você tem um conhecimento profundo sobre seus sistemas e projetos que nenhum gerente pode ter. Com esse conhecimento vem a responsabilidade para agir.

B I B L I O G R A F I A

[McConnell87]: Malcolm McConnell, Desafiador ‘Um mau funcionamento grave’, Novo Iorque, NY: Simon & Schuster, 1987

[Wiki-Challenger]: “Desastre do ônibus espacial Challenger,”
http://en.wikipedia.org/wiki/Space_Shuttle_Challenger_disaster

PREFÁCIO

AC KNOWLEDGMENTS

Minha carreira tem sido uma série de colaborações e esquemas. Embora eu tenha tido muitos sonhos e aspirações particulares, sempre parecia encontrar alguém para compartilhar eles com. Nesse sentido, me sinto um pouco como os Sith: “Sempre há dois”.

A primeira colaboração que pude considerar profissional foi com John Marchese aos 13 anos. Ele e eu planejamos construir computadores juntos. Eu era o cérebro e ele a força. Eu mostrei a ele onde soldar um fio e ele o soldou. Mostrei a ele onde montar um relé e ele montou isso. Foi muito divertido e passamos centenas de horas nisso. Na verdade, construímos alguns objetos de aparência impressionante com relés, botões, luzes e até Teletipos! É claro que nenhum deles realmente fez nada, mas foram muito impressionante e trabalhamos muito duro neles. Para João: Obrigado!

No primeiro ano do ensino médio, conheci Tim Conrad na minha aula de alemão.

Tim era inteligente. Quando nos unimos para construir um computador, ele era o cérebro e Eu era o músculo. Ele me ensinou eletrônica e me deu minha primeira introdução ao um PDP-8. Ele e eu construímos uma calculadora binária eletrônica funcional de 18 bits fora dos componentes básicos. Poderia somar, subtrair, multiplicar e dividir. Isso nos levou um ano de fins de semana e todas as férias de primavera, verão e Natal. Nós trabalhamos furiosamente sobre isso. No final, funcionou muito bem. Para Tim: Obrigado!

AGRADECIMENTOS

Tim e eu aprendemos a programar computadores. Isso não foi fácil de fazer em 1968, mas conseguimos. Temos livros sobre montador PDP-8, Fortran, Cobol, PL/1, entre outros. Nós os devoramos. Escrevemos programas que não tínhamos esperança de executando porque não tínhamos acesso a um computador. Mas nós os escrevemos de qualquer maneira, por puro amor.

Nossa escola secundária iniciou um currículo de ciência da computação em nosso segundo ano. Eles conectaram um teletipo ASR-33 a um modem dial-up de 110 bauds. Eles tinham uma conta no sistema de compartilhamento de tempo Univac 1108 no Illinois Institute of Tecnologia. Tim e eu imediatamente nos tornamos os operadores de fato desse máquina. Ninguém mais poderia chegar perto dele.

O modem foi conectado pegando o telefone e discando o número. Quando você ouviu o barulho do modem respondendo, você apertou o botão "orig" botão no teletipo fazendo com que o modem de origem emita seu próprio guincho. Então você desligou o telefone e a conexão de dados foi estabelecida.

O telefone tinha uma trava no mostrador. Somente os professores tinham a chave. Mas isso não aconteceu importa, porque aprendemos que você pode discar um telefone (qualquer telefone) tocando o número de telefone no gancho do interruptor. Eu era baterista, então tive bastante bom timing e reflexos. Eu poderia discar para aquele modem, com a trava no lugar, em menos de 10 segundos.

Tínhamos dois teletipos no laboratório de informática. Uma delas era a máquina online e a outra era uma máquina offline. Ambos foram usados pelos alunos para escrever suas programas. Os alunos digitavam seus programas nos teletipos com o perfurador de fita de papel ativado. Cada pressionamento de tecla foi gravado na fita. Os estudantes escreveram seus programas em IITran, uma linguagem interpretada extremamente poderosa.

Os alunos deixavam suas fitas de papel em uma cesta perto dos Teletipos.

Depois da escola, Tim e eu ligávamos para o computador (tocando, é claro), carregue as fitas no sistema em lote IITran e desligue. Com 10 caracteres por segundo, este não foi um procedimento rápido. Mais ou menos uma hora depois, ligaríamos de volta e obtenha as impressões, novamente com 10 caracteres por segundo. O Teletipo não separe as listagens dos alunos ejetando as páginas. Apenas imprimiu um após o outro

AGRADECIMENTOS

depois do próximo, então nós os cortamos usando uma tesoura, prendemos sua entrada com um clipe de papel

fita de papel em sua listagem e coloque-os na cesta de saída.

Tim e eu éramos os mestres e deuses desse processo. Até os professores nos deixaram sozinho quando estávamos naquela sala. Estávamos fazendo o trabalho deles e eles sabiam disso.

Eles nunca nos pediram para fazer isso. Eles nunca nos disseram que poderíamos. Eles nunca nos deram a chave do telefone. Acabamos de nos mudar e eles se mudaram - e nos deram

uma coleira muito longa. Aos meus professores de matemática, Sr. McDermit, Sr. Fogel e Sr.

Robien: Obrigado!

Então, depois de todos os trabalhos de casa dos alunos terem sido feitos, íamos brincar. Nós escrevemos programa após programa para fazer uma série de coisas malucas e estranhas. Nós escrevemos programas que representavam graficamente círculos e parábolas em ASCII em um teletipo. Nós escrevemos

programas de caminhada aleatória e geradores de palavras aleatórias. Calculamos 50 fatoriais até o último dígito. Passamos horas e horas inventando programas para escrever e em seguida, fazê-los trabalhar.

Dois anos depois, Tim, nosso compadre Richard Lloyd e eu fomos contratados como programadores da ASC Tabulating em Lake Bluff, Illinois. Tim e eu tínhamos 18 anos na época tempo. Havíamos decidido que a faculdade era uma perda de tempo e que deveríamos começar nossas carreiras imediatamente. Foi aqui que conhecemos Bill Hohri, Frank Ryder, Big

Jim Carlin e John Miller. Eles deram a alguns jovens a oportunidade de

aprenda o que era a programação profissional. A experiência não foi toda

positivos e nem todos negativos. Certamente foi educativo. Para todos eles, e para

Richard, que catalisou e conduziu grande parte desse processo: Obrigado.

Depois de desistir e derreter aos 20 anos, trabalhei como cortador de grama

reparador trabalhando para meu cunhado. Eu era tão ruim nisso que ele teve que atirar eu. Obrigado, Wes!

Mais ou menos um ano depois, acabei trabalhando na Outboard Marine Corporation. Por desta vez eu era casado e tinha um filho a caminho. Eles me demitiram também. Obrigado, John, Ralf e Tom!

AGRADECIMENTOS

Depois fui trabalhar na Teradyne onde conheci Russ Ashdown, Ken Finder, Bob Copithorne, Chuck Studee e CK Srithran (agora Kris Iyer). Ken era meu chefe. Chuck e CK eram meus amigos. Apreendi muito com todos eles. Obrigado, pessoal! Depois houve Mike Carew. Na Teradyne, ele e eu nos tornamos uma dupla dinâmica. Escrevemos vários sistemas juntos. Se você quisesse fazer algo, e feito rápido, você pediu que Bob e Mike fizessem isso. Nós nos divertimos muito juntos. Obrigado, Mike!

Jerry Fitzpatrick também trabalhou na Teradyne. Nós nos conhecemos enquanto jogávamos Dungeons & Dragões juntos, mas rapidamente formaram uma colaboração. Escrevemos software em um Commodore 64 para oferecer suporte aos usuários de D&D. Também iniciamos um novo projeto em Teradyne chamou de “A Recepcionista Eletrônica”. Trabalhamos juntos por vários anos, e ele se tornou, e continua sendo, um grande amigo. Obrigado, Jerry! Passei um ano na Inglaterra enquanto trabalhava para Teradyne. Lá eu me juntei Mike Kergozou. Ele e eu planejam juntos todo tipo de coisas, embora a maioria desses esquemas tinha a ver com bicicletas e pubs. Mas ele era um dedicado programador que estava muito focado em qualidade e disciplina (embora, talvez ele discordaria). Obrigado, Mike!

Retornando da Inglaterra em 1987, comecei a planejar com Jim Newkirk. Nós ambos deixaram a Teradyne (com meses de diferença) e se juntaram a uma start-up chamada Clear Comunicações. Passamos vários anos juntos lá, trabalhando para fazer o milhões que nunca vieram. Mas continuamos nossos planos. Obrigado, Jim! No final, fundamos juntos a Object Mentor. Jim é o mais direto, pessoa disciplinada e focada com quem tive o privilégio de trabalhar. Ele me ensinou tantas coisas que não posso enumerá-las aqui. Em vez disso, eu tenho dedicado este livro a ele.

Há tantos outros com quem planejei, tantos outros com quem colaborei com tantos outros que tiveram impacto na minha vida profissional: Lowell Lindstrom, Dave Thomas, Michael Feathers, Bob Koss, Brett Schuchert, Dean Wampler, Pascal Roy, Jeff Langr, James Grenning, Brian Button, Alan Francis,

AGRADECIMENTOS

Mike Hill, Eric Meade, Ron Jeffries, Kent Beck, Martin Fowler, Grady Booch, e uma lista interminável de outros. Obrigado a todos.

É claro que a maior colaboradora da minha vida foi minha adorável esposa, Ann Maria. Casei-me com ela quando tinha 20 anos, três dias depois que ela completou 18. Durante 38 anos ela tem sido minha companheira constante, meu leme e vela, meu amor e minha vida. eu estamos ansiosos por mais quatro décadas com ela.

E agora, meus colaboradores e parceiros de maquinação são meus filhos. eu trabalho estreitamente com minha filha mais velha, Angela, minha adorável mãe galinha e intrépida assistente. Ela me mantém no caminho certo e estreito e nunca me deixa esquecer um data ou compromisso. Elaborei planos de negócios com meu filho Micah, o fundador de 8thlight. com. Sua cabeça para os negócios é muito melhor do que a minha. Nosso o mais recente empreendimento, cleancoders.com, é muito emocionante!

Meu filho mais novo, Justin, acaba de começar a trabalhar com Micah na 8th Light. Meu a filha mais nova, Gina, é engenheira química e trabalha para a Honeywell. Com esses dois, a conspiração séria apenas começou!

Ninguém em sua vida lhe ensinará mais do que seus filhos. Obrigado, crianças!

Esta página foi intencionalmente deixada em branco

xxxx

SOBRE O AUTOR

Robert C. Martin (“Tio Bob”) é programador desde 1970. Ele é fundador e presidente da Object Mentor, Inc., uma empresa internacional de alto desenvolvedores e gerentes de software experientes, especializados em ajudar empresas realizarem seus projetos. Object Mentor oferece melhoria de processos consultoria, consultoria em design de software orientado a objetos, treinamento e habilidade serviços de desenvolvimento para grandes corporações em todo o mundo.

Martin publicou dezenas de artigos em diversas revistas especializadas e é um palestrante regular em conferências e feiras internacionais.

Ele é autor e editou muitos livros, incluindo:

- Projetando aplicativos C++ orientados a objetos usando o método Booch
- Linguagens de Padrões de Design de Programa 3

xxx

- Mais joias C++
- Programação Extrema na Prática
- Desenvolvimento Ágil de Software: Princípios, Padrões e Práticas
- UML para programadores Java
- Código Limpo

Líder na indústria de desenvolvimento de software, Martin atuou por três anos como editor-chefe do Relatório C++ e atuou como o primeiro presidente do Aliança Ágil.

Robert também é o fundador da Uncle Bob Consulting, LLC e cofundador da seu filho Micah Martin, da The Clean Coders LLC.

SOBRE O AUTOR

NA COBERTURA

A imagem impressionante na capa, que lembra o olho de Sauron, é M1, o Caranguejo Nebulosa. M1 está localizado em Taurus, cerca de um grau à direita de Zeta Tauri, o estrela na ponta do chifre esquerdo do touro. A nebulosa do caranguejo é o remanescente de uma super nova que explodiu por todo o céu na data bastante auspiciosa de 4 de julho, 1054 anúncio. A uma distância de 6.500 anos-luz, aquela explosão apareceu aos chineses

observadores como uma nova estrela, aproximadamente tão brilhante quanto Júpiter. Na verdade, foi visível durante

o dia! Nos seis meses seguintes, ele desapareceu lentamente da vista a olho nu.

A imagem da capa é uma composição de luz visível e de raios X. A imagem visível foi tirada pelo telescópio Hubble e forma o envelope externo. O objeto interno

que parece um alvo de tiro com arco azul foi obtido pelo telescópio de raios X Chandra.

A imagem visível mostra uma nuvem de poeira e gás em rápida expansão, misturada com elementos pesados que sobraram da explosão da supernova. Essa nuvem agora tem 11 anos-luz de diâmetro, pesa 4,5 massas solares e está se expandindo a uma velocidade taxa furiosa de 1.500 quilômetros por segundo. A energia cinética daquele velho a explosão é impressionante, para dizer o mínimo.

Bem no centro do alvo há um ponto azul brilhante. É onde está o pulsar. Isso foi a formação do pulsar que causou a explosão da estrela em primeiro lugar.

Quase uma massa solar de material no núcleo da estrela condenada implodiu em um esfera de nêutrons com cerca de 30 quilômetros de diâmetro. A energia cinética disso implosão, juntamente com a incrível barragem de neutrinos criada quando todos aqueles nêutrons se formaram, rasgaram a estrela e a explodiram para o reino vindouro.

O pulsar gira cerca de 30 vezes por segundo; e pisca enquanto gira. Nós podemos vê-lo piscando em nossos telescópios. Esses pulsos de luz são a razão pela qual chamamos é um pulsar, abreviação de Pulsating Star.

NA CAPA

PRÉ-REQUISITO

INTRODUÇÃO

(Não pule isso, você vai precisar.)

Presumo que você tenha comprado este livro porque é um computador programador e ficam intrigados com a noção de profissionalismo. Você deveria estar.

Profissionalismo é algo que nossa profissão precisa urgentemente.

Eu também sou programador. Sou programador há 421 anos; e nesse tempo—

deixe-me dizer: eu já vi de tudo. Eu fui demitido. Fui elogiado. Eu tenho sido um

líder de equipe, um gerente, um grunhido e até um CEO. Eu trabalhei com brilhantes

1. Não entre em pânico.

INTRODUÇÃO DO PRÉ-REQUISITO

programadores e eu trabalhamos com slugs.2 Trabalhei em corte de alta tecnologia sistemas de software/hardware embarcados de ponta e trabalhei em empresas sistemas de folha de pagamento. Programei em COBOL, FORTRAN, BAL, PDP-8, PDP-11, C, C++, Java, Ruby, Smalltalk e uma infinidade de outras linguagens e sistemas. Trabalhei com ladrões de salário não confiáveis e trabalhei com profissionais consumados. É essa última classificação que é o tema deste livro.

Nas páginas deste livro tentarei definir o que significa ser um profissional programador. Descreverei as atitudes, disciplinas e ações que considero ser essencialmente profissional.

Como posso saber quais são essas atitudes, disciplinas e ações? Porque eu tinha para aprendê-los da maneira mais difícil. Veja, quando consegui meu primeiro emprego como programador, profissional seria a última palavra que você usaria para me descrever.

O ano era 1969. Eu tinha 17 anos. Meu pai havia atormentado uma empresa local chamada ASC para me contratar como programador temporário de meio período. (Sim, meu pai poderia fazer coisas assim. Uma vez eu o vi sair na frente de um carro em alta velocidade carro com a mão estendida ordenando “Pare!” O carro parou. Ninguém disse “não” para meu pai.) A empresa me colocou para trabalhar na sala onde todos os IBM manuais de computador foram mantidos. Eles me fizeram colocar anos e anos de atualizações em os manuais. Foi aqui que vi pela primeira vez a frase: “Esta página saiu intencionalmente em branco.”

Depois de alguns dias atualizando manuais, meu supervisor me pediu para escrever um programa EasyCoder3 simples. Fiquei emocionado ao ser questionado. Eu nunca tinha escrito um programa para um computador real antes. Eu tinha, no entanto, inalado o Autocoder livros e tinha uma vaga noção de como começar.

O programa consistia simplesmente em ler registros de uma fita e substituir os IDs de esses registros com novos IDs. Os novos IDs começaram em 1 e foram incrementados em

2. Termo técnico de origem desconhecida.

3. EasyCoder foi o montador do computador Honeywell H200, que era semelhante ao Autocodificador para o computador IBM 1401.

INTRODUÇÃO DO PRÉ-REQUISITO

1 para cada novo registro. Os registros com os novos IDs deveriam ser gravados em um fita nova.

Meu supervisor me mostrou uma prateleira que continha muitas pilhas de livros vermelhos e azuis cartões perfurados. Imagine que você comprou 50 baralhos de cartas, 25 cartas vermelhas decks e 25 decks azuis. Então você empilhou esses baralhos um em cima do outro.

Era assim que se pareciam essas pilhas de cartas. Eles eram listrados de vermelho e azul, e as listras tinham cerca de 200 cartas cada. Cada uma dessas listras continha o código-fonte da biblioteca de sub-rotinas que os programadores normalmente usavam. Os programadores simplesmente retirariam o deck superior da pilha, certificando-se de que eles pegaram nada além de cartões vermelhos ou azuis e depois colocaram isso no final de suas plataforma de programas.

Escrevi meu programa em alguns formulários de codificação. Os formulários de codificação eram grandes folhas de papel retangulares divididas em 25 linhas e 80 colunas. Cada linha representava uma carta. Você escreveu seu programa no formulário de codificação usando bloco letras maiúsculas e um lápis nº 2. Nas últimas 6 colunas de cada linha você escreveu um número de sequência com aquele lápis nº 2. Normalmente você incrementou a sequência número por 10 para que você possa inserir cartões mais tarde.

O formulário de codificação foi para os perfuradores. Esta empresa tinha várias dezenas mulheres que pegaram formulários de codificação de uma grande caixa de entrada e depois os “digitaram” em máquinas perfuradoras. Essas máquinas eram muito parecidas com máquinas de escrever, exceto que os caracteres foram perfurados em cartões em vez de impressos em papel.

No dia seguinte, os digitadores me devolveram meu programa por correio interno.

Meu pequeno baralho de cartas perfuradas estava embrulhado com meus formulários de codificação e um elástico. Examinei os cartões em busca de erros de digitação. Não havia nenhum. Então então coloquei o deck da biblioteca de sub-rotinas no final do meu deck de programas e em seguida, subi o convés até os operadores de computador.

Os computadores estavam atrás de portas trancadas em um ambiente controlado sala com piso elevado (para todos os cabos). Bati na porta e um o operador pegou meu baralho com austeridade e o colocou em outra caixa de entrada dentro da sala de informática. Quando eles chegassem a esse ponto, eles executariam meu convés.

INTRODUÇÃO DO PRÉ-REQUISITO

No dia seguinte, recebi meu deck de volta. Estava embrulhado em uma lista dos resultados do correr e mantido junto com um elástico. (Usamos muitos elásticos aqueles dias!)

Abri a listagem e vi que minha compilação falhou. As mensagens de erro em a listagem era muito difícil de entender, então levei-a para o meu supervisor. Ele examinou, murmurou baixinho, fez alguns comentários rápidos notas na listagem, pegou meu baralho e me disse para segui-lo.

Ele me levou até a sala da máquina furadora e sentou-se diante de uma máquina furadora vazia. Um por um, ele corrigiu os cartões que estavam errados e acrescentou um ou dois outras cartas. Ele rapidamente explicou o que estava fazendo, mas tudo passou como um piscar.

Ele levou o novo deck até a sala de informática e bateu na porta. Ele disse algumas palavras mágicas a um dos operadores e depois entrou no sala de informática atrás dele. Ele acenou para que eu o seguisse. A operadora configurou as unidades de fita e carregamos o deck enquanto assistíamos. As fitas giraram, o a impressora tagarelou e então acabou. O programa funcionou.

No dia seguinte, meu supervisor me agradeceu pela ajuda e encerrou meu emprego. Aparentemente, o ASC não sentiu que tinha tempo para nutrir um 17 anos.

Mas minha ligação com a ASC mal havia terminado. Alguns meses depois, obtive uma visão completa trabalho temporário de segundo turno na ASC operando impressoras off-line. Estas impressoras imprimiam lixo eletrônico de imagens impressas que foram armazenadas em fita. Meu trabalho era carregar o impressoras com papel, carregar as fitas nas unidades de fita, corrigir atolamentos de papel e caso contrário, apenas observe as máquinas funcionarem.

O ano era 1970. A faculdade não era uma opção para mim, nem tinha qualquer atrativos específicos. A guerra do Vietname ainda estava em curso e os campi eram caóticos. Continuei inalando livros sobre COBOL, Fortran, PL/1, PDP-8 e IBM 360 Assembler. Minha intenção era evitar a escola e dirigir o mais rápido possível. o máximo que pude para conseguir um emprego de programação.

INTRODUÇÃO DO PRÉ-REQUISITO

Doze meses depois alcancei esse objetivo. Fui promovido a tempo integral programador na ASC. Eu e dois de meus bons amigos, Richard e Tim, também de 19 anos, trabalhou com uma equipe de três outros programadores escrevendo um relatório de contabilidade em tempo real

sistema para um sindicato de caminhoneiros. A máquina era uma Varian 620i. Foi um simples minicomputador semelhante em arquitetura a um PDP-8, exceto que tinha um processador de 16 bits palavra e dois registros. A linguagem era assembler.

Escrevemos cada linha de código desse sistema. E quero dizer cada linha. Nós escrevemos o sistema operacional, os cabeçotes de interrupção, os drivers IO, o sistema de arquivos para o discos, o swapper de sobreposição e até mesmo o vinculador relocável. Sem mencionar todos o código do aplicativo. Escrevemos tudo isso em 8 meses trabalhando 70 e 80 horas por semana para cumprir um prazo infernal. Meu salário era de US\$ 7.200 por ano.

Nós entregamos esse sistema. E então desistimos.

Desistimos de repente e com malícia. Veja, depois de todo esse trabalho, e depois de ter entregamos um sistema bem-sucedido, a empresa nos deu um aumento de 2%. Nós nos sentimos enganados

e abusado. Vários de nós conseguimos empregos em outros lugares e simplesmente pedimos demissão.

Eu, no entanto, adotei uma abordagem diferente e muito infeliz. Eu e um amigo

invadiram o escritório do chefe e saíram juntos bem alto. Isso foi

emocionalmente muito satisfatório – por um dia.

No dia seguinte, percebi que não tinha emprego. Eu tinha 19 anos, estava desempregado, sem

grau. Fui entrevistado para alguns cargos de programação, mas essas entrevistas não

não vai bem. Então, trabalhei na oficina de conserto de cortadores de grama do meu cunhado por quatro meses. Infelizmente eu era um péssimo reparador de cortadores de grama. Ele finalmente teve para me deixar ir. Eu caí em um medo desagradável.

Eu ficava acordado até as 3 da manhã todas as noites comendo pizza e assistindo filmes antigos de monstros

na velha TV em preto e branco com orelhas de coelho dos meus pais. Apenas alguns dos fantasmas onde personagens dos filmes. Fiquei na cama até às 13h porque não queria

enfrentar meus dias tristes. Fiz um curso de cálculo em uma faculdade comunitária local e

falhou. Eu estava um desastre.

INTRODUÇÃO DO PRÉ-REQUISITO

Minha mãe me chamou de lado e me disse que minha vida estava uma bagunça e que eu tinha fui um idiota por pedir demissão sem ter um novo emprego, e por desistir tão emocionalmente e por desistir junto com meu amigo. Ela me disse que você nunca desista sem ter um novo emprego, e você sempre desiste com calma, frieza e sozinho. Ela me disse que eu deveria ligar para meu antigo chefe e implorar por meu antigo emprego de volta.

Ela disse: “Você precisa comer uma torta humilde”.

Garotos de dezenove anos não são conhecidos por seu apetite por tortas modestas, e eu não foi exceção. Mas as circunstâncias afetaram meu orgulho. No

No final, liguei para meu chefe e dei uma grande mordida naquela humilde torta. E funcionou. Ele ficou feliz em me recontratar por US\$ 6.800 por ano e fiquei feliz em aceitá-lo.

Passei mais dezoito meses trabalhando lá, observando meus Ps e Qs

e tentando ser um funcionário tão valioso quanto possível. fui recompensado com promoções e aumentos e um contracheque regular. A vida era boa. Quando eu deixei isso empresa, estava em boas condições e com uma oferta de emprego melhor no bolso.

Você pode pensar que aprendi minha lição; que agora eu era um profissional. Longe disso. Essa foi apenas a primeira de muitas lições que precisei aprender. Na vinda anos eu seria demitido de um emprego por perder datas críticas descuidadamente, e quase demitido de outro por vazar inadvertidamente informações confidenciais para um cliente. Eu assumiria a liderança de um projeto condenado e o levaria até o fim.

chão sem pedir a ajuda que eu sabia que precisava. eu agressivamente

defender minhas decisões técnicas, mesmo que elas vão contra o

necessidades dos clientes. Eu contrataria uma pessoa totalmente desqualificada, sobrecarregando meu empregador com uma enorme responsabilidade para lidar. E o pior de tudo, eu pegaria dois outras pessoas foram demitidas por causa da minha incapacidade de liderar.

Então pense neste livro como um catálogo dos meus próprios erros, um mata-borrão dos meus próprios crimes,

e um conjunto de diretrizes para você evitar andar com meus primeiros sapatos.

7

1

PROFESSORALISM

"Oh, ria, Curtin, meu velho. É uma grande piada pregada em nós pelo Senhor, ou pelo destino, ou natureza, o que você preferir. Mas quem ou o que jogou certamente tinha senso de humor! Rá!"

- Howard, O Tesouro da Sierra Madre

CAPÍTULO 1 PROFISSIONALISMO

8

Então, você quer ser um desenvolvedor de software profissional, não é? Você quer segurar cabeça erguida e declare ao mundo: “Eu sou um profissional!” Você quer que as pessoas olhar para você com respeito e tratá-lo com deferência. Você quer mães apontando para você e dizendo aos filhos para serem como você. Você quer tudo. Certo?

CUIDADO COM O QUE VOCÊ PEDE

Profissionalismo é um termo carregado. Certamente é uma medalha de honra e orgulho, mas é também um marcador de responsabilidade e prestação de contas. Os dois andam de mãos dadas, é claro. Você não pode se orgulhar e honrar algo que você não pode ser responsabilizado.

É muito mais fácil ser um não profissional. Os não profissionais não precisam aceitar responsabilidade pelo trabalho que realizam – eles deixam isso para seus empregadores. Se um não profissional comete um erro, o empregador limpa a bagunça. Mas quando um profissional erra, ele limpa a bagunça.

O que aconteceria se você permitisse que um bug passasse por um módulo e isso custasse sua empresa \$ 10.000? O não profissional encolheria os ombros, diria “coisas acontecem” e comece a escrever o próximo módulo. O profissional seria assinar um cheque de US\$ 10.000 para a empresa!¹

Sim, parece um pouco diferente quando é o seu próprio dinheiro, não é? Mas isso sentimento é o sentimento que um profissional tem o tempo todo. Na verdade, esse sentimento é o essência do profissionalismo. Porque, veja você, profissionalismo tem tudo a ver com responsabilidade.

TAKING RESPONSIBILITY

Você leu a introdução, certo? Se não, volte e faça isso agora; ele define o contexto para tudo o que se segue neste livro.

Apreendi como assumir responsabilidades sofrendo as consequências de não aceitar.

1. Esperamos que ele tenha uma boa política de Erros e Omissões!

ASSUMINDO A RESPONSABILIDADE

9

Em 1979 eu trabalhava para uma empresa chamada Teradyne. Eu era o “responsável engenheiro” para o software que controlava um mini e microcomputador sistema que mediu a qualidade das linhas telefônicas. O minicomputador central foi conectado através de linhas telefônicas dedicadas ou dial-up de 300 bauds a dezenas de microcomputadores satélites que controlavam o hardware de medição. O código foi tudo escrito em assembler.

Nossos clientes eram gerentes de serviços de grandes companhias telefônicas. Cada tinha a responsabilidade por 100.000 linhas telefônicas ou mais. Meu sistema ajudou esses gerentes de área de serviço encontram e reparam defeitos e problemas no linhas telefônicas antes que seus clientes percebessem. Isso reduziu o cliente taxas de reclamação que as comissões de serviços públicos mediram e usaram para regular as tarifas que as companhias telefônicas poderiam cobrar. Em suma, estes sistemas eram incrivelmente importantes.

Todas as noites esses sistemas executavam uma “rotina noturna” na qual a central minicomputador disse a cada um dos microcomputadores satélite para testar cada linha telefônica sob seu controle. Todas as manhãs o computador central retire a lista de linhas defeituosas, juntamente com suas características de falha. O os gerentes da área de serviço usariam este relatório para agendar reparadores para consertar o falhas antes que os clientes pudessem reclamar.

Certa ocasião, enviei um novo lançamento para várias dezenas de clientes. “Navio” é exatamente a palavra certa. Eu escrevi o software em fitas e enviei essas fitas aos clientes. Os clientes carregaram as fitas e reinicializaram o sistemas.

A nova versão corrigiu alguns pequenos defeitos e adicionou um novo recurso que nosso os clientes estavam exigindo. Havíamos dito a eles que forneceríamos aquele novo recurso em uma determinada data. Eu mal consegui passar as fitas durante a noite para que elas chegar na data prometida.

Dois dias depois recebi um telefonema do nosso gerente de serviço de campo, Tom. Ele me disse isso vários clientes reclamaram que a “rotina noturna” não havia sido concluída, e que eles não receberam nenhum relatório. Meu coração afundou porque para enviar o software dentro do prazo, deixei de testar a rotina. Eu testei grande parte do

CAPÍTULO 1 PROFISSIONALISMO

10

outras funcionalidades do sistema, mas testar a rotina levou horas, e eu necessário para enviar o software. Nenhuma das correções de bugs estava no código de rotina, então eu me senti seguro.

Perder um relatório noturno era um grande problema. Isso significava que os reparadores tinham menos que

fazer e seria lotado mais tarde. Isso significava que alguns clientes poderiam notar um culpar e reclamar. Perder dados de uma noite é suficiente para obter um serviço gerente de área para ligar para Tom e criticá-lo.

Liguei nosso sistema de laboratório, carreguei o novo software e então iniciei uma rotina. Isso demorou várias horas, mas depois abortou. A rotina falhou. Se eu tivesse feito esse teste antes do envio, as áreas de serviço não teriam perdido dados e a área de serviço os gerentes não estariam criticando Tom agora.

Liguei para Tom para dizer que poderia duplicar o problema. Ele me disse que a maioria dos outros clientes telefonaram para ele com a mesma reclamação. Então ele me perguntou quando eu poderia consertar isso. Eu disse a ele que não sabia, mas que estava trabalhando nisso. No entanto, eu disse a ele que os clientes deveriam voltar para o software antigo. Ele ficou com raiva de mim dizendo que isso foi um golpe duplo para os clientes, já que eles perderam uma noite inteira de dados e não puderam usar o novo recurso que estavam prometido.

O bug foi difícil de encontrar e os testes levaram várias horas. A primeira correção não trabalho. Nem o segundo. Levei várias tentativas e, portanto, vários dias, para descobrir o que deu errado. O tempo todo, Tom me ligava de vez em quando horas me perguntando quando eu consertaria isso. Ele também se certificou de que eu soubesse sobre o as broncas que ele recebia dos gerentes da área de serviço e como

Foi embaraçoso para ele dizer-lhes para colocarem as fitas antigas de volta.

No final encontrei o defeito, enviei as fitas novas e tudo voltou

normal. Tom, que não era meu chefe, se acalmou e colocamos tudo episódio atrás de nós. Meu chefe veio até mim quando tudo acabou e disse: “Aposto que você não vamos fazer isso de novo.” Eu concordei.

Após refletir, percebi que o envio sem testar a rotina havia sido

irresponsável. A razão pela qual negligenciei o teste foi para poder dizer que havia enviado

PRIMEIRO, NÃO FAÇA DANO

11

na hora certa. Era sobre eu salvar a face. Eu não estava preocupado com o cliente, nem sobre meu empregador. Eu só estava preocupado com o meu próprio reputação. Eu deveria ter assumido a responsabilidade antecipadamente e dito ao Tom que os testes não estavam completos e que eu não estava preparado para enviar o software no prazo. Isso teria sido difícil e Tom ficaria chateado. Mas nenhum cliente teria perdido dados e nenhum gerente de serviço teria ligado.

PRIMEIRO, NÃO FAÇA DANO

Então, como assumimos a responsabilidade? Existem alguns princípios. Desenhando do O juramento de Hipócrates pode parecer arrogante, mas que fonte melhor existe? E, na verdade, não faz sentido que a primeira responsabilidade e o primeiro objetivo de um aspirante a profissional é usar seus poderes para o bem?

Que mal um desenvolvedor de software pode causar? Do ponto de vista puramente de software, ele ou ela pode prejudicar tanto a função quanto a estrutura do software. Nós vamos explorar como evitar fazer exatamente isso.

NÃO PREJUDE A FUNCIONAMENTO

Claramente, queremos que nosso software funcione. Na verdade, a maioria de nós somos programadores hoje porque conseguimos que algo funcionasse uma vez e queremos essa sensação novamente. Mas não somos os únicos que queremos que o software funcione. Nossos clientes e os empregadores também querem que funcione. Na verdade, eles estão nos pagando para criar software que funciona do jeito que eles querem.

Prejudicamos o funcionamento do nosso software quando criamos bugs. Portanto, para para sermos profissionais, não devemos criar bugs.

“Mas espere!” Eu ouço você dizer. “Isso não é razoável. O software é muito complexo para ser criado sem bugs.”

Claro que você está certo. O software é muito complexo para ser criado sem bugs.

Infelizmente isso não o deixa fora de perigo. O corpo humano também é complexo de entender na íntegra, mas os médicos ainda juram não fazer prejudicar. Se eles não se livrarem dessa situação, como poderemos nós?

CAPÍTULO 1 PROFISSIONALISMO

12

“Você está nos dizendo que devemos ser perfeitos?” Eu ouvi você se opor?

Não, estou lhe dizendo que você deve ser responsável por suas imperfeições. O

O fato de que certamente ocorrerão bugs em seu código não significa que você não esteja responsável por eles. O fato de que a tarefa de escrever software perfeito é virtualmente impossível não significa que você não seja responsável pela imperfeição.

É função de um profissional ser responsável pelos erros, mesmo que os erros sejam praticamente certo. Então, meu aspirante a profissional, a primeira coisa que você deve praticar está se desculpando. As desculpas são necessárias, mas insuficientes. Você não pode simplesmente manter

cometendo os mesmos erros repetidas vezes. À medida que você amadurece em sua profissão, sua taxa de erro deve diminuir rapidamente em direção à assíntota de zero. Isso nunca vai acontecer chegar a zero, mas é sua responsabilidade chegar o mais próximo possível dele.

O controle de qualidade não deve encontrar nada

Portanto, ao lançar seu software, você deve esperar que o controle de qualidade não encontre problemas. É extremamente pouco profissional enviar propositalmente código que você saiba que está com defeito no controle de qualidade. E qual código você sabe que está com defeito?

Qualquer código

você não tem certeza!

Algumas pessoas usam o controle de qualidade como coletores de bugs. Eles enviam um código que eles não enviaram

cuidadosamente verificado. Eles dependem do controle de qualidade para encontrar os bugs e reportá-los para os desenvolvedores. Na verdade, algumas empresas recompensam o controle de qualidade com base no número de

insetos que eles encontram. Quanto mais bugs, maior será a recompensa.

Não importa que este seja um comportamento desesperadamente caro que prejudica o

empresa e o software. Não importa que esse comportamento arruíne horários e

mina a confiança da empresa na equipe de desenvolvimento. Nunca

lembre-se de que esse comportamento é simplesmente preguiçoso e irresponsável. Liberando código para controle de qualidade

que você não sabe que funciona não é profissional. Isso viola a regra de “não causar danos”.

O controle de qualidade encontrará bugs? Provavelmente, então prepare-se para se desculpar – e então descubra

por que esses bugs escaparam da sua atenção e fizeram algo para evitá-los

aconteça novamente.

PRIMEIRO, NÃO FAÇA DANO

13

Cada vez que o controle de qualidade, ou pior, um usuário, encontra um problema, você deve se surpreender,

decepcionado e determinado a evitar que isso aconteça novamente.

Você deve saber que funciona

Como você pode saber se seu código funciona? Isso é fácil. Teste. Teste novamente. Teste.

Teste. Teste de sete maneiras até domingo!

Talvez você esteja preocupado com o fato de que testar tanto seu código levará muito tempo tempo. Afinal, você tem horários e prazos a cumprir. Se você gastar todo o seu teste de tempo, você nunca conseguirá escrever mais nada. Bom ponto! Então, automatize seus testes. Escreva testes de unidade que você possa executar a qualquer momento e execute esses testes sempre que puder.

Quanto do código deve ser testado com esses testes unitários automatizados? Faça

Eu realmente preciso responder a essa pergunta? Tudo isso! Todos. De. Isto.

Estou sugerindo 100% de cobertura de teste? Não, não estou sugerindo isso. Estou exigindo isso.

Cada linha de código que você escreve deve ser testada. Período.

Isso não é irrealista? Claro que não. Você só escreve código porque espera para ser executado. Se você espera que ele seja executado, você deve saber que funciona. A única maneira de saber isso é testando.

Sou o principal contribuidor e committer de um projeto de código aberto chamado

FitNesse. No momento em que este livro foi escrito, havia 60ksloc no FitNesse. 26 desses 60 estão escritos em mais de 2.000 testes de unidade. Emma relata que a cobertura desses testes de 2.000 é de aproximadamente 90%.

Por que minha cobertura de código não é maior? Porque Emma não consegue ver todas as linhas de código que está sendo executado! Acredito que a cobertura seja muito maior que isso.

A cobertura é 100%? Não, 100% é uma assíntota.

Mas algum código não é difícil de testar? Sim, mas apenas porque esse código foi projetado para ser difícil de testar. A solução para isso é projetar seu código para ser fácil para testar. E a melhor maneira de fazer isso é escrever seus testes primeiro, antes de escrever o código que os transmite.

CAPÍTULO 1 PROFISSIONALISMO

14

Esta é uma disciplina conhecida como Test Driven Development (TDD), que iremos falaremos mais sobre isso em um capítulo posterior.

Controle de qualidade automatizado

Todo o procedimento de controle de qualidade do FitNesse é a execução da unidade e aceitação testes. Se esses testes passarem, eu envio. Isso significa que meu procedimento de controle de qualidade leva cerca de três

minutos, e posso executá-lo por capricho.

Agora, é verdade que ninguém morre se houver um bug no FitNesse. Ninguém perde milhões de dólares também. Por outro lado, FitNesse tem milhares de usuários e um lista de bugs muito pequena.

Certamente alguns sistemas são tão críticos que um pequeno teste automatizado é insuficiente para determinar a prontidão para implantação. Por outro lado, você como desenvolvedor precisa de um mecanismo relativamente rápido e confiável para saber que o código que você

escreveu trabalhos e não interfere no resto do sistema. Então, no mesmo pelo menos, seus testes automatizados devem informar que é muito provável que o sistema passe no controle de qualidade.

NÃO PREJUDIQUE A ESTRUTURA

O verdadeiro profissional sabe que entregar função em detrimento da estrutura é uma tarefa tola. É a estrutura do seu código que permite que ele seja flexível. Se você compromete a estrutura, você compromete o futuro.

A suposição fundamental subjacente a todos os projetos de software é que o software é fácil de mudar. Se você violar esta suposição criando estruturas inflexíveis, então você enfraquece o modelo econômico no qual toda a indústria se baseia.

Resumindo: você deve ser capaz de fazer alterações sem custos exorbitantes.

Infelizmente, muitos projetos ficam atolados em um poço de alcatrão de má estrutura.

Tarefas que costumavam levar dias começam a levar semanas e depois meses. Gestão, desesperado para recuperar o impulso perdido, contrata mais desenvolvedores para acelerar as coisas para cima. Mas estes promotores simplesmente contribuem para o pântano, aprofundando a situação estrutural.

danos e aumentando o impedimento.

PRIMEIRO, NÃO FAÇA DANO

15

Muito tem sido escrito sobre os princípios e padrões de design de software que estruturas de suporte flexíveis e de fácil manutenção.² Software profissional os desenvolvedores guardam essas coisas na memória e se esforçam para adaptar seu software para eles. Mas há um truque que poucos desenvolvedores de software seguem: se você quer que seu software seja flexível, você precisa flexibilizá-lo!

A única maneira de provar que seu software é fácil de mudar é facilitar mudanças nele. E quando você descobrir que as mudanças não são tão fáceis quanto você pensava, você refina o design para que a próxima mudança seja mais fácil.

Quando você faz essas mudanças fáceis? O tempo todo! Cada vez que você olha para um módulo, você faz alterações pequenas e leves nele para melhorar sua estrutura.

Cada vez que você lê o código, você ajusta a estrutura.

Essa filosofia às vezes é chamada de refatoração impiedosa. Eu chamo isso de “o escoteiro regra”: Sempre faça check-in de um limpador de módulo do que quando você fez check-out. Sempre faça algum ato aleatório de gentileza com o código sempre que o vir.

Isso é completamente contrário à maneira como a maioria das pessoas pensa sobre software. Eles acho que fazer uma série contínua de mudanças no software funcional é

perigoso. Não! O que é perigoso é permitir que o software permaneça estático. Se você não o está flexionando, então, quando precisar alterá-lo, você o achará rígido.

Por que a maioria dos desenvolvedores tem medo de fazer alterações contínuas em seu código? Eles tem medo que eles quebrem! Por que eles têm medo de quebrá-lo? Porque eles não tenho testes.

Tudo se resume aos testes. Se você tiver um conjunto automatizado de testes que cubra praticamente 100% do código, e se esse conjunto de testes puder ser executado rapidamente em por capricho, então você simplesmente não terá medo de alterar o código. Como você prova você não tem medo de mudar o código? Você muda isso o tempo todo.

Os desenvolvedores profissionais têm tanta certeza de seus códigos e testes que ficam irritantemente casual em fazer mudanças aleatórias e oportunistas. Eles vão mude o nome de uma turma, por capricho. Eles perceberão um método demorado enquanto

2. [PPP2001]

CAPÍTULO 1 PROFISSIONALISMO

16

ler um módulo e reparticioná-lo normalmente. Eles vão transformar uma instrução switch em implantação polimórfica ou recolher um hierarquia de herança em uma cadeia de comando. Em suma, eles tratam software da mesma forma que um escultor trata a argila - eles a modelam e moldam continuamente.

TRABALHO ETH I C

Sua carreira é sua responsabilidade. Não é responsabilidade do seu empregador certifique-se de que você é comercializável. Não é responsabilidade do seu empregador treinar você, ou para enviá-lo para conferências, ou para comprar livros. Essas coisas são suas responsabilidade. Ai do desenvolvedor de software que confia sua carreira aos seus empregador.

Alguns empregadores estão dispostos a comprar livros para você e enviá-lo para aulas de treinamento e conferências. Tudo bem, eles estão lhe fazendo um favor. Mas nunca caia no armadilha de pensar que isso é responsabilidade do seu empregador. Se o seu empregador não faz essas coisas por você, você deve encontrar uma maneira de fazê-las sozinho. Também não é responsabilidade do seu empregador conceder-lhe o tempo necessário para aprenda. Alguns empregadores podem fornecer esse tempo. Alguns empregadores podem até exigir que você reserve um tempo. Mas, novamente, eles estão lhe fazendo um favor, e você deve ser apropriadamente grato. Tais favores não são algo que você deveria espere.

Você deve ao seu empregador uma certa quantidade de tempo e esforço. Por uma questão de argumento, vamos usar o padrão dos EUA de 40 horas por semana. Essas 40 horas deve ser gasto nos problemas do seu empregador, não nos seus problemas.

Você deve planejar trabalhar 60 horas por semana. Os primeiros 40 são para você empregador. Os 20 restantes são para você. Durante as 20 horas restantes você deve ser ler, praticar, aprender e aprimorar sua carreira.

Posso ouvir você pensando: "Mas e minha família? E minha vida? Estou Eu deveria sacrificá-los pelo meu empregador?"

Não estou falando de todo o seu tempo livre aqui. Estou falando de cerca de 20 horas extras por semana. Isso é cerca de três horas por dia. Se você usa a hora do almoço para leia, ouça podcasts no seu trajeto e gaste 90 minutos por dia aprendendo um novo idioma, você terá tudo sob controle.

Faça as contas. Em uma semana são 168 horas. Dê 40 ao seu empregador e à sua carreira outros 20. Isso deixa 108. Outros 56 para dormir deixam 52 para todo o resto.

Talvez você não queira assumir esse tipo de compromisso. Tudo bem, mas você não deveria então pensar em si mesmo como um profissional. Profissionais gastam tempo cuidando de sua profissão.

Talvez você pense que o trabalho deveria ficar no trabalho e que você não deveria trazê-lo casa. Concordo! Você não deveria trabalhar para seu empregador durante esses 20 horas. Em vez disso, você deveria estar trabalhando em sua carreira.

Às vezes, esses dois estão alinhados um com o outro. Às vezes, o trabalho que você faz para o seu empregador é muito benéfico para a sua carreira. Nesse caso, gastar algumas dessas 20 horas são razoáveis. Mas lembre-se, essas 20 horas são para você. Eles devem ser usados para tornar você mais valioso como profissional.

Talvez você pense que esta é uma receita para o esgotamento. Pelo contrário, é uma receita para evite o esgotamento. Presumivelmente, você se tornou um desenvolvedor de software porque é apaixonado por software e seu desejo de ser um profissional é motivado por essa paixão. Durante essas 20 horas você deveria estar fazendo aquelas coisas que reforçar essa paixão. Essas 20 horas devem ser divertidas!

CONHEÇA SEU CAMPO

Você sabe o que é um gráfico de Nassi-Shneiderman? Se não, por que não? Você sabe a diferença entre uma máquina de estado Mealy e uma máquina de estado Moore? Você deve. Você poderia escrever um quicksort sem pesquisar? Você sabe qual é o termo “Análise de Transformação” significa? Você poderia realizar uma decomposição funcional com diagramas de fluxo de dados? O que significa o termo “Dados Tramp”? Você tem ouvido o termo “Consciência”? O que é uma mesa Parnas?

CAPÍTULO 1 PROFISSIONALISMO

18

Uma riqueza de ideias, disciplinas, técnicas, ferramentas e terminologias decoram o últimos cinquenta anos do nosso campo. Quanto disso você sabe? Se você quer ser um profissional, você deve conhecer uma parte considerável dele e estar constantemente aumentando o tamanho desse pedaço.

Por que você deveria saber essas coisas? Afinal, nosso campo não está progredindo tanto rapidamente que todas estas ideias antigas se tornaram irrelevantes? A primeira parte disso a consulta parece óbvia na superfície. Certamente nosso campo está progredindo e em um ritmo feroz. Curiosamente, porém, é que o progresso é, em muitos aspectos, periférico. É verdade que não esperamos 24 horas pelo retorno da compilação mais. É verdade que escrevemos sistemas com tamanho de gigabytes. É verdade que nós trabalhar no meio de uma rede global que fornece acesso instantâneo a informação. Por outro lado, estamos escrevendo as mesmas instruções if e while que estávamos escrevendo há 50 anos. Muita coisa mudou. Muito não aconteceu. A segunda parte da pergunta certamente não é verdadeira. Muito poucas ideias dos últimos 50 anos tornaram-se irrelevantes. Alguns foram marginalizados, é verdade. A noção de fazer o desenvolvimento em cascata certamente caiu em desuso. Mas isso não significa que não deveríamos saber o que é e quais são seus pontos positivos e negativos. No geral, porém, a grande maioria das ideias duramente conquistadas nos últimos 50 anos são tão valiosos hoje quanto eram naquela época. Talvez eles sejam ainda mais valiosos agora. Lembre-se da maldição de Santayana: “Aqueles que não conseguem se lembrar do passado são condenado a repeti-lo.”

Aqui está uma lista mínima das coisas que todo profissional de software deve ser familiarizado com:

- Padrões de projeto. Você deve ser capaz de descrever todos os 24 padrões do livro GOF e ter conhecimento prático de muitos dos padrões dos livros POSA.
- Princípios de design. Você deve conhecer os princípios SOLID e ter um bom compreensão dos princípios dos componentes.
- Métodos. Você deve entender XP, Scrum, Lean, Kanban, Waterfall, Análise Estruturada e Projeto Estruturado.

- Disciplinas. Você deve praticar TDD, design orientado a objetos, estruturado Programação, integração contínua e programação em pares.

- Artefatos: Você deve saber usar: UML, DFDs, Gráficos Estruturais, Petri Redes, diagramas e tabelas de transição de estado, fluxogramas e tabelas de decisão.

APRENDIZAGEM CONTINUA

A taxa frenética de mudanças em nossa indústria significa que os desenvolvedores de software deve continuar a aprender grandes quantidades apenas para acompanhar. Ai dos arquitetos que param de codificar, rapidamente se descobrirão irrelevantes. Ai do programadores que param de aprender novas linguagens - eles observarão como a indústria passa por eles. Ai dos desenvolvedores que não conseguem aprender novas disciplinas e técnicas – seus pares se destacarão à medida que declinarem.

Você visitaria um médico que não se mantivesse atualizado com as revistas médicas?

Você contrataria um advogado tributarista que não se mantivesse atualizado com as leis tributárias e precedentes? Por que os empregadores deveriam contratar desenvolvedores que não se mantêm atualizados?

Leia livros, artigos, blogs, tweets. Vá para conferências. Vá para grupos de usuários.

Participe de grupos de leitura e estudo. Aprenda coisas que estão fora do seu zona de conforto. Se você é um programador .NET, aprenda Java. Se você é um Java programador, aprenda Ruby. Se você é um programador C, aprenda Lisp. Se você quiser realmente dobre seu cérebro, aprenda Prolog and Forth!

PRÁTICA

Prática de profissionais. Os verdadeiros profissionais trabalham duro para manter suas habilidades afiadas e pronto. Não basta simplesmente fazer o seu trabalho diário e chamar isso de prática.

Fazer seu trabalho diário é desempenho, não prática. A prática é quando você exercitar especificamente suas habilidades fora do desempenho de seu trabalho para o único propósito de refinar e aprimorar essas habilidades.

O que isso poderia significar para um desenvolvedor de software praticar? No início pensei que o conceito parecia absurdo. Mas pare e pense por um momento. Considere como os músicos dominam seu ofício. Não é atuando. É praticando. E como eles praticam? Entre outras coisas, eles têm exercícios especiais que executar. Escalas, estudos e corridas. Eles fazem isso repetidamente para treinar seus dedos e sua mente, e para manter o domínio de suas habilidades.

CAPÍTULO 1 PROFISSIONALISMO

20

Então, o que os desenvolvedores de software poderiam fazer para praticar? Há um capítulo inteiro em este livro dedicado a diferentes técnicas de prática, por isso não vou entrar muito detalhe aqui. Uma técnica que uso com frequência é a repetição de exercícios simples como o jogo de boliche ou fatores primos. Eu chamo esses exercícios de Kata. Existem muitos desses kata para escolher.

Um kata geralmente vem na forma de um problema de programação simples para resolver, como como escrever a função que calcula os fatores primos de um número inteiro. O ponto de fazer o kata não é descobrir como resolver o problema; você sabe como fazer isso já. O objetivo do kata é treinar os dedos e o cérebro.

Farei um ou dois kata todos os dias, muitas vezes como parte da preparação para o trabalho. Eu poderia fazer isso

em Java, ou em Ruby, ou em Clojure, ou em alguma outra linguagem para a qual eu queira manter minhas habilidades. Usarei o kata para aprimorar uma habilidade específica, como manter meus dedos costumavam apertar teclas de atalho ou usar certas refatorações.

Pense no kata como um exercício de aquecimento de 10 minutos pela manhã e um exercício de 10 minutos pela manhã.

esfriar à noite.

CO LL ABO R ATI O N

A segunda melhor maneira de aprender é colaborar com outras pessoas. Profissional desenvolvedores de software fazem um esforço especial para programar juntos, praticar juntos, projetar e planejar juntos. Ao fazer isso, eles aprendem muito uns com os outros e faça mais com mais rapidez e menos erros.

Isso não significa que você tenha que gastar 100% do seu tempo trabalhando com outras pessoas. O tempo sozinho também é muito importante. Por mais que eu goste de combinar programas com outros, fico louco se não consigo sair sozinho de vez em quando.

M E NTO R I N G

A melhor maneira de aprender é ensinando. Nada irá introduzir fatos e valores em seu cabeça mais rápido e mais difícil do que ter que comunicá-los às pessoas que você é responsável por. Portanto, o benefício do ensino é fortemente a favor do professor.

Da mesma forma, não há melhor maneira de trazer novas pessoas para uma organização do que sentar-se com eles e mostrar-lhes o que fazer. Profissionais levam para o lado pessoal responsabilidade de orientar os juniores. Eles não vão deixar um júnior se debater sem supervisão.

CONHEÇA SEU D O M A I N

É responsabilidade de todo profissional de software entender o domínio das soluções que estão programando. Se você estiver escrevendo um sistema de contabilidade, você deve conhecer a área de contabilidade. Se você estiver escrevendo um formulário de viagem, você deve conhecer a indústria de viagens. Você não precisa ser um especialista em domínio, mas há uma quantidade razoável de devida diligência que você deve realizar.

Ao iniciar um projeto em um novo domínio, leia um ou dois livros sobre o assunto.

Entreviste seu cliente e usuários sobre os fundamentos e princípios básicos do domínio. Passe algum tempo com os especialistas e tente entender seus princípios e valores.

É o pior tipo de comportamento não profissional simplesmente codificar a partir de uma especificação sem entender por que essa especificação faz sentido para o negócio. Em vez disso, você deve saber o suficiente sobre o domínio para ser capaz de reconhecer e desafiar erros de especificação.

I D E N T I F Y COM SEU PLOY E R /CU STO M E R

Os problemas do seu empregador são os seus problemas. Você precisa entender o que esses problemas são e trabalhar para encontrar as melhores soluções. À medida que você desenvolve um sistema

você precisa se colocar no lugar do seu empregador e certificar-se de que o

os recursos que você está desenvolvendo realmente atenderão às necessidades do seu empregador.

É fácil para os desenvolvedores se identificarem. É fácil cair em um nós

versus eles atitude com seu empregador. Os profissionais evitam isso a todo custo.

CAPÍTULO 1 PROFISSIONALISMO

22

H U M I L I T Y

Programar é um ato de criação. Quando escrevemos código, estamos criando algo do nada. Estamos corajosamente impondo ordem ao caos. Nós somos comandando com confiança, em detalhes precisos, os comportamentos de uma máquina que caso contrário, poderia causar danos incalculáveis. E assim, programar é um ato de arrogância suprema.

Os profissionais sabem que são arrogantes e não são falsamente humildes. Um profissional conhece seu trabalho e se orgulha de seu trabalho. Um profissional está confiante em suas habilidades e assume riscos ousados e calculados com base nessa confiança. Um profissional não é tímido.

Porém, um profissional também sabe que haverá momentos em que ele irá falhar, seus cálculos de risco estarão errados, suas habilidades serão insuficientes; ele vai olhar no espelho e ver um tolo arrogante sorrindo de volta para ele.

Então, quando um profissional for alvo de uma piada, ele será o primeiro a rir.

Ele nunca ridicularizará os outros, mas aceitará o ridículo quando for merecido e rirá quando não é. Ele não rebaixará outra pessoa por cometer um erro, porque ele sabe que pode ser o próximo a falhar.

Um profissional entende sua arrogância suprema e que o destino acabará por observá-lo e nivelá-lo. Quando esse objetivo se conecta, o melhor que você pode fazer é aproveitar. Conselho de Howard: Ria.

B I B L I O G R A F I A

[PPP2001]: Robert C. Martin, Princípios, Padrões e Práticas de Software Ágil
Desenvolvimento, Upper Saddle River, NJ: Prentice Hall, 2002.

23

2

DIZENDO NÃO

"Faça; ou não. Não há como tentar."

-Yoda

No início dos anos 70, eu e dois amigos meus de dezenove anos estávamos trabalhando em um sistema de contabilidade em tempo real para o sindicato dos caminhoneiros em Chicago para uma empresa denominado ASC. Se nomes como Jimmy Hoffa vêm à mente, deveriam. Você não mexer com os caminhoneiros em 1971.

Nosso sistema deveria entrar no ar em uma determinada data. Muito dinheiro estava andando naquela data. Nossa equipe vinha trabalhando 60, 70 e 80 horas por semana para tentar mantenha esse cronograma.

CAPÍTULO 2 DIZENDO NÃO

24

Uma semana antes da data de lançamento, finalmente montamos o sistema em sua totalidade. Havia muitos bugs e problemas para resolver, e nós freneticamente trabalhamos na lista. Mal havia tempo para comer e dormir, muito menos para pensar. Frank, o gerente do ASC, era coronel aposentado da Força Aérea. Ele era um daqueles tipo de gerente barulhento e direto. Era o seu caminho ou a estrada, e ele colocou você naquela rodovia, lançando-o de 10.000 pés sem pára-quadras. Nós jovens de dezenove anos mal conseguíamos fazer contato visual com ele.

Frank disse que isso tinha que ser feito até a data. Isso era tudo que havia para fazer. A data viria e estaríamos prontos. Período. Nenhuma discussão. Acabou e acabou.

Meu chefe, Bill, era um cara simpático. Ele trabalhava com Frank há alguns anos e entendi o que era possível com Frank e o que não era. Ele disse nós que iríamos ao vivo no encontro, não importa o que acontecesse.

Então entramos ao vivo no encontro. E foi um desastre terrível.

Havia uma dúzia de terminais half-duplex de 300 bauds que conectavam o Teamster sede em Chicago até nossa máquina, trinta milhas ao norte, nos subúrbios. Cada desses terminais trancados a cada 30 minutos ou mais. Nós vimos esse problema antes, mas não havia simulado o tráfego que os digitadores sindicais estavam de repente batendo em nosso sistema.

Para piorar a situação, as folhas rasgadas impressas nos teletipos ASR35 que também estivessem conectados ao nosso sistema por linhas telefônicas de 110 baud congelariam no meio da impressão.

A solução para esses congelamentos foi reiniciar. Então eles teriam que pegar todo mundo cujo terminal ainda estava ativo para terminar o trabalho e depois parar. Quando todos foram interrompidos, então eles nos chamariam para reiniciar. As pessoas que foram congeladas teriam que recomeçar. E isso estava acontecendo mais de uma vez por hora.

Depois de meio dia assim, o gerente do escritório do Teamster nos disse para desligar o sistema para baixo e não trazê-lo novamente até que esteja funcionando. Enquanto isso, eles haviam perdido meio dia de trabalho e teríamos que reinserir tudo usando o sistema antigo.

DIZENDO NÃO

Ouvimos os lamentos e rugidos de Frank por todo o prédio. Eles continuaram por um muito, muito tempo. Então Bill e o analista do nosso sistema, Jalil, vieram até nós e perguntaram quando poderíamos ter o sistema estável. Eu disse, “quatro semanas”.

A expressão em seus rostos era de horror e depois de determinação. “Não”, eles disseram, “isso deve estar funcionando até sexta-feira.

Então eu disse: “Olha, mal conseguimos fazer esse sistema funcionar na semana passada. Nós precisa sacudir os problemas e questões. Precisamos de quatro semanas.

Mas Bill e Jalil foram inflexíveis. “Não, realmente tem que ser sexta-feira. Você pode pelo menos tente?

Então o líder da nossa equipe disse: “OK, vamos tentar”.

Sexta-feira foi uma boa escolha. A carga do fim de semana foi bem menor. Nós conseguimos encontre mais problemas e corrija-os antes que chegue segunda-feira. Mesmo assim, todo castelo de cartas quase desabou novamente. Os problemas de congelamento continuaram em acontecer uma ou duas vezes por dia. Houve outros problemas também. Mas gradualmente, depois de mais algumas semanas, chegamos ao ponto em que o sistema as reclamações diminuíram e a vida normal parecia que poderia realmente ser possível.

E então, como eu disse na introdução, todos desistimos. E eles ficaram com uma verdadeira crise nas suas mãos. Eles tiveram que contratar um novo grupo de programadores para tentar

para lidar com o enorme fluxo de problemas provenientes do cliente.

A quem podemos culpar este desastre? Claramente, o estilo de Frank faz parte do problema. Suas intimidações dificultaram que ele ouvisse a verdade.

Certamente Bill e Jalil deveriam ter reprimido Frank com muito mais força do que eles fizeram. Certamente o nosso líder de equipe não deveria ter cedido à sexta-feira demanda. E certamente eu deveria ter continuado a dizer “não” em vez de ficar na fila atrás do líder da nossa equipe.

Profissionais falam a verdade ao poder. Os profissionais têm a coragem de dizer não seus gerentes.

CAPÍTULO 2 DIZENDO NÃO

26

Como você diz não ao seu chefe? Afinal, é o seu chefe! Você não deveria fazer o que seu chefe diz?

Não. Não se você for um profissional.

Os escravos não podem dizer não. Os trabalhadores podem hesitar em dizer não. Mas espera-se que os profissionais digam não. Na verdade, bons gestores desejam alguém que tem coragem de dizer não. É a única maneira de realmente fazer qualquer coisa.

A DVE R SAR IAL RO LE S

Um dos revisores deste livro realmente odiou este capítulo. Ele disse que quase fez com que ele largasse o livro. Ele construiu equipes onde não havia adversários relacionamentos; as equipes trabalharam juntas em harmonia e sem confrontos.

Estou feliz por este revisor, mas me pergunto se seus times são realmente tão confrontadores livre como ele supõe. E se forem, pergunto-me se são tão eficientes como poderia ser. Minha própria experiência tem sido que as decisões difíceis são melhor tomadas através do confronto de papéis adversários.

Os gerentes são pessoas com uma função a cumprir, e a maioria dos gerentes sabe como fazer isso trabalho muito bem. Parte desse trabalho é perseguir e defender seus objetivos como agressivamente quanto puderem.

Da mesma forma, os programadores também são pessoas com um trabalho a fazer, e a maioria eles sabem como fazer esse trabalho muito bem. Se eles são profissionais, eles perseguirão e defenderão os seus objectivos tão agressivamente quanto possível.

Quando seu gerente lhe disser que a página de login deve estar pronta amanhã, ele está perseguindo e defendendo um de seus objetivos. Ele está fazendo seu trabalho. Se você sabemos muito bem que é impossível concluir a página de login até amanhã, então você não estará fazendo seu trabalho se disser “OK, vou tentar”. A única maneira de fazer o seu trabalho, nesse ponto, é dizer “Não, isso é impossível”.

Mas você não precisa fazer o que seu gerente diz? Não, seu gerente é contando com você para defender seus objetivos tão agressivamente quanto ele defende os dele. É assim que vocês dois chegarão ao melhor resultado possível.

O melhor resultado possível é o objetivo que você e seu gerente compartilham. O

O truque é encontrar esse objetivo, e isso geralmente requer negociação.

A negociação às vezes pode ser agradável.

Mike: "Paula, preciso que a página de login esteja pronta amanhã."

Paula: "Nossa! Isso em breve? Bom, ok, vou tentar."

Mike: "OK, isso é ótimo. Obrigado!"

Essa foi uma pequena conversa agradável. Todo confronto foi evitado. Ambas as partes saiu sorrindo. Legal.

Mas ambas as partes estavam se comportando de maneira pouco profissional. Paula sabe muito bem que o página de login vai demorar mais de um dia, então ela está apenas mentindo. Ela pode não pense nisso como uma mentira. Talvez ela pense que realmente vai tentar, e talvez ela mantém alguma esperança de que ela realmente consiga fazer isso. Mas no final, é ainda é uma mentira.

Mike, por outro lado, aceitou o "Vou tentar" como "Sim". Isso é apenas uma coisa idiota fazer. Ele deveria saber que Paula estava tentando evitar o confronto, então ele deveria ter pressionado a questão dizendo: "Você parece hesitante. Tem certeza de que posso fazer isso amanhã?"

Aqui está outra conversa agradável.

Mike: "Paula, preciso que a página de login esteja pronta amanhã."

Paula: "Oh, desculpe Mike, mas vou levar mais tempo do que isso."

Mike: "Quando você acha que pode fazer isso?"

Paula: "Que tal daqui a duas semanas?"

Mike: (rabisca algo em seu cronômetro) "OK, obrigado."

Por mais agradável que fosse, também era terrivelmente disfuncional e totalmente pouco profissional. Ambas as partes falharam na busca pelo melhor resultado possível.

Em vez de perguntar se duas semanas estariam bem, Paula deveria ter ficado mais assertivo: "Vou levar duas semanas, Mike".

CAPÍTULO 2 DIZENDO NÃO

28

Mike, por outro lado, simplesmente aceitou a data sem questionar, como se seus próprios objetivos não importavam. É de se perguntar se ele não vai simplesmente denunciar de volta ao seu chefe que a demonstração do cliente terá que ser adiada por causa de Paula. Esse tipo de comportamento passivo-agressivo é moralmente repreensível.

Em todos estes casos, nenhuma das partes perseguiu um objectivo comum aceitável. Nenhum dos dois partido tem procurado o melhor resultado possível. Vamos tentar isso.

Mike: "Paula, preciso que a página de login esteja pronta amanhã."

Paula: "Não, Mike, é um trabalho de duas semanas."

Mike: "Duas semanas? Os arquitetos estimaram em três dias e é já tenho cinco anos!"

Paula: "Os arquitetos estavam errados, Mike. Eles fizeram suas estimativas antes o marketing do produto atendeu aos requisitos. Eu tenho pelo menos mais dez dias de trabalho para fazer nisso. Você não viu meu atualizado estimativa no wiki?"

Mike: (parecendo severo e tremendo de frustração) "Isso não é aceitável

Paula. Os clientes virão para uma demonstração amanhã e preciso mostre a eles a página de login funcionando."

Paula: "Em que parte da página de login você precisa trabalhar amanhã?"

Mike: "Preciso da página de login! Preciso conseguir fazer login."

Paula: "Mike, posso lhe dar um modelo da página de login que permitirá que você faça login. Estou trabalhando agora. Na verdade, não verificará seu nome de usuário e senha, e não enviará por e-mail uma senha esquecida para você. Não terá o banner de notícias da empresa "Times-quaring" na parte superior, e o botão de ajuda e o texto suspenso não serão trabalho. Ele não armazenará um cookie para lembrar de você na próxima vez e não imporá nenhuma restrição de permissão a você. Mas você será capaz de faça login. Isso basta?"

Mike: "Conseguirei fazer login?"

Paula: "Sim, você poderá fazer login."

Mike: "Isso é ótimo Paula, você salva vidas!" (sai bombeando o ar e dizendo "Sim!")

Eles alcançaram o melhor resultado possível. Eles fizeram isso dizendo não e então encontrar uma solução que fosse mutuamente aceitável para ambos. Eles estavam agindo como profissionais. A conversa foi um pouco contraditória e houve alguns momentos desconfortáveis, mas isso é de se esperar quando duas pessoas assertivamente perseguir objetivos que não estão em perfeito alinhamento.

O QUE É E POR QUÊ?

Talvez você pense que Paula deveria ter explicado por que a página de login foi vai demorar muito mais. Minha experiência é que o porquê é muito menos importante do que o fato. O fato é que a página de login levará duas semanas.

Por que levará duas semanas é apenas um detalhe.

Ainda assim, saber porquê pode ajudar Mike a compreender e, portanto, a aceitar, a

fato. Justo. E em situações em que Mike tem conhecimento técnico e

temperamento para entender, tais explicações podem ser úteis. Por outro

Por outro lado, Mike pode discordar da conclusão. Mike pode decidir que Paula

estava fazendo tudo errado. Ele pode dizer a ela que ela não precisa de todos aqueles testes,

ou toda essa revisão, ou a etapa 12 poderia ser omitida. Fornecendo muito

o detalhe pode ser um convite para o microgerenciamento.

STAKE ALTO

O momento mais importante para dizer não é quando os riscos são maiores. Quanto mais alto quanto mais está em jogo, mais valioso o não se torna.

Isto deveria ser evidente. Quando o custo do fracasso é tão alto que a sobrevivência

da sua empresa depende disso, você deve estar absolutamente determinado a dar

seus gerentes as melhores informações que puder. E isso muitas vezes significa dizer não.

Don (Diretor de Desenvolvimento): “Então, nossa estimativa atual para conclusão

do projeto Golden Goose é daqui a doze semanas, com um

incerteza de mais ou menos cinco semanas.”

Charles (CEO): (fica olhando fixamente por quinze segundos enquanto seu rosto fica vermelho) “Faça

você quer sentar aí e me dizer que podemos completar dezessete semanas

desde a entrega?”

CAPÍTULO 2 DIZENDO NÃO

30

Don: “Isso é possível, sim.”

Charles: (levanta-se, Don se levanta um segundo depois) “Droga, Don! Isso deveria ter sido feito há três semanas! Eu tenho Galitron me ligando todos os dias perguntando onde está o sistema deles.

Não vou dizer-lhes que terão de esperar mais quatro meses? Você tem que fazer melhor.”

Don: Chuck, eu te disse há três meses, depois de todas as demissões, que teríamos preciso de mais quatro meses. Quero dizer, Cristo Chuck, você cortou meu cado vinte por cento! Você disse a Galitron que chegaríamos atrasados?

Charles: “Você sabe muito bem que não. Não podemos nos dar ao luxo de perder esse pedido

Don. (Charles faz uma pausa, seu rosto fica branco) Sem Galitron, estamos

realmente mangueira. Você sabe disso, não é? E agora com esse atraso,

Estou com medo. . . O que direi ao conselho? (Ele se senta lentamente sentado, tentando não desmoronar.) Don, você precisa fazer melhor.”

Don: “Não há nada que eu possa fazer, Chuck. Já passamos por isso.

Galitron não cortará escopo e não aceitará qualquer provisório

lançamentos. Eles querem fazer a instalação uma vez e pronto

isso. Simplesmente não consigo fazer isso mais rápido. Isso não vai acontecer.”

Carlos: “Droga. Suponho que não importaria se eu lhe contasse seu trabalho estava em jogo.”

Don: “Me despedir não vai mudar a estimativa, Charles.”

Charles: “Terminamos aqui. Volte para sua equipe e mantenha este projeto em movimento. Tenho alguns telefonemas muito difíceis para fazer.

Claro, Charles deveria ter contado a Galitron há menos de três meses, quando ele primeiro

soube da nova estimativa. Pelo menos agora ele está fazendo a coisa certa ao

ligando para eles (e para o conselho). Mas se Don não tivesse se mantido firme, essas ligações poderia ter sido adiado ainda mais.

SENDU UM “JOGADOR DE EQUIPE”

Todos nós já ouvimos como é importante ser um “jogador de equipe”. Ser um jogador de equipe significa jogar sua posição da melhor maneira possível e ajudar seus

companheiros de equipe quando eles entram em apuros. Um jogador de equipe se comunica com frequência,

fica de olho em seus companheiros de equipe e executa seus próprios responsabilidades da melhor maneira possível.

SER UM “JOGADOR DE EQUIPE”

31

Um jogador de equipe não é alguém que diz sim o tempo todo. Considere este cenário:

Paula: "Mike, tenho essas estimativas para você. A equipe concorda que estaremos pronto para fazer uma demonstração em cerca de oito semanas, mais ou menos uma semana."

Mike: "Paula, já agendamos a demonstração para daqui a seis semanas."

Paula: "Sem ouvir de nós primeiro? Vamos Mike, você não pode forçar isso em nós."

Mike: "Já está feito."

Paula: (suspiro) "OK, olha, vou voltar para a equipe e descobrir o que podemos entregar com segurança em seis semanas, mas não será todo o sistema. Haverá faltarão alguns recursos e o carregamento de dados ficará incompleto."

Mike: "Paula, o cliente espera ver uma demonstração completa."

Paula: "Isso não vai acontecer, Mike."

Mike: "Droga. OK, elabore o melhor plano que puder e me avise amanhã."

Paula: "Isso eu posso fazer."

Mike: "Não há algo que você possa fazer para trazer esse encontro? Talvez existe uma maneira de trabalhar de maneira mais inteligente e criativa."

Paula: "Somos todos muito criativos, Mike. Temos um bom controle sobre o problema, e a data será de oito ou nove semanas, não seis."

Mike: "Você poderia fazer horas extras."

Paula: "Isso só nos faz ir mais devagar, Mike. Lembre-se da bagunça que fizemos última vez que exigimos horas extras?"

Mike: "Sim, mas isso não precisa acontecer desta vez."

Paula: "Será como da última vez, Mike. Confie em mim. Serão oito ou nove semanas, não seis."

Mike: "OK, me dê seu melhor plano, mas continue pensando em como consegui-lo feito em seis semanas. Eu sei que vocês vão descobrir alguma coisa."

Paula: "Não, Mike, não vamos. Vou conseguir um plano para seis semanas, mas será faltando muitos recursos e dados. É assim que vai ser."

Mike: "OK, Paula, mas aposto que vocês podem fazer milagres se tentarem."

(Paula se afasta balançando a cabeça.)

Mais tarde, na reunião de estratégia do Diretor...

CAPÍTULO 2 DIZENDO NÃO

32

Don: "OK Mike, como você sabe, o cliente virá para uma demonstração em seis semanas. Eles estão esperando ver tudo funcionando."

Mike: "Sim, e estaremos prontos. Minha equipe está se esforçando muito nisso e nós vamos fazer isso. Teremos que fazer algumas horas extras, e seja bem criativo, mas nós faremos acontecer!"

Don: "É ótimo que você e sua equipe trabalhem em equipe."

Quem foram os verdadeiros jogadores da equipe neste cenário? Paula estava jogando pelo time, porque ela representava o que poderia e não poderia ser feito da melhor maneira possível habilidade. Ela defendeu agressivamente sua posição, apesar da bajulação e bajulando Mike. Mike estava jogando em um time de um só. Mike é para Mike. Ele é claramente não está na equipe de Paula porque ele simplesmente a comprometeu com algo que ela disse explicitamente que não poderia fazer. Ele também não está no time de Don (embora ele discordo) porque ele simplesmente mentiu descaradamente.

Então, por que Mike fez isso? Ele queria que Don o visse como um jogador de equipe, e ele tem fé em sua capacidade de persuadir e manipular Paula para tentar os seis prazo da semana. Mike não é mau; ele está muito confiante em sua capacidade de conseguir pessoas façam o que ele quiser.

TENTANDO

A pior coisa que Paula poderia fazer em resposta às manipulações de Mike seria dizer "OK, vamos tentar. Detesto canalizar Yoda aqui, mas neste caso ele está correto. Há sem tentar.

Talvez você não goste dessa ideia? Talvez você pense que tentar é uma coisa positiva para fazer. Afinal, Colombo teria descoberto a América se não tivesse tentado?

A palavra tentar tem muitas definições. A definição que discordo aqui é "para aplique um esforço extra." Que esforço extra Paula poderia aplicar para preparar a demonstração em hora? Se houver um esforço extra que ela possa aplicar, ela e sua equipe não devem têm aplicado todos os seus esforços antes. Eles deviam estar segurando algum esforço na reserva.¹

1. Como Foghorn Leghorn: "Sempre mantenho minhas penas numeradas exatamente para esse tipo de emergência."

SER UM “JOGADOR DE EQUIPE”

33

A promessa de tentar é uma admissão de que você está se segurando, que você tem um reservatório de esforço extra que você pode aplicar. A promessa de tentar é uma admissão que o objectivo é alcançável através da aplicação deste esforço extra; além disso, é um compromisso de aplicar esse esforço extra para atingir a meta. Portanto, por prometendo tentar você está se comprometendo a ter sucesso. Isso coloca o fardo sobre você. Se a sua “tentativa” não levar ao resultado desejado, você terá falhado.

Você tem um reservatório extra de energia que está retendo? Se você aplique essas reservas, você conseguirá cumprir a meta? Ou, prometendo tentar você está simplesmente se preparando para o fracasso?

Ao prometer tentar, você está prometendo mudar seus planos. Afinal, os planos você tinha eram insuficientes. Ao prometer tentar você está dizendo que tem um novo plano. Qual é esse novo plano? Que mudança você fará em seu comportamento? Que coisas diferentes você fará porque agora está “tentando”?

Se você não tiver um novo plano, se não mudar seu comportamento, se você faz tudo exatamente como faria antes de prometer “tentar”, então o que significa tentar?

Se você não está guardando alguma energia de reserva, se não tem um novo plano, se você não vai mudar seu comportamento e se está razoavelmente confiante na sua estimativa original, então prometer tentar é fundamentalmente desonesto. Você estão mentindo. E provavelmente você está fazendo isso para salvar a aparência e evitar um confronto. A abordagem de Paula foi muito melhor. Ela continuou a lembrar a Mike que o a estimativa da equipe era incerta. Ela sempre dizia “oito ou nove semanas”. Ela enfatizou a incerteza e nunca recuou. Ela nunca sugeriu que houvesse pode ser algum esforço extra, ou algum novo plano, ou alguma mudança de comportamento que poderia reduzir essa incerteza.

Três semanas depois...

Mike: “Paula, a demonstração será daqui a três semanas e os clientes estão exigindo ver o upload de arquivos funcionando.”

Paula: “Mike, isso não está na lista de recursos com os quais concordamos.”

CAPÍTULO 2 DIZENDO NÃO

34

Mike: "Eu sei, mas eles estão exigindo isso".

Paula: "OK, isso significa que o Single Sign-on ou o Backup terão que ser retirado da demonstração.

Mike: "Absolutamente não! Eles esperam ver esses recursos funcionando também!"

Paula: "Então eles estão esperando ver todos os recursos funcionando. o que você está me dizendo? Eu te disse que isso não iria acontecer."

Mike: "Sinto muito, Paula, mas o cliente simplesmente não cede. quero ver tudo."

Paula: "Isso não vai acontecer, Mike. Simplesmente não vai acontecer."

Mike: "Qual é Paula, vocês não podem pelo menos tentar?"

Paula: "Mike, eu poderia tentar levitar. Eu poderia tentar transformar chumbo em ouro. Eu poderia tentar atravessar o Atlântico a nado. Você acha que eu teria sucesso?"

Mike: "Agora você está sendo irracional. Não estou pedindo o impossível."

Paula: "Sim, Mike, você é."

(Mike sorri, acena com a cabeça e se vira para ir embora.)

Mike: "Tenho fé em você Paula; sei que você não vai me decepcionar."

Paula: (falando para as costas de Mike) "Mike, você está sonhando. Isso não vai terminar bem."

(Mike apenas acena sem se virar.)

AGG R E S S Ã O P S S I V

Paula tem uma decisão interessante a tomar. Ela suspeita que Mike não está contando Don sobre suas estimativas. Ela poderia simplesmente deixar Mike caminhar até o fim do penhasco. Ela poderia garantir que cópias de todos os memorandos apropriados estivessem arquivadas, para que quando o desastre acontecer ela poderá mostrar o que disse a Mike, e quando ela disse a ele. Isso é agressão passiva. Ela simplesmente deixou Mike se enforcar. Ou ela poderia tentar evitar o desastre comunicando-se diretamente com Don. Isso é arriscado, com certeza, mas é também o que realmente significa ser um jogador de equipe. Quando um trem de carga está vindo em sua direção e você é o único que pode ver, você pode sair silenciosamente da pista e ver todos os outros correrem acabou, ou você pode gritar "Trem! Saia da pista!"

SER UM “JOGADOR DE EQUIPE”

35

Dois dias depois...

Paula: "Mike, você contou a Don sobre minhas estimativas? Ele contou ao cliente que a demonstração não terá o recurso de upload de arquivos funcionando?"

Mike: "Paula, você disse que faria isso funcionar para mim."

Paula: "Não, Mike, não fiz isso. Eu disse que era impossível. Aqui está uma cópia do memorando que lhe enviei depois da nossa conversa."

Mike: "Sim, mas você ia tentar de qualquer maneira, certo?"

Paula: "Já tivemos essa discussão, Mike. Lembra, ouro e chumbo?"

Mike: (suspira) "Olha, Paula, você simplesmente tem que fazer isso. Você apenas tem que fazer. Por favor faça o que for preciso, mas você só precisa fazer isso acontecer para mim."

Paula: "Mike. Você está errado. Não preciso fazer isso acontecer para você.

O que tenho que fazer, se você não fizer isso, é contar a Don.

Mike: "Isso seria passar da minha cabeça, você não faria isso."

Paula: "Eu não quero Mike, mas farei se você me forçar."

Mike: "Ah, Paula..."

Paula: "Olha, Mike, os recursos não vão ficar prontos a tempo para o demonstração. Você precisa colocar isso na sua cabeça. Pare de tentar convencer que eu trabalhe mais. Pare de se iludir pensando que de alguma forma eu irei tirar um coelho da cartola. Encare o fato de que você precisa contar a Don, e você tem que contar a ele hoje.

Mike: (olhos arregalados) "Hoje?"

Paula: "Sim, Mike. Hoje. Porque amanhã espero ter uma reunião com você e Don sobre quais recursos incluir na demonstração. Se essa reunião não acontecer amanhã, então serei forçado a ir para Don mesmo. Aqui está uma cópia do memorando que explica exatamente isso."

Mike: "Você está apenas protegendo sua bunda!"

Paula: "Mike, estou tentando nos proteger. Você pode imaginar o desastre se o cliente vier aqui esperando uma demonstração completa e não conseguirmos entregar?"

O que acontece no final com Paula e Mike? Vou deixar para você resolver possibilidades. A questão é que Paula se comportou de maneira muito profissional. Ela tem disse não nos momentos certos e da maneira certa. Ela disse não quando empurrada

CAPÍTULO 2 DIZENDO NÃO

36

para alterar suas estimativas. Ela disse não quando manipulada, bajulada e implorada. E, o mais importante, ela disse não à autoilusão e à inação de Mike. Paula estava jogando pelo time. Mike precisava de ajuda, e ela usou todos os meios ao seu alcance. poder para ajudá-lo.

O CUSTO DE DIZER SIM

Na maioria das vezes queremos dizer sim. Na verdade, equipes saudáveis se esforçam para encontrar uma maneira

para dizer sim. Gerentes e desenvolvedores em equipes bem administradas negociarão entre si outro até que cheguem a um plano de ação mutuamente acordado.

Mas, como vimos, às vezes a única maneira de chegar ao sim certo é ser sem medo, então diga não.

Considere a seguinte história que John Blanco postou em seu blog.² É reimpresso aqui com permissão. Ao lê-lo, pergunte-se quando e como ele deveria ter dito não.

2. <http://raptureinvenice.com/?p=63>

É B O O D C O D E I M P S S I B L E?

Quando você chega à adolescência, decide que quer ser desenvolvedor de software. Durante nos anos do ensino médio, você aprende a escrever software usando princípios orientados a objetos. Ao se formar na faculdade, você aplica todos os princípios que aprendeu em áreas como inteligência artificial ou gráficos 3D.

E quando você entra no circuito profissional, você começa sua busca sem fim para escrever código de qualidade comercial, sustentável e “perfeito” que resistirá ao teste do tempo.

Qualidade comercial. Huh. Isso é muito engraçado.

Eu me considero sortudo, adoro padrões de design. Gosto de estudar a teoria da codificação perfeição. Não tenho nenhum problema em iniciar uma discussão de uma hora sobre por que meu XP a escolha do parceiro quanto à hierarquia de herança está errada - que HAS-A é melhor que IS-A nesse sentido

muitos casos. Mas algo está me incomodando ultimamente e estou me perguntando uma coisa. . .

. . . Um bom código é impossível no desenvolvimento de software moderno?

O CUSTO DE DIZER SIM

37

A proposta típica de projeto

Como desenvolvedor contratado em tempo integral (e meio período), passo meus dias (e noites) desenvolvendo

aplicativos móveis para clientes. E o que aprendi ao longo dos muitos anos em que faço isso

é que as demandas do trabalho do cliente me impedem de escrever os aplicativos de qualidade real que eu gostaria.

Antes de começar, deixe-me apenas dizer que não é por falta de tentativa. Adoro o tópico de código limpo. eu

Não conheço ninguém que busque aquele design de software perfeito como eu. É a execução que

Acho mais evasivo, e não pelo motivo que você pensa.

Aqui, deixe-me contar uma história.

No final do ano passado, uma empresa bastante conhecida lançou uma RFP (Solicitação de

Proposta) para ter um aplicativo construído para eles. Eles são um grande varejista, mas por uma questão de anonimato

vamos chamá-los de Gorilla Mart. Eles dizem que precisam criar uma presença no iPhone e gostariam de ter uma

aplicativo produzido para eles pela Black Friday. O problema? Já é 1º de novembro. Isso deixa apenas menos de 4 semanas para criar o aplicativo. Ah, e neste momento a Apple ainda está demorando duas semanas para aprovar

aplicativos. (Ah, os bons e velhos tempos.) Então, espere, este aplicativo tem que ser escrito em . . . DUAS SEMANAS?!?!

Sim. Temos duas semanas para escrever este aplicativo. E, infelizmente, ganhamos a licitação. (Em negócios, a importância do cliente é importante.) Isso vai acontecer.

“Mas está tudo bem”, diz o executivo nº 1 do Gorilla Mart. “O aplicativo é simples. Basta mostrar usuários alguns produtos de nosso catálogo e permitem que eles pesquisem os locais das lojas. Nós já faça isso em nosso site. Forneceremos os gráficos também. Você provavelmente pode - o que é a palavra - sim, codifique-a!

O executivo nº 2 do Gorilla Mart entra na conversa. “E só precisamos de alguns cupons que o usuário pode mostrar na caixa registradora. O aplicativo será descartável. Vamos tirá-lo pela porta, e então, para a Fase II, faremos algo maior e melhor do zero.”

E então está acontecendo. Apesar de anos de lembretes constantes de que cada recurso solicitado por um cliente

pois sempre será mais complexo escrever do que explicar, você vai em frente. Você realmente acredita que desta vez isso realmente pode ser feito em duas semanas. Sim! Nós podemos fazer isso! Desta vez é diferente!

São apenas alguns gráficos e uma chamada de serviço para obter a localização da loja. XML! Sem suor. Nós podemos fazer

isso. Estou animado! Vamos!

Leva apenas um dia para você e a realidade se conhecerem novamente.

Eu: Então, você pode me dar as informações necessárias para ligar para o serviço web de localização da sua loja?

O Cliente: O que é um serviço web?

Eu:

Continua

CAPÍTULO 2 DIZENDO NÃO

38

E foi exatamente assim que aconteceu. O serviço de localização da loja, encontrado exatamente onde está deveria estar no canto superior direito do site, não é um serviço da web. É gerado por Código Java. Ix-não com a API-ay. E para começar, é hospedado por um parceiro estratégico do Gorilla Mart.

Digite o nefasto “terceiro”.

Em termos de cliente, um “terceiro” é semelhante a Angelina Jolie. Apesar da promessa de que você será capaz de

ter uma conversa esclarecedora durante uma boa refeição e, com sorte, sair depois... desculpe, isso não está acontecendo. Você apenas terá que fantasiar sobre isso enquanto cuida dos negócios sozinho.

No meu caso, a única coisa que consegui arrancar do Gorilla Mart foi uma foto atual de seu listagens de lojas atuais em um arquivo Excel. Tive que escrever o código de pesquisa de localização da loja do zero.

O golpe duplo veio mais tarde naquele dia: eles queriam os dados do produto e do cupom online então pode ser alterado semanalmente. Lá se vai a codificação! Duas semanas para escrever um aplicativo para iPhone

agora são duas semanas para escrever um aplicativo para iPhone, um backend PHP e integrá-los juntos—. . . O que? Eles querem que eu cuide do controle de qualidade também?

Para compensar o trabalho extra, a codificação terá que ser um pouco mais rápida. Esqueça esse resumo fábrica. Use um loop for grande e grosso em vez do composto, não há tempo!

Um bom código tornou-se impossível.

Duas semanas para conclusão

Deixe-me dizer-lhe que duas semanas foram bastante miseráveis. Primeiro, dois dos dias foram eliminados devido às reuniões de dia inteiro para meu próximo projeto. (Isso amplifica o quão curto foi o período vai ser.) No final das contas, eu realmente tive oito dias para fazer as coisas. Na primeira semana em que trabalhei

74 horas e na próxima semana. . . Deus . . . Eu nem me lembro, foi erradicado da minha sinapses. Provavelmente uma coisa boa.

Passei aqueles oito dias escrevendo código furioso. Usei todas as ferramentas disponíveis para obtê-lo feito: copiar e colar (também conhecido como código reutilizável), números mágicos (evitando a duplicação de

definindo constantes e então, suspiro!, redigitando-as) e absolutamente NENHUM teste de unidade! (Quem precisa de barras vermelhas em um momento como este, isso só me desmotivaria!)

Era um código muito ruim e nunca tive tempo de refatorar. Considerando o prazo, no entanto, era realmente muito estelar e, afinal, era um código “descartável”, certo? Algum disso parece familiar? Bem, espere, vai melhorar.

Enquanto eu estava dando os retoques finais no aplicativo (os retoques finais foram escrever todo o código do servidor), comecei a olhar a base de código e me perguntei se talvez valesse a pena.

Afinal, o aplicativo estava pronto. Eu sobrevivi!

“Ei, acabamos de contratar Bob, ele está muito ocupado e não pôde fazer a ligação, mas ele disse deveríamos exigir que os usuários fornecessem seus endereços de e-mail para receber os cupons. Ele

ainda não viu o aplicativo, mas acha que seria uma ótima ideia! Também queremos um relatório sistema para obter esses e-mails do servidor. Um que seja bom e não muito caro.

(Espere, a última parte foi Monty Python.) Falando em cupons, eles precisam ser capazes de expirar após um número de dias que especificamos. Ah, e...”

Vamos recuar. O que sabemos sobre o que é um bom código? Um bom código deve ser extensível. Sustentável. Deve ser passível de modificação. Deveria ser lido como prosa.

Bem, este não era um bom código.

Outra coisa. Se você quer ser um desenvolvedor melhor, você deve sempre manter isso inevitavelmente em mente.

mente: O cliente sempre estenderá o prazo. Eles sempre vão querer mais recursos. Eles sempre vai querer mudanças – TARDE. E aqui está a fórmula do que esperar:

(nº de executivos)²

+ 2 * # de Novos Executivos

+ # dos filhos de Bob

= DIAS ADICIONADOS NO ÚLTIMO HORA

Agora, os executivos são pessoas decentes. Eu acho. Eles sustentam a família (presumindo que Satanás tenha

aprovado por terem um). Eles querem que o aplicativo tenha sucesso (é hora da promoção!). O

O problema é que todos querem reivindicar diretamente o sucesso do projeto. Quando tudo estiver dito e feito,

todos eles querem apontar algum recurso ou decisão de design que cada um possa chamar de seu.

Então, voltando à história, adicionamos mais alguns dias ao projeto e obtivemos o recurso de e-mail feito. E então desmaiei de exaustão.

Os clientes nunca se importam tanto quanto você

Os clientes, apesar dos seus protestos, apesar da sua aparente urgência, nunca se importam tanto quanto você faz para que o aplicativo chegue na hora certa. Na tarde em que dobrei o aplicativo, enviei um e-mail com a construção final para todos os stakeholders, Executivos (sibilos!), Gerentes e assim por diante.

“ESTÁ FEITO! TRAGO PARA VOCÊ V1.0! LOUVADO O TEU NOME.” Apertei Enviar, recostei-me na cadeira,

e com um sorriso maroto comecei a fantasiar como a empresa iria me levar até eles

ombros e liderei uma procissão pela 42nd Street enquanto fui coroado “Maior Desenvolvedor

Eva. No mínimo, meu rosto estaria em todas as propagandas deles, certo?

Engraçado, eles não pareciam concordar. Na verdade, eu não tinha certeza do que eles pensavam. Eu não ouvi nada. Não é um

espiar. Acontece que o pessoal do Gorilla Mart estava ansioso e já havia passado para o próximo passo.

Você acha que eu minto? Confira isso. Fui até a loja da Apple sem preencher um aplicativo

descrição. Eu havia solicitado um do Gorilla Mart, e eles não me responderam e

não havia tempo para esperar. (Veja o parágrafo anterior.) Eu os escrevi novamente. E novamente. eu tenho

Continua

CAPÍTULO 2 DIZENDO NÃO

40

parte de nossa própria administração sobre isso. Duas vezes recebi resposta e duas vezes me disseram: “O que você

precisa de novo?” **PRECISO DA DESCRIÇÃO DO APLICATIVO!**

Uma semana depois, a Apple começou a testar o aplicativo. Geralmente é um momento de alegria, mas foi em vez disso, é um momento de pavor mortal. Como esperado, no final do dia o aplicativo foi rejeitado. Foi sobre a desculpa mais triste e pobre para permitir uma rejeição que posso imaginar: “Falta um aplicativo no aplicativo

descrição.” Funcionalmente perfeito; nenhuma descrição do aplicativo. E por esta razão Gorilla Mart não tinham o aplicativo pronto para a Black Friday. Fiquei muito chateado.

Eu sacrifiquei minha família por um super sprint de duas semanas, e ninguém no Gorilla Mart poderia estar se preocupou em criar uma descrição do aplicativo com uma semana de antecedência. Eles nos deram uma hora depois

a rejeição – aparentemente esse foi o sinal para começar a trabalhar.

Se eu estivesse chateado antes, ficaria lívido uma semana e meia depois disso. Você vê, eles ainda não nos deu dados reais. Os produtos e cupons no servidor eram falsos. Imaginário.

O código do cupom era 1234567890. Você sabe, bobagem falsa. (Bolonha é escrito bobagem quando usado nesse contexto, aliás.)

E foi naquela manhã fatídica que verifiquei o Portal e **A APLICAÇÃO ESTAVA DISPONÍVEL!**

Dados falsos e tudo! Eu gritei de horror e chamei quem pude e gritei:

“PRECISO DOS DADOS!” e a mulher do outro lado me perguntou se eu precisava de bombeiros ou polícia, então desliguei no 911. Mas então liguei para o Gorilla Mart e disse: “PRECISO DE DADOS!” E eu vou nunca se esqueça da resposta:

Ah, olá, João. Temos um novo vice-presidente e decidimos não divulgar. Retire-o do App Store, sim?

No final, descobriu-se que pelo menos 11 pessoas cadastraram seus endereços de e-mail no banco de dados, o que significava que havia 11 pessoas que poderiam entrar em um Gorilla Mart com um cupom falso do iPhone a reboque. Rapaz, isso pode ficar feio.

Quando tudo foi dito e feito, o cliente havia dito uma coisa corretamente o tempo todo: o código estava um descartável. O único problema é que ele nunca foi lançado.

Resultado? Apressado para concluir, lento para chegar ao mercado

A lição da história é que suas partes interessadas, sejam clientes externos ou internos gerenciamento, descobriram como fazer com que os desenvolvedores escrevam código rapidamente. Efetivamente? Não.

Rapidamente? Sim. Veja como funciona:

- Diga ao desenvolvedor que o aplicativo é simples. Isto serve para pressionar o equipe de desenvolvimento em um falso estado de espírito. Também faz com que os desenvolvedores para começar a trabalhar mais cedo, onde eles...

CÓDIGO IMPOSSÍVEL

CÓDIGO DE IMPSSIBLE

Na história, quando John pergunta “É impossível um bom código?”, ele está realmente perguntando “É profissionalismo impossível?” Afinal, não foi só o código que sofreu em sua história de disfunção. Eram sua família, seu empregador, seu cliente e os usuários.

Todo mundo perdeu³ nesta aventura. E eles perderam por falta de profissionalismo.

Então, quem estava agindo de forma não profissional? John deixa claro que ele acha que foi os executivos do Gorilla Mart. Afinal, seu manual era bastante claro acusação de seu mau comportamento. Mas o comportamento deles foi ruim? Eu não acho.

3. Com a possível exceção do empregador direto de John, embora eu aposto que eles também perderam.

- Adicione recursos criticando a equipe por não reconhecer sua necessidade. Em neste caso, o conteúdo codificado exigiria atualizações do aplicativo para mudar. Como eu poderia não perceber isso? Eu fiz, mas recebi uma falsa prometi antes, é por isso. Ou um cliente contratará “um cara novo” que é reconheceu que há alguma omissão óbvia. Um dia um cliente dirá que acabei de contratar Steve Jobs e podemos adicionar alquimia ao aplicativo? Então eles vão...

- Empurre o prazo. Repetidamente. Os desenvolvedores trabalham mais rápido e mais difíceis (e, aliás, são mais propensos a erros, mas quem se importa com isso, certo?) com alguns dias para cumprir o prazo. Por que dizer a eles que você podem adiar ainda mais a data enquanto eles estão sendo tão produtivos? Pegue vantagem disso! E assim vai, acrescentam-se alguns dias, acrescenta-se uma semana, apenas quando você trabalhou em um turno de 20 horas para deixar tudo certo. É como um burro e uma cenoura, exceto que você não é tratado tão bem quanto o burro. É um manual brilhante. Você pode culpá-los por pensarem que funciona? Mas eles não veem o Código horrível. E assim acontece repetidamente, apesar dos resultados. Numa economia globalizada, onde as empresas estão sujeitas ao todo-poderoso dólar e aumentando o o preço das ações envolve demissões, equipes sobrecarregadas e terceirização, esta estratégia que mostrei a você reduzir os custos do desenvolvedor está tornando um bom código obsoleto. Como desenvolvedores, seremos questionados/ instruídos/enganados a escrever o dobro do código na metade do tempo se não tomarmos cuidado.

CAPÍTULO 2 DIZENDO NÃO

42

O pessoal do Gorilla Mart queria a opção de ter um aplicativo para iPhone no Black Sexta-feira. Eles estavam dispostos a pagar para ter essa opção. Eles encontraram alguém disposto a fornecer essa opção. Então, como você pode culpá-los?

Sim, é verdade, houve algumas falhas de comunicação. Aparentemente o executivos não sabiam o que realmente era um serviço web, e havia todos os questões normais de uma parte de uma grande corporação sem saber o que a outra parte está fazendo. Mas tudo isso deveria ser esperado. John até admite isso quando ele diz: “Apesar de anos de lembretes constantes de que cada recurso de um cliente pede será sempre mais complexo de escrever do que de explicar. . .”

Então, se o culpado não foi o Gorilla Mart, quem foi?

Talvez tenha sido o empregador direto de John. João não disse isso explicitamente, mas há foi uma dica quando ele disse, entre parênteses: “Nos negócios, a importância do cliente importa.” Então, o empregador de John fez promessas irracionais ao Gorilla Mart?

Eles pressionaram John, direta ou indiretamente, para que fizesse essas promessas? se tornar realidade? João não diz isso, então só podemos nos perguntar.

Mesmo assim, onde está a responsabilidade de João em tudo isso? Eu coloquei a culpa diretamente João. John foi quem aceitou o prazo inicial de duas semanas, sabendo bem, os projetos geralmente são mais complexos do que parecem. João é aquele que aceitou a necessidade de escrever o servidor PHP. João foi quem aceitou o registro do e-mail e o vencimento do cupom. John é quem trabalhou Dias de 20 horas e semanas de 90 horas. John é quem se subtraiu de sua família e sua vida para cumprir esse prazo.

E por que João fez isso? Ele nos diz em termos inequívocos: “Apertei Enviar, coloquei de volta na minha cadeira e com um sorriso maroto comecei a fantasiar como a empresa me colocariam em seus ombros e liderariam uma procissão pela 42nd Street enquanto fui coroado “Maior Desenvolvedor de Todos”. Em suma, John estava tentando ser um herói. Ele viu sua chance de receber elogios e foi em frente. Ele se inclinou e agarrou o anel de latão.

Os profissionais costumam ser heróis, mas não porque tentam ser. Os profissionais tornam-se heróis quando realizam um trabalho bem feito, dentro do prazo e do orçamento. Ao tentar se tornar o homem do momento, o salvador do dia, John não estava agindo como um profissional.

CÓDIGO IMPOSSÍVEL

43

John deveria ter dito não ao prazo original de duas semanas. Ou se não, então ele deveria ter dito não quando descobriu que não havia serviço web. Ele deveria ter disse não ao pedido de cadastro de e-mail e vencimento de cupom. Ele deveria disseram não a qualquer coisa que exigisse horas extras e sacrifícios horríveis.

Mas acima de tudo, John deveria ter dito não à sua própria decisão interna de que o a única maneira de terminar esse trabalho a tempo era fazer uma grande bagunça. Observe o que João disse sobre bons códigos e testes de unidade:

"Para compensar o trabalho extra, a codificação terá que ser um pouco mais rápida. Esqueça aquela fábrica abstrata. Use um loop for grande e grosso em vez do composto, não há hora!"

E novamente:

"Passei aqueles oito dias escrevendo código furioso. Usei todas as ferramentas disponíveis para eu para fazer isso: copiar e colar (também conhecido como código reutilizável), números mágicos (evitando a duplicação de definição de constantes e então, suspiro!, redigitando-as), e absolutamente NENHUM teste de unidade! (Quem precisa de barras vermelhas num momento como este, seria me desmotiva!)"

Dizer sim a essas decisões foi o verdadeiro cerne do fracasso. João aceitou isso a única maneira de ter sucesso era se comportar de maneira pouco profissional, então ele colheu o recompensa apropriada.

Isso pode parecer duro. Não foi planejado dessa forma. Nos capítulos anteriores eu descrevi como cometi o mesmo erro em minha carreira, mais de uma vez. O a tentação de ser um herói e "resolver o problema" é enorme. O que todos nós temos que percebemos é que dizer sim ao abandono de nossas disciplinas profissionais não é o caminho para resolver problemas. Abandonar essas disciplinas é a maneira como você cria problemas.

Com isso, posso finalmente responder à pergunta inicial de John:

"É impossível um bom código? O profissionalismo é impossível?"

Resposta: eu digo não.

Esta página foi intencionalmente deixada em branco

DIZENDO SIM

Você sabia que eu inventei o correio de voz? É verdade. Na verdade, eram três nós, que detemos a patente do correio de voz. Ken Finder, Jerry Fitzpatrick e eu. no início dos anos 80, e trabalhávamos para uma empresa chamada Teradyne. Nosso CEO nos contratou para criar um novo tipo de produto, e inventamos “A Recepcionista Eletrônica” ou ER, abreviadamente.

CAPÍTULO 3 DIZENDO SIM

46

Todos vocês sabem o que é ER. ER é uma daquelas máquinas horríveis que responde às telefona para empresas e faz todos os tipos de perguntas idiotas que você precisa responder pressionando botões. (“Para inglês, pressione 1.”)

Nosso pronto-socorro atenderia o telefone de uma empresa e pediria que você discasse o nome de a pessoa que você queria. Ele pediria que você pronunciasse seu nome e então ligaria para a pessoa em questão. Anunciaria a chamada e perguntaria se deveria ser aceito. Nesse caso, a chamada seria conectada e desligada.

Você poderia dizer ao pronto-socorro onde estava. Você poderia fornecer vários números de telefone para tente. Então, se você estivesse no escritório de outra pessoa, o pronto-socorro poderia encontrá-lo. Se você estivesse em

para casa, o pronto-socorro pode encontrar você. Se você estivesse em uma cidade diferente, o pronto-socorro poderia encontrá-lo.

E, no final, se ER não conseguisse encontrar você, seria necessária uma mensagem. É onde a mensagem de voz chegou.

Curiosamente, Teradyne não conseguiu descobrir como vender ER. O projeto correu fora do orçamento e se transformou em algo que sabíamos vender - CDS,

O Craft Dispatch System, para enviar reparadores de telefones para o próximo trabalho. E Teradyne também retirou a patente sem nos avisar. (!) O atual titular da patente depositada três meses depois de nós. (!!)

Muito depois da transformação do ER em CDS, mas muito antes de eu descobrir que o a patente foi retirada. Esperei em uma árvore pelo CEO da empresa. Nós tinha um grande carvalho na frente do prédio. Eu subi e esperei seu Jaguar para estacionar. Encontrei-o na porta e pedi alguns minutos. Ele obrigado.

Eu disse a ele que realmente precisávamos reiniciar o projeto de emergência. Eu disse a ele que estava certeza de que poderia ganhar dinheiro. Ele me surpreendeu ao dizer: “OK, Bob, prepare um plano. Mostre-me como posso ganhar dinheiro. Se você fizer isso, e eu acreditar, vou começar Pronto-socorro novamente.

Eu não esperava isso. Eu esperava que ele dissesse: "Você está certo, Bob. Vou começar esse projeto novamente e vou descobrir como ganhar dinheiro

1. Não que a patente valesse algum dinheiro para mim. Eu o vendi para Teradyne por US\$ 1, de acordo com meu emprego contrato (e não recebi o dólar).

UMA LINGUAGEM DE COMPROMISSO

47

isso.” Mas não. Ele colocou o fardo de volta em mim. E foi um fardo eu era ambivalente sobre. Afinal, eu era um cara de software, não de dinheiro. Eu queria trabalhar no Projeto ER, não será responsável por lucros e perdas. Mas eu não queria mostrar meu ambivalência. Então agradeci e saí do escritório com estas palavras:

“Obrigado, Russ. Estou comprometido. . . Eu acho.”

Com isso, deixe-me apresentar Roy Osherove, que lhe dirá como patética essa afirmação foi.

A L A N G U A G E D E C O M M I T M E N T

Por Roy Osherove

Diga. Quer dizer. Faça.

Existem três partes para assumir um compromisso.

1. Você diz que fará isso.
2. Você está falando sério.
3. Você realmente faz isso.

Mas quantas vezes encontramos outras pessoas (não nós mesmos, é claro!) que nunca vai até o fim com esses três estágios?

- Você pergunta ao cara de TI por que a rede está tão lenta e ele diz “Sim. Nós realmente preciso adquirir alguns roteadores novos.” E você sabe que nada vai acontecer em essa categoria.

- Você pede a um membro da equipe que execute alguns testes manuais antes de fazer check-in no código-fonte, e ele responde: “Claro. Espero chegar lá até o final do dia.” E de alguma forma você sente que precisará perguntar amanhã se algum teste realmente ocorreu antes do check-in.

- Seu chefe entra na sala e murmura: “temos que ir mais rápido”. E você sabe que ele realmente quer dizer que VOCÊ precisa se mover mais rápido. Ele não vai fazer nada sobre isso.

CAPÍTULO 3 DIZENDO SIM

48

Há muito poucas pessoas que, quando dizem alguma coisa, falam sério e depois realmente fazer isso. Há alguns que dirão coisas e serão sinceros, mas eles nunca conseguem fazer isso. E há muito mais pessoas que prometem coisas e nem pretendo fazê-los. Já ouviu alguém dizer: “Cara, eu realmente preciso perder algum peso”, e você sabia que eles não fariam nada a respeito? Isso acontece o tempo todo.

Por que continuamos tendo aquela estranha sensação de que, na maioria das vezes, as pessoas não estão realmente comprometido em fazer algo?

Pior ainda, muitas vezes a nossa intuição pode falhar. Às vezes gostaríamos de acreditar em alguém realmente significa o que eles dizem quando realmente não o fazem. Gostaríamos de acreditar em um desenvolvedor quando eles dizem, pressionados até o canto, que podem terminar aqueles dois tarefa semanal em uma semana, mas não deveríamos.

Em vez de confiar em nossos instintos, podemos usar alguns truques relacionados à linguagem para tentar descobrir se as pessoas realmente querem dizer o que dizem. E ao mudar o que dizemos, nós podemos começar a cuidar das etapas 1 e 2 da lista anterior por conta própria. Quando nós digamos que nos comprometeremos com algo e precisamos ser sinceros.

RECONHECIMENTO DA FALTA DE CO M M ITM E NT

Deveríamos olhar para a linguagem que usamos quando nos comprometemos a fazer algo, como o sinal revelador do que está por vir. Na verdade, é mais uma questão de procurar palavras específicas no que dizemos. Se você não consegue encontrar essas pequenas palavras mágicas, é provável que

será que não queremos dizer o que dizemos ou podemos não acreditar que isso seja viável.

Aqui estão alguns exemplos de palavras e frases que você deve procurar e que são sinais reveladores de não compromisso:

- Necessidade/deveria. “Precisamos fazer isso.” “Eu preciso perder peso.” “Alguém deveria fazer isso acontecer.”
- Esperança\desejo. “Espero terminar isso amanhã.” “Espero que possamos nos encontrar novamente algum dia.” “Eu gostaria de ter tempo para isso.” “Eu gostaria que este computador fosse mais rápido.”
- Vamos. (não seguido por “eu...”) “Vamos nos encontrar algum dia.” “Vamos terminar isso.”

UMA LINGUAGEM DE COMPROMISSO

49

Ao começar a procurar por essas palavras, você verá que começa a identificá-las em quase todos os lugares ao seu redor e até mesmo nas coisas que você diz aos outros. Você descobrirá que tendemos a estar muito ocupados, não assumindo a responsabilidade pelas coisas. E não está tudo bem quando você ou outra pessoa confia nessas promessas como parte do trabalho. Você deu o primeiro passo: comece a reconhecer a falta de compromisso ao seu redor e em você.

Ouvimos como é o descompromisso. Como reconhecemos o real compromisso?

O QUE S O E S COM M I T M E N D S O U N D I K E?

O que é comum nas frases da seção anterior é que elas presumir que as coisas estão fora de “minhas” mãos ou eles não assumem responsabilidade pessoal. Em cada um destes casos, as pessoas comportam-se como se fossem vítimas de uma situação em vez de controlá-lo.

A verdade é que você, pessoalmente, SEMPRE tem algo que está sob seu controle. controle, então sempre há algo que você pode se comprometer totalmente a fazer.

O ingrediente secreto para reconhecer o compromisso real é procurar sentenças isso soa assim: eu vou. . . por . . . (exemplo: terminarei isso na terça-feira.)

O que há de importante nesta frase? Você está declarando um fato sobre algo que VOCÊ fará com um horário de término claro. Você não está falando de mais ninguém além de você mesmo. Você está falando sobre uma ação que irá realizar. Você não “possivelmente” aceitará, ou “pode chegar lá”; você vai conseguir isso.

Não há (tecnicamente) saída para esse compromisso verbal. Você disse que faria e agora apenas um resultado binário é possível - ou você consegue ou não.

Se você não fizer isso, as pessoas poderão fazer com que você cumpra suas promessas. Você vai se sentir ruim por não fazer isso. Você se sentirá estranho em contar a alguém que não tem feito (se alguém ouviu você prometer que faria).

Assustador, não é?

CAPÍTULO 3 DIZENDO SIM

50

Você está assumindo total responsabilidade por algo, diante de um público de pelo menos uma pessoa. Não é só você ficar na frente do espelho ou do computador tela. É você enfrentando outro ser humano e dizendo que fará isso. Esse é o início do compromisso. Colocar-se na situação que o obriga a fazer alguma coisa.

Você mudou a linguagem que usa para uma linguagem de compromisso, e isso irá ajudá-lo a passar pelos próximos dois estágios: ser sincero e seguir através.

Aqui estão uma série de razões pelas quais você pode não querer dizer isso ou seguir em frente com algumas soluções.

Não funcionaria porque confio na pessoa X para fazer isso.

Você só pode se comprometer com coisas sobre as quais tem total controle. Por exemplo, se seu objetivo é terminar um módulo que também depende de outra equipe, você não pode comprometa-se a finalizar o módulo com total integração com a outra equipe. Mas você pode se comprometer com ações específicas que o levarão ao seu objetivo. Você poderia:

- Converse por uma hora com Gary, da equipe de infraestrutura, para entender suas dependências.
- Crie uma interface que abstraia a dependência do seu módulo dos outros infraestrutura da equipe.
- Reúna-se pelo menos três vezes esta semana com o responsável pela construção para garantir que seu as mudanças funcionam bem no sistema de construção da empresa.
- Crie sua própria compilação pessoal que execute seus testes de integração para o módulo.

Veja a diferença?

Se o objetivo final depender de outra pessoa, você deve se comprometer com ações específicas que o aproximam do objetivo final.

UMA LINGUAGEM DE COMPROMISSO

51

Não funcionaria porque realmente não sei se isso pode ser feito.

Se isso não puder ser feito, você ainda pode se comprometer com ações que o aproximarão para o alvo. Descobrir se isso pode ser feito pode ser uma das ações a comprometa-se!

Em vez de se comprometer a corrigir todos os 25 bugs restantes antes do lançamento (que pode não ser possível), você pode se comprometer com essas ações específicas que lhe trazem mais perto desse objetivo:

- Analise todos os 25 bugs e tente recriá-los.
- Converse com o controle de qualidade que encontrou cada bug para ver uma reprodução desse bug.
- Gaste todo o tempo que tiver esta semana tentando consertar cada bug.

Não funcionaria porque às vezes eu simplesmente não consigo.

Isso acontece. Algo inesperado pode acontecer, e isso é a vida. Mas você ainda quero corresponder às expectativas. Nesse caso, é hora de mudar as expectativas, o mais rápido possível.

Se você não consegue cumprir o seu compromisso, o mais importante é levantar um vermelho sinalizar o mais rápido possível para quem você se comprometeu.

Quanto mais cedo você levantar a bandeira para todas as partes interessadas, maior será a probabilidade de haver

momento para a equipe parar, reavaliar as ações atuais que estão sendo tomadas e decidir se algo pode ser feito ou mudado (em termos de prioridades, por exemplo). Por fazendo isso, seu compromisso ainda poderá ser cumprido ou você poderá mudar para um compromisso diferente.

Alguns exemplos são:

- Se você marcar uma reunião para o meio-dia em um café no centro da cidade com um colega e ficar preso no trânsito, você duvida que conseguirá seguir em frente compromisso de chegar na hora certa. Você pode ligar para seu colega assim que perceba que você pode estar atrasado e avise-os. Talvez você possa encontrar um mais próximo local para se reunir, ou talvez adiar a reunião.

- Se você se comprometeu a resolver um bug que considerava solucionável e percebe em algum momento o bug é muito mais horrível do que se pensava, você pode levantar a bandeira. A equipe pode então decidir sobre um curso de ação para fazer esse compromisso (emparelhamento, aumento de soluções potenciais, brainstorming) ou mude a prioridade e passe para outro bug mais simples.

Um ponto importante aqui é: se você não contar a ninguém sobre o potencial problema o mais rápido possível, você não está dando a ninguém a chance de ajudá-lo cumprir seu compromisso.

RESUMO

Criar uma linguagem de compromisso pode parecer um pouco assustador, mas pode ajudar a resolver muitos dos problemas de comunicação que os programadores enfrentam hoje – estimativas, prazos e contratempos de comunicação face a face. Você será levado como um sério desenvolvedor que cumpre sua palavra, e essa é uma das melhores coisas que você pode esperamos em nossa indústria.

~~~

### APRENDA A DIZER “SIM”

Pedi a Roy que contribuísse com aquele artigo porque ele tocou em mim. eu tenho tenho pregado sobre aprender a dizer não há algum tempo. Mas é tão importante aprender a dizer sim.

### O OUTRO LADO DE “TENTAR”

Vamos imaginar que Peter seja o responsável por algumas modificações na classificação motor. Ele estima, em particular, que essas modificações levarão cinco ou seis dias. Ele também acha que escrever a documentação para as modificações irá demorar algumas horas. Na manhã de segunda-feira, sua gerente, Marge, pede status. Marge: “Peter, você terminará os mods do mecanismo de classificação até sexta-feira?” Peter: “Acho que isso é factível.”



## APRENDENDO A DIZER “SIM”

53

Marge: “Isso incluirá a documentação?”

Peter: “Vou tentar fazer isso também.”

Talvez Marge não consiga ouvir a hesitação nas declarações de Peter, mas ele certamente está não assumindo muito compromisso. Marge está fazendo perguntas que exigem respostas booleanas, mas as respostas booleanas de Peter são confusas.

Observe o abuso da palavra tentar. No último capítulo usamos o “esforço extra” definição de tentativa. Aqui, Peter está usando a definição “talvez, talvez não”.

Seria melhor que Peter respondesse assim:

Marge: “Peter, você terminará os mods do mecanismo de classificação até sexta-feira?”

Peter: “Provavelmente, mas pode ser segunda-feira.”

Marge: “Isso incluirá a documentação?”

Peter: “A documentação vai demorar mais algumas horas, então segunda-feira é possível, mas pode ser até terça-feira.”

Neste caso, a linguagem de Pedro é mais honesta. Ele está descrevendo o seu próprio incerteza para Marge. Marge pode ser capaz de lidar com essa incerteza. No por outro lado, ela talvez não.

## COMMITTING COM DISCIPLINE

Marge: “Peter, preciso de um sim ou não definitivo. Você terá o mecanismo de classificação concluído e documentado até sexta-feira?”

Esta é uma pergunta perfeitamente justa para Marge fazer. Ela tem um cronograma para manter, e ela precisa de uma resposta binária sobre sexta-feira. Como Pedro deveria responder?

Peter: “Nesse caso, Marge, terei que dizer não. O mais rápido que puder ter certeza que terminarei os mods e os documentos é terça-feira.

Marge: “Você está se comprometendo com terça-feira?”

Peter: “Sim, terei tudo pronto na terça.”

Mas e se Marge realmente precisar das modificações e da documentação feita por Sexta-feira?

Marge: "Peter, terça-feira me traz um problema real. Willy, nosso redator de tecnologia, irá estar disponível na segunda-feira. Ele tem cinco dias para finalizar o usuário guia. Se eu não tiver os documentos do mecanismo de classificação até segunda-feira de manhã, ele nunca terminará o manual a tempo. Você pode fazer a documentação primeiro?"

Peter: "Não, os mods têm que vir primeiro, porque nós geramos os documentos a partir da saída das execuções de teste."

Marge: "Bem, não há alguma maneira de você terminar os mods e o documentos antes de segunda-feira de manhã?"

Agora Peter tem uma decisão a tomar. Há uma boa chance de que ele termine com o avaliar as modificações do motor na sexta-feira, e ele pode até conseguir terminar o documentos antes de ir para casa no fim de semana. Ele poderia trabalhar algumas horas Sábado também se as coisas demorarem mais do que ele espera. Então, o que ele deveria dizer a Marge?

Peter: "Olha, Marge, há uma boa chance de eu conseguir fazer tudo até segunda-feira de manhã, se eu dedicar algumas horas extras no sábado.

Isso resolve o problema de Marge? Não, isso simplesmente muda as probabilidades, e isso é o que Peter precisa dizer a ela.

Marge: "Posso contar na manhã de segunda-feira, então?"

Peter: "Provavelmente, mas não definitivamente."

Isso pode não ser bom o suficiente para Marge.

Marge: "Olha, Peter, eu realmente preciso de uma definição definitiva sobre isso. Existe alguma maneira de você

pode se comprometer a fazer isso antes de segunda-feira de manhã?"

Peter pode ficar tentado a quebrar a disciplina neste momento. Ele pode ser capaz de conseguir feito mais rápido se ele não escrever seus testes. Ele poderia ser capaz de terminar mais rápido se não refatora. Ele pode conseguir terminar mais rápido se não executar o trabalho completo conjunto de regressão.

É aqui que o profissional traça o limite. Em primeiro lugar, Peter está simplesmente errado sobre suas suposições. Ele não terminará mais rápido se não escrever seus testes. Ele não será feito mais rápido se ele não refatorar. Ele não terminará mais rápido se omitir o conjunto completo de regressão. Anos de experiência nos ensinaram que quebrar disciplinas apenas nos atrasam.

Mas em segundo lugar, como profissional, ele tem a responsabilidade de manter certos padrões. Seu código precisa ser testado e precisa de testes. Seu código precisa estar limpo. E ele tem que ter certeza de que não quebrou mais nada o sistema.

Peter, como profissional, já assumiu o compromisso de manter esses padrões. Todos os outros compromissos que ele assumir devem estar subordinados a isso. Então toda esta linha de raciocínio precisa ser abortada.

Peter: “Não, Marge, realmente não tenho como ter certeza sobre qualquer data antes de terça-feira. Sinto muito se isso atrapalha sua agenda, mas é apenas a realidade que enfrentamos.

Marge: “Droga, eu realmente estava contando em trazer esse aqui mais cedo.

Você tem certeza?”

Peter: “Tenho certeza que pode ser até terça-feira, sim.”

Marge: “OK, acho que vou falar com Willy para ver se ele consegue reorganizar sua agenda.”

Neste caso, Marge aceitou a resposta de Peter e começou a procurar outras opções. Mas e se todas as opções de Marge tiverem sido esgotadas? E se Pedro foram a última esperança?

Marge: “Peter, olha, eu sei que isso é uma grande imposição, mas eu realmente preciso de você para encontrar uma maneira de fazer tudo isso até segunda-feira de manhã. É realmente crítico. Não há algo que você possa fazer?”

Então agora Peter começa a pensar em fazer horas extras significativas, e provavelmente a maior parte do fim de semana. Ele precisa ser muito honesto consigo mesmo sobre sua resistência e reservas. É fácil dizer que você fará muitas coisas nos finais de semana, é muito mais difícil reunir energia suficiente para realizar um trabalho de alta qualidade.

## CAPÍTULO 3 DIZENDO SIM

56

Os profissionais conhecem seus limites. Eles sabem quantas horas extras podem aplicar-se eficazmente e sabem qual será o custo.

Neste caso, Peter sente-se bastante confiante de que algumas horas extras durante a semana e algum tempo no fim de semana será suficiente.

Peter: "OK, Marge, vou te dizer uma coisa. Vou ligar para casa e esclarecer algumas coisas. horas extras com minha família. Se eles concordarem com isso, então eu atendo tarefa concluída na manhã de segunda-feira. Eu até irei na segunda-feira manhã para garantir que tudo corra bem com Willy. Mas então irei para casa e só voltarei na quarta-feira. Combinado?

Isto é perfeitamente justo. Peter sabe que pode conseguir as modificações e documentos feitos se ele fizer horas extras. Ele também sabe que será inútil por um alguns dias depois disso.

### CO N CLU S I O N

Os profissionais não são obrigados a dizer sim a tudo o que lhes é pedido.

No entanto, devem trabalhar arduamente para encontrar formas criativas de tornar o "sim" possível.

Quando os profissionais dizem sim, eles usam a linguagem do compromisso para que haja não há dúvida sobre o que eles prometeram.

## CODIFICAÇÃO

Em um livro anterior<sup>1</sup> escrevi bastante sobre a estrutura e a natureza do Código Limpo.

Este capítulo discute o ato de codificação e o contexto que envolve esse ato.

Quando eu tinha 18 anos eu sabia digitar razoavelmente bem, mas precisava olhar as teclas.

Eu não conseguia digitar às cegas. Então, uma noite, passei algumas longas horas em um IBM 029 keypunch recusando-se a olhar para meus dedos enquanto eu digitava um programa que havia escrito em vários formulários de codificação. Examinei cada cartão depois de digitá-lo e descartei aqueles que foram digitados errado.

1. [Martin09]

## CAPÍTULO 4 CODIFICAÇÃO

58

No começo, digitei alguns por engano. No final da noite eu estava digitando tudo com quase perfeição. Percebi, durante aquela longa noite, que digitar às cegas é tudo sobre confiança. Meus dedos sabiam onde estavam as chaves, eu só precisava ganhar a confiança de que não estava cometendo um erro. Uma das coisas que ajudou com essa confiança é que pude sentir quando estava cometendo um erro. No final da noite, se eu cometesse um erro, eu sabia quase instantaneamente e simplesmente ejetou o cartão sem olhar para ele.

Ser capaz de sentir seus erros é muito importante. Não apenas na digitação, mas também tudo. Ter senso de erro significa que você fecha o feedback muito rapidamente faça um loop e aprenda com seus erros ainda mais rapidamente. Eu estudei e dominei, diversas disciplinas desde aquele dia no 029. Descobri que em cada caso que o a chave para o domínio é a confiança e o senso de erro.

Este capítulo descreve meu conjunto pessoal de regras e princípios para codificação. Essas regras e os princípios não dizem respeito ao meu código em si; eles são sobre meu comportamento, humor e atitude ao escrever código. Eles descrevem meu próprio estado mental, moral e emocional contexto para escrever código. Estas são as raízes da minha confiança e senso de erro.

Você provavelmente não concordará com tudo o que digo aqui. Afinal, isso é profundamente pessoal coisas. Na verdade, você pode discordar violentamente de algumas das minhas atitudes e princípios. Tudo bem – elas não pretendem ser verdades absolutas para ninguém além de mim.

O que são é a abordagem de um homem para ser um programador profissional.

Talvez, ao estudar e contemplar meu ambiente pessoal de codificação, você posso aprender a arrancar a pedra da minha mão.

### PR E PAR E D N E S S

A codificação é uma atividade intelectualmente desafiadora e exaustiva. Requer um nível de concentração e foco que poucas outras disciplinas exigem. A razão para o fato é que a codificação exige que você faça malabarismos com muitos fatores concorrentes ao mesmo tempo.

1. Primeiro, seu código deve funcionar. Você deve entender qual problema você está resolver e entender como resolver esse problema. Você deve garantir que o o código que você escreve é uma representação fiel dessa solução. Você deve gerenciar

## PREPARAÇÃO

59

cada detalhe dessa solução, mantendo-se consistente dentro da linguagem, plataforma, arquitetura atual e todas as verrugas do sistema atual.

2. Seu código deve resolver o problema definido pelo cliente. Muitas vezes os requisitos do cliente não resolvem realmente os problemas do cliente. É cabe a você ver isso e negociar com o cliente para garantir que as verdadeiras necessidades do cliente são atendidas.

3. Seu código deve se encaixar bem no sistema existente. Não deveria aumentar a rigidez, fragilidade ou opacidade desse sistema. As dependências devem estar bem gerenciado. Resumindo, seu código precisa seguir princípios sólidos de engenharia.<sup>2</sup>

4. Seu código deve ser legível por outros programadores. Isto não é simplesmente um questão de escrever bons comentários. Em vez disso, requer que você crie o código em de tal forma que revele sua intenção. Isso é difícil de fazer. Na verdade, isso pode ser a coisa mais difícil que um programador pode dominar.

Fazer malabarismos com todas essas preocupações é difícil. É fisiologicamente difícil manter a concentração e foco necessários por longos períodos de tempo. Adicione a isso os problemas e distrações do trabalho em equipe, em uma organização e os cuidados e preocupações da vida cotidiana. O resultado final é que a oportunidade para a distração é alta.

Quando você não consegue se concentrar e focar o suficiente, o código que você escreve será errado. Terá bugs. Terá a estrutura errada. Será opaco e complicado. Não resolverá os problemas reais dos clientes. Em suma, terá para ser retrabalhado ou refeito. Trabalhar distraído cria desperdício.

Se você estiver cansado ou distraído, não codifique. Você apenas acabará refazendo o que você fez. Em vez disso, encontre uma maneira de eliminar as distrações e acalmar sua mente.

### 3h CO D E

O pior código que já escrevi foi às 3 da manhã. O ano era 1988 e eu estava trabalhando em uma start-up de telecomunicações chamada Clear Communications. Estávamos todos dedicando muitas horas para construir “equidade de suor”. Estávamos, é claro, todos sonhando em ser rico.

2. [Martin03]

## CAPÍTULO 4 CODIFICAÇÃO

60

Numa noite bem tarde – ou melhor, numa manhã bem cedo, para resolver um problema problema de tempo – meu código enviou uma mensagem para si mesmo por meio do evento sistema de despacho (chamamos isso de “envio de correspondência”). Esta foi a solução errada, mas às 3 da manhã parecia muito bom. Na verdade, após 18 horas de codificação sólida (sem mencionar as semanas de 60-70 horas) foi tudo que consegui pensar.

Lembro-me de me sentir muito bem comigo mesmo durante as longas horas que trabalhei. Lembro-me de me sentir dedicado. Lembro-me de pensar que trabalhar às 3 da manhã é o que profissionais sérios fazem. Como eu estava errado!

Esse código voltou para nos atormentar repetidas vezes. Instituiu um design defeituoso estrutura que todos usaram, mas consistentemente tiveram que contornar. Isso causou tudo tipos de erros de tempo estranhos e ciclos de feedback estranhos. Entraríamos no infinito mail faz um loop quando uma mensagem faz com que outra seja enviada, e depois outra, infinitamente. Nunca tivemos tempo de reescrever esse maço (assim pensamos), mas sempre parecia ter tempo para adicionar outra verruga ou adesivo para contornar isso. O lixo cresceu e cresceu, cercando aquele código das 3 da manhã com cada vez mais bagagem e lado efeitos. Anos depois, isso se tornou uma piada de equipe. Sempre que eu estava cansado ou frustrado eles diziam: "Cuidado! Bob está prestes a enviar uma correspondência para si mesmo!"

A moral desta história é: não escreva código quando estiver cansado. Dedicção e profissionalismo tem mais a ver com disciplina do que com horas. Certifique-se de que seu sono, saúde e estilo de vida são ajustados para que você possa dedicar oito boas horas por dia.

### CÓDIGO DE PREOCUPAÇÃO

Você já teve uma grande briga com seu cônjuge ou amigo e depois tentou codificar? Você notou que havia um processo em segundo plano em execução no seu se importa em tentar resolver, ou pelo menos rever a luta? Às vezes você pode sentir o estresse desse processo de fundo em seu peito ou na boca do estômago.

Pode fazer você se sentir ansioso, como quando você toma muito café ou faz dieta cocaína. É uma distração.

Quando estou preocupado com uma discussão com minha esposa, ou com uma crise de cliente, ou com uma

criança doente, não consigo manter o foco. Minha concentração vacila. eu me encontro com meus olhos na tela e meus dedos no teclado, sem fazer nada. Catatônico.



## PREPARAÇÃO

61

Paralisado. A um milhão de milhas de distância, resolvendo o problema no histórico, em vez de realmente resolver o problema de codificação que está diante de mim.

Às vezes me forço a pensar sobre o código. Eu poderia me dirigir até escreva uma linha ou duas. Posso me esforçar para fazer um ou dois testes para passar. Mas eu não posso continue assim. Inevitavelmente, encontro-me caindo numa insensibilidade estupefata, vendo nada através dos meus olhos abertos, agitando-me interiormente com a preocupação de fundo. Aprendi que não é hora de codificar. Qualquer código que eu produza será lixo. Então em vez de codificar, preciso resolver a preocupação.

É claro que existem muitas preocupações que simplesmente não podem ser resolvidas em uma hora ou dois. Além disso, é pouco provável que os nossos empregadores tolerem durante muito tempo a nossa incapacidade de

trabalhar enquanto resolvemos nossos problemas pessoais. O truque é aprender como desligar processo em segundo plano, ou pelo menos reduzir sua prioridade para que não seja um distração contínua.

Eu faço isso particionando meu tempo. Em vez de me forçar a codificar enquanto o a preocupação de fundo está me incomodando, vou gastar um período dedicado de tempo, talvez uma hora, trabalhando na questão que está criando a preocupação. Se meu filho estiver doente, ligarei para casa e verificarei como está. Se tiver brigado com minha esposa, ligarei ela e conversar sobre os problemas. Se eu tiver problemas financeiros, gastarei tempo pensando em como posso lidar com as questões financeiras. Eu sei que provavelmente não resolver os problemas nesta hora, mas é muito provável que eu consiga reduzir a ansiedade e silenciar o processo em segundo plano.

Idealmente, o tempo gasto lutando com questões pessoais seria um tempo pessoal. Isso seria uma pena passar uma hora no escritório dessa maneira. Desenvolvedores profissionais alocar seu tempo pessoal para garantir que o tempo gasto no escritório seja tão produtivo quanto possível. Isso significa que você deve reservar especificamente um tempo em casa para resolver suas ansiedades para que você não as leve para o escritório.

Por outro lado, se você estiver no escritório e ao fundo ansiedades estão minando sua produtividade, então é melhor gastar uma hora acalmá-los do que usar força bruta para escrever código que você apenas terá que jogue fora mais tarde (ou pior, viva com).

### O FLUXO ZONE

Muito tem sido escrito sobre o estado hiperprodutivo conhecido como “fluxo”.

Alguns programadores chamam isso de “a Zona”. Qualquer que seja o nome, você provavelmente está familiarizado com isso. É o estado de consciência altamente focado e com visão de túnel que os programadores podem entrar enquanto escrevem código. Neste estado eles se sentem produtivo. Neste estado eles se sentem infalíveis. E então eles desejam alcançar isso estado, e muitas vezes medem seu valor próprio por quanto tempo eles podem gastar lá.

Aqui vai uma pequena dica de alguém que esteve lá e voltou: Evite a Zona.

Este estado de consciência não é realmente hiperprodutivo e certamente não é infalível. Na verdade, é apenas um estado meditativo suave em que certas as faculdades são diminuídas em favor de uma sensação de velocidade.

Deixe-me ser claro sobre isso. Você escreverá mais código na Zona. Se você estiver praticando TDD, você percorrerá o loop vermelho/verde/refatorar mais rapidamente.

E você sentirá uma leve euforia ou uma sensação de conquista. O problema é que você perde um pouco da visão geral enquanto está na Zona, então provavelmente tome decisões que mais tarde você terá que voltar atrás e reverter. Código escrito em a Zona pode sair mais rápido, mas você voltará para visitá-la mais.

Hoje em dia, quando me sinto entrando na Zona, afasto-me por alguns minutos.

Eu limpo minha cabeça respondendo alguns e-mails ou olhando alguns tweets. Se estiver perto chega ao meio-dia, farei uma pausa para o almoço. Se estou trabalhando em equipe, encontrarei um par parceiro.

Um dos grandes benefícios da programação em pares é que é virtualmente impossível um par para entrar na Zona. A Zona é um estado pouco comunicativo, enquanto o emparelhamento requer comunicação intensa e constante. Na verdade, uma das queixas que

O que ouço com frequência sobre o emparelhamento é que ele bloqueia a entrada na Zona. Bom! A Zona não é onde você quer estar.

Bem, isso não é bem verdade. Há momentos em que a Zona é exatamente onde você quero ser. Quando você está praticando. Mas falaremos sobre isso em outro capítulo.

## MUSIC

Na Teradyne, no final dos anos 70, eu tinha um escritório particular. Eu era o administrador do sistema do nosso PDP 11/60, e por isso fui um dos poucos programadores autorizados a ter um terminal privado. Esse terminal era um VT100 rodando a 9600 baud e conectado ao PDP 11 com 80 pés de cabo RS232 que coloquei nas placas do teto do meu escritório para a sala de informática.

Eu tinha um sistema estéreo em meu escritório. Era um antigo toca-discos, amplificador e chão alto-falantes. Eu tinha uma coleção significativa de vinis, incluindo Led Zeppelin, Pink Floyd, e.... Bem, você entendeu.

Eu costumava ligar o aparelho de som e depois escrever o código. Eu pensei que isso ajudou meu concentração. Mas eu estava errado.

Um dia voltei para um módulo que estava editando enquanto ouvia o sequência de abertura de The Wall. Os comentários nesse código continham letras da peça e anotações editoriais sobre bombardeiros de mergulho e bebês chorando. Foi quando me dei conta. Como leitor do código, eu estava aprendendo mais sobre o coleção de músicas do autor (eu) do que estava aprendendo sobre o problema que o código estava tentando resolver.

Percebi que simplesmente não codifico bem enquanto ouço música. A música faz não me ajude a focar. Na verdade, o ato de ouvir música parece consumir alguma recurso vital que minha mente precisa para escrever código limpo e bem projetado. Talvez não funcione assim para você. Talvez a música ajude você a escrever código. eu conheço muitas pessoas que codificam usando fones de ouvido. Eu aceito que a música pode ajudá-los, mas também suspeito que o que realmente está acontecendo é que o a música os está ajudando a entrar na Zona.

## INTERUPTIONS

Visualize-se enquanto codifica em sua estação de trabalho. Como você responde quando alguém lhe faz uma pergunta? Você ataca eles? Você olha? Será que o seu linguagem corporal diz a eles para irem embora porque você está ocupado? Resumindo, você é rude?

## CAPÍTULO 4 CODIFICAÇÃO

64

Ou você para o que está fazendo e ajuda educadamente alguém que está preso? Faça você os trata como gostaria que eles o tratassem se você estivesse preso?

A resposta rude geralmente vem da Zona. Você pode se ressentir de ser arrastado fora da Zona, ou você pode ficar ressentido com a interferência de alguém em sua tentativa de entrar na Zona. De qualquer forma, a grosseria geralmente vem do seu relacionamento com a Zona.

Às vezes, porém, a culpa não é da Zona, é apenas que você está tentando para entender algo complicado que requer concentração. Existem várias soluções para isso.

O emparelhamento pode ser muito útil como forma de lidar com interrupções. Seu par parceiro pode manter o contexto do problema em questão, enquanto você atende uma ligação, ou uma pergunta de um colega de trabalho. Quando você retorna ao seu parceiro, ele rapidamente ajuda a reconstruir o contexto mental que você tinha antes da interrupção.

TDD é outra grande ajuda. Se você tiver um teste com falha, esse teste contém o contexto de onde você está. Você pode retornar a ele após uma interrupção e continuar a fazer aquela falha no teste.

No final, é claro, haverá interrupções que o distrairão e o farão perder tempo. Quando eles acontecerem, lembre-se que da próxima vez você pode ser o único que precisa interromper outra pessoa. Então a atitude profissional é educada disposição para ser útil.

### BLOCO DO ESCRITO R

Às vezes, o código simplesmente não vem. Isso já aconteceu comigo e eu vi isso acontecer com os outros. Você senta em sua estação de trabalho e nada acontece.

Muitas vezes você encontrará outro trabalho para fazer. Você lerá o e-mail. Você lerá tweets. Você vai folhear livros, agendas ou documentos. Você convocará reuniões. Você vai iniciar conversas com outras pessoas. Você fará qualquer coisa para não precisar enfrentar aquela estação de trabalho e observe como o código se recusa a aparecer.

## BLOCO DO ESCRITO

65

O que causa esses bloqueios? Já falamos sobre muitos dos fatores.

Para mim, outro fator importante é o sono. Se não estou dormindo o suficiente, simplesmente não consigo codificar. Outros são preocupação, medo e depressão.

Curiosamente, existe uma solução muito simples. Funciona quase sempre. É fácil fazer e pode fornecer o impulso para escrever muitos códigos.

A solução: Encontre um parceiro.

É incrível como isso funciona bem. Assim que você se sentar ao lado de outra pessoa, os problemas que estavam bloqueando você desaparecem. Há uma mudança fisiológica que acontece quando você trabalha com alguém. Não sei o que é, mas posso definitivamente sentir isso. Há algum tipo de mudança química em meu cérebro ou corpo que me quebra o bloqueio e me faz andar novamente.

Esta não é uma solução perfeita. Às vezes a mudança dura uma ou duas horas, apenas ser seguido por uma exaustão tão severa que tenho que me separar do meu par parceiro e encontrar algum buraco para se recuperar. Às vezes, mesmo quando sentado com alguém, não posso fazer mais do que apenas concordar com o que essa pessoa está fazendo. Mas para mim, a reação típica ao emparelhamento é uma recuperação do meu ímpeto.

## ENTRADAS CRIATIVAS

Há outras coisas que faço para evitar o bloqueio. Eu aprendi há muito tempo que a produção criativa depende da contribuição criativa.

Eu leio muito e leio todo tipo de material. Li material sobre software, política, biologia, astronomia, física, química, matemática e muito mais. No entanto, Acho que o que melhor estimula a produção criativa é a ciência ficção.

Para você, pode ser outra coisa. Talvez um bom romance de mistério, ou poesia, ou até mesmo um romance. Acho que a verdadeira questão é que a criatividade gera criatividade. Há também um elemento de escapismo. As horas que passo longe do meu habitual problemas, enquanto é ativamente estimulado por ideias desafiadoras e criativas, resulta em uma pressão quase irresistível para criar algo sozinho.

## CAPÍTULO 4 CODIFICAÇÃO

66

Nem todas as formas de contribuição criativa funcionam para mim. Assistir TV geralmente não ajuda eu crio. Ir ao cinema é melhor, mas só um pouco. Ouvir música faz não me ajuda a criar código, mas me ajuda a criar apresentações, palestras e vídeos. De todas as formas de contribuição criativa, nada funciona melhor para mim do que a boa e velha ópera espacial.

### DE BUGG I N G

Uma das piores sessões de depuração da minha carreira aconteceu em 1972. O terminal conectado ao sistema de contabilidade dos Teamsters usados para congelar uma ou duas vezes por dia. Não havia como forçar isso a acontecer. O erro não preferiu quaisquer terminais específicos ou quaisquer aplicações específicas. Não importava o que usuário estava fazendo antes do congelamento. Um minuto o terminal estava funcionando tudo bem, e no minuto seguinte estava irremediavelmente congelado.

Demorou semanas para diagnosticar esse problema. Enquanto isso, os Teamsters estavam ficando cada vez mais chateado. Cada vez que havia um congelamento, a pessoa naquele momento terminal teria que parar de funcionar e esperar até que pudessem coordenar todos os outros usuários concluíam suas tarefas. Então eles nos ligariam e reiniciá-los. Foi um pesadelo.

Passamos as primeiras semanas apenas coletando dados entrevistando as pessoas que vivenciaram os bloqueios. Perguntávamos a eles o que estavam fazendo a hora e o que eles fizeram anteriormente. Perguntamos a outros usuários se eles notaram algo em seus terminais no momento do congelamento. Estes as entrevistas foram todas feitas por telefone porque os terminais estavam localizados em no centro de Chicago, enquanto trabalhávamos 30 milhas ao norte nos campos de milho. Não tínhamos logs, nem contadores, nem depuradores. Nosso único acesso ao interior do o sistema eram luzes e interruptores no painel frontal. Poderíamos parar o computador e, em seguida, espiar na memória uma palavra por vez. Mas nós não pude fazer isso por mais de cinco minutos porque os Teamsters precisavam de seus backup do sistema.

Passamos alguns dias escrevendo um inspetor simples em tempo real que poderia ser operado do teletipo ASR-33 que serviu de console. Com isso poderíamos espiar

e vasculhar a memória enquanto o sistema estava em execução. Adicionamos registro mensagens impressas no teletipo em momentos críticos. Criamos na memória contadores que contavam eventos e lembravam a história do estado que poderíamos inspecionar com o inspetor. E, claro, tudo isso teve que ser escrito a partir de zero em assembler e testado à noite, quando o sistema não estava em uso.

Os terminais foram acionados por interrupção. Os caracteres sendo enviados para os terminais foram mantidos em buffers circulares. Cada vez que uma porta serial termina de enviar um caractere, uma interrupção seria acionada e o próximo caractere no buffer circular seria preparado para envio.

Finalmente descobrimos que quando um terminal travava era porque as três variáveis que gerenciavam o buffer circular estavam fora de sincronia. Não tínhamos ideia de por que isso acontecia acontecendo, mas pelo menos era uma pista. Em algum lugar no 5 KSLOC de supervisão código, havia um bug que manipulava incorretamente um desses ponteiros.

Este novo conhecimento também nos permitiu descongelar terminais manualmente! Nós poderíamos insira valores padrão nessas três variáveis usando o inspetor, e o terminais iriam magicamente começar a funcionar novamente. Eventualmente escrevemos um pequeno hack

que examinaria todos os contadores para ver se eles estavam desalinhados e consertá-los. A princípio, invocamos esse hack pressionando um botão especial de interrupção do usuário ligue o painel frontal sempre que os Teamsters ligarem para relatar um congelamento.

Mais tarde, simplesmente executamos o utilitário de reparo uma vez a cada segundo.

Mais ou menos um mês depois, o problema do congelamento estava resolvido, na opinião dos Teamsters. preocupado. Ocasionalmente, um de seus terminais parava por meio segundo ou então, mas a uma taxa básica de 30 caracteres por segundo, ninguém pareceu notar.

Mas por que os contadores estavam desalinhados? Eu tinha dezenove anos e estava determinado para descobrir.

O código de supervisão foi escrito por Richard, que desde então partiu para faculdade. Nenhum de nós estava familiarizado com esse código porque Richard tinha sido bastante possessivo com isso. Esse código era dele e não tínhamos permissão para saber isso. Mas agora que Richard se foi, então peguei a lista de centímetros de espessura e comecei a repasse página por página.

## CAPÍTULO 4 CODIFICAÇÃO

68

As filas circulares naquele sistema eram apenas estruturas de dados FIFO, ou seja, filas. Os programas aplicativos empurravam caracteres para uma extremidade da fila até a fila estava cheia. As cabeças de interrupção tiraram os personagens do outro lado da fila quando a impressora estiver pronta para recebê-los. Quando a fila estava vazia, a impressora iria parar. Nosso bug fez com que os aplicativos pensassem que a fila estava cheio, mas fez com que os controladores de interrupção pensassem que a fila estava vazia. Os cabeçotes de interrupção são executados em um “thread” diferente de todos os outros códigos. Então contadores e

variáveis que são manipuladas por cabeçalhos de interrupção e outros códigos devem ser protegido de atualização simultânea. No nosso caso, isso significou virar o interrompe qualquer código que manipule essas três variáveis. Pelo vez que me sentei com aquele código, eu sabia que estava procurando algum lugar no código que tocou nas variáveis, mas não desativou as interrupções primeiro.

Hoje em dia, é claro, usaríamos a infinidade de ferramentas poderosas à nossa disposição para encontre todos os lugares onde o código tocou nessas variáveis. Em segundos teríamos conheça cada linha de código que os tocou. Em poucos minutos saberíamos qual não desativou as interrupções. Mas isso era 1972 e eu não tinha nenhuma ferramenta como isso. O que eu tinha eram meus olhos.

Examinei cada página desse código, procurando as variáveis. Infelizmente, as variáveis foram usadas em todos os lugares. Quase todas as páginas os tocaram de uma maneira ou outro. Muitas dessas referências não desativaram as interrupções porque eram referências somente leitura e, portanto, inofensivas. O problema era, naquele montador específico, não havia uma boa maneira de saber se uma referência foi lida. apenas sem seguir a lógica do código. Sempre que uma variável foi lida, ela posteriormente poderá ser atualizado e armazenado. E se isso aconteceu enquanto as interrupções foram ativados, as variáveis podem ser corrompidas.

Levei dias de estudo intenso, mas no final consegui. Ali, no meio do código, era um local onde uma das três variáveis estava sendo atualizada enquanto as interrupções estavam habilitadas.

Eu fiz as contas. A vulnerabilidade durou cerca de dois microssegundos. Havia uma dúzia de terminais, todos rodando a 30 cps, portanto, uma interrupção a cada 3 ms ou mais. Dado o tamanho do supervisor e a taxa de clock da CPU, esperaríamos um congelamento desta vulnerabilidade uma ou duas vezes por dia. Bingo!



Resolvi o problema, claro, mas nunca tive coragem de desligar o hack automático que inspecionou e consertou os contadores. Até hoje não estou convencido de que não havia outro buraco.

#### DEBU G I N G TIME

Por alguma razão, os desenvolvedores de software não pensam no tempo de depuração como codificação tempo. Eles pensam na depuração do tempo como um chamado da natureza, algo que acabou de acontecer.

para ser feito. Mas o tempo de depuração é tão caro para a empresa quanto a codificação o tempo é e, portanto, qualquer coisa que possamos fazer para evitá-lo ou diminuí-lo é bom.

Hoje em dia gasto muito menos tempo depurando do que há dez anos. eu não tenho medi a diferença, mas acredito que seja um fator de dez. Eu consegui isso redução verdadeiramente radical no tempo de depuração, adotando a prática de Teste Desenvolvimento Orientado (TDD), que discutiremos em outro capítulo.

Quer você adote TDD ou alguma outra disciplina de igual eficácia,<sup>3</sup> é incumbe a você, como profissional, reduzir o tempo de depuração o mais próximo possível para zero quanto você puder. Claramente zero é uma meta assintótica, mas é a meta mesmo assim.

Os médicos não gostam de reabrir os pacientes para consertar algo que fizeram de errado. Advogados não gosto de repetir casos que eles erraram. Um médico ou advogado que fez isso muitas vezes não seria considerado profissional. Da mesma forma, um desenvolvedor de software quem cria muitos bugs está agindo de forma pouco profissional.

#### PACANDO VOCÊ MESMO

O desenvolvimento de software é uma maratona, não um sprint. Você não pode vencer a corrida tentando correr o mais rápido possível desde o início. Você ganha conservando seu recursos e controlando seu próprio ritmo. Uma maratonista cuida de seu corpo antes e durante a corrida. Programadores profissionais conservam sua energia e criatividade com o mesmo cuidado.

3. Não conheço nenhuma disciplina que seja tão eficaz quanto o TDD, mas talvez você conheça.

## CAPÍTULO 4 CODIFICAÇÃO

70

### SAIBA QUANDO SE AFASTAR

Não pode ir para casa até resolver este problema? Ah, sim, você pode, e provavelmente deveria! Criatividade e inteligência são estados mentais passageiros. Quando você está cansados, eles vão embora. Se você bater em seu cérebro que não funciona por uma hora depois tarde da noite tentando resolver um problema, você simplesmente se tornará mais cansado e reduzir a chance de que o banho, ou o carro, o ajude a resolver o problema.

Quando você estiver preso, quando estiver cansado, desligue-se por um tempo. Dê o seu subconsciente criativo uma solução para o problema. Você fará mais em menos tempo e com menos esforço se você tiver o cuidado de economizar seus recursos. Ritmo você e sua equipe. Aprenda seus padrões de criatividade e brilho, e tirar vantagem deles em vez de trabalhar contra eles.

### D R I V I N G H O M E

Um lugar onde resolvi vários problemas foi meu carro a caminho de casa do trabalho. Dirigir requer muitos recursos mentais não criativos. Você deve dedique seus olhos, mãos e partes de sua mente à tarefa; portanto, você deve se desligar dos problemas no trabalho. Há algo sobre desengajamento que permite à sua mente procurar soluções de uma forma diferente e maneira mais criativa.

### O SHOW E R

Resolvi um número excessivo de problemas no chuveiro. Talvez isso borriço de água de manhã cedo me acorda e me faz revisar todos os soluções que meu cérebro inventou enquanto eu dormia.

Quando você está trabalhando em um problema, às vezes você chega tão perto dele que não consigo ver todas as opções. Você sente falta de soluções elegantes porque a parte criativa sua mente é suprimida pela intensidade do seu foco. Às vezes a melhor maneira resolver um problema é ir para casa, jantar, assistir TV, ir para a cama e depois acorde na manhã seguinte e tome um banho.

## ESTAR ATRASADO

71

## ESTAR ATRASO

Você vai se atrasar. Acontece com os melhores de nós. Acontece com os mais dedicados nós. Às vezes simplesmente estragamos nossas estimativas e acabamos atrasados.

O truque para gerenciar atrasos é a detecção precoce e a transparência. O pior

O cenário ocorre quando você continua contando a todos, até o final,

que você chegará na hora certa - e então decepcionará todos eles. Não faça isso. Em vez disso,

meça regularmente seu progresso em relação ao seu objetivo e encontre três

datas de término baseadas em fatos: melhor caso, caso nominal e pior caso. Seja tão honesto quanto você pode sobre todas as três datas. Não incorpore esperança em suas estimativas!

Apresente todos os três números para sua equipe e partes interessadas. Atualize estes números diariamente.

## ESPERANÇA

E se esses números mostrarem que você pode perder um prazo? Por exemplo, vamos digamos que haverá uma feira daqui a dez dias e precisamos ter nosso produto lá.

Mas digamos também que sua estimativa de três números para o recurso que você está trabalhando é 12/08/20.

Não espere que você consiga fazer tudo em dez dias! A esperança é a assassina do projeto.

A esperança destrói horários e arruína reputações. Espero que você se aprofunde

problema. Se a feira for daqui a dez dias e sua estimativa nominal for 12, você

não vão conseguir. Certifique-se de que a equipe e as partes interessadas

entenda a situação e não desista até que haja um plano alternativo. Não

deixe qualquer outra pessoa ter esperança.

## R U S H I N G

E se o seu gerente sentar com você e pedir que você tente cumprir o prazo?

E se o seu gerente insistir que você “faça o que for preciso”? Mantenha suas estimativas!

Suas estimativas originais são mais precisas do que quaisquer alterações feitas durante

4. Há muito mais sobre isso no capítulo Estimativa.

seu chefe está confrontando você. Diga ao seu chefe que você já considerou o opções (porque você tem) e que a única maneira de melhorar o cronograma é reduzir o escopo. Não fique tentado a se apressar.

Ai do pobre desenvolvedor que cede à pressão e concorda em tentar faça o prazo. Esse desenvolvedor começará a usar atalhos e trabalhar mais horas na vã esperança de fazer um milagre. Esta é uma receita para o desastre porque isso dá falsas esperanças a você, sua equipe e suas partes interessadas. Permite que todos evita enfrentar o problema e atrasa as decisões difíceis necessárias.

Não há como ter pressa. Você não pode codificar mais rápido. Você não pode fazer você mesmo resolve os problemas mais rapidamente. Se você tentar, você apenas vai desacelerar e faça uma bagunça que atrase todos os outros também.

Portanto, você deve responder ao seu chefe, à sua equipe e às partes interessadas privando eles de esperança.

### O V E R T I M E

Então seu chefe diz: "E se você trabalhar duas horas extras por dia? E se você trabalhar no sábado? Vamos lá, só tem que haver uma maneira de aproveitar horas suficientes para concluir o recurso a tempo.

Horas extras podem funcionar e às vezes são necessárias. Às vezes você pode fazer um data de outra forma impossível, colocando alguns dias de dez horas, e um sábado ou dois. Mas isso é muito arriscado. Não é provável que você consiga realizar 20% mais trabalho trabalhando 20% mais horas. Além do mais, as horas extras certamente falharão se continuarem por mais de duas ou três semanas.

Portanto, você não deve concordar em fazer horas extras, a menos que (1) você possa pessoalmente pagar, (2) é de curto prazo, duas semanas ou menos, e (3) seu chefe tem uma alternativa planejar caso o esforço de horas extras falhe.

Esse último critério é um obstáculo. Se o seu chefe não consegue articular com você o que ele fará se o esforço de horas extras falhar, então você não deve concordar em trabalhar horas extras.

AJUDA

73

## FAL S E D E V E R Y

De todos os comportamentos pouco profissionais que um programador pode ter, talvez o pior de tudo é dizer que você terminou quando sabe que não. Às vezes isso é apenas uma mentira aberta, e isso é ruim o suficiente. Mas o caso muito mais insidioso é quando conseguimos racionalizar uma nova definição de “pronto”. Nós convencemos nós mesmos que já fizemos o suficiente e passar para a próxima tarefa. Nós racionalizamos que qualquer trabalho que reste possa ser feito mais tarde, quando tivermos mais tempo. Esta é uma prática contagiosa. Se um programador fizer isso, outros verão e siga o exemplo. Um deles ampliará ainda mais a definição de “pronto” e todos os outros adotarão a nova definição. Eu vi isso ser levado a um nível horrível extremos. Na verdade, um de meus clientes definiu “concluído” como “check-in”. O código nem precisou compilar. É muito fácil “fazer” se nada precisa funcionar! Quando uma equipe cai nessa armadilha, os gestores ouvem que tudo está indo bem. Todos relatórios de status mostram que todos estão na hora certa. É como se cegos fizessem um piquenique nos trilhos: ninguém vê o trem de carga de obras inacabadas carregando sobre eles até que seja tarde demais.

## DEFINIR “D O N E”

Você evita o problema da falsa entrega criando uma definição independente de “feito.” A melhor maneira de fazer isso é ter seus analistas de negócios e testadores crie testes de aceitação automatizados<sup>5</sup> que devem ser aprovados antes que você possa dizer que está feito. Esses testes devem ser escritos em uma linguagem de teste como FitNesse, Selênio, RobotFX, Pepino e assim por diante. Os testes devem ser compreensíveis pelas partes interessadas e empresários e deve ser executado com frequência.

## AJUDA

Programar é difícil. Quanto mais jovem você é, menos acredita nisso. Afinal, é apenas um monte de declarações if e while. Mas à medida que você ganha experiência você começa a perceber que a maneira como você combina essas declarações if e while é criticamente

5. Consulte o Capítulo 7, “Teste de aceitação”.

## CAPÍTULO 4 CODIFICAÇÃO

74

importante. Você não pode simplesmente juntá-los e torcer pelo melhor. Em vez disso, você tem que particionar cuidadosamente o sistema em pequenas unidades compreensíveis que têm o mínimo possível a ver um com o outro - e isso é difícil.

Programar é tão difícil, na verdade, que está além da capacidade de uma pessoa para fazê-lo bem. Não importa o quão habilidoso você seja, você certamente se beneficiará pensamentos e ideias de outro programador.

### AJUDANDO OS OUTROS

Por isso, é responsabilidade dos programadores estarem disponíveis para ajudar um ao outro. É uma violação da ética profissional isolar-se num cubículo ou escritório e recusar as perguntas dos outros. Seu trabalho não é tão importante que você não pode emprestar parte do seu tempo para ajudar os outros. Na verdade, como profissional você tem a obrigação de oferecer essa ajuda sempre que for necessário.

Isso não significa que você não precise de algum tempo sozinho. Claro que sim. Mas você tem que ser justo e educado sobre isso. Por exemplo, você pode divulgar que entre 10h e meio-dia você não deve ser incomodado, mas das 13h às 15h sua porta está aberta.

Você deve estar consciente do status de seus companheiros de equipe. Se você ver alguém que parece estar com problemas, você deve oferecer sua ajuda. Você provavelmente ficará bastante surpreso com o efeito profundo que sua ajuda pode ter. Não é que você seja tão muito mais inteligente do que a outra pessoa, só que uma nova perspectiva pode ser uma profundo catalisador para a resolução de problemas.

Ao ajudar alguém, sente-se e escreva o código juntos. Planeje gastar o melhor parte de uma hora ou mais. Pode demorar menos do que isso, mas você não quer parecer estar apressados. Resigne-se à tarefa e faça um esforço sólido. Você provavelmente sairá tendo aprendido mais do que deu.

### B E I N G H E L P E D

Quando alguém se oferecer para ajudá-lo, seja gentil. Aceite a ajuda agradecidamente e entregue-se a essa ajuda. Não proteja seu território. Não empurre

## AJUDA

75

a ajuda porque você está sob a arma. Espere trinta minutos ou mais. Se por aquela hora a pessoa não está ajudando muito, então desculpe educadamente você mesmo e encerre a sessão com agradecimento. Lembre-se, assim como você é a honra é obrigada a oferecer ajuda, você é obrigado a aceitar ajuda.

Aprenda como pedir ajuda. Quando você está preso, confuso ou simplesmente não consegue embrulhar sua mente em torno de um problema, peça ajuda a alguém. Se você está sentado em uma equipe sala, você pode simplesmente sentar e dizer: “Preciso de ajuda”. Caso contrário, use o yammer, ou Twitter, ou e-mail, ou o telefone da sua mesa. Peça ajuda. Novamente, este é um questão de ética profissional. Não é profissional ficar preso quando a ajuda é facilmente acessível.

A esta altura você pode estar esperando que eu comece um refrão de Kumbaya enquanto coelhinhos peludos saltam nas costas dos unicórnios e todos nós voamos alegremente arco-íris de esperança e mudança. Não, não exatamente. Veja bem, os programadores tendem a ser introvertidos arrogantes e egocêntricos. Não entramos neste negócio porque como pessoas. A maioria de nós começou a programar porque preferimos focar profundamente em minúcias estereis, fazer malabarismos com muitos conceitos simultaneamente e, em geral, provar para nós mesmos que temos cérebros do tamanho de um planeta, sem precisar interagir com as complexidades confusas de outras pessoas.

Sim, isso é um estereótipo. Sim, é generalização com muitas exceções. Mas o a realidade é que os programadores não tendem a ser colaboradores.<sup>6</sup> E ainda assim a colaboração é fundamental para uma programação eficaz. Portanto, como para muitos de nós a colaboração não é um instinto, necessitamos de disciplinas que nos levem a colaborar.

## M E N T O R I N G

Tenho um capítulo inteiro sobre esse assunto posteriormente no livro. Por enquanto, deixe-me simplesmente dizer

que a formação de programadores menos experientes é responsabilidade daqueles que têm mais experiência. Os cursos de treinamento não resolvem isso. Os livros não resolvem.

Nada pode levar um jovem desenvolvedor de software ao alto desempenho mais rapidamente

6. Isto é muito mais verdadeiro para os homens do que para as mulheres. Tive uma conversa maravilhosa com @desi (Desi McAdam,

fundadora da DevChix) sobre o que motiva as mulheres programadoras. Eu disse a ela isso quando recebi um programa

trabalhando, era como matar a grande fera. Ela me disse que para ela e outras mulheres com quem ela havia conversado,

o ato de escrever código foi um ato de nutrir a criação.

## CAPÍTULO 4 CODIFICAÇÃO

76

do que sua própria motivação e orientação eficaz por parte de seus superiores. Portanto, uma vez novamente, é uma questão de ética profissional que os programadores seniores gastem tempo colocar programadores mais jovens sob sua proteção e orientá-los. Pelo

Da mesma forma, esses programadores mais jovens têm o dever profissional de procurar essa orientação de seus idosos.

### B I B L I O G R A P H Y

[Martin09]: Robert C. Martin, Código Limpo, Upper Saddle River, NJ: Prentice Salão, 2009.

[Martin03]: Robert C. Martin, Desenvolvimento Ágil de Software: Princípios, Padrões, e Práticas, Upper Saddle River, NJ: Prentice Hall, 2003.



## TESTE D RIVEN

## D E VE LOPM E NT

Já se passaram mais de dez anos desde que o Test Driven Development (TDD) fez sua estreia na indústria. Surgiu como parte da onda Extreme Programming (XP), mas desde então, foi adotado pelo Scrum e praticamente todos os outros métodos ágeis.

Mesmo equipes não ágeis praticam TDD.

Quando, em 1998, ouvi falar pela primeira vez de “Test First Programming”, fiquei cético. Quem não seria? Escreva seus testes de unidade primeiro? Quem faria uma coisa boba dessas?

Mas eu já era programador profissional há trinta anos e já tinha visto as coisas vêm e vão na indústria. Eu sabia que não deveria descartar nada de mão, especialmente quando alguém como Kent Beck diz isso.

Assim, em 1999, viajei para Medford, Oregon, para me encontrar com Kent e aprender o disciplina dele. Toda a experiência foi um choque!

Kent e eu sentamos em seu escritório e começamos a codificar algum probleminha simples em Java. Eu queria apenas escrever aquela coisa boba. Mas Kent resistiu e me levou, passo passo a passo, através do processo. Primeiro ele escreveu uma pequena parte de um teste unitário, mal suficiente para se qualificar como código. Então ele escreveu código suficiente para fazer esse teste compilar. Então ele escreveu mais um pouco de teste e depois mais código.

O tempo do ciclo estava completamente fora da minha experiência. Eu estava acostumado a escrever código por quase uma hora antes de tentar compilá-lo ou executá-lo. Mas Kent estava literalmente executando seu código a cada trinta segundos ou mais. Fiquei pasmo!

Além do mais, reconheci o tempo do ciclo! Foi o tipo de tempo de ciclo que eu usei anos antes, quando criança<sup>1</sup>, programava jogos em linguagens interpretadas como Basic ou Logotipo. Nessas linguagens não há tempo de construção, então basta adicionar uma linha de código e então execute. Você percorre o ciclo muito rapidamente. E por causa disso, você pode ser muito produtivo nesses idiomas.

Mas na programação real esse tipo de tempo de ciclo era absurdo. De verdade programação, você tinha que gastar muito tempo escrevendo código e muito mais tempo para compilá-lo. E ainda mais tempo para depurá-lo. Eu era um C++ programador, caramba! E em C++ tivemos tempos de construção e link que levaram minutos, às vezes horas. Os tempos de ciclo de trinta segundos eram inimagináveis.

No entanto, lá estava Kent, cozinhando este programa Java em ciclos de trinta segundos e sem qualquer indício de que ele iria desacelerar tão cedo. Então amanheceu para mim, enquanto estava sentado no escritório de Kent, que usando essa disciplina simples eu poderia código em linguagens reais com o tempo de ciclo do Logo! Fiquei viciado!

1. Do meu ponto de vista na época, uma criança é qualquer pessoa com menos de 35 anos. Durante meus vinte anos, passei um tempo significativo

Não posso perder tempo escrevendo joguinhos bobos em linguagens interpretadas. Eu escrevi jogos de guerra espacial, aventura

jogos, jogos de corrida de cavalos, jogos de cobra, jogos de azar, você escolhe.

## AS TRÊS LEIS DO TDD

### O JÚRI ISIN

Desde então aprendi que TDD é muito mais que um simples truque para encurtar meu tempo de ciclo. A disciplina tem todo um repertório de benefícios que irei descrever nos parágrafos seguintes.

Mas primeiro preciso dizer o seguinte:

- O júri chegou!
- A polêmica acabou.
- GOTO é prejudicial.
- E o TDD funciona.

Sim, tem havido muitos blogs e artigos controversos escritos sobre TDD ao longo dos anos e ainda existem. Nos primeiros dias, foram tentativas sérias de crítica e compreensão. Hoje em dia, porém, são apenas desabafos. O fundo

A linha é que o TDD funciona e todo mundo precisa superar isso.

Eu sei que isso parece estridente e unilateral, mas dado o histórico, não acho os cirurgiões deveriam defender a lavagem das mãos, e não acho que os programadores deveria ter que defender o TDD.

Como você pode se considerar um profissional se não sabe que tudo seu código funciona? Como você pode saber que todo o seu código funciona se não o testar toda vez que você faz uma mudança? Como você pode testá-lo toda vez que fizer um mudar se você não tiver testes de unidade automatizados com cobertura muito alta? Como pode você obter testes de unidade automatizados com cobertura muito alta sem praticar TDD?

Essa última frase requer alguma elaboração. O que é TDD?

### OS TRÊS REE LAWS DO TDD

1. Você não tem permissão para escrever nenhum código de produção até que tenha escrito primeiro um teste de unidade com falha.
2. Você não tem permissão para escrever mais testes de unidade do que o suficiente para falhar - e não compilar está falhando.

3. Você não tem permissão para escrever mais código de produção que seja suficiente para passar o teste de unidade atualmente falhando.

Essas três leis prendem você a um ciclo que dura, talvez, trinta segundos. Você comece escrevendo uma pequena parte de um teste de unidade. Mas dentro de alguns segundos você deve mencionar o nome de alguma classe ou função que você ainda não escreveu, fazendo com que o teste de unidade falhe na compilação. Então você deve escrever a produção código que faz o teste ser compilado. Mas você não pode escrever mais do que isso, então você comece a escrever mais código de teste de unidade.

Você dá voltas e mais voltas no ciclo. Adicionando um pouco ao código de teste. Adicionando um pouco ao código de produção. Os dois fluxos de código crescem simultaneamente em complementares componentes. Os testes se ajustam ao código de produção como um anticorpo se ajusta a um antígeno.

### O L I T A N Y O F B E N E F I T S

#### Certeza

Se você adotar o TDD como disciplina profissional, você escreverá dezenas de testes todos os dias, centenas de testes todas as semanas e milhares de testes todos os anos.

E você manterá todos esses testes em mãos e os executará sempre que fizer qualquer alterações no código.

Sou o principal autor e mantenedor do FitNesse,<sup>2</sup> um software de aceitação baseado em Java ferramenta de teste. No momento em que este livro foi escrito, FitNesse tinha 64.000 linhas de código, das quais 28.000

estão contidos em pouco mais de 2.200 testes unitários individuais. Esses testes cobrem pelo menos 90% do código de produção<sup>3</sup> e leva cerca de 90 segundos para ser executado.

Sempre que faço uma alteração em qualquer parte do FitNesse, simplesmente executo os testes de unidade. Se

eles passam, tenho quase certeza de que a mudança que fiz não quebrou nada.

Quão certo é “quase certo”? Certo o suficiente para enviar!

O processo de controle de qualidade para FitNesse é o comando: ant release. Esse comando constrói o FitNesse do zero e depois executa todos os testes de unidade e aceitação.

Se todos esses testes passarem, eu envio.

2. <http://fitnesse.org>

3. Noventa por cento é o mínimo. O número é realmente maior que isso. A quantidade exata é difícil de calcular porque as ferramentas de cobertura não conseguem ver o código executado em processos externos ou em blocos catch.

### Taxa de injeção de defeito

Agora, FitNesse não é um aplicativo de missão crítica. Se houver um bug, ninguém morre e ninguém perde milhões de dólares. Então posso me dar ao luxo de enviar com base em nada além de passar nos testes. Por outro lado, FitNesse tem milhares de usuários, e apesar da adição de 20.000 novas linhas de código no ano passado, minha lista de bugs apenas tem 17 bugs (muitos dos quais são de natureza cosmética). Então eu conheço meu defeito taxa de injeção é muito baixa.

Este não é um efeito isolado. Existem vários relatórios<sup>4</sup> e estudos<sup>5</sup> que descrever redução significativa de defeitos. Da IBM à Microsoft, do Sabre à Symantec, empresa após empresa e equipe após equipe experimentaram defeitos reduções de 2X, 5X e até 10X. São números que nenhum profissional deveria ignorar.

### Coragem

Por que você não conserta um código incorreto quando o vê? Sua primeira reação ao ver um A função bagunçada é “Isso está uma bagunça, precisa ser limpo”. Sua segunda reação é “Não vou tocar nisso!” Por que? Porque você sabe que se tocar você corre o risco quebrando-o; e se você quebrá-lo, ele se tornará seu.

Mas e se você pudesse ter certeza de que sua limpeza não quebrou nada? O que se você tivesse o tipo de certeza que acabei de mencionar? E se você pudesse clicar em um botão e saiba em 90 segundos que suas alterações não quebraram nada, e só tinha feito o bem?

Este é um dos benefícios mais poderosos do TDD. Quando você tem um conjunto de testes em que você confia, você perde todo o medo de fazer alterações. Quando você vê mal código, basta limpá-lo no local. O código se torna argila que você pode com segurança esculpir em estruturas simples e agradáveis.

Quando os programadores perdem o medo de limpar, eles limpam! E código limpo é mais fácil de entender, mais fácil de mudar e mais fácil de estender. Os defeitos tornam-se

4. [http://www.objectmentor.com/omSolutions/agile\\_customers.html](http://www.objectmentor.com/omSolutions/agile_customers.html)

5. [Maximilien], [George2003], [Janzen2005], [Nagappan2008]

ainda menos provável porque o código fica mais simples. E a base de código de forma constante melhora em vez da podridão normal a que a nossa indústria se habituou.

Que programador profissional permitiria que a podridão continuasse?

Documentação

Você já usou uma estrutura de terceiros? Frequentemente, o terceiro enviará para você um manual bem formatado, escrito por escritores de tecnologia. O manual típico emprega 27 fotografias coloridas brilhantes de oito por dez com círculos e setas e um parágrafo no verso de cada um explicando como configurar, implantar, manipular e de outra forma usar essa estrutura. No final, no apêndice, geralmente há uma pequena seção feia que contém todos os exemplos de código. Qual é o primeiro lugar que você acessa nesse manual? Se você é um programador, você vai aos exemplos de código. Você acessa o código porque sabe que o código dirá você a verdade. As 27 fotografias brilhantes coloridas de oito por dez com círculos e setas e um parágrafo no verso podem ser bonitos, mas se você quiser saber como usar o código, você precisa ler o código.

Cada um dos testes unitários que você escreve ao seguir as três leis é um exemplo, escrito em código, descrevendo como o sistema deve ser usado. Se você seguir o três leis, então haverá um teste unitário que descreve como criar cada objeto no sistema, todas as maneiras pelas quais esses objetos podem ser criados. Haverá uma unidade teste que descreve como chamar todas as funções do sistema de todas as maneiras que essas funções podem ser chamadas de forma significativa. Para qualquer coisa que você precisa saber como fazer, haverá um teste de unidade que o descreve em detalhes.

Os testes unitários são documentos. Eles descrevem o design de nível mais baixo do sistema. Eles são inequívocos, precisos e escritos em uma linguagem que o público entende e são tão formais que executam. Eles são os melhores tipo de documentação de baixo nível que pode existir. Que profissional não fornecer essa documentação?

Projeto

Quando você segue as três leis e escreve seus testes primeiro, você se depara com uma dilema. Muitas vezes você sabe exatamente qual código deseja escrever, mas os três

## O QUE TDD NÃO É

83

as leis dizem para você escrever um teste de unidade que falha porque esse código não existe! Isto significa que você precisa testar o código que está prestes a escrever.

O problema com o teste de código é que você precisa isolá-lo. Muitas vezes é difícil testar uma função se essa função chamar outras funções. Para escrever isso teste, você precisa descobrir uma maneira de dissociar a função de todos os outros. Em outras palavras, a necessidade de testar primeiro força você a pensar em boas projeto.

Se você não escrever seus testes primeiro, não haverá força impedindo você de acoplar as funções juntas em uma massa não testável. Se você escrever seus testes mais tarde, você pode ser capaz de testar as entradas e as saídas da massa total, mas será provavelmente será muito difícil testar as funções individuais.

Portanto, seguir as três leis e escrever seus testes primeiro cria uma força que leva você a um design melhor desacoplado. Que profissional não empregam ferramentas que os levaram a projetos melhores?

“Mas posso escrever meus testes mais tarde”, você diz. Não, você não pode. Na verdade. Ah, você pode escreva alguns testes mais tarde. Você pode até abordar uma cobertura alta mais tarde se for cuidadoso para medi-lo. Mas os testes que você escreve depois do fato são de defesa. Os testes que você escreve primeiro são ofensas. Os testes posteriores são escritos por alguém que já está investido no código e já sabe como o problema foi resolvido. Simplesmente não há como esses testes podem ser tão incisivos quanto os testes escritos primeiro.

## A L O P T I O N P R O F E S S I O N

O resultado de tudo isso é que o TDD é a opção profissional. É uma disciplina que aumenta a certeza, a coragem, a redução de defeitos, a documentação e o design. Com tudo isso acontecendo, pode ser considerado pouco profissional não usá-lo.

## O QUE TDD NÃO É

Apesar de todos os seus pontos positivos, o TDD não é uma religião ou uma fórmula mágica. Seguindo o três leis não garante nenhum desses benefícios. Você ainda pode escrever código incorreto mesmo se você escrever seus testes primeiro. Na verdade, você pode escrever testes ruins.

Da mesma forma, há momentos em que seguir as três leis é simplesmente impraticável ou inapropriado. Essas situações são raras, mas existem. Não desenvolvedor profissional deve sempre seguir uma disciplina quando essa disciplina não mais mal do que bem.

### B I B L I O G R A P H Y

[Maximilien]: E. Michael Maximilien, Laurie Williams, “Avaliando Test-Driven Desenvolvimento na IBM”, [http://collaboration.csc.ncsu.edu/laurie/Papers/MAXIMILIEN\\_WILLIAMS.PDF](http://collaboration.csc.ncsu.edu/laurie/Papers/MAXIMILIEN_WILLIAMS.PDF)

[George2003]: B. George e L. Williams, “Uma investigação inicial de testes Desenvolvimento Impulsionado na Indústria”, <http://collaboration.csc.ncsu.edu/laurie/Artigos/TDDpaperv8.pdf>

[Janzen2005]: D. Janzen e H. Saiedian, “Conceitos de desenvolvimento orientado a testes, taxonomia e direção futura”, IEEE Computer, Volume 38, Edição 9, págs. 43–50.

[Nagappan2008]: Nachiappan Nagappan, E. Michael Maximilien, Thirumalesh Bhat e Laurie Williams, “Realizando a melhoria da qualidade por meio de testes desenvolvimento impulsionado: resultados e experiências de quatro equipes industriais”, Springer Science + Business Media, LLC 2008: [http://research.microsoft.com/en-us/projects/esm/nagappan\\_tdd.pdf](http://research.microsoft.com/en-us/projects/esm/nagappan_tdd.pdf)



## PR ATUAÇÃO

Todos os profissionais praticam sua arte participando de exercícios de aprimoramento de habilidades. Músicos ensaiam escalas. Jogadores de futebol passam por pneus. Médicos praticam suturas e técnicas cirúrgicas. Os advogados praticam argumentos. Soldados ensaiam missões. Quando o desempenho é importante, os profissionais praticam. Este capítulo é tudo sobre as maneiras pelas quais os programadores podem praticar sua arte.

## CAPÍTULO 6 PRÁTICA

86

### ALGUMAS BAC KG RO U N D O N PRÁTICA

Praticar não é um conceito novo no desenvolvimento de software, mas não reconhecê-lo como praticado até logo após a virada do milênio. Talvez a primeira instância formal de um programa prático foi impressa na página 6 do [K&R-C].

```
principal()
```

```
{  
    printf("Olá, mundo\n");  
}
```

Quem entre nós não escreveu esse programa de uma forma ou de outra? Nós usamos isso como forma de provar um novo ambiente ou uma nova linguagem. Escrevendo e executando esse programa é a prova de que podemos escrever e executar qualquer programa.

Quando eu era muito mais jovem, um dos primeiros programas que escrevi sobre um novo computador era SQINT, os quadrados de inteiros. Eu escrevi em assembler, BASIC, FORTRAN, COBOL e um zilhão de outras linguagens. Mais uma vez, foi uma forma de provar que eu poderia fazer o computador fazer o que eu queria.

No início dos anos 80, os computadores pessoais começaram a aparecer nos departamentos lojas. Sempre que passava por um, como um VIC-20 ou um Commodore-64, ou um TRS-80, Eu escreveria um pequeno programa que imprimisse um fluxo infinito de caracteres ‘\’ e ‘/’ na tela. Os padrões que este programa produziu eram agradáveis à vista e pareciam muito mais complexos do que o pequeno programa que os gerou.

Embora esses pequenos programas fossem certamente programas práticos, os programas mers em geral não praticavam. Francamente, esse pensamento nunca nos ocorreu.

Estávamos muito ocupados escrevendo código para pensar em praticar nossas habilidades. E além disso, qual teria sido o objetivo? Durante esses anos a programação não exigia reações rápidas ou dedos ágeis. Não usamos tela editores até o final dos anos 70. Passamos muito do nosso tempo esperando por compilações ou depurar trechos de código longos e horríveis. Ainda não tínhamos inventado o ciclos curtos de TDD, por isso não exigimos o ajuste fino que a prática poderia trazer.

## ALGUNS ANTECEDENTES DA PRÁTICA

### DOIS Z E RO S

Mas as coisas mudaram desde os primeiros dias da programação. Algumas coisas têm mudado muito. Outras coisas não mudaram muito.

Uma das primeiras máquinas para as quais escrevi programas foi uma PDP-8/I. Esta máquina teve um tempo de ciclo de 1,5 microssegundos. Tinha 4.096 palavras de 12 bits na memória central. Era do tamanho de uma geladeira e consumia uma quantidade significativa de eletricidade poder. Ele tinha uma unidade de disco que podia armazenar 32K de palavras de 12 bits, e conversamos para ele com um teletipo de 10 caracteres por segundo. Achamos que isso era um poderoso máquina, e nós a usamos para fazer milagres.

Acabei de comprar um novo laptop Macbook Pro. Possui um processador dual core de 2,8 GHz, 8 GB de RAM, um SSD de 512 GB e uma tela LED 1920 × 1200 de 17 polegadas. Eu carrego isso minha mochila. Ele fica no meu colo. Consome menos de 85 watts.

Meu laptop é oito mil vezes mais rápido, tem dois milhões de vezes mais memória, tem dezesseis milhões de vezes mais armazenamento off-line, requer 1% da energia, ocupa 1% do o espaço e custa um vigésimo quinto do preço do PDP-8/I. Vamos fazer as contas:

$$8.000 \cdot 2.000.000 \cdot 16.000.000 \cdot 100 \cdot 100 \cdot 25 = 6,4 \cdot 10^{22}$$

Este número é grande. Estamos falando de 22 ordens de grandeza! É assim existem muitos angstroms entre aqui e Alfa Centauri. Isso é quantos elétrons existem em um dólar de prata. Essa é a massa da Terra em unidades de Michael Moore. Este é um número muito grande. E está sentado no meu colo, e provavelmente o seu também!

E o que estou fazendo com esse aumento de potência de 22 fatores de dez? estou fazendo praticamente o que eu estava fazendo com aquele PDP-8/I. Estou escrevendo declarações if, while loops e atribuições.

Ah, tenho ferramentas melhores para escrever essas declarações. E eu tenho línguas melhores para escrever essas declarações. Mas a natureza das declarações não mudou em todo esse tempo. O código em 2010 seria reconhecível para um programador do década de 1960. A argila que manipulamos não mudou muito nessas quatro décadas.

### TURN NARO U N D TIME

Mas a forma como trabalhamos mudou drasticamente. Nos anos 60 eu poderia esperar um dia ou dois para ver os resultados de uma compilação. No final dos anos 70, um programa de 50.000 linhas pode levar 45 minutos para compilar. Mesmo na década de 90, longos tempos de construção eram a norma.

Os programadores de hoje não esperam por compilações.<sup>1</sup> Os programadores de hoje têm essas imenso poder sob seus dedos que eles podem girar em torno do vermelho-verde-refatorar loop em segundos.

Por exemplo, trabalho em um projeto Java de 64.000 linhas chamado FitNesse. Uma construção completa, incluindo todos os testes unitários e de integração, é executado em menos de 4 minutos. Se aqueles os testes passarem, estou pronto para enviar o produto. Portanto, todo o processo de controle de qualidade, desde a origem

código para implantação, requer menos de 4 minutos. As compilações quase não tomam medidas tempo disponível. Testes parciais requerem segundos. Então eu posso literalmente girar em torno do compile/teste loop dez vezes por minuto!

Nem sempre é sensato ir tão rápido. Muitas vezes é melhor desacelerar e apenas pensar.<sup>2</sup>

Mas há outras ocasiões em que girar em torno desse loop o mais rápido possível é altamente produtivo.

Fazer qualquer coisa rapidamente requer prática. Girando em torno do loop de código/teste rapidamente exige que você tome decisões muito rápidas. Tomar decisões rapidamente significa ser capaz de reconhecer um vasto número de situações e problemas e simplesmente saiba o que fazer para resolvê-los.

Considere dois artistas marciais em combate. Cada um deve reconhecer o que o outro está tentando e responde adequadamente em milissegundos. Em um combate situação, você não pode se dar ao luxo de congelar o tempo, estudar as posições e deliberar sobre a resposta apropriada. Numa situação de combate você simplesmente tem para reagir. Na verdade, é o seu corpo que reage enquanto a sua mente está trabalhando em um estratégia de nível superior.

1. O fato de alguns programadores esperarem por compilações é trágico e indicativo de descuido. No mundo de hoje

os tempos de construção devem ser medidos em segundos, não em minutos e certamente não em horas.

2. Esta é uma técnica que Rich Hickey chama de HDD, ou Desenvolvimento Orientado a Rede.

## O DOJO DE CODIFICAÇÃO

89

Quando você gira o loop de código/teste várias vezes por minuto, é seu corpo que sabe quais teclas apertar. Uma parte primordial da sua mente reconhece a situação e reage em milissegundos com a solução apropriada enquanto sua mente está livre para se concentrar no problema de nível superior.

Tanto no caso das artes marciais quanto no caso da programação, a velocidade depende prática. E em ambos os casos a prática é semelhante. Escolhemos um repertório de pares problema/solução e executá-los repetidamente até sabermos eles com frio.

Considere um guitarrista como Carlos Santana. A música em sua cabeça simplesmente vem fora os dedos. Ele não se concentra nas posições dos dedos ou na técnica de palhetada. Seu mente está livre para planejar melodias e harmonias de nível superior enquanto seu corpo traduz esses planos em movimentos dos dedos de nível inferior.

Mas obter esse tipo de facilidade de jogo requer prática. Músicos praticam escalas e estudos e riffs repetidamente até que eles os conheçam completamente.

### O C O D I N G D O J O

Desde 2001 venho realizando uma demonstração de TDD que chamo de The Bowling Jogo.<sup>3</sup> É um pequeno exercício adorável que leva cerca de trinta minutos. Ele experimenta conflito no design, chega ao clímax e termina com uma surpresa. Eu escrevi um capítulo inteiro sobre este exemplo em [PPP2003].

Ao longo dos anos realizei esta demonstração com centenas, talvez milhares, de vezes. Fiquei muito bom nisso! Eu poderia fazer isso dormindo. Minimizei as teclas digitadas, ajustou os nomes das variáveis e ajustou a estrutura do algoritmo até que fosse apenas certo. Embora eu não soubesse na época, este foi meu primeiro kata.

Em 2005 participei da Conferência XP2005 em Sheffield, Inglaterra. Eu participei de um sessão com o nome Coding Dojo liderada por Laurent Bossavit e Emmanuel

Gaillot. Eles fizeram com que todos abrissem seus laptops e codificassem junto com eles enquanto

3. Este se tornou um kata muito popular, e uma pesquisa no Google encontrará muitos exemplos dele. O original é

aqui: <http://butunclebob.com/ArticleS.UncleBob.TheBowlingGameKata>.

usou TDD para escrever Game of Life de Conway. Eles chamaram isso de “Kata” e creditaram “Pragmático” Dave Thomas<sup>4</sup> com a ideia original.<sup>5</sup>

Desde então, muitos programadores adotaram uma metáfora das artes marciais para sessões práticas. O nome Coding Dojo<sup>6</sup> parece ter pegado. Às vezes um grupo de programadores se reunirá e praticará juntos como artistas marciais fazer. Outras vezes, os programadores praticarão sozinhos, novamente como fazem os artistas marciais. Cerca de um ano atrás eu estava ensinando um grupo de desenvolvedores em Omaha. No almoço eles me convidou para participar do Coding Dojo. Observei vinte desenvolvedores abrirem seus laptops e, tecla por tecla, seguido pelo líder que estava fazendo

O jogo de boliche Kata.

Existem vários tipos de atividades que acontecem em um dojo. Aqui estão alguns:

### K ATA

Nas artes marciais, um kata é um conjunto preciso de movimentos coreografados que simula um lado de um combate. O objetivo, que é abordado assintoticamente, é a perfeição. O artista se esforça para ensinar seu corpo a fazer cada movimento com perfeição e a reunir esses movimentos em uma encenação fluida. Kata bem executados são lindos para assistir.

Por mais bonitos que sejam, o propósito de aprender um kata não é executá-lo estágio. O objetivo é treinar sua mente e corpo como reagir em um determinado situação de combate. O objetivo é tornar os movimentos aperfeiçoados automáticos e instintivo para que eles estejam lá quando você precisar deles.

Um kata de programação é um conjunto preciso de pressionamentos de teclas e mouse coreografados. movimentos que simulam a resolução de algum problema de programação. Você não estão realmente resolvendo o problema porque você já conhece a solução.

Em vez disso, você está praticando os movimentos e decisões envolvidos na resolução do problema. problema.

4. Usamos o prefixo “Pragmático” para desambiguá-lo do “Big” Dave Thomas da OTI.

5. <http://codekata.pragprog.com>

6. <http://codingdojo.org/>

A assíntota da perfeição é mais uma vez o objetivo. Você repete o exercício e novamente para treinar seu cérebro e dedos como se mover e reagir. Como você pratica, você poderá descobrir melhorias sutis e eficiências em seu movimentos ou na própria solução.

Praticar um conjunto de katas é uma boa maneira de aprender teclas de atalho e navegação expressões idiomáticas. Também é uma boa forma de aprender disciplinas como TDD e CI. Mas a maioria mais importante ainda, é uma boa maneira de inserir pares comuns de problema/solução em seu subconsciente, para que você simplesmente saiba como resolvê-los ao enfrentá-los em programação de verdade.

Como qualquer artista marcial, um programador deve conhecer vários kata e pratique-os regularmente para que não desapareçam da memória. Muitos katas estão registrados em <http://katas.softwarecraftsmanship.org>. Outros podem ser encontrados em <http://codekata.pragprog.com>. Alguns dos meus favoritos são:

- O jogo de boliche: [http://butunclebob.com/ArticleS.UncleBob.TheBowling-Jogo Kata](http://butunclebob.com/ArticleS.UncleBob.TheBowling-JogoKata)
- Fatores principais: <http://butunclebob.com/ArticleS.UncleBob.ThePrimeFactors-Kata>
- Quebra de texto: <http://thecleancoder.blogspot.com/2010/10/craftsman-62-dark-caminho.html>

Para um verdadeiro desafio, tente aprender um kata tão bem que você possa musicá-lo. Fazer isso bem é difícil.<sup>7</sup>

### WA SA

Quando estudei jiu-jitsu, passamos grande parte do tempo no dojo praticando em duplas. nossa wasa. Wasa é muito parecido com um kata de dois homens. As rotinas são precisamente memorizado e reproduzido. Um parceiro desempenha o papel de agressor e o outro parceiro é o defensor. Os movimentos são repetidos inúmeras vezes enquanto o os profissionais trocam de papéis.

7. <http://katas.softwarecraftsmanship.org/?p=71>

Os programadores podem praticar de maneira semelhante usando um jogo conhecido como ping-pong.<sup>8</sup> Os dois parceiros escolhem um kata, ou um problema simples. Um programador escreve um teste de unidade e o outro deve fazê-lo passar. Então eles invertem papéis.

Se os parceiros escolherem um kata padrão, então o resultado é conhecido e o programadores estão praticando e criticando a digitação e a digitação uns dos outros. técnicas de mouse e quão bem eles memorizaram o kata. Por outro

Por outro lado, se os parceiros escolherem um novo problema para resolver, o jogo pode ficar um pouco mais interessante. O programador que escreve um teste tem uma quantidade excessiva de controle sobre como o problema será resolvido. Ele também tem uma quantidade significativa de poder para estabelecer restrições. Por exemplo, se os programadores optarem por implementar um algoritmo de classificação, o redator do teste pode facilmente impor restrições à velocidade e espaço de memória que desafiará seu parceiro. Isso pode tornar o jogo bastante competitivo. . . e divertido.

### RAN DOR I

Randori é um combate de forma livre. Em nosso dojo de jiu-jitsu, montamos uma variedade de cenários de combate e depois aplicá-los. Às vezes, uma pessoa era instruída a defender, enquanto cada um de nós o atacaria em sequência. Às vezes nós faríamos colocar dois ou mais atacantes contra um único defensor (geralmente o sensei, que quase sempre venceu). Às vezes fazíamos dois contra dois e assim por diante.

O combate simulado não se adapta bem à programação; no entanto, há um jogo que é jogado em muitos dojos de codificação chamados randori. É muito parecido com dois homens wasa em que os parceiros estão resolvendo um problema. No entanto, é jogado com muitas pessoas e as regras têm uma diferença. Com a tela projetada na parede, uma pessoa escreve um teste e depois se senta. A próxima pessoa faz o teste passar e então escreve o próximo teste. Isso pode ser feito em sequência ao redor da mesa, ou as pessoas podem simplesmente fazer fila quando se sentirem emocionadas. Em ambos os casos, esses exercícios pode ser muito divertido.

8. <http://c2.com/cgi/wiki?PairProgrammingPingPongPattern>



## AMPLIANDO SUA EXPERIÊNCIA

93

É notável o quanto você pode aprender com essas sessões. Você pode ganhar um imenso insight sobre a maneira como outras pessoas resolvem problemas. Esses insights podem servir apenas para ampliar sua própria abordagem e melhorar suas habilidades.

## AMPLIANDO SUA EXPERIÊNCIA

Os programadores profissionais muitas vezes sofrem com a falta de diversidade nos tipos de problemas que eles resolvem. Os empregadores muitas vezes impõem um único idioma, plataforma e domínio em que seus programadores devem trabalhar. Sem uma ampliação influência, isso pode levar a uma restrição muito prejudicial ao seu currículo e ao seu mentalidade. Não é incomum que tais programadores se encontrem despreparados pelas mudanças que periodicamente varrem a indústria.

## O P E N S O U R C E

Uma forma de se manter à frente da curva é fazer o que os advogados e os médicos fazem: assumir algum trabalho pro bono, contribuindo para um projeto de código aberto. Existem muitos por aí, e provavelmente não há melhor maneira de aumentar seu repertório de habilidades do que realmente trabalhar em algo que interessa a outra pessoa.

Então se você é um programador Java, contribua com um projeto Rails. Se você escreve muito de C++ para seu empregador, encontre um projeto Python e contribua com ele.

## PRÁTICA E ETH I C S

Programadores profissionais praticam em seu próprio tempo. Não é do seu empregador trabalho para ajudá-lo a manter suas habilidades afiadas para você. Não é função do seu empregador ajudá-lo a manter seu currículo atualizado. Os pacientes não pagam aos médicos para praticarem suturas. Os fãs de futebol (geralmente) não pagam para ver os jogadores passarem pelos pneus. Frequentadores de concertos

não pague para ouvir músicos tocando escalas. E os empregadores de programadores não tenho que pagar pelo seu tempo de prática.

Como o seu tempo de prática é seu, você não precisa usar os mesmos idiomas ou plataformas que você usa com seu empregador. Escolha qualquer idioma que você goste e mantenha suas habilidades políglotas afiadas. Se você trabalha em uma loja .NET, pratique um pouco de Java ou Ruby no almoço ou em casa.

## CAPÍTULO 6 PRÁTICA

94

### CONCLUSÃO

De uma forma ou de outra, todos os profissionais praticam. Eles fazem isso porque se preocupam em fazer o melhor trabalho possível. Além do mais, eles praticam seu próprio tempo porque percebem que é sua responsabilidade - e não sua do empregador - para manter suas habilidades afiadas. Praticar é o que você faz quando não está sendo pago. Você faz isso para ser pago, e bem pago.

### BIBLIOGRAPHY

[K&R-C]: Brian W. Kernighan e Dennis M. Ritchie, A Programação C Idioma, Upper Saddle River, NJ: Prentice Hall, 1975.

[PPP2003]: Robert C. Martin, Desenvolvimento Ágil de Software: Princípios, Padrões, e Práticas, Upper Saddle River, NJ: Prentice Hall, 2003.

## ACC EPTANC E TESTE

O papel do desenvolvedor profissional é tanto de comunicação quanto de desenvolvimento. papel de implementação. Lembre-se de que a entrada/saída de lixo também se aplica aos programadores, portanto, os programadores profissionais têm o cuidado de garantir que sua comunicação com outros membros da equipe e da empresa são precisos e saudáveis.

### R EQUISITOS DE COMUNICAÇÃO

Um dos problemas de comunicação mais comuns entre programadores e o negócio são os requisitos. Os empresários declaram o que acreditam que precisam, e então os programadores constroem o que acreditam que o negócio descreveu. Pelo menos é assim que deveria funcionar. Na realidade, a comunicação de requisitos é extremamente difícil e o processo está repleto de erros.

Em 1979, enquanto trabalhava na Teradyne, recebi a visita de Tom, o gerente da instalação e serviço de campo. Ele me pediu para mostrar a ele como usar o ED-402 editor de texto para criar um sistema simples de identificação de problemas.

ED-402 era um editor proprietário escrito para o computador M365, que foi Clone PDP-8 de Teradyne. Como editor de texto, era muito poderoso. Tinha um embutido linguagem de script que usamos para todos os tipos de aplicativos de texto simples.

Tom não era um programador. Mas a aplicação que ele tinha em mente era simples, então ele pensou que eu poderia ensiná-lo rapidamente e então ele poderia escrever o requerimento ele mesmo. Na minha ingenuidade pensei a mesma coisa. Afinal, a linguagem de script era pouco mais que uma linguagem macro para os comandos de edição, com muito decisão rudimentar e construções de looping.

Então nos sentamos juntos e perguntei o que ele queria que sua inscrição fizesse.

Ele começou com a tela de entrada inicial. Mostrei a ele como criar um arquivo de texto que conteria as instruções do script e como digitar o simbólico representação dos comandos de edição nesse script. Mas quando olhei para seus olhos, não havia nada olhando para trás. Minha explicação simplesmente não fazia sentido para ele.

Esta foi a primeira vez que encontrei isso. Para mim foi uma coisa simples de representam comandos do editor simbolicamente. Por exemplo, para representar um controle-B comando (o comando que coloca o cursor no início do atual linha) você simplesmente digitou ^B no arquivo de script. Mas isso não fazia sentido para Tom. Ele não conseguia passar da edição de um arquivo para a edição de um arquivo que editava um arquivo. Tom não era burro. Acho que ele simplesmente percebeu que isso seria muito mais envolvido do que pensava inicialmente e não queria investir tempo e energia mental necessária para aprender algo tão terrivelmente complicado como usando um editor para comandar um editor.

Então, pouco a pouco, me vi implementando esse aplicativo enquanto ele estava sentado lá e assisti. Nos primeiros vinte minutos ficou claro que sua ênfase havia mudou de aprender como fazer sozinho para ter certeza de que o que eu fiz foi o que ele queria.

Levamos um dia inteiro. Ele descreveria um recurso e eu o implementaria enquanto ele observava. O tempo do ciclo foi de cinco minutos ou menos, então não havia razão para ele se levantar e fazer qualquer outra coisa. Ele me pedia para fazer X, e dentro de cinco minutos eu tinha X funcionando.

Muitas vezes ele desenhava o que queria em um pedaço de papel. Algumas das coisas ele queria eram difíceis de fazer no ED-402, então eu proporia outra coisa. Nós eventualmente concordaria em algo que funcionaria, e então eu faria funcionar.

Mas então tentávamos e ele mudava de ideia. Ele diria algo como: “Sim, isso simplesmente não tem o fluxo que procuro. Vamos tentar de uma maneira diferente.”

Hora após hora, mexemos, cutucamos e colocamos aquele aplicativo em forma.

Tentamos uma coisa, depois outra e depois outra. Ficou muito claro para mim que ele era o escultor e eu era a ferramenta que ele empunhava.

No final, ele conseguiu o aplicativo que procurava, mas não tinha ideia de como fazer isso. comece a construir o próximo para si mesmo. Eu, por outro lado, aprendi um lição poderosa sobre como os clientes realmente descobrem o que precisam. eu aprendi que a sua visão das características nem sempre sobrevive ao contato real com o computador.

### PREMATUREREPRECISION

Tanto as empresas como os programadores são tentados a cair na armadilha da precisão. Os empresários querem saber exatamente o que vão conseguir antes eles autorizam um projeto. Os desenvolvedores querem saber exatamente o que devem entregar antes de estimarem o projeto. Ambos os lados querem uma precisão que simplesmente não pode ser alcançado e muitas vezes estão dispostos a desperdiçar uma fortuna tentando alcançá-lo.

#### O Princípio da Incerteza

O problema é que as coisas parecem diferentes no papel e no ambiente de trabalho. sistema. Quando a empresa realmente vê o que especificou em execução em um sistema, eles percebem que não era isso que eles queriam. Depois que eles virem o requisito realmente correndo, eles têm uma ideia melhor do que realmente querem - e é geralmente não é o que eles estão vendo.

## CAPÍTULO 7 TESTE DE ACEITAÇÃO

98

Há uma espécie de efeito observador, ou princípio de incerteza, em jogo. Quando você demonstrar um recurso para a empresa, isso fornece mais informações do que eles tinham antes, e essas novas informações impactam a forma como eles veem todo o sistema. No final, quanto mais precisos forem os seus requisitos, menos relevantes eles serão. tornar-se à medida que o sistema é implementado.

Ansiedade de estimativa

Os desenvolvedores também podem cair na armadilha da precisão. Eles sabem que devem estimar o sistema e muitas vezes pensam que isso requer precisão. Isso não acontece.

Primeiro, mesmo com informações perfeitas, suas estimativas terão uma variação enorme.

Em segundo lugar, o princípio da incerteza transforma a precisão inicial em hash. O os requisitos mudarão, tornando essa precisão discutível.

Os desenvolvedores profissionais entendem que as estimativas podem e devem ser feitas com base em requisitos de baixa precisão e reconhecem que essas estimativas são estimativas. Para reforçar isso, os desenvolvedores profissionais sempre incluem barras de erro com suas estimativas para que a empresa entenda a incerteza. (Veja

Capítulo 10, “Estimativa”).

LATE AMBIGUITY

A solução para a precisão prematura é adiar a precisão o máximo possível.

Os desenvolvedores profissionais não especificam um requisito até que estejam quase para desenvolvê-lo. No entanto, isso pode levar a outro mal: a ambiguidade tardia.

Muitas vezes as partes interessadas discordam. Quando o fizerem, poderão achar mais fácil escrever palavras

contornar o desacordo em vez de resolvê-lo. Eles encontrarão alguma maneira de formular a exigência com a qual todos podem concordar, sem realmente resolver a disputa. Certa vez ouvi Tom DeMarco dizer: “Uma ambigüidade em um documento de requisitos representa um argumento entre as partes interessadas.”<sup>1</sup>

É claro que não é preciso uma discussão ou desacordo para criar ambiguidade.

Às vezes, as partes interessadas simplesmente presumem que os seus leitores sabem o que querem dizer.

1. Imersão XP 3, maio de 2000. <http://c2.com/cgi/wiki?TomsTalkAtXpImmersionThree>

Pode ser perfeitamente claro para eles no seu contexto, mas significa algo completamente diferente para o programador que o lê. Esse tipo de contexto a ambiguidade também pode ocorrer quando clientes e programadores estão falando cara a cara. enfrentar.

Sam (parte interessada): "OK, agora é necessário fazer backup desses arquivos de log."

Paula: "OK, com que frequência?"

Sam: "Diariamente".

Paula: "Certo. E onde você quer que ele seja salvo?"

Sam: "O que você quer dizer?"

Paula: "Você quer que eu salve em um subdiretório específico?"

Sam: "Sim, isso seria bom."

Paula: "Como devemos chamá-lo?"

Sam: "Que tal 'backup'?"

Paula: "Claro, tudo bem. Então, vamos gravar o arquivo de log no backup diretório todos os dias. A que horas?"

Sam: "Todos os dias."

Paula: "Não, quero dizer, a que horas do dia você quer que seja escrito?"

Sam: "A qualquer hora."

Paula: "Meio-dia?"

Sam: "Não, não durante o horário de negociação. Meia-noite seria melhor."

Paula: "OK, meia-noite então."

Sam: "Ótimo, obrigado!"

Paula: "Sempre um prazer."

Mais tarde, Paula conta a seu companheiro Peter sobre a tarefa.

Paula: "OK, precisamos copiar o arquivo de log para um subdiretório chamado backup todas as noites à meia-noite."

Peter: "OK, que nome de arquivo devemos usar?"

Paula: "log.backup deveria servir."

Pedro: "Você acertou."

## CAPÍTULO 7 TESTE DE ACEITAÇÃO

100

Em um escritório diferente, Sam está ao telefone com seu cliente.

Sam: “Sim, sim, os arquivos de log serão salvos.”

Carl: “OK, é vital que nunca percamos nenhum registro. Precisamos voltar através de todos esses arquivos de log, mesmo meses ou anos depois, sempre que há uma interrupção, evento ou disputa.”

Sam: “Não se preocupe, acabei de falar com Paula. Ela salvará os registros em um diretório chamado backup todas as noites à meia-noite.”

Carl: “OK, isso parece bom.”

Presumo que você tenha detectado a ambigüidade. O cliente espera que todos os arquivos de log sejam salvos, e Paula simplesmente pensou que queriam salvar o arquivo de registro da noite anterior. Quando o cliente procura backups de arquivos de log para meses, eles simplesmente encontre o de ontem à noite.

Neste caso, Paula e Sam deixaram cair a bola. É responsabilidade de desenvolvedores profissionais (e partes interessadas) para garantir que toda ambigüidade seja removido dos requisitos.

Isso é difícil e só há uma maneira que conheço de fazer isso.

### TESTES DE AC E PTAN C E

O termo teste de aceitação está sobrecarregado e usado em demasia. Algumas pessoas assumem que esses são os testes que os usuários executam antes de aceitarem uma versão. Outras pessoas acho que estes são testes de controle de qualidade. Neste capítulo definiremos testes de aceitação como testes

escrito por uma colaboração das partes interessadas e dos programadores, a fim de definir quando um requisito é concluído.

### A D E F I N I T I O N D E “D O N E”

Uma das ambigüidades mais comuns que enfrentamos como profissionais de software é a ambigüidade de “pronto”. Quando um desenvolvedor diz que concluiu uma tarefa, o que isso significa? quer dizer? O desenvolvedor está pronto no sentido de que está pronto para implantar o recurso com total confiança? Ou ele quer dizer que está pronto para o controle de qualidade? Ou talvez ele esteja terminei de escrevê-lo e consegui executá-lo uma vez, mas ainda não o testei.



## TESTES DE ACEITAÇÃO

101

Trabalhei com equipes que tinham uma definição diferente para as palavras “pronto” e “completo”. Uma equipe específica usou os termos “pronto” e “pronto”.

Os desenvolvedores profissionais têm uma única definição de pronto: Feito significa feito.

Concluído significa que todo o código foi escrito, todos os testes foram aprovados, o controle de qualidade e as partes interessadas

aceito. Feito.

Mas como você pode atingir esse nível de conclusão e ainda progredir rapidamente iteração em iteração? Você cria um conjunto de testes automatizados que, quando aprovados, atenda a todos os critérios acima! Quando os testes de aceitação do seu recurso forem aprovados, você terminou.

Os desenvolvedores profissionais conduzem a definição de seus requisitos até testes de aceitação automatizados. Eles trabalham com as partes interessadas e com o controle de qualidade para garantir

que esses testes automatizados são uma especificação completa do que foi feito.

Sam: “OK, agora é necessário fazer backup desses arquivos de log.”

Paula: “OK, com que frequência?”

Sam: “Diariamente”.

Paula: “Certo. E onde você quer que ele seja salvo?”

Sam: “O que você quer dizer?”

Paula: “Você quer que eu salve em um subdiretório específico?”

Sam: “Sim, isso seria bom.”

Paula: “Como devemos chamá-lo?”

Sam: “Que tal ‘backup’?”

Tom (testador): “Espere, backup é um nome muito comum. O que você realmente quer?”  
armazenando neste diretório?”

Sam: “Os backups.”

Tom: “Backups de quê?”

Sam: “Os arquivos de log.”

Paula: “Mas só há um arquivo de log.”

Sam: “Não, são muitos. Um para cada dia.”

Tom: “Você quer dizer que há um arquivo de log ativo e muitos arquivos de log  
cópias de segurança?”

## CAPÍTULO 7 TESTE DE ACEITAÇÃO

102

Sam: "Claro."

Paula: "Ah! Achei que você só queria um backup temporário."

Sam: "Não, o cliente quer ficar com todos eles para sempre."

Paula: "Essa é uma novidade para mim. OK, que bom que esclarecemos isso."

Tom: "Portanto, o nome do subdiretório deve nos dizer exatamente o que há nele."

Sam: "Tem todos os registros inativos antigos."

Tom: "Então vamos chamá-lo de `old_inactive_logs`."

Sam: "Ótimo."

Tom: "Então, quando esse diretório é criado?"

Sam: "Hein?"

Paula: "Devemos criar o diretório quando o sistema iniciar, mas somente se o diretório ainda não existe."

Tom: "OK, aí está o nosso primeiro teste. Vou precisar iniciar o sistema e ver se o diretório `old_inactive_logs` é criado. Então vou adicionar um arquivo a isso diretório. Então vou desligar e começar de novo, e ter certeza de que ambos o diretório e o arquivo ainda estão lá."

Paula: "Esse teste vai demorar muito para ser executado. A inicialização do sistema é já 20 segundos, e crescendo. Além disso, eu realmente não quero ter construir todo o sistema sempre que executo os testes de aceitação."

Tom: "O que você sugere?"

Paula: "Vamos criar uma classe `SystemStarter`. O programa principal irá carregar isso starter com um grupo de objetos `StartupCommand`, que seguirão o Padrão de comando. Então, durante a inicialização do sistema, o `SystemStarter` simplesmente instruirá todos os objetos `StartupCommand` a serem executados. Um daqueles Os derivados do `StartupCommand` criarão os `old_inactive_logs` diretório, mas apenas se ele ainda não existir."

Tom: "Oh, OK, então tudo que preciso testar é aquele derivado do `StartupCommand`. Posso escrever um teste `FitNesse` simples para isso."

Tom vai para o quadro.

"A primeira parte será mais ou menos assim":

dado o comando `LogFileDirectoryStartupCommand`

dado que o diretório `old_inactive_logs` não existe

## TESTES DE ACEITAÇÃO

103

quando o comando é executado

então o diretório old\_inactive\_logs deve existir

e deve estar vazio

“A segunda parte ficará assim”:

dado o comando LogFileDirectoryStartupCommand

dado que o diretório old\_inactive\_logs existe

e que contém um arquivo chamado x

Quando o comando é executado

Então o diretório old\_inactive\_logs ainda deve existir

e ainda deve conter um arquivo chamado x

Paula: “Sim, isso deve bastar.”

Sam: “Uau, tudo isso é realmente necessário?”

Paula: “Sam, qual dessas duas afirmações não é importante o suficiente para especificar?”

Sam: “Só quero dizer que parece muito trabalhoso pensar e escrever tudo esses testes.”

Tom: “É, mas não dá mais trabalho do que escrever um plano de teste manual. E é muito mais trabalhoso executar repetidamente um teste manual.”

## COMUNICAÇÃO

O objetivo dos testes de aceitação é comunicação, clareza e precisão. Por concordando com eles, os desenvolvedores, partes interessadas e testadores entendem o que o plano para o comportamento do sistema é. Alcançar esse tipo de clareza é responsabilidade de todas as partes. Os desenvolvedores profissionais assumem a responsabilidade de trabalhar com partes interessadas e testadores para garantir que todas as partes saibam o que está prestes a ser construído.

## AUTO M AÇÃO

Os testes de aceitação devem sempre ser automatizados. Há um lugar para manual testes em outras partes do ciclo de vida do software, mas esses tipos de testes nunca devem ser manual. A razão é simples: custo.

Considere a imagem na Figura 7-1. As mãos que você vê ali pertencem ao QA gerente de uma grande empresa de Internet. O documento que ele está segurando é a tabela de

conteúdo para seu plano de teste manual. Ele tem um exército de testadores manuais em off-shore locais que executam este plano uma vez a cada seis semanas. Custa-lhe mais de um milhão dólares todas as vezes. Ele está me segurando porque acabou de voltar de uma reunião em que seu gerente lhe disse que precisava cortar seu orçamento em 50%. Sua pergunta para mim é: “Qual metade desses testes não devo realizar?”

Figura 7-1 Plano de teste manual

Chamar isso de desastre seria um eufemismo grosseiro. O custo de funcionamento do plano de teste manual é tão grande que eles decidiram sacrificá-lo e simplesmente conviva com o fato de que eles não saberão se metade de seu produto funciona! Os desenvolvedores profissionais não permitem que esse tipo de situação aconteça. O custo de automatizar testes de aceitação é tão pequeno em comparação com o custo de execução planos de teste manuais que não faz sentido econômico escrever scripts para humanos para executar. Os desenvolvedores profissionais assumem a responsabilidade por sua parte em garantir que os testes de aceitação sejam automatizados.

## TESTES DE ACEITAÇÃO

105

Existem muitas ferramentas comerciais e de código aberto que facilitam a automação de testes de aceitação. FitNesse, Cucumber, cuke4duke, estrutura de robô e Selênio, só para citar alguns. Todas essas ferramentas permitem especificar testes em um formato que não-programadores possam ler, entender e até mesmo criar.

### EXTRA TRABALHO

O argumento de Sam sobre o trabalho é compreensível. Parece muito trabalho extra para escrever testes de aceitação como este. Mas dada a Figura 7-1 podemos ver que não é realmente trabalho extra. Escrever esses testes é simplesmente o trabalho de especificar o sistema. Especificar neste nível de detalhe é a única maneira de nós, como programadores, podermos saber o que significa “pronto”. Especificar neste nível de detalhe é a única maneira de as partes interessadas podem garantir que o sistema pelo qual estão pagando realmente faz o que eles precisam. E especificar neste nível de detalhe é a única maneira de obter sucesso automatizar os testes. Portanto, não encare esses testes como um trabalho extra. Olhe para eles como enorme economia de tempo e dinheiro. Esses testes impedirão que você implemente o sistema errado e permitirá que você saiba quando terminar.

### QUEM ESCRIVE TESTES DE ACEITAÇÃO E QUANDO?

Em um mundo ideal, as partes interessadas e o controle de qualidade colaborariam para redigir estes testes, e os desenvolvedores os revisariam quanto à consistência. No mundo real, as partes interessadas raramente têm tempo ou disposição para mergulhar no nível exigido de detalhes. Por isso, muitas vezes, eles delegam a responsabilidade a analistas de negócios, controle de qualidade ou até mesmo desenvolvedores. Se acontecer que os desenvolvedores devem escrever esses testes, então tome cuidado para que o desenvolvedor que escreve o teste não seja o mesmo que o desenvolvedor que implementa o recurso testado.

Normalmente, os analistas de negócios escrevem as versões do “caminho feliz” dos testes, porque esses testes descrevem os recursos que têm valor comercial. O controle de qualidade normalmente escreve os testes de “caminho infeliz”, as condições de limite, exceções e casos extremos. Isso ocorre porque o trabalho do QA é ajudar a pensar sobre o que pode dar errado. Seguindo o princípio da “precisão tardia”, os testes de aceitação devem ser escritos o mais tarde possível, normalmente alguns dias antes da implementação do recurso. Em Ágil projetos, os testes são escritos após os recursos terem sido selecionados para o próximo Iteração ou Sprint.

Os primeiros testes de aceitação devem estar prontos no primeiro dia da iteração.

Mais devem ser concluídos a cada dia até o ponto médio da iteração, quando todos deles devem estar prontos. Se todos os testes de aceitação não estiverem prontos na metade do a iteração, então alguns desenvolvedores terão que contribuir para finalizá-los. Se isso acontece com frequência, então mais BAs e/ou QAs devem ser adicionados à equipe.

### O PAPEL DO D E V E LOPE R

O trabalho de implementação de um recurso começa quando os testes de aceitação desse recurso recurso está pronto. Os desenvolvedores executam os testes de aceitação do novo recurso e veja como eles falham. Em seguida, eles trabalham para conectar o teste de aceitação ao o sistema e, em seguida, comece a fazer o teste passar implementando o desejado recurso.

Paula: "Peter, você poderia me ajudar com essa história?"

Pedro: "Claro, Paula, e aí?"

Paula: "Aqui está o teste de aceitação. Como vocês podem ver, está falhando."

dado o comando `LogFileDirectoryStartupCommand`

dado que o diretório `old_inactive_logs` não existe

quando o comando é executado

então o diretório `old_inactive_logs` deve existir

e deve estar vazio

Peter: "Sim, tudo vermelho. Nenhum dos cenários está escrito. Deixe-me escrever o primeiro."

|cenário|dado o comando `_|cmd|`

|criar comando|@cmd|

Paula: "Já temos uma operação `createCommand`?"

Peter: "Sim, está no `CommandUtilitiesFixture` que escrevi na semana passada."

Paula: "OK, então vamos fazer o teste agora."

Peter: (executa o teste). "Sim, a primeira linha é verde, vamos passar para a próxima."

Não se preocupe muito com cenários e luminárias. Esses são apenas alguns

o encanamento que você precisa escrever para conectar os testes ao sistema que está sendo testado.

Basta dizer que todas as ferramentas fornecem alguma maneira de usar a correspondência de padrões para

reconhecer e analisar as instruções do teste e, em seguida, chamar funções que alimentar os dados do teste no sistema que está sendo testado. A quantidade de esforço é pequeno, e os cenários e acessórios são reutilizáveis em muitos testes diferentes.

A questão de tudo isso é que é trabalho do desenvolvedor conectar a aceitação testes no sistema e, em seguida, fazer com que esses testes sejam aprovados.

### TESTE A NEGOCIAÇÃO E A AGRESSÃO PASSIVA

Os autores dos testes são humanos e cometem erros. Às vezes, os testes conforme escritos não fazem muito sentido quando você começa a implementá-los. Eles podem ser também complicado. Eles podem ser estranhos. Eles podem conter suposições tolas.

Ou eles podem simplesmente estar errados. Isso pode ser muito frustrante se você for o desenvolvedor que precisa fazer o teste passar.

Como desenvolvedor profissional, é sua função negociar com o autor do teste um melhor teste. O que você nunca deve fazer é escolher a opção passivo-agressiva e diga a si mesmo: "Bem, é isso que o teste diz, então é isso que vou fazer".

Lembre-se, como profissional, é seu trabalho ajudar sua equipe a criar o melhor software que puderem. Isso significa que todos precisam estar atentos aos erros e deslizos e trabalhar juntos para corrigi-los.

Paula: "Tom, este teste não está certo."

certifique-se de que a pós-operação termine em 2 segundos.

Tom: "Parece-me bom. Nosso requisito é que os usuários não tenham esperar mais de dois segundos. Qual é o problema?"

Paula: "O problema é que só podemos dar essa garantia numa base estatística sentido."

Tom: "Huh? Isso soa como palavrão. O requisito é dois segundos."

Paula: "Certo, e podemos conseguir isso 99,5% das vezes."

Tom: "Paula, esse não é o requisito."

Paula: "Mas é a realidade. Não tenho como fazer a garantia de outra forma."

## CAPÍTULO 7 TESTE DE ACEITAÇÃO

108

Tom: "Sam vai ter um ataque."

Paula: "Não, na verdade, já falei com ele sobre isso. Ele está bem, desde que já que a experiência normal do usuário dura dois segundos ou menos."

Tom: "OK, então como faço para escrever este teste? Não posso simplesmente dizer que o post a operação geralmente termina em dois segundos."

Paula: "Você diz isso estatisticamente."

Tom: "Você quer dizer que quer que eu execute mil pós-operações e faça certeza de que não mais que cinco são mais que dois segundos? Isso é um absurdo."

Paula: "Não, isso levaria quase uma hora para ser executado. Que tal isso?"

execute 15 transações posteriores e acumule tempos.

garantir que a pontuação Z por 2 segundos seja de pelo menos 2,57

Tom: "Uau, o que é pontuação Z?"

Paula: "Só algumas estatísticas. Aqui, que tal isso?"

execute 15 transações posteriores e acumule tempos.

garanta que as chances sejam de 99,5% de que o tempo será inferior a 2 segundos.

Tom: "Sim, isso é legível, mas posso confiar na matemática por trás do cenar?"

Paula: "Vou fazer questão de mostrar todos os cálculos intermediários na prova relatório para que você possa verificar a matemática se tiver alguma dúvida."

Tom: "OK, isso funciona para mim."

### TESTE DE ACEITAÇÃO E TESTE DE U N I T

Os testes de aceitação não são testes unitários. Os testes unitários são escritos por programadores para programadores. São documentos formais de projeto que descrevem o nível mais baixo estrutura e comportamento do código. O público são programadores, não empresas.

Os testes de aceitação são escritos pela empresa para a empresa (mesmo quando você, o desenvolvedor, acabe escrevendo-os). São documentos de requisitos formais que especificar como o sistema deve se comportar do ponto de vista do negócio. O público é a empresa e os programadores.



## TESTES DE ACEITAÇÃO

109

Pode ser tentador tentar eliminar o “trabalho extra” assumindo que os dois tipos de testes são redundantes. Embora seja verdade que os testes unitários e de aceitação muitas vezes testam as mesmas coisas, elas não são redundantes.

Primeiro, embora possam testar as mesmas coisas, fazem-no através de diferentes mecanismos e caminhos. Os testes unitários investigam as entranhas do sistema, criando chamadas para métodos em classes específicas. Os testes de aceitação invocam muito o sistema mais longe, na API ou às vezes até no nível da interface do usuário. Portanto, os caminhos de execução que esses testes realizam são muito diferentes.

Mas a verdadeira razão pela qual esses testes não são redundantes é que sua função principal é não testando. O fato de serem testes é incidental. Testes unitários e aceitação os testes são documentos primeiro e depois os testes. Seu principal objetivo é formalizar documentar o design, a estrutura e o comportamento do sistema. O fato de eles verificar automaticamente se o design, a estrutura e o comportamento que eles especificam são extremamente útil, mas a especificação é seu verdadeiro propósito.

### GUI S E OUTRAS CO M PLIC AÇÕES

É difícil especificar GUIs antecipadamente. Isso pode ser feito, mas raramente é bem feito. O A razão é que a estética é subjetiva e, portanto, volátil. As pessoas querem mexer com GUIs. Eles querem massageá-los e manipulá-los. Eles querem tentar diferentes fontes, cores, layouts de página e fluxos de trabalho. As GUIs estão em constante mudança. Isso torna um desafio escrever testes de aceitação para GUIs. O truque é projete o sistema para que você possa tratar a GUI como se fosse uma API, em vez do que um conjunto de botões, controles deslizantes, grades e menus. Isto pode parecer estranho, mas é realmente apenas um bom design.

Existe um princípio de design chamado Princípio de Responsabilidade Única (SRP). Isto princípio afirma que você deve separar as coisas que mudam para diferentes razões e agrupar aquelas coisas que mudam pelas mesmas razões.

GUIs não são exceção.

O layout, formato e fluxo de trabalho da GUI mudarão por questões estéticas e razões de eficiência, mas a capacidade subjacente da GUI permanecerá a mesma

apesar dessas mudanças. Portanto, ao escrever testes de aceitação para uma GUI você aproveite as abstrações subjacentes que não mudam com muita frequência.

Por exemplo, pode haver vários botões em uma página. Em vez de criar testes que clicam nesses botões com base em suas posições na página, você pode estar capaz de clicar neles com base em seus nomes. Melhor ainda, talvez cada um deles tenha um ID exclusivo que você pode usar. É muito melhor escrever um teste que selecione o botão cujo ID é `ok_button` do que selecionar o botão na coluna 3 da linha 4 da grade de controle.

Testando através da interface certa

Melhor ainda é escrever testes que invoquem os recursos do sistema subjacente através de uma API real em vez de através da GUI. Esta API deve ser a mesma API usada pela GUI. Isso não é novidade. Especialistas em design têm nos dito durante décadas para separar nossas GUIs de nossas regras de negócios.

Testar através da GUI é sempre problemático, a menos que você esteja testando apenas o GUI. A razão é que a GUI provavelmente mudará, tornando os testes muito frágeis.

Quando cada alteração na GUI interromper mil testes, você iniciará jogando fora os testes ou você vai parar de mudar a GUI. Nenhum dos dois essas são boas opções. Então escreva seus testes de regras de negócios para passar por uma API logo abaixo da GUI.

Alguns testes de aceitação especificam o comportamento da própria GUI. Esses testes devem ir através da GUI. Entretanto, esses testes não testam regras de negócios e, portanto, não exigem que as regras de negócios estejam conectadas à GUI. Portanto, é uma boa ideia dissociar a GUI e as regras de negócios e substituir o negócio regras com stubs enquanto testa a própria GUI.

Mantenha os testes da GUI no mínimo. Eles são frágeis porque a GUI é volátil. Quanto mais testes de GUI você tiver, menor será a probabilidade de mantê-los.

### C N T I N U O U S I N T E G R A Ç Ã O

Certifique-se de que todos os seus testes de unidade e testes de aceitação sejam executados várias vezes por dia em um sistema de integração contínua. Este sistema deve ser acionado pelo seu

## CONCLUSÃO

111

sistema de controle de código-fonte. Cada vez que alguém envia um módulo, o CI o sistema deve iniciar uma compilação e, em seguida, executar todos os testes no sistema. O os resultados dessa execução devem ser enviados por e-mail para todos da equipe.

Pare as prensas

É muito importante manter os testes de CI em execução o tempo todo. Eles nunca deveriam falhar. Se falharem, toda a equipe deve parar o que está fazendo e se concentrar em fazer com que os testes quebrados passassem novamente. Uma compilação quebrada no sistema CI deve ser visto como uma emergência, um evento de “parar as impressoras”.

Prestei consultoria para equipes que não levaram a sério os testes quebrados. Eles eram “muito ocupados” para consertar os testes quebrados, então eles os deixaram de lado, prometendo consertá-los

mais tarde. Em um caso, a equipe realmente retirou os testes quebrados da compilação porque foi tão inconveniente vê-los falhar. Mais tarde, após liberar para o cliente, eles perceberam que haviam esquecido de colocar esses testes de volta na construção. Eles aprendi isso porque um cliente irritado estava ligando para eles relatando erros.

## CONCLUSÃO

A comunicação sobre detalhes é difícil. Isto é especialmente verdadeiro para programadores e partes interessadas comunicando-se sobre os detalhes de uma aplicação. É também fácil para cada parte acenar com a mão e assumir que a outra parte entende. Muitas vezes ambas as partes concordam que entendem e deixam com ideias completamente diferentes.

A única maneira que conheço de eliminar efetivamente erros de comunicação entre programadores e partes interessadas é escrever testes de aceitação automatizados. Estes os testes são tão formais que são executados. Eles são completamente inequívocos e eles não podem ficar fora de sincronia com o aplicativo. Eles são os perfeitos documento de requisitos.

Esta página foi intencionalmente deixada em branco

## ESTRATÉGIAS DE TESTE

Desenvolvedores profissionais testam seu código. Mas testar não é simplesmente uma questão de escrever alguns testes unitários ou alguns testes de aceitação. Escrever esses testes é uma boa coisa, mas está longe de ser suficiente. O que toda equipe de desenvolvimento profissional necessita é uma boa estratégia de teste.

Em 1989, eu estava trabalhando na Rational no primeiro lançamento do Rose. Todo mês ou então nosso gerente de controle de qualidade convocaria um dia de “caça aos bugs”. Todos da equipe, desde programadores, gerentes, secretárias, administradores de banco de dados, sentariam-se desanimados com Rose e tentavam fazê-lo falhar. Foram atribuídos prêmios a vários tipos de

insetos. A pessoa que encontrasse um bug poderia ganhar um jantar para dois. O quem encontrar mais bugs pode ganhar um fim de semana em Monterrey.

### Q A DEVE ENCONTRAR NADA

Eu já disse isso antes e direi novamente. Apesar de sua empresa pode ter um grupo de controle de qualidade separado para testar o software, esse deve ser o objetivo do grupo de desenvolvimento que o controle de qualidade não encontrou nada de errado. É claro que não é provável que este objetivo seja constantemente alcançado. Afinal, quando você tem um grupo de pessoas inteligentes unidas e determinadas a encontrar todas as rugas e deficiências em um produto, provavelmente encontrarão alguns. Ainda assim, cada vez que o controle de qualidade encontrar algo, a equipe de desenvolvimento deverá reagir com horror. Eles

deveriam perguntar-se como isso aconteceu e tomar medidas para evitá-lo no futuro.

O controle de qualidade faz parte da equipe

A seção anterior pode ter feito parecer que o controle de qualidade e o desenvolvimento estão em discordam um do outro, que seu relacionamento é contraditório. Esta não é a intenção.

Em vez disso, o controle de qualidade e o desenvolvimento deveriam trabalhar juntos para garantir a qualidade

do sistema. A melhor função para a parte de controle de qualidade da equipe é atuar como especificadores e caracterizadores.

Controle de qualidade como especificadores

Deveria ser função do controle de qualidade trabalhar com as empresas para criar a aceitação automatizada

testes que se tornam o verdadeiro documento de especificações e requisitos para o sistema. Iteração por iteração, eles reúnem os requisitos do negócio e traduzi-los em testes que descrevem aos desenvolvedores como o sistema deve comportar-se (Veja Capítulo 7, “Teste de Aceitação”). Em geral, a empresa escreve o testes de caminho feliz, enquanto o controle de qualidade escreve os testes de canto, limite e caminho infeliz.

Controle de qualidade como caracterizadores

A outra função do QA é usar a disciplina de testes exploratórios<sup>1</sup> para caracterizar o verdadeiro comportamento do sistema em execução e relatar esse comportamento

1. [http://www.satisfice.com/articles/what\\_is\\_et.shtml](http://www.satisfice.com/articles/what_is_et.shtml)

de volta ao desenvolvimento e aos negócios. Nesta função, o controle de qualidade não está interpretando o

requisitos. Em vez disso, eles estão identificando os comportamentos reais do sistema.

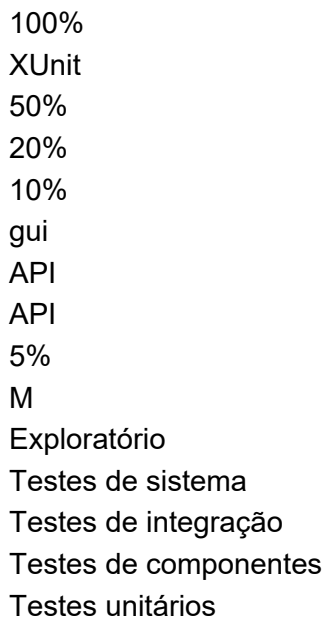
### O TESTE AUTOMATIZADO

Desenvolvedores profissionais empregam a disciplina de Desenvolvimento Orientado a Testes para criar testes de unidade. As equipes de desenvolvimento profissional usam testes de aceitação para especificar seu sistema e integração contínua (Capítulo 7, página 110) para evitar a regressão. Mas estes testes são apenas parte da história. Por melhor que seja temos um conjunto de testes unitários e de aceitação, também precisamos de testes de nível superior para certifique-se de que o controle de qualidade não encontre nada. A Figura 8-1 mostra a Pirâmide de Automação de Teste,<sup>2</sup>

uma representação gráfica dos tipos de testes que um desenvolvimento profissional necessidades da organização.

2. [COHN09] pp.

Figura 8-1 A pirâmide de automação de testes



### U N I T TESTES

Na base da pirâmide estão os testes unitários. Esses testes são escritos por programadores, para programadores, na linguagem de programação do sistema.

A intenção desses testes é especificar o sistema no nível mais baixo. Desenvolvedores escrever esses testes antes de escrever o código de produção como uma forma de especificar o que eles estão prestes a escrever. Eles são executados como parte da Integração Contínua para garantir que a intenção dos programadores seja mantida.

Os testes de unidade fornecem uma cobertura tão próxima de 100% quanto possível. Geralmente isso o número deve estar em algum lugar na década de 90. E deve ser uma cobertura verdadeira como oposto a testes falsos que executam código sem afirmar seu comportamento.

### TESTES DE COMPONENTES

Estes são alguns dos testes de aceitação mencionados no capítulo anterior.

Geralmente eles são escritos em componentes individuais do sistema. O componentes do sistema encapsulam as regras de negócios, então os testes para esses componentes são os testes de aceitação para essas regras de negócios

Conforme ilustrado na Figura 8-2, um teste de componente envolve um componente. Ele passa entrada dados no componente e coleta dados de saída dele. Ele testa que o a saída corresponde à entrada. Quaisquer outros componentes do sistema são desacoplados o teste usando técnicas apropriadas de zombaria e duplicação de teste.

Figura 8-2 Teste de aceitação de componentes

Componente

Teste de aceitação



Os testes de componentes são escritos por QA e Business com assistência do desenvolvimento. Eles são compostos em um ambiente de teste de componentes como FitNesse, JBehave ou Pepino. (Os componentes da GUI são testados com ambientes de teste da GUI como Selenium ou Watir.) A intenção é que a empresa seja capaz de ler e interpretar esses testes, se não for autor deles.

Os testes de componentes cobrem aproximadamente metade do sistema. Eles são direcionados mais para situações de caminho feliz e cantos, limites e caminhos alternativos muito óbvios. A grande maioria dos casos de caminhos infelizes são cobertos por testes unitários e não fazem sentido no nível dos testes de componentes.

### TESTES DE INTEGRAÇÃO

Esses testes só têm significado para sistemas maiores que possuem muitos componentes.

Conforme mostrado na Figura 8-3, esses testes reúnem grupos de componentes e testam quanto bem eles se comunicam entre si. Os outros componentes do sistema são desacoplados como de costume com simulações e testes duplos apropriados.

Os testes de integração são testes de coreografia. Eles não testam regras de negócios. Em vez disso, eles testam quanto bem a montagem dos componentes dança juntos. Eles são testes de encaixe que garantem que os componentes estão conectados corretamente e podem se comunicar claramente uns com os outros.

Figura 8-3 Teste de integração

Componente

Componente

Teste de integração

Componente

Componente

Os testes de integração são normalmente escritos pelos arquitetos do sistema, ou designers líderes, do sistema. Os testes garantem que a estrutura arquitetônica do sistema seja som. É neste nível que poderemos ver testes de desempenho e rendimento.

Os testes de integração normalmente são escritos na mesma linguagem e ambiente como testes de componentes. Eles normalmente não são executados como parte do Contínuo Conjunto de integração, porque geralmente têm tempos de execução mais longos. Em vez disso, esses testes

são executados periodicamente (noturno, semanalmente, etc.) conforme considerado necessário por seus autores.

### TESTE DO SISTEMA

São testes automatizados executados em todo o sistema integrado.

Eles são os testes de integração definitivos. Eles não testam regras de negócios diretamente.

Em vez disso, eles testam se o sistema foi conectado corretamente e se suas peças interoperar de acordo com o plano. Esperamos ver o rendimento e testes de desempenho neste conjunto.

Esses testes são escritos pelos arquitetos do sistema e líderes técnicos. Normalmente eles são escritos na mesma linguagem e ambiente que os testes de integração para a interface do usuário. Eles são executados com relativa pouca frequência, dependendo de sua duração, mas quanto mais frequentemente, melhor.

Os testes do sistema cobrem talvez 10% do sistema. Isso ocorre porque sua intenção não é para garantir o comportamento correto do sistema, mas a construção correta do sistema. O correto o comportamento do código e dos componentes subjacentes já foi verificado pelas camadas inferiores da pirâmide.

### MANUAL E XPLO R A T O R Y T E S S

É aqui que os humanos colocam as mãos nos teclados e os olhos no

telas. Esses testes não são automatizados nem programados. A intenção destes

testes é explorar o sistema em busca de comportamentos inesperados enquanto confirma o esperado comportamentos. Para esse fim, precisamos de cérebros humanos, com criatividade humana, trabalhando para investigar e explorar o sistema. Criação de um plano de teste escrito para esse tipo de teste anula o propósito.

## BIBLIOGRAFIA

Algumas equipes terão especialistas para fazer esse trabalho. Outras equipes simplesmente declararão um ou dois dias de “caça aos insetos” em que o maior número de pessoas possível, incluindo gerentes, secretárias, programadores, testadores e redatores de tecnologia, “batem” no sistema para ver se eles conseguem quebrá-lo.

O objetivo não é a cobertura. Não vamos provar todas as regras de negócios e cada caminho de execução com esses testes. Pelo contrário, o objectivo é garantir que o sistema se comporta bem sob operação humana e encontrar criativamente o máximo “peculiaridades” possíveis.

## CONCLUSÃO

TDD é uma disciplina poderosa e os testes de aceitação são formas valiosas de expressar e impor requisitos. Mas eles são apenas parte de uma estratégia total de testes. Para cumprir a meta de que “o controle de qualidade não deve encontrar nada”, as equipes de desenvolvimento precisam

trabalhar lado a lado com o controle de qualidade para criar uma hierarquia de unidade, componente, testes de integração, sistema e exploratórios. Esses testes devem ser executados tão freqüentemente quanto

possível fornecer feedback máximo e garantir que o sistema permaneça continuamente limpo.

## BIBLIOGRAPHY

[COHN09]: Mike Cohn, Sucesso com Agile, Boston, MA: Addison-Wesley, 2009.

Esta página foi intencionalmente deixada em branco

## GESTÃO DO TEMPO

Oito horas é um período de tempo notavelmente curto. São apenas 480 minutos ou 28.800 segundos. Como profissional, você espera usar esses poucos preciosos segundos da forma mais eficiente e eficaz possível. Que estratégia você pode usar para garantir que você não perca o pouco tempo que tem? Como você pode efetivamente administrar seu tempo?

Em 1986 eu morava em Little Sandhurst, Surrey, Inglaterra. eu estava gerenciando um departamento de desenvolvimento de software com 15 pessoas para a Teradyne em Bracknell. Meu

## CAPÍTULO 9 GESTÃO DO TEMPO

122

os dias foram agitados com telefonemas, reuniões improvisadas, questões de serviço de campo, e interrupções. Então, para poder realizar qualquer trabalho, tive que adotar algumas lindas disciplinas drásticas de gerenciamento de tempo.

- Eu acordava às 5 horas da manhã e ia de bicicleta até o escritório em Bracknell

6 da manhã. Isso me deu 2- 1 \_

2 horas de silêncio antes do caos do dia começou.

- Ao chegar eu escrevia um cronograma no meu quadro. Eu dividi o tempo em incrementos de 15 minutos e preenchi a atividade que eu trabalharia durante esse bloco de tempo.

- Preenchi completamente as primeiras 3 horas desse horário. A partir das 9h comecei deixando um intervalo de 15 minutos por hora; dessa forma eu poderia empurrar mais rapidamente interrupções em um desses slots abertos e continue trabalhando.

- Deixei o horário depois do almoço sem agendamento porque sabia que a essa altura seria um inferno teria me soltado e eu teria que ficar em modo reativo pelo resto do dia. Durante aqueles raros períodos da tarde em que o caos não se intrometia, Eu simplesmente trabalhei na coisa mais importante até que isso acontecesse. Este esquema nem sempre deu certo. Acordar às 5 da manhã nem sempre era viável, e às vezes o caos rompeu todas as minhas estratégias cuidadosas e consumiu meu dia. Mas na maior parte do tempo consegui manter minha cabeça acima da água.

### ENCONTROS

As reuniões custam cerca de US\$ 200 por hora por participante. Isto leva em conta salários, benefícios, custos de instalações e assim por diante. Na próxima vez que você estiver em um reunião, calcule o custo. Você pode se surpreender.

Existem duas verdades sobre o encontro.

1. As reuniões são necessárias.

2. As reuniões são uma grande perda de tempo.

Muitas vezes estas duas verdades descrevem igualmente o mesmo encontro. Alguns presentes pode considerá-los inestimáveis; outros podem considerá-los redundantes ou inúteis.

## REUNIÕES

123

Os profissionais estão cientes do alto custo das reuniões. Eles também estão cientes de que seu próprio tempo é precioso; eles têm código para escrever e cronogramas para cumprir. Portanto, eles resistem ativamente a participar de reuniões que não tenham um impacto imediato e benefício significativo.

### DECLINING

Você não precisa comparecer a todas as reuniões para as quais foi convidado. Na verdade, é pouco profissional ir a muitas reuniões. Você precisa usar seu tempo com sabedoria. Então tenha muito cuidado com quais reuniões você participa e quais você recusa educadamente. A pessoa que o convida para uma reunião não é responsável pela gestão da sua tempo. Só você pode fazer isso. Então, quando você receber um convite para uma reunião, não aceitar, a menos que seja uma reunião para a qual sua participação seja imediata e significativamente necessário para o trabalho que você está fazendo agora.

Às vezes a reunião será sobre algo que lhe interessa, mas não é imediatamente necessário. Você terá que escolher se pode pagar o tempo. Seja cuidado – pode haver reuniões mais do que suficientes para consumir seus dias.

Às vezes, a reunião será sobre algo em que você pode contribuir, mas não é imediatamente significativo para o que você está fazendo atualmente. Você terá que escolha se a perda para o seu projeto vale o benefício para o deles. Isso pode parece cínico, mas sua responsabilidade é primeiro com seus projetos. Ainda assim, é frequente bom para uma equipe ajudar outra, então você pode querer discutir sua participação com sua equipe e gerente.

Às vezes, sua presença na reunião será solicitada por alguém em autoridade, como um engenheiro sênior em outro projeto ou o gerente de um projeto diferente. Você terá que escolher se essa autoridade supera a sua horário de trabalho. Novamente, sua equipe e seu supervisor podem ajudar a tomar essa decisão.

Uma das funções mais importantes do seu gerente é mantê-lo fora das reuniões. Um bom gerente estará mais do que disposto a defender sua decisão de recusar presença porque esse gerente está tão preocupado com o seu tempo quanto você.

### L E AVI N G

As reuniões nem sempre acontecem como planejado. Às vezes você se encontra sentado em um reunião que você teria recusado se soubesse mais. Às vezes novos tópicos são adicionados ou a implicância de alguém domina a discussão. Acima dos anos desenvolvi uma regra simples: quando a reunião ficar chata, vá embora. Novamente, você tem a obrigação de administrar bem o seu tempo. Se você se encontrar preso em uma reunião que não é um bom uso do seu tempo, você precisa encontrar uma maneira de sair educadamente dessa reunião.

É claro que você não deve sair furioso de uma reunião exclamando “Isso é chato!”

Não há necessidade de ser rude. Você pode simplesmente perguntar, no momento oportuno, se a sua presença ainda é necessária. Você pode explicar que não pode gastar muito mais tempo, e pergunte se existe uma maneira de agilizar a discussão ou embaralhar a agenda.

O importante a perceber é que permanecer numa reunião que se tornou uma perda de tempo para você e para a qual você não pode mais contribuir significativamente, não é profissional. Você tem a obrigação de usar sabiamente o tempo do seu empregador e dinheiro, por isso não é falta de profissionalismo escolher um momento apropriado para negociar sua saída.

### TENHA UMA AGÊNCIA E UM OBJETIVO

A razão pela qual estamos dispostos a suportar o custo das reuniões é que às vezes precisa dos participantes juntos em uma sala para ajudar a alcançar um objetivo específico. Para usar o tempo dos participantes com sabedoria, a reunião deve ter uma agenda clara, com horários para cada tópico e uma meta declarada.

Se você for convidado a ir a uma reunião, certifique-se de saber quais discussões estão acontecendo nela, quanto tempo está reservado para elas e qual objetivo deve ser alcançado. Se você não consegue obter uma resposta clara sobre essas coisas, então recuse educadamente comparecer. Se você for a uma reunião e descobrir que a agenda foi manipulada ou abandonada, deverá solicitar que o novo tema seja apresentado e a ordem do dia seja seguido. Se isso não acontecer, você deve sair educadamente quando possível.



## REUNIÕES

125

### STAND-UP MEETINGS

Essas reuniões fazem parte do canhão Agile. Seu nome vem do fato que se espera que os participantes permaneçam de pé durante a reunião. Cada participante se reveza para responder três perguntas:

1. O que eu fiz ontem?
2. O que vou fazer hoje?
3. O que está no meu caminho?

Isso é tudo. Cada pergunta não deve levar mais de vinte segundos, então cada participante não deve demorar mais do que um minuto. Mesmo em grupo de dez pessoas, esta reunião deve terminar bem antes de dez minutos decorrido.

### REUNIÕES DE PLANEJAMENTO DE ITERAÇÃO

Essas são as reuniões mais difíceis de realizar bem no cânone Agile. Concluído mal, eles levam muito tempo. É preciso habilidade para fazer essas reuniões acontecerem bem, uma habilidade que vale a pena aprender.

As reuniões de planejamento de iteração têm como objetivo selecionar os itens do backlog que serão executado na próxima iteração. As estimativas já devem ser feitas para os candidatos itens de data. A avaliação do valor do negócio já deve ser feita. Em muito bom organizações, os testes de aceitação/componentes já estarão escritos, ou pelo menos esboçado.

A reunião deve prosseguir rapidamente com cada item da lista de pendências do candidato sendo brevemente discutido e então selecionado ou rejeitado. Não mais que cinco ou dez minutos devem ser gastos em qualquer item. Se for necessária uma discussão mais longa, deve ser agendado para outro horário com um subconjunto da equipe.

Minha regra é que a reunião não deve ocupar mais do que 5% do total tempo na iteração. Então, para uma iteração de uma semana (quarenta horas), a reunião deve terminar dentro de duas horas.

### ITERATIÃO RETROSPECTIVA E DEMO

Essas reuniões são realizadas ao final de cada iteração. Membros da equipe discutir o que deu certo e o que deu errado. As partes interessadas veem uma demonstração do recursos recém-funcionados. Estas reuniões podem ser alvo de abusos graves e podem absorver uma muito tempo, então agende-os 45 minutos antes do horário de encerramento no último dia de a iteração. Aloque no máximo 20 minutos para retrospectiva e 25 minutos para a demonstração. Lembre-se, faz apenas uma semana ou duas, então não deveria haver muito o que falar.

### ARGUMENTOS / DISAGREEMENTS

Kent Beck uma vez me disse algo profundo: “Qualquer argumento que não possa ser resolvido em cinco minutos não pode ser resolvido discutindo. A razão pela qual isso dura tanto tempo é que não há evidências claras que apoiem nenhum dos lados. O argumento é provavelmente religioso, em oposição a factual.

As divergências técnicas tendem a atingir a estratosfera. Cada festa tem tudo tipos de justificativas para sua posição, mas raramente quaisquer dados. Sem dados, qualquer argumento que não chega a um acordo em poucos minutos (em algum lugar entre cinco e trinta anos) simplesmente nunca chegarão a um acordo. A única coisa a fazer é ir buscar alguns dados.

Algumas pessoas tentarão vencer uma discussão pela força de caráter. Eles podem gritar, ou ir na sua cara, ou agir com condescendência. Não importa; força de vontade não resolver divergências por muito tempo. Os dados sim.

Algumas pessoas serão passivo-agressivas. Eles concordarão apenas para encerrar a discussão, e depois sabotar o resultado recusando-se a empenhar-se na solução. Eles dirão para eles mesmos: “É assim que eles queriam, e agora eles vão conseguir o que eles queriam.” Este é provavelmente o pior tipo de comportamento não profissional que existe é. Nunca, jamais faça isso. Se você concorda, então você deve se envolver.

Como você obtém os dados necessários para resolver um desacordo? Às vezes você pode execute experimentos ou faça alguma simulação ou modelagem. Mas às vezes o melhor alternativa é simplesmente jogar uma moeda para escolher um dos dois caminhos em questão.

Se as coisas derem certo, então esse caminho era viável. Se você tiver problemas, você pode voltar e desça pelo outro caminho. Seria sensato chegar a acordo sobre um momento tão bem como um conjunto de critérios para ajudar a determinar quando o caminho escolhido deve ser abandonado.

Cuidado com reuniões que são apenas um local para desabafar um desentendimento e para reunir apoio para um lado ou outro. E evite aqueles onde apenas um dos argumentadores estão apresentando.

Se um argumento deve realmente ser resolvido, então peça a cada um dos argumentadores que apresente seu caso para a equipe em cinco minutos ou menos. Depois faça a equipe votar. O toda a reunião levará menos de quinze minutos.

#### FOCO S-M AN NA

Perdoe-me se esta seção parece cheirar a metafísica da Nova Era, ou talvez a Masmorras e Dragões. Só que é assim que penso sobre esse assunto.

A programação é um exercício intelectual que requer longos períodos de concentração e foco. O foco é um recurso escasso, como o maná.<sup>1</sup> Depois você gastou seu maná de foco, você tem que recarregar fazendo coisas sem foco atividades por uma hora ou mais.

Não sei o que é esse maná-foco, mas tenho a sensação de que é um efeito físico. substância (ou possivelmente sua falta) que afeta a alteridade e a atenção. Seja lá o que for pode ser, você pode sentir quando ele está lá e pode sentir quando ele se foi.

Os desenvolvedores profissionais aprendem a administrar seu tempo para aproveitar suas vantagens. foco-maná. Escrevemos código quando nosso maná de foco é alto; e fazemos outros, coisas menos produtivas quando não é.

O Focus-maná também é um recurso decadente. Se você não usá-lo quando estiver lá, você provavelmente o perderão. Essa é uma das razões pelas quais as reuniões podem ser tão

1. Maná é uma mercadoria comum em jogos de fantasia e RPG como Dungeons & Dragons. Cada jogador tem uma certa quantidade de maná, que é uma substância mágica gasta sempre que um jogador lança um feitiço mágico

feitiço. Quanto mais potente o feitiço, mais maná daquele jogador é consumido. Manna recarrega lentamente,

diária fixa. Portanto, é fácil usar tudo isso em algumas sessões de lançamento de feitiços.

## CAPÍTULO 9 GESTÃO DO TEMPO

128

devastador. Se você gastar todo o seu maná de foco em uma reunião, você não terá qualquer sobra para codificação.

Preocupações e distrações também consomem foco-maná. A briga que você teve com seu cônjuge ontem à noite, o amassado que você fez no para-choque esta manhã ou a conta que você se esqueceu de pagar na semana passada, tudo isso sugará seu maná de foco rapidamente.

### S L E E P

Não consigo enfatizar isso com força suficiente. Eu tenho mais foco depois de uma boa noite de sono. Sete horas de sono muitas vezes me dão oito horas completas vale a pena focar-maná. Desenvolvedores profissionais gerenciam seu horário de sono para garantir que eles tenham completado seu maná de foco quando chegarem ao trabalho pela manhã.

### C A F F E I N E

Não há dúvida de que alguns de nós podemos fazer uso mais eficiente do nosso foco-maná consumindo quantidades moderadas de cafeína. Mas tome cuidado. Cafeína também coloca uma estranha “tremor” no seu foco. Muito disso pode desviar seu foco em direções muito estranhas. Um zumbido de cafeína muito forte pode fazer com que você desperdice um dia inteiro hiperfocado em todas as coisas erradas.

O uso e a tolerância à cafeína são uma coisa pessoal. Minha preferência pessoal é uma única xícara de café forte pela manhã e uma coca diet no almoço.

Às vezes dobro essa dose, mas raramente faço mais do que isso.

### R E C H A R G I N G

O Focus-maná pode ser parcialmente recarregado pela desfocagem. Uma boa e longa caminhada, uma conversa com amigos, um momento apenas olhando pela janela pode ajudar a bombeie o foco-maná de volta.

Algumas pessoas meditam. Outras pessoas tiram uma soneca revigorante. Outros ouvirão um podcast ou folhear uma revista.

Descobri que uma vez que o maná acaba, você não pode forçar o foco. Você pode ainda escrever código, mas é quase certo que você terá que reescrevê-lo no dia seguinte ou viver com uma massa apodrecida por semanas ou meses. Então é melhor levar trinta, ou até sessenta minutos para desfocar.

#### M U S C L E F O C U S

Há algo peculiar em praticar disciplinas físicas como artes marciais, artes, tai-chi ou ioga. Embora essas atividades exijam um foco significativo, é um tipo diferente de foco da codificação. Não é intelectual, é músculo. E de alguma forma, o foco muscular ajuda a recarregar o foco mental. É mais que um simples recarregar embora. Acho que um regime regular de foco muscular aumenta meu capacidade de foco mental.

Minha forma escolhida de foco físico é andar de bicicleta. Vou cavalgar por uma ou duas horas, às vezes cobrindo vinte ou trinta milhas. Eu ando em uma trilha paralela ao Des Rio Plaines, então não tenho que lidar com carros.

Enquanto ando, ouço podcasts sobre astronomia ou política. Às vezes eu apenas ouvir minha música favorita. E às vezes eu simplesmente desligo os fones de ouvido e ouço a natureza.

Algumas pessoas dedicam tempo para trabalhar com as mãos. Talvez eles gostem carpintaria, ou maquetes de construção, ou jardinagem. Seja qual for a atividade, há algo sobre atividades que se concentram nos músculos que aumenta a capacidade de trabalhe com sua mente.

#### I N P U T V E R S U S O U T P U T

Outra coisa que considero essencial para o foco é equilibrar minha produção com entrada apropriada. Escrever software é um exercício criativo. Acho que sou o mais criativo quando estou exposto à criatividade de outras pessoas. Então eu li muita ciência ficção. A criatividade desses autores estimula de alguma forma a minha própria criatividade sucos para software.

### TIME BOXING E TOMATES

Uma maneira muito eficaz que usei para gerenciar meu tempo e foco é usar o conhecida Técnica Pomodoro,<sup>2</sup> também conhecida como tomate. A ideia básica é muito simples. Você define um timer de cozinha padrão (tradicionalmente em forma de tomate) por 25 minutos. Enquanto o cronômetro estiver funcionando, você não deixa nada interferir com o que você está fazendo. Se o telefone tocar você atende e pergunta educadamente se você pode ligar de volta dentro de 25 minutos. Se alguém parar para fazer uma pergunta, você pergunte educadamente se você pode entrar em contato com eles em 25 minutos. Independentemente da interrupção, você simplesmente adia até o cronômetro tocar. Afinal, poucas interrupções são tão terrivelmente urgentes que não conseguem esperar 25 minutos!

Quando o cronômetro do tomate apita, você para o que está fazendo imediatamente. Você lidar com quaisquer interrupções que ocorreram durante o tomate. Então você pega um intervalo de cinco minutos ou mais. Então você ajusta o cronômetro para mais 25 minutos e comece o próximo tomate. A cada quarto tomate você faz uma pausa maior de 30 minutos ou mais.

Há bastante coisa escrita sobre essa técnica e recomendo que você a leia.

No entanto, a descrição acima deve fornecer a essência da técnica.

Usando esta técnica, seu tempo é dividido em tempo de tomate e tempo sem tomate.

A hora do tomate é produtiva. É dentro dos tomates que você realiza o verdadeiro trabalho.

O tempo fora do tomate pode ser distrações, reuniões, intervalos ou outro momento que não é gasto trabalhando em suas tarefas.

Quantos tomates você consegue fazer em um dia? Em um dia bom você pode conseguir 12 ou até 14 tomates prontos. Em um dia ruim, você pode fazer apenas duas ou três tarefas.

Se você contá-los e mapeá-los, terá uma ideia bem rápida de quanto

seu dia você gasta de forma produtiva e quanto você gasta lidando com “coisas”.

Algumas pessoas ficam tão confortáveis com a técnica que estimam suas tarefas

em tomates e depois medir a velocidade semanal do tomate. Mas isso é apenas glaciê

no bolo. O verdadeiro benefício da Técnica Pomodoro é que 25 minutos

janela de tempo produtivo que você defende agressivamente contra todas as interrupções.

2. <http://www.pomodortechnique.com/>

## Becos Cegos

### AVOIDANCE

Às vezes, seu coração simplesmente não está em seu trabalho. Pode ser que aquilo que precisa fazer é assustador, desconfortável ou chato. Talvez você pense que isso forçará você em um confronto ou levá-lo a um buraco de rato inevitável. Ou talvez você simplesmente não quero fazer isso.

### PRIORITY INVERSION

Seja qual for o motivo, você encontra maneiras de evitar fazer o trabalho real. Você convence você mesmo que algo mais é mais urgente, e você faz isso em vez disso. Isto é chamada inversão de prioridade. Você aumenta a prioridade de uma tarefa para poder adiar a tarefa que tem a verdadeira prioridade. As inversões de prioridades são uma mentira que contamos a nós mesmos.

Não conseguimos enfrentar o que precisa ser feito, então nos convencemos de que outra tarefa é mais importante. Sabemos que não é, mas mentimos para nós mesmos.

Na verdade, não estamos mentindo para nós mesmos. O que estamos realmente a fazer é preparar-nos para

a mentira que contaremos quando alguém nos perguntar o que estamos fazendo e por que estamos fazendo

isso. Estamos construindo uma defesa para nos proteger do julgamento dos outros.

Claramente este é um comportamento pouco profissional. Profissionais avaliam a prioridade de cada tarefa, desconsiderando seus medos e desejos pessoais, e executar essas tarefas em ordem de prioridade.

### BLIND ALLEYS

Becos sem saída são uma realidade para todos os artesãos de software. Às vezes você vai tome uma decisão e percorra um caminho técnico que não leva a lugar nenhum.

Quanto mais investido você estiver em sua decisão, mais tempo você vagará no deserto. Se você apostou em sua reputação profissional, você vagará para sempre.

A prudência e a experiência o ajudarão a evitar certos becos sem saída, mas você nunca evite todos eles. Portanto, a verdadeira habilidade que você precisa é perceber rapidamente quando você está

em um, e tenha a coragem de desistir. Isso às vezes é chamado de A Regra de

Buracos: Quando você estiver em um, pare de cavar.

Os profissionais evitam ficar tão envolvidos com uma ideia que não conseguem abandoná-la e vire-se. Eles mantêm a mente aberta sobre outras ideias, de modo que, quando atingem um beco sem saída eles ainda têm outras opções.

### MAR S H E S, BOG S, SWAM PS, E D O T E R M E S S E S

Pior que becos sem saída são bagunças. A bagunça te atrasa, mas não te impede.

A bagunça impede seu progresso, mas você ainda pode progredir por meio de pura força bruta. As bagunças são piores que os becos sem saída porque você sempre pode ver o caminho a seguir, e sempre parece mais curto que o caminho de volta (mas não é).

Já vi produtos arruinados e empresas destruídas por bagunças de software. eu tenho vi a produtividade das equipes diminuir de jitterbug para canto fúnebre em apenas alguns meses. Nada tem um efeito negativo mais profundo ou duradouro sobre o produtividade de uma equipe de software do que uma bagunça. Nada.

O problema é que começar uma confusão, como entrar num beco sem saída, é inevitável. A experiência e a prudência podem ajudá-lo a evitá-los, mas eventualmente você tomará uma decisão que o levará a uma bagunça.

A progressão de tal confusão é insidiosa. Você cria uma solução para um simples problema, tomando cuidado para manter o código simples e limpo. À medida que o problema cresce em escopo e complexidade, você estende essa base de código, mantendo-a tão limpa quanto você pode. Em algum momento você percebe que fez uma escolha errada de design quando iniciado e que seu código não se adapta bem na direção em que o os requisitos estão mudando.

Este é o ponto de inflexão! Você ainda pode voltar e consertar o design. Mas você pode também continue avançando. Voltar parece caro porque você terá que retrabalhar o código existente, mas voltar nunca será mais fácil do que é agora. Se você avança, você levará o sistema a um pântano do qual ele nunca poderá sair escapar.



## CONCLUSÃO

133

Os profissionais temem muito mais bagunça do que becos sem saída. Eles estão sempre à procura de bagunças que começam a crescer sem limites e vão gastar todo esforço necessário para escapar deles o mais cedo e o mais rápido possível.

Avançar em um pântano, quando você sabe que é um pântano, é o pior uma espécie de inversão de prioridades. Ao seguir em frente você está mentindo para si mesmo, mentindo para

sua equipe, mentindo para sua empresa e mentindo para seus clientes. Você está dizendo que tudo ficará bem, quando na verdade você está caminhando para uma desgraça compartilhada.

## CONCLUSÃO

Os profissionais de software são diligentes no gerenciamento de seu tempo e de seus foco. Eles compreendem as tentações da inversão de prioridades e combatem-na como uma questão de honra. Eles mantêm suas opções abertas, mantendo uma mente aberta sobre soluções alternativas. Eles nunca ficam tão envolvidos em uma solução que não possam abandone-o. E eles estão sempre à procura de confusão crescente, e eles limpe-os assim que forem reconhecidos. Não há visão mais triste do que uma equipe de desenvolvedores de software trabalhando inutilmente em um pântano cada vez mais profundo.

Esta página foi intencionalmente deixada em branco

## E ESTIMAÇÃO

A estimativa é uma das atividades mais simples, porém mais assustadoras que o software pode realizar. profissionais enfrentam. Muito valor comercial depende disso. Tanto do nosso reputações dependem disso. Grande parte da nossa angústia e fracasso são causados por isso. É a principal barreira que foi criada entre empresários e desenvolvedores. É a fonte de quase toda a desconfiança que rege essa relação.

## CAPÍTULO 10 ESTIMATIVA

Em 1978, fui o principal desenvolvedor de um programa Z-80 incorporado de 32K escrito em linguagem assembly. O programa foi gravado em 32 1K ´ 8 EEprom fichas. Esses 32 chips foram inseridos em três placas, cada uma contendo 12 fichas.

Tínhamos centenas de dispositivos em campo, instalados em centrais telefônicas em todos os Estados Unidos. Sempre que corrigimos um bug ou adicionamos um recurso, tenho que enviar técnicos de serviço de campo para cada uma dessas unidades e substituí-los todas as 32 fichas!

Isto foi um pesadelo. Os chips e as placas eram frágeis. Os pinos dos chips podem entortar e quebrar. A flexão constante das tábuas pode danificar juntas de solda. O risco de quebra e erro era enorme. O custo para a empresa era muito alto.

Meu chefe, Ken Finder, veio até mim e me pediu para consertar isso. O que ele queria era uma maneira de fazer uma alteração em um chip que não exigia que todos os outros chips fossem mudar. Se você leu meus livros ou ouviu minhas palestras, sabe que eu discuto muito sobre capacidade de implantação independente. Foi aqui que aprendi essa lição pela primeira vez. Nosso problema era que o software era um único executável vinculado. Se uma nova linha de código foi adicionado ao programa, todos os endereços das linhas seguintes do código alterado. Como cada chip continha apenas 1K do espaço de endereço, o conteúdo de praticamente todos os chips mudariam.

A solução foi bem simples. Cada chip teve que ser desacoplado de todos os outros. Cada um teve que ser transformado em uma unidade de compilação independente que pudesse ser queimado independentemente de todos os outros.

Então medi os tamanhos de todas as funções do aplicativo e escrevi um programa simples que os encaixa, como um quebra-cabeça, em cada uma das fichas, deixando cerca de 100 bytes de espaço para expansão. No início de cada ficha Coloquei uma tabela de ponteiros para todas as funções desse chip. Na inicialização, estes ponteiros foram movidos para a RAM. Todo o código do sistema foi alterado para que as funções foram chamadas apenas através desses vetores RAM e nunca diretamente.

## ESTIMATIVA

Sim, você entendeu. Os chips eram objetos, com vtables. Todas as funções eram polimorficamente implantado. E, sim, foi assim que aprendi alguns dos princípios da OOD, muito antes de eu saber o que era um objeto.

Os benefícios foram enormes. Não só poderíamos implantar chips individuais, como também também poderia fazer patches em campo movendo funções para RAM e redirecionando os vetores. Isso tornou a depuração de campo e o hot patching muito mais fáceis. Mas eu discordo. Quando Ken veio até mim e me pediu para resolver esse problema, ele sugeriu algo sobre ponteiros para funções. Passei um dia ou dois formalizando a ideia e depois apresentei-lhe um plano detalhado. Ele me perguntou quanto tempo levaria, e respondi que levaria cerca de um mês.

Demorou três meses.

Só fiquei bêbado duas vezes na vida e só fiquei realmente bêbado uma vez. Foi à festa de Natal da Teradyne em 1978. Eu tinha 26 anos.

A festa foi realizada no escritório da Teradyne, que era principalmente um laboratório aberto. Todos chegaram cedo e então houve uma forte nevasca que impediu o banda e o fornecedor cheguem lá. Felizmente havia muita bebida.

Não me lembro muito daquela noite. E o que me lembro, gostaria de não ter lembrado. Mas vou compartilhar um momento comovente com você.

Eu estava sentado de pernas cruzadas no chão com Ken (meu chefe, que tinha 29 anos anos na época e não bêbado) chorando sobre quanto tempo a vetorização o trabalho estava me levando. O álcool libertou meus medos e inseguranças reprimidos sobre minha estimativa. Não creio que minha cabeça estivesse no colo dele, mas minha memória apenas não é muito claro sobre esse tipo de detalhe.

Lembro-me de perguntar a ele se ele estava bravo comigo e se ele achava que estava demorando. eu por muito tempo. Embora a noite tenha sido um borrão, sua resposta permaneceu clara pelas décadas seguintes. Ele disse: “Sim, acho que você demorou muito, mas posso ver que você está trabalhando duro nisso e fazendo bons progressos. É algo que realmente precisamos. Então, não, não estou bravo.”

## CAPÍTULO 10 ESTIMATIVA

138

### O QUE É UMA ESTIMATIVA?

O problema é que vemos as estimativas de maneiras diferentes. As empresas gostam de ver estimativas como compromissos. Os desenvolvedores gostam de ver as estimativas como suposições. O a diferença é profunda.

### UM CO M M ITM E NT

Um compromisso é algo que você deve alcançar. Se você se comprometer a conseguir algo feito até uma determinada data, então você simplesmente precisa concluí-lo até essa data. Se isso significa que você tem que trabalhar 12 horas por dia, nos finais de semana, faltando à família férias, então que assim seja. Você assumiu o compromisso e precisa honrá-lo.

Os profissionais não assumem compromissos a menos que saibam que podem alcançá-los.

É tão simples assim. Se você for solicitado a se comprometer com algo que você não tem certeza do que pode fazer, então sua honra está fadada a recusar. Se você for questionado comprometer-se com uma data que você sabe que pode alcançar, mas que exigiria muito tempo horas, fins de semana e férias em família perdidas, a escolha é sua; mas é melhor você estar disposto a fazer o que for preciso.

Compromisso tem a ver com certeza. Outras pessoas aceitarão seus compromissos e fazer planos com base neles. O custo de não cumprir esses compromissos, para eles, e para a sua reputação, é enorme. Perder um compromisso é um ato de a desonestidade é apenas um pouco menos onerosa do que uma mentira aberta.

### UMA N E ESTIMATIVA

Uma estimativa é uma suposição. Nenhum compromisso está implícito. Nenhuma promessa é feita.

Perder uma estimativa não é de forma alguma desonroso. A razão pela qual fazemos estimativas é porque não sabemos quanto tempo algo vai demorar.

Infelizmente, a maioria dos desenvolvedores de software são péssimos estimadores. Isto não é porque há alguma habilidade secreta para estimar – não há. A razão pela qual muitas vezes somos tão ruins na estimativa é porque não entendemos a verdadeira natureza de uma estimativa.

Uma estimativa não é um número. Uma estimativa é uma distribuição. Considerar:

## O QUE É UMA ESTIMATIVA?

Mike: “Qual é a sua estimativa para completar a tarefa Frazzle?”

Pedro: “Três dias”.

Peter realmente vai terminar em três dias? É possível, mas qual é a probabilidade?

A resposta para isso é: não temos ideia. O que Pedro quis dizer e o que

Mike aprendeu? Se Mike voltar em três dias, ele deveria ficar surpreso se Peter não está feito? Por que ele estaria? Peter não assumiu um compromisso. Pedro tem não disse a ele qual a probabilidade de três dias versus quatro dias ou cinco dias.

O que teria acontecido se Mike tivesse perguntado a Peter qual a probabilidade de sua estimativa de três dias foi?

Mike: “Qual é a probabilidade de você terminar em três dias?”

Pedro: “Muito provavelmente.”

Mike: “Você pode colocar um número nisso?”

Peter: “Cinquenta ou sessenta por cento.”

Mike: “Portanto, há uma boa chance de que você demore quatro dias.”

Peter: “Sim, na verdade posso levar até cinco ou seis, embora eu duvide.”

Mike: “Quanto você duvida disso?”

Peter: “Oh, não sei... tenho noventa e cinco por cento de certeza de que terminarei antes que seis dias se passem.”

Mike: “Você quer dizer que pode demorar sete dias?”

Peter: “Bem, só se tudo der errado. Caramba, se tudo correr errado, poderia levar dez ou até onze dias. Mas não é muito é provável que muita coisa dê errado.”

Agora estamos começando a aprimorar a verdade. A estimativa de Peter é uma probabilidade distribuição. Em sua mente, Peter vê a probabilidade de conclusão como o que é mostrada é a Figura 10-1.

Você pode ver por que Pedro deu a estimativa original de três dias. É o mais alto barra no gráfico. Portanto, na mente de Peter, esta é a duração mais provável para a tarefa. Mas Mike vê as coisas de forma diferente. Ele olha para a extremidade direita do gráfico e teme que Peter possa levar onze dias para terminar.

CAPÍTULO 10 ESTIMATIVA

140

Mike deveria estar preocupado com isso? Claro! Murphy1 fará o que quiser  
Peter, então algumas coisas provavelmente vão dar errado.

I M P L I E D C O M M I T M E N T S

Então agora Mike tem um problema. Ele não tem certeza sobre o tempo que Peter levará para realizar a tarefa. Para minimizar essa incerteza, ele pode pedir a Peter um compromisso. Isto é algo que Pedro não está em posição de dar.

Mike: “Peter, você pode me dar uma data difícil quando terminar?”

Peter: "Não, Mike. Como eu disse, provavelmente será feito em três, talvez quatro, dias.”

Mike: “Podemos dizer quatro então?”

Peter: “Não, podem ser cinco ou seis.”

Até agora, todos estão se comportando de maneira justa. Mike pediu um compromisso e Peter recusou-se cuidadosamente a dar-lhe um. Então Mike tenta uma abordagem diferente:

1. A Lei de Murphy afirma que se algo pode dar errado, dará errado.

Figura 10-1 Distribuição de probabilidade

- 50%
- 45%
- 40%
- 35%
- 30%
- 25%
- 20%
- 15%
- 10%
- 5%
- 0%
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10
- 11



Mike: “OK, Peter, mas você pode tentar não demorar mais do que seis dias?”

O apelo de Mike parece bastante inocente e Mike certamente não tem más intenções.

Mas o que exatamente Mike está pedindo a Peter para fazer? O que significa “tentar”?

Já falamos sobre isso no Capítulo 2. A palavra tentar é um termo carregado. Se

Peter concorda em “tentar”, então ele se compromete com seis dias. Não há outra maneira de interpretá-lo. Concordar em tentar é concordar em ter sucesso.

Que outra interpretação poderia haver? O que é, precisamente, que Pedro está

vai fazer para “experimental”? Ele vai trabalhar mais de oito horas? Isso é

claramente implícito. Ele vai trabalhar nos finais de semana? Sim, isso também está implícito. Ele vai

pular as férias em família? Sim, isso também faz parte da implicação. Todas essas coisas

fazem parte de “tentar”. Se Peter não fizer essas coisas, Mike poderá acusar

ele de não se esforçar o suficiente.

Os profissionais fazem uma distinção clara entre estimativas e compromissos.

Eles não se comprometem a menos que tenham certeza de que terão sucesso. Eles são

cuidado para não assumir quaisquer compromissos implícitos. Eles comunicam a probabilidade

distribuição de suas estimativas da forma mais clara possível, para que os gestores possam

fazer planos apropriados.

## PERT

Em 1957, a Técnica de Avaliação e Revisão de Programas (PERT) foi criada para

apoiar o projeto do submarino Polaris da Marinha dos EUA. Um dos elementos do PERT é

a forma como as estimativas são calculadas. O esquema fornece uma solução muito simples, mas muito

forma eficaz de converter estimativas em distribuições de probabilidade adequadas para gestores.

Ao estimar uma tarefa, você fornece três números. Isso é chamado de trivariado

análise:

- O: Estimativa Otimista. Este número é extremamente otimista. Você só poderia realizar a tarefa rapidamente se tudo der certo. Na verdade, para que a matemática funcione, esse número deve ter muito menos que um

## CAPÍTULO 10 ESTIMATIVA

142

1% de chance de ocorrência.<sup>2</sup> No caso de Peter, isso seria 1 dia, conforme mostrado em Figura 10-1.

- N: Estimativa Nominal. Esta é a estimativa com maior chance de sucesso.

Se você desenhasse um gráfico de barras, seria a barra mais alta, como mostrado na Figura 10-1. São 3 dias.

- P: Estimativa Pessimista. Mais uma vez, isto é extremamente pessimista. Deveria incluir tudo, exceto furacões, guerra nuclear, buracos negros perdidos e outras catástrofes. Novamente, a matemática só funciona se esse número tiver muito menos do que 1% de chance de sucesso. No caso de Peter, esse número está fora do gráfico a direita. Então, 12 dias.

Dadas essas três estimativas, podemos descrever a distribuição de probabilidade como segue:

- $\mu =$

+

+

Ó

N

P

4

6

$\mu$  é a duração esperada da tarefa. No caso de Peter é  $(1+12+12)/6$ , ou cerca de 4,2 dias. Para a maioria das tarefas este será um número um tanto pessimista porque a cauda direita da distribuição é mais longa que a cauda esquerda cauda.<sup>3</sup>

- $\sigma =$

-

P

Ó

6

$\sigma$  é o desvio padrão<sup>4</sup> da distribuição de probabilidade da tarefa. É uma medida de quão incerta é a tarefa. Quando esse número é grande, a incerteza também é grande. Para Peter, esse número é  $(12 - 1)/6$ , ou cerca de 1,8 dias. Dada a estimativa de Peter de 4,2/1,8, Mike entende que esta tarefa provavelmente será feita em cinco dias, mas também pode levar 6 ou até 9 dias para ser concluído.

2. O número exato para uma distribuição normal é 1:769, ou 0,13%, ou 3 sigma. As probabilidades de uma em mil são provavelmente seguro.

3. O PERT presume que isto se aproxima de uma distribuição beta. Isto faz sentido, uma vez que a duração mínima

para uma tarefa é muitas vezes muito mais certo do que o máximo. [McConnell2006] Figura 1-3.

4. Se você não sabe o que é um desvio padrão, deverá encontrar um bom resumo de probabilidade e estatística.

tiques. O conceito não é difícil de entender e será muito útil para você.

PERT

143

Mas Mike não gerencia apenas uma tarefa. Ele está gerenciando um projeto com muitas tarefas.

Peter tem três dessas tarefas nas quais deve trabalhar em sequência. Pedro tem estimado essas tarefas conforme mostrado na Tabela 10-1.

Tabela 10-1 Tarefas de Pedro

Tarefa

Otimista

Nominal

Pessimista

$\mu$

$\sigma$

Alfa

1

3

12

4.2

1,8

Beta

1

1,5

14

3.5

2.2

Gama

3

6,25

11

6,5

1.3

O que há com essa tarefa “beta”? Parece que Peter está bastante confiante sobre isso, mas algo poderia dar errado e o atrapalharia significativamente.

Como Mike deveria interpretar isso? Quanto tempo Mike deve planejar para Peter completar todas as três tarefas?

Acontece que, com alguns cálculos simples, Mike consegue combinar todos os valores de Peter. tarefas e chegar a uma distribuição de probabilidade para todo o conjunto de tarefas. O matemática é bastante simples:

•  $\mu$

$\mu$

sequência

tarefa

$=\sum$

Para qualquer sequência de tarefas, a duração esperada dessa sequência é a soma simples de todas as durações esperadas das tarefas nessa sequência. Então se Peter tem três tarefas para completar, e suas estimativas são 4,2/1,8, 3,5/2,2, e 6.5/1.3, então Peter provavelmente terminará com todos os três em cerca de 14 dias: 4,2 + 3,5 + 6,5.

•  $\sigma$

$\sigma$

sequência =  $\sum$

tarefa

2

O desvio padrão da sequência é a raiz quadrada da soma dos quadrados dos desvios padrão das tarefas. Portanto, o desvio padrão para todas as três tarefas de Peter são cerca de 3.

(1,8

2.2

1.3)

(3.24

2,48

1.69)

9,77

2

2

2 1/2

1/2

1/2

+

+

=

+

+

=

= 3,13

Isso diz a Mike que as tarefas de Peter provavelmente levarão 14 dias, mas podem muito bem demorar 17 dias (1s) e pode até levar 20 dias (2s). Poderia até levar mais tempo, mas isso é bastante improvável.

Veja novamente a tabela de estimativas. Você pode sentir a pressão para conseguir todos os três tarefas feitas em cinco dias? Afinal, as estimativas do melhor caso são 1, 1 e 3. Mesmo o as estimativas nominais somam apenas 10 dias. Como chegamos aos 14 dias, com possibilidade de 17 ou 20? A resposta é que a incerteza naqueles as tarefas são compostas de uma forma que adiciona realismo ao plano.

Se você é um programador com mais de alguns anos de experiência, provavelmente já viu projetos que foram estimados de forma otimista e que demoraram de três a cinco vezes mais do que o esperado. O esquema PERT simples que acabamos de mostrar é uma maneira razoável para ajudar a evitar o estabelecimento de expectativas otimistas. Profissionais de software são muito cuidadosos ao estabelecer expectativas razoáveis, apesar da pressão para tentar ir rápido.

### TAREFAS DE ESTIMAÇÃO

Mike e Peter estavam cometendo um erro terrível. Mike estava perguntando a Peter quanto tempo suas tarefas levariam. Peter deu respostas honestas e trivariadas, mas e quanto ao opiniões de seus companheiros de equipe? Eles poderiam ter uma ideia diferente?

O recurso de estimativa mais importante que você possui são as pessoas ao seu redor.

Eles podem ver coisas que você não vê. Eles podem ajudá-lo a estimar melhor suas tarefas com precisão do que você pode estimá-los por conta própria.

### WI D E BAN D D E LPH I

Na década de 1970, Barry Boehm nos apresentou uma técnica de estimativa chamada “delphi de banda larga”.<sup>5</sup> Houve muitas variações ao longo dos anos. Alguns são formais, alguns são informais; mas todos eles têm uma coisa em comum: o consenso. A estratégia é simples. Uma equipe de pessoas se reúne, discute uma tarefa, estima o tarefa e itere a discussão e a estimativa até que cheguem a um acordo.

5. [Boehm81]

## ESTIMANDO TAREFAS

145

A abordagem original delineada por Boehm envolveu diversas reuniões e documentos que envolvem muita cerimônia e sobrecarga para o meu gosto. eu prefiro abordagens simples de baixa sobrecarga, como as seguintes.

### Dedos voadores

Todos se sentam em volta de uma mesa. As tarefas são discutidas uma de cada vez. Para cada tarefa há discussão sobre o que a tarefa envolve, o que pode confundir ou complicá-lo e como pode ser implementado. Então os participantes colocaram suas mãos abaixo da mesa e levantam de 0 a 5 dedos com base em quanto tempo eles acho que a tarefa vai demorar. O moderador conta 1-2-3 e todos os participantes mostre suas mãos imediatamente.

Se todos concordarem, eles passam para a próxima tarefa. Caso contrário eles continuam a discussão para determinar por que eles discordam. Eles repetem isso até concordarem. O acordo não precisa ser absoluto. Enquanto as estimativas estiverem próximas, é bom o suficiente. Assim, por exemplo, um punhado de 3 e 4 é concordância. No entanto se todos levantarem 4 dedos, exceto uma pessoa que levantar 1 dedo, então eles têm algo para conversar.

A escala da estimativa é decidida no início da reunião. Poderia ser o número de dias para uma tarefa ou pode ser alguma escala mais interessante como “dedos vezes três” ou “dedos ao quadrado”.

A simultaneidade de exibição dos dedos é importante. Não queremos pessoas mudando suas estimativas com base no que veem outras pessoas fazerem.

### Planejando o pôquer

Em 2002, James Grenning escreveu um artigo maravilhoso<sup>6</sup> descrevendo “Planning Poker”. Esta variação do delphi de banda larga tornou-se tão popular que vários as empresas usaram a ideia para fazer brindes de marketing na forma de planejando baralhos de pôquer.<sup>7</sup> Existe até um site chamado Planningpoker.com que você pode usar para planejar pôquer na Internet com equipes distribuídas.

6. [Grenning2002]

7. <http://store.mountangoatsoftware.com/products/planning-poker-cards>

## CAPÍTULO 10 ESTIMATIVA

146

A ideia é muito simples. Para cada membro da equipe de estimativa, dê uma mão de cartões com números diferentes. Os números de 0 a 5 funcionam bem, e tornar este sistema logicamente equivalente a dedos voadores.

Escolha uma tarefa e discuta-a. Em algum momento, o moderador pede a todos que escolham um cartão. Os membros da equipe retiram um cartão que corresponde à sua estimativa e segure-o com as costas voltadas para fora, para que ninguém mais possa ver o valor de o cartão. Em seguida, o moderador pede a todos que mostrem seus cartões.

O resto é como dedos voadores. Se houver acordo, então a estimativa é aceito. Caso contrário, as cartas são devolvidas à mão e os jogadores continuar a discutir a tarefa.

Muita “ciência” tem sido dedicada a escolher os valores corretos das cartas para um mão. Algumas pessoas chegaram ao ponto de usar cartas baseadas em uma série de Fibonacci. Outros incluíram cartões para o infinito e pontos de interrogação. Pessoalmente, acho cinco cartas marcadas com 0, 1, 3, 5, 10 são suficientes.

### Estimativa de afinidade

Uma variação particularmente única do delphi de banda larga foi mostrada para mim vários anos atrás por Lowell Lindstrom. Eu tive muita sorte com isso abordagem com vários clientes e equipes.

Todas as tarefas são escritas em cartões, sem exibição de estimativas. O a equipe de estimativa fica em volta de uma mesa ou parede com os cartões espalhados aleatoriamente. Os membros da equipe não falam, simplesmente começam a separar as cartas em relação um ao outro. As tarefas que demoram mais são movidas para a direita. Menor as tarefas se movem para a esquerda.

Qualquer membro da equipe pode mover qualquer carta a qualquer momento, mesmo que já tenha sido movido por outro membro. Qualquer carta movida mais de h vezes é reservada para discussão.

Eventualmente, a classificação silenciosa desaparece e a discussão pode começar. Desentendimentos sobre a ordem das cartas são exploradas. Pode haver algum design rápido sessões ou algumas molduras rápidas desenhadas à mão para ajudar a obter consenso.



## CONCLUSÃO

A próxima etapa é desenhar linhas entre os cartões que representam os tamanhos dos baldes.

Esses intervalos podem estar em dias, semanas ou pontos. Cinco baldes em um Fibonacci a sequência (1, 2, 3, 5, 8) é tradicional.

### Estimativas Trivariadas

Essas técnicas Delphi de banda larga são boas para escolher um único valor nominal. estimativa para uma tarefa. Mas como afirmamos anteriormente, na maioria das vezes queremos três estimativas para que possamos criar uma distribuição de probabilidade. O otimista e valores pessimistas para cada tarefa podem ser gerados muito rapidamente usando qualquer um dos variantes delphi de banda larga. Por exemplo, se você estiver usando o Planning Poker, você simplesmente peça à equipe para mostrar as cartas de sua estimativa pessimista e então pegue o mais alto. Você faz o mesmo para a estimativa otimista e escolhe a mais baixa.

### A LEI DOS GRANDES NÚMEROS

As estimativas estão repletas de erros. É por isso que são chamadas de estimativas. Uma maneira de gerenciar erros é tirar vantagem da Lei dos Grandes Números.<sup>8</sup> Uma implicação desta lei é que se você dividir uma tarefa grande em muitas tarefas menores e estimar independentemente, a soma das estimativas das pequenas tarefas será mais preciso do que uma única estimativa da tarefa maior. A razão deste aumento precisão é que os erros nas pequenas tarefas tendem a se integrar.

Francamente, isso é otimista. Erros nas estimativas tendem à subestimação e não é superestimação, portanto a integração dificilmente é perfeita. Dito isto, dividir tarefas grandes em pequenas e estimar as pequenas de forma independente ainda é uma boa técnica. Alguns dos erros se integram, e quebrando o tarefas é uma boa maneira de entender melhor essas tarefas e descobrir surpresas.

## CONCLUSÃO

Os desenvolvedores de software profissionais sabem como fornecer à empresa estimativas práticas que a empresa pode usar para fins de planejamento. Eles não fazem promessas que não podem cumprir e não assumem compromissos que eles não têm certeza se podem se encontrar.

8. [http://en.wikipedia.org/wiki/Law\\_of\\_large\\_numbers](http://en.wikipedia.org/wiki/Law_of_large_numbers)

Quando os profissionais assumem compromissos, eles fornecem números concretos e depois eles fazem esses números. Contudo, na maioria dos casos os profissionais não fazem tais compromissos. Em vez disso, eles fornecem estimativas probabilísticas que descrevem o tempo de conclusão esperado e a variação provável.

Os desenvolvedores profissionais trabalham com os outros membros de sua equipe para alcançar consenso sobre as estimativas que são dadas à administração.

As técnicas descritas neste capítulo são exemplos de algumas das diferentes maneiras pelas quais os desenvolvedores profissionais criam estimativas práticas. Estes não são os apenas essas técnicas e não são necessariamente as melhores. Eles são simplesmente técnicas que descobri que funcionam bem para mim.

### B I B L I O G R A F I A

[McConnell2006]: Steve McConnell, Estimativa de Software: Desmistificando o Negro Arte, Redmond, WA: Microsoft Press, 2006.

[Boehm81]: Barry W. Boehm, Economia de Engenharia de Software, Upper Saddle River, NJ: Prentice Hall, 1981.

[Grenning2002]: James Grenning, "Planejando o pôquer ou como evitar análises Paralisia durante o planejamento de lançamento", abril de 2002, <http://renaissancesoftware.net/papers/14-papers/44-planing-poker.html>

## P R E S S U R E

Imagine que você está tendo uma experiência extracorpórea, observando-se uma mesa de operação enquanto um cirurgião realiza uma cirurgia cardíaca aberta em você. Isso cirurgião está tentando salvar sua vida, mas o tempo é limitado, então ele está operando sob um prazo – um prazo literal.

## CAPÍTULO 11 PRESSÃO

150

Como você quer que aquele médico se comporte? Você quer que ele pareça calmo e coletado? Você quer que ele emita ordens claras e precisas para sua equipe de suporte? Você quer que ele siga seu treinamento e adira às suas disciplinas?

Ou você quer que ele sue e pragueje? Você quer que ele bata e instrumentos de arremesso? Você quer que ele culpe a administração por ações irrealistas expectativas e reclamando continuamente do tempo? Você quer que ele comportando-se como um profissional ou como um desenvolvedor típico?

O desenvolvedor profissional é calmo e decidido sob pressão. Como a pressão cresce ele adere aos seus treinos e disciplinas, sabendo que são os melhores maneira de cumprir os prazos e compromissos que o pressionam.

Em 1988 eu trabalhava na Clear Communications. Esta foi uma start-up que nunca bastante começou. Gastámos a nossa primeira ronda de financiamento e depois tivemos ir por um segundo e depois por um terceiro.

A visão inicial do produto parecia boa, mas a arquitetura do produto poderia nunca parecem ficar de castigo. No início, o produto era tanto software quanto hardware. Depois passou a ser apenas software. A plataforma de software mudou de PCs para Sparcstations. Os clientes mudaram de high-end para low-end.

Eventualmente, até mesmo a intenção original do produto mudou conforme a empresa tentava para encontrar algo que gerasse receita. Nos quase quatro anos que passei lá, não acho que a empresa tenha obtido um centavo de receita.

Escusado será dizer que nós, desenvolvedores de software, estávamos sob pressão significativa. Lá foram algumas noites muito longas e fins de semana ainda mais longos passados no escritório no terminal. As funções foram escritas em C com 3.000 linhas. Lá eram discussões com gritos e xingamentos. Houve intriga e subterfúgio. Houve socos perfurados nas paredes, canetas atiradas com raiva em quadros brancos, caricaturas de colegas irritantes gravadas nas paredes com o pontas de lápis, e havia um suprimento interminável de raiva e estresse.

Os prazos eram determinados pelos acontecimentos. Os recursos tiveram que ser preparados para feiras comerciais

ou demonstrações de clientes. Qualquer coisa que um cliente pedisse, por mais bobo que fosse, nós esteja pronto para a próxima demonstração. O tempo sempre foi muito curto. O trabalho sempre foi atrás. Os horários eram sempre esmagadores.

## EVITANDO PRESSÃO

151

Se você trabalhasse 80 horas por semana, poderia ser um herói. Se você hackeou algum bagunçar tudo para uma demonstração do cliente, você pode ser um herói. Se você fez isso o suficiente, você pode ser promovido. Caso contrário, você poderá ser demitido. Foi uma start-up - foi era tudo sobre a “equidade do suor”. E em 1988, com quase 20 anos de experiência sob meu cinto, eu comprei.

Eu era o gerente de desenvolvimento contando aos programadores que trabalhavam para mim que eles tinham que trabalhar mais e mais rápido. Eu era um dos caras das 80 horas, escrevendo 3.000 linhas C funcionam às 2 da manhã enquanto meus filhos dormiam em casa sem os pai em casa. Fui eu quem jogou as canetas e gritou. Eu tenho pessoas demitidos se não se adaptassem. Foi horrível. Eu fui horrível.

Então chegou o dia em que minha esposa me obrigou a dar uma boa olhada no espelho. Não gostei do que vi. Ela me disse que eu simplesmente não era muito agradável de estar por perto.

Eu tive que concordar. Mas eu não gostei, então saí de casa furioso e comecei a caminhar sem destino. Caminhei por cerca de trinta minutos, fervendo enquanto eu caminhava; e então começou a chover.

E algo clicou dentro da minha cabeça. Eu comecei a rir. Eu ri da minha loucura.

Eu ri do meu estresse. Eu ri do homem no espelho, o pobre idiota que estava tornando a vida miserável para si e para os outros em nome de... o quê?

Tudo mudou naquele dia. Parei as horas malucas. Eu parei o alto estilo de vida estressante. Parei de jogar canetas e escrever funções C de 3.000 linhas.

Decidi que iria aproveitar minha carreira fazendo-a bem, e não fazendo isso estupidamente.

Deixei esse emprego da maneira mais profissional que pude e me tornei consultor. Desde então dia em que nunca chamei outra pessoa de “chefe”.

## EVITE D I N G P R E S S U R E

A melhor maneira de manter a calma sob pressão é evitar as situações que causam pressão. Essa evitação pode não eliminar completamente a pressão, mas pode percorrer um longo caminho para minimizar e encurtar a alta pressão períodos.

## CAPÍTULO 11 PRESSÃO

152

### COMMITMENTOS

Como descobrimos no Capítulo 10, é importante evitar comprometer-se com prazos que não temos certeza se podemos cumprir. A empresa sempre vai querer isso compromissos porque querem eliminar riscos. O que devemos fazer é fazer certifique-se de que o risco seja quantificado e apresentado ao negócio para que eles possam gerenciá-lo adequadamente. Aceitar compromissos irrealistas frustra esse objetivo e presta um péssimo serviço tanto à empresa quanto a nós mesmos.

Às vezes, compromissos são assumidos por nós. Às vezes descobrimos que nosso negócio fez promessas aos clientes sem nos consultar. Quando isso acontecer

temos o dever de honra de ajudar a empresa a encontrar uma maneira de atender a esses compromissos. No entanto, não temos obrigação de honra de aceitar os compromissos.

A diferença é importante. Os profissionais sempre ajudarão a empresa a encontrar um maneira de atingir seus objetivos. Mas os profissionais não necessariamente aceitam compromissos feitos para eles pela empresa. No final, se não conseguirmos encontrar uma maneira de nos encontrar

as promessas feitas pela empresa, depois as pessoas que fizeram as promessas deve aceitar a responsabilidade.

Isto é fácil de dizer. Mas quando o seu negócio está falindo e o seu salário é atrasado por causa de compromissos não cumpridos, é difícil não sentir a pressão. Mas se você se comportou profissionalmente, pelo menos você pode manter a cabeça erguida enquanto procurar um novo emprego.

### MANTENDO-SE LIMPO

A maneira de ir rápido e manter os prazos sob controle é manter-se limpo. Profissionais não sucumba à tentação de criar uma bagunça para agir rapidamente.

Os profissionais percebem que “rápido e sujo” é um oxímoro. Sujo sempre significa devagar!

Podemos evitar pressão mantendo nossos sistemas, nosso código e nosso design como limpo possível. Isto não significa que passamos horas intermináveis polindo código. Significa simplesmente que não toleramos bagunças. Sabemos que a bagunça vai nos atrasa, fazendo com que percamos datas e quebreemos compromissos. Então fazemos o melhor trabalho que pudermos e manter nossa produção o mais limpa possível.

## CRISIS DISCIPLINE

Você sabe no que acredita ao se observar em uma crise. Se em uma crise você siga suas disciplinas, então você realmente acredita nessas disciplinas. Por outro lado, se você mudar seu comportamento em uma crise, então você não acredita verdadeiramente em seu comportamento normal.

Se você seguir a disciplina de Desenvolvimento Orientado a Testes em tempos sem crise mas abandoná-lo durante uma crise, você não confiará realmente que o TDD seja útil.

Se você mantiver seu código limpo em tempos normais, mas fizer bagunça em uma crise, então você realmente não acredita que a bagunça o atrapalhe. Se você emparelhar em um crise, mas normalmente não forma pares, então você acredita que o emparelhamento é mais eficiente do que não emparelhamento.

Escolha disciplinas que você se sinta confortável em seguir durante uma crise. Então siga eles o tempo todo. Seguir essas disciplinas é a melhor maneira de evitar ficar em uma crise.

Não mude seu comportamento quando a crise chegar. Se suas disciplinas são as melhor forma de trabalhar, então devem ser seguidas mesmo no auge de uma crise.

## HANDLING PRESSURE

Prevenir, mitigar e eliminar a pressão é muito bom, mas às vezes a pressão surge apesar de todas as suas melhores intenções e prevenções.

Às vezes, o projeto leva mais tempo do que se pensava. Às vezes o design inicial está simplesmente errado e deve ser retrabalhado. Às vezes você perde um membro valioso da equipe ou cliente. Às vezes você assume um compromisso que você simplesmente não consegue manter. Então o que?

## NÃO PANIC

Gerencie seu estresse. Noites sem dormir não o ajudarão a terminar mais rápido. Sentado e a preocupação também não vai ajudar. E a pior coisa que você pode fazer é se apressar! Resista a essa tentação a todo custo. A pressa só irá levá-lo mais fundo no buraco.

## CAPÍTULO 11 PRESSÃO

154

Em vez disso, diminua a velocidade. Pense bem no problema. Trace um curso para o melhor resultado possível e, em seguida, dirigir-se para esse resultado de uma forma razoável e ritmo constante.

### COMUNICATO

Deixe sua equipe e seus superiores saberem que você está com problemas. Diga a eles o seu melhores planos para sair de problemas. Peça-lhes sua opinião e orientação.

Evite criar surpresas. Nada deixa as pessoas mais irritadas e menos racionais do que surpresas. As surpresas multiplicam a pressão por dez.

### CONFIE EM SEUS DISCIPLINAS

Quando as coisas ficarem difíceis, confie em suas disciplinas. A razão pela qual você tem disciplinas é fornecer orientação em momentos de alta pressão. Estes são os vezes para prestar atenção especial a todas as suas disciplinas. Estes não são os tempos para questioná-los ou abandoná-los.

Em vez de procurar em pânico por algo, qualquer coisa, que possa ajudar você termina mais rápido, se torna mais deliberado e dedicado a seguir suas disciplinas escolhidas. Se você seguir o TDD, escreva ainda mais testes do que habitual. Se você é um refatorador impiedoso, refatore ainda mais. Se você continuar suas funções pequenas e mantenha-as ainda menores. A única maneira de passar a panela de pressão é confiar no que você já sabe que funciona – o seu disciplinas.

### OBTENHA AJUDA

Par! Quando a temperatura estiver alta, encontre um associado que esteja disposto a emparelhar o programa com

você. Você terminará mais rápido, com menos defeitos. Seu parceiro irá ajudar você mantém sua disciplina e evita entrar em pânico. Seu parceiro irá identificar coisas que você sente falta, terá ideias úteis e compensará quando você perde o foco.



## CONCLUSÃO

155

Da mesma forma, quando você vir alguém que está sob pressão, ofereça-se para emparelhar com eles. Ajude-os a sair do buraco em que estão.

## CONCLUSÃO

O truque para lidar com a pressão é evitá-la quando puder e resistir quando você não pode. Você evita isso gerenciando compromissos, seguindo suas disciplinas, e mantendo-se limpo. Você resiste mantendo a calma, comunicando-se, seguindo suas disciplinas e obter ajuda.

Esta página foi intencionalmente deixada em branco

## COLLABORAÇÃO

A maior parte do software é criada por equipes. As equipes são mais eficazes quando a equipe os membros colaboram profissionalmente. Não é profissional ser um solitário ou um recluso em uma equipe.

Em 1974, eu tinha 22 anos. Meu casamento com minha maravilhosa esposa, Ann Marie, mal tinha seis anos.

meses de idade. Faltava ainda um ano para o nascimento da minha primeira filha, Angela. E eu trabalhou em uma divisão da Teradyne conhecida como Chicago Laser Systems.

Trabalhando ao meu lado estava meu colega de colégio, Tim Conrad. Tim e eu tivemos operou alguns milagres em nosso tempo. Construímos computadores juntos em seu porão. Construímos as escadas de Jacob nas minhas. Nós ensinamos um ao outro como programar PDP-8s e como conectar circuitos integrados e transistores em calculadoras funcionando.

Éramos programadores trabalhando em um sistema que usava lasers para ajustar componentes como resistores e capacitores com precisão extremamente alta. Para Por exemplo, recortámos o cristal do primeiro relógio digital, o Motorola Pulsar.

O computador que programamos foi o M365, clone PDP-8 da Teradyne. Nós escrevi em linguagem assembly, e nossos arquivos de origem foram mantidos em fita magnética cartuchos. Embora pudéssemos editar em uma tela, o processo foi bastante complicado, então usamos listagens impressas para a maior parte de nossa leitura de código e edição.

Não tínhamos nenhuma facilidade para pesquisar a base de código. Não havia como encontrar todos os lugares onde uma determinada função foi chamada ou uma determinada constante foi usado. Como você pode imaginar, isso foi um grande obstáculo.

Então, um dia, Tim e eu decidimos que escreveríamos um gerador de referência cruzada. Isto programa leria nossas fitas de origem e imprimiria uma lista de cada símbolo, junto com os números do arquivo e da linha onde esse símbolo foi usado.

O programa inicial era bastante simples de escrever. Ele simplesmente lê a fita de origem, analisou a sintaxe do assembler, criou uma tabela de símbolos e adicionou referências ao entradas. Funcionou muito bem, mas foi terrivelmente lento. Demorou mais de uma hora para processar nosso Programa Operacional Mestre (o MOP).

A razão pela qual foi tão lento foi que estávamos segurando a crescente tabela de símbolos em um único buffer de memória. Sempre que encontramos uma nova referência nós a inserimos em o buffer, movendo o restante do buffer alguns bytes para baixo para liberar espaço.

Tim e eu não éramos especialistas em estruturas de dados e algoritmos. Nós nunca tínhamos ouvido falar de tabelas hash ou pesquisas binárias. Não tínhamos ideia de como fazer um algoritmo rápido. Sabíamos apenas que o que estávamos fazendo era muito lento.

Então tentamos uma coisa após a outra. Tentamos colocar as referências em links listas. Tentamos deixar lacunas no array e aumentar o buffer apenas quando o lacunas preenchidas. Tentamos criar listas vinculadas de lacunas. Tentamos todos os tipos de ideias malucas.

Ficamos diante do quadro branco em nosso escritório e desenhamos diagramas de nossos dados estruturas e realizou cálculos para prever o desempenho. Chegaríamos ao escritório todos os dias com outra ideia nova. Colaboramos como demônios.

Algumas das coisas que tentamos aumentaram o desempenho. Alguns desaceleraram. Isso era enlouquecedor. Foi quando descobri como é difícil otimizar software e quão não intuitivo é o processo.

No final conseguimos reduzir o tempo para menos de 15 minutos, o que foi muito próximo quanto tempo levou simplesmente para ler a fita de origem. Então ficamos satisfeitos.

### PROGRAMADORES VERSUS PESSOAS

Não nos tornamos programadores porque gostamos de trabalhar com pessoas. Como regra achamos os relacionamentos interpessoais confusos e imprevisíveis. Nós gostamos do limpo e comportamento previsível das máquinas que programamos. Estamos mais felizes quando ficamos sozinhos em uma sala por horas nos concentrando profundamente em algo realmente problema interessante.

OK, isso é uma generalização enorme e há muitas exceções. Existem muitos programadores que são bons em trabalhar com pessoas e gostam do desafio. Mas a média do grupo ainda tende na direção que afirmei. Nós, programadores, aproveitem a leve privação sensorial e a imersão semelhante a um casulo de foco.

### PROGRAMADORES E EMPLOYEES

Nas décadas de setenta e oitenta, enquanto trabalhava como programador na Teradyne, aprendi a ser muito bom em depuração. Adorei o desafio e lançaria resolver problemas com vigor e entusiasmo. Nenhum bug poderia se esconder por muito tempo de mim!

Quando resolvi um bug, foi como conquistar uma vitória ou matar o Jabberwock!

Eu iria até meu chefe, Ken Finder, com a lâmina Vorpall na mão e apaixonadamente descrevia para ele o quão interessante era o bug. Um dia Ken finalmente entrou em erupção frustração: "Bugs não são interessantes. Bugs só precisam ser corrigidos!"

Aprendi algo naquele dia. É bom ser apaixonado pelo que fazemos. Mas também é bom ficar de olho nos objetivos das pessoas que pagam para você.

A primeira responsabilidade do programador profissional é atender às necessidades de seu empregador. Isso significa colaborar com seus gerentes, negócios analistas, testadores e outros membros da equipe para compreender profundamente o negócio objetivos. Isso não significa que você precisa se tornar um especialista em negócios. Isso significa que você precisa entender por que está escrevendo o código que está escrevendo e como a empresa que emprega você se beneficiará com isso.

A pior coisa que um programador profissional pode fazer é enterrar-se alegremente em uma tumba de tecnologia enquanto o negócio quebra e queima ao seu redor. Seu trabalho é manter o negócio funcionando!

Portanto, os programadores profissionais dedicam algum tempo para entender o negócio. Eles converse com os usuários sobre o software que eles estão usando. Eles conversam com vendas e marketing

pessoas sobre os problemas e questões que elas têm. Eles conversam com seus gerentes para compreender os objetivos de curto e longo prazo da equipe.

Em suma, prestam atenção ao navio em que navegam.

A única vez que fui demitido de um emprego de programação foi em 1976. Eu estava trabalhando para Outboard Marine Corp. na época. Eu estava ajudando a escrever uma fábrica sistema de automação que usou IBM System/7s para monitorar dezenas de alumínio máquinas fundidas no chão de fábrica.

Tecnicamente, este foi um trabalho desafiador e gratificante. A arquitetura do O System/7 era fascinante, e o próprio sistema de automação de fábrica era realmente interessante.

Também tínhamos uma boa equipe. O líder da equipe, John, era competente e motivado.

Meus dois colegas de equipe de programação foram agradáveis e prestativos. Tínhamos um laboratório

dedicado ao nosso projeto, e todos nós trabalhamos naquele laboratório. O parceiro de negócios estava envolvido e no laboratório conosco. Nosso gerente, Ralph, era competente, focado e no comando.

Tudo deveria ter sido ótimo. O problema era eu. eu estava entusiasmado o suficiente sobre o projeto e sobre a tecnologia, mas na idade avançada de 24 Eu simplesmente não conseguia me preocupar com o negócio ou com sua estrutura política interna.

Meu primeiro erro foi no primeiro dia. Apareci sem gravata. eu tinha usado uma na minha entrevista e vi que todo mundo usava gravata, mas não conseguiu fazer a conexão. Então, no meu primeiro dia, Ralph veio até mim e claramente disse: “Nós usamos gravata aqui”.

Eu não posso te dizer o quanto eu me resenti disso. Isso me incomodou profundamente. eu usava a gravata todos os dias, e eu odiava isso. Mas por que? Eu sabia no que estava me metendo. eu sabia as convenções que haviam adotado. Por que eu ficaria tão chateado? Porque eu era um idiota egoísta e narcisista.

Eu simplesmente não conseguia chegar ao trabalho na hora certa. E eu pensei que isso não importava. Afinal,

Eu estava fazendo “um bom trabalho”. E era verdade, eu estava fazendo um ótimo trabalho escrevendo meus programas. Eu era facilmente o melhor programador técnico da equipe. eu poderia escrever código mais rápido e melhor que os outros. Eu poderia diagnosticar e resolver problemas mais rapidamente. Eu sabia que era valioso. Então, horários e datas não importavam muito para mim.

A decisão de me demitir foi tomada um dia, quando não compareci a tempo por um marco. Aparentemente, John havia nos contado que queria uma demonstração de trabalho apresentada na próxima segunda-feira. Tenho certeza de que sabia disso, mas datas e horários simplesmente não eram importantes para mim.

Estávamos em desenvolvimento ativo. O sistema não estava em produção. Havia não havia razão para deixar o sistema funcionando quando não havia ninguém no laboratório. Eu devo ter sido o último a sair naquela sexta-feira e, aparentemente, deixei o sistema em um estado de não funcionamento. O fato de segunda-feira ser importante simplesmente não estava preso em meu cérebro.

Cheguei uma hora atrasado naquela segunda-feira e vi todo mundo reunido, taciturno, ao redor um sistema que não funciona. John me perguntou: “Por que o sistema não está funcionando hoje, Bob?” Minha resposta: “Não sei”. E sentei-me para depurá-lo. eu estava ainda não tenho ideia da demonstração de segunda-feira, mas pude perceber pelo corpo de todos os outros linguagem que algo estava errado. Então John veio e sussurrou no meu ouvido: “E se Stenberg tivesse decidido visitar?” Então ele foi embora em nojo.

Stenberg era o vice-presidente responsável pela automação. Hoje em dia o chamaríamos de CIO. A pergunta não tinha significado para mim. “E daí?” Eu pensei. “O sistema não na produção, qual é o problema?”

Recebi minha primeira carta de advertência mais tarde naquele dia. Ele me disse que eu tinha que mudar meu

atitude imediata ou “rescisão rápida será o resultado”. eu estava horrorizado!

Levei algum tempo para analisar meu comportamento e comecei a perceber o que eu estava fazendo fazendo errado. Conversei com John e Ralph sobre isso. Eu decidi virar eu e meu trabalho por aí.

E eu fiz! Parei de chegar tarde. Comecei a prestar atenção ao interno política. Comecei a entender por que John estava preocupado com Stenberg. eu comecei para ver a situação ruim em que o coloquei por não ter aquele sistema funcionando Segunda-feira.

Mas era muito pouco, muito tarde. A sorte estava lançada. Recebi uma segunda carta de advertência mês depois por um erro trivial que cometi. Eu deveria ter percebido naquele momento que as cartas eram uma formalidade e que a decisão de me demitir tinha já foi feito. Mas eu estava determinado a resgatar a situação. Então eu trabalhei ainda mais difícil.

A reunião de rescisão ocorreu algumas semanas depois.

Fui para casa naquele dia, para minha esposa grávida de 22 anos, e tive que contar a ela isso Eu fui demitido. Essa não é uma experiência que eu queira repetir.



PROGRAMERS VERSUS PROGRAMMERS

Os programadores muitas vezes têm dificuldade em trabalhar em estreita colaboração com outros programadores.

Isso leva a alguns problemas realmente terríveis.

Código de propriedade

Um dos piores sintomas de uma equipe disfuncional é quando cada programador constrói um muro em torno de seu código e se recusa a permitir que outros programadores o toquem.

Já estive em lugares onde os programadores nem deixavam outros os programadores veem seu código. Esta é uma receita para o desastre.

Certa vez, fui consultor de uma empresa que fabricava impressoras de última geração. Essas máquinas possuem muitos componentes diferentes, como alimentadores, impressoras, empilhadores, grampeadores, cortadores e assim por diante. A empresa valorizou cada um desses dispositivos de forma diferente.

Alimentadores

eram mais importantes que os empilhadores, e nada era mais importante que o impressora.

Cada programador trabalhou em seu dispositivo. Um cara escreveria o código para o alimentador, outro cara escreveria o código do grampeador. Cada um deles manteve sua tecnologia para si mesmos e impediram que qualquer outra pessoa mexesse em seu código.

A influência política que estes programadores exerciam estava diretamente relacionada com a forma como a empresa valorizou muito o dispositivo. O programador que trabalhou no impressora era inexpugnável.

Isso foi um desastre para a tecnologia. Como consultor pude perceber que havia houve uma duplicação massiva no código e que as interfaces entre os módulos estavam completamente distorcidos. Mas nenhum argumento da minha parte poderia convencer os programadores (ou a empresa) mudem seus hábitos. Afinal, seu salário as avaliações estavam vinculadas à importância dos dispositivos que mantinham.

Propriedade Coletiva

É muito melhor quebrar todas as barreiras de propriedade do código e ter a própria equipe todo o código. Prefiro equipes nas quais qualquer membro da equipe possa verificar qualquer módulo e fazer as alterações que acharem apropriadas. Eu quero que a equipe possui o código, não os indivíduos.

Os desenvolvedores profissionais não impedem que outras pessoas trabalhem no código. Eles não constroem muros de propriedade em torno do código. Em vez disso, eles trabalham uns com os outros tanto quanto possível do sistema. Eles aprendem uns com os outros trabalhando entre si em outras partes do sistema.

### Emparelhamento

Muitos programadores não gostam da ideia de programação em pares. Eu acho isso estranho já que a maioria dos programadores forma pares em emergências. Por que? Porque é claramente a maneira mais eficiente de resolver o problema. Isso apenas remonta ao velho ditado:

Duas cabeças pensam melhor que uma. Mas se o emparelhamento é a maneira mais eficiente de resolver um problema em uma emergência, por que não é a maneira mais eficiente de resolver um período problemático?

Não vou citar estudos seus, embora existam alguns que poderiam ser citados. Não vou contar nenhuma anedota, embora haja muitas que eu poderia digitar. Não vou nem dizer quanto você deve emparelhar. Tudo que eu vou

te digo é que essa dupla de profissionais. Por que? Porque pelo menos para alguns problemas é a maneira mais eficiente de resolvê-los. Mas esse não é o único motivo.

Os profissionais também fazem dupla porque é a melhor forma de compartilhar conhecimento com cada um outro. Profissionais não criam silos de conhecimento. Em vez disso, eles aprendem os diferentes partes do sistema e do negócio emparelhando-se entre si. Eles reconhecem que embora todos os membros da equipe tenham posição para jogar, todos os membros da equipe devem também ser capazes de jogar em outra posição em apuros.

Os profissionais fazem dupla porque é a melhor forma de revisar o código. Nenhum sistema deve consistir em código que não foi revisado por outros programadores. Existem muitas maneiras de conduzir revisões de código; a maioria delas é terrivelmente ineficiente.

A maneira mais eficiente e eficaz de revisar o código é colaborar na sua escrita.

### C E R E B E L L U M S

Peguei o trem para Chicago em uma manhã de 2000, no auge do ponto com boom. Ao descer do trem para a plataforma, fui assaltado por um enorme outdoor pendurado acima das portas de saída. O sinal era para um conhecido

## CEREBELOS

165

empresa de software que estava recrutando programadores. Dizia: Venha esfregar cerebelos com o melhor.

Fiquei imediatamente impressionado com a estupidez de um sinal como esse. Esses pobres publicitários sem noção tentavam apelar para um público altamente técnico, inteligente e população conhecedora de programadores. Esse é o tipo de pessoa que não sofre estupidez particularmente bem. Os anunciantes estavam tentando evocar o imagem de compartilhamento de conhecimento com outras pessoas altamente inteligentes. Infelizmente eles se referiam a uma parte do cérebro, o cerebelo, que lida com músculos finos controle, não inteligência. Então, as mesmas pessoas que eles estavam tentando atrair eram zombando de um erro tão bobo.

Mas algo mais me intrigou sobre esse sinal. Isso me fez pensar em um grupo de pessoas tentando esfregar cerebelos. Como o cerebelo está na parte posterior do cérebro, a melhor maneira de esfregar os cerebelos é ficar de costas um para o outro. eu imaginei uma equipe de programadores em cubículos, sentados nos cantos, de costas um para o outro, olhando para as telas enquanto usam fones de ouvido. É assim que você esfrega cerebelos. Isso também não é uma equipe.

Os profissionais trabalham juntos. Vocês não podem trabalhar juntos enquanto estão sentados cantos usando fones de ouvido. Então eu quero vocês sentados em mesas de frente para cada um outro. Quero que vocês sejam capazes de sentir o cheiro do medo um do outro. Eu quero que você seja capaz

ouvir os murmúrios frustrados de alguém. Eu quero uma comunicação fortuita, linguagem verbal e corporal. Quero que vocês se comuniquem como uma unidade.

Talvez você acredite que trabalha melhor quando trabalha sozinho. Isso pode seja verdade, mas isso não significa que a equipe funcione melhor quando você trabalha sozinho. E, na verdade, é altamente improvável que você trabalhe melhor quando trabalha sozinho.

Há momentos em que trabalhar sozinho é a coisa certa a fazer. Há momentos quando você simplesmente precisa pensar muito sobre um problema. Há momentos quando a tarefa é tão trivial que seria um desperdício ter outra pessoa trabalhando com você. Mas, em geral, é melhor colaborar estreitamente com outras pessoas e emparelhar com eles uma grande fração do tempo.

CONCLUSÃO

Talvez não tenhamos entrado na programação para trabalhar com pessoas. Azar para nós. Programar tem tudo a ver com trabalhar com pessoas. Precisamos trabalhar com nossos negócios, e precisamos trabalhar uns com os outros.

Eu sei, eu sei. Não seria ótimo se eles nos trancassem em uma sala com seis pessoas? telas enormes, um canal T3, um conjunto paralelo de processadores super-rápidos, ilimitado carneiro e disco, e um suprimento interminável de cola diet e salgadinhos de milho picantes? Infelizmente, não é para ser. Se realmente quisermos passar nossos dias programando, vamos ter que aprender a conversar com - pessoas.<sup>1</sup>

1. Uma referência à última palavra do filme *Soylent Green*.

## EQUIPE E PROJETOS

E se você tiver muitos pequenos projetos para realizar? Como você deve alocar esses projetos para os programadores? E se você tiver um projeto realmente grande para terminar?

## CAPÍTULO 13 EQUIPES E PROJETOS

### D O E S I T B L E N D?

Tenho prestado consultoria para vários bancos e seguradoras ao longo dos anos.

Uma coisa que eles parecem ter em comum é a maneira estranha como particionam os projetos.

Muitas vezes, um projeto em um banco será um trabalho relativamente pequeno que requer um ou dois programadores por algumas semanas. Este projeto geralmente será composto por um projeto gerente, que também gerencia outros projetos. Será composto por uma empresa analista, que também está fornecendo requisitos para outros projetos. Será dotado de pessoal com alguns programadores que também estão trabalhando em outros projetos. Um testador ou dois serão designados e também trabalharão em outros projetos.

Veja o padrão? O projeto é tão pequeno que nenhum indivíduo pode ser designado para ele em tempo integral. Todo mundo está trabalhando no projeto aos 50, ou mesmo aos 25, por cento.

Agora, aqui está uma regra: não existe metade de uma pessoa.

Não faz sentido dizer a um programador para dedicar metade do seu tempo ao projeto A e o resto do tempo para o projeto B, especialmente quando os dois projetos têm dois diferentes gerentes de projeto, diferentes analistas de negócios, diferentes programadores, e testadores diferentes. Como na cozinha do Inferno você pode chamar uma monstruosidade dessas de equipe? Isso não é uma equipe, é algo que saiu de um liquidificador Waring.

### A EQUIPE G E L L E D

Leva tempo para uma equipe se formar. Os membros da equipe começam a formar relacionamentos.

Eles aprendem como colaborar uns com os outros. Eles aprendem as peculiaridades um do outro, pontos fortes e fracos. Eventualmente, a equipe começa a se unir.

Há algo verdadeiramente mágico em uma equipe formada. Eles podem fazer milagres.

Eles se antecipam, se protegem, se apoiam e

exigir o melhor um do outro. Eles fazem as coisas acontecerem.

Uma equipe formada geralmente consiste em cerca de uma dúzia de pessoas. Poderia ser tantos quanto vinte ou apenas três, mas o melhor número é provavelmente cerca de doze. O

## MISTURA?

169

a equipe deve ser composta por programadores, testadores e analistas. E deveria ter um gerente de projeto.

A proporção de programadores para testadores e analistas pode variar muito, mas 2:1 é uma proporção bom número. Portanto, uma equipe bem formada de doze pessoas pode ter sete programadores, dois testadores, dois analistas e um gerente de projeto.

Os analistas desenvolvem os requisitos e escrevem testes de aceitação automatizados para eles. Os testadores também escrevem testes de aceitação automatizados. A diferença entre os dois são perspectiva. Ambos são requisitos de escrita. Mas os analistas se concentram em valor comercial; os testadores se concentram na correção. Os analistas escrevem os casos do caminho feliz;

os testadores se preocupam com o que pode dar errado e escrevem a falha e o limite casos.

O gerente de projeto acompanha o progresso da equipe e garante que a equipe entende os cronogramas e prioridades.

Um dos membros da equipe pode desempenhar um papel de treinador ou mestre em tempo parcial, com responsabilidade pela defesa do processo e das disciplinas da equipe. Eles atuam como consciência da equipe quando a equipe é tentada a sair do processo por causa de pressão de cronograma.

### Fermentação

Leva tempo para uma equipe como essa resolver suas diferenças, chegar a um acordo uns com os outros, e realmente gel. Pode levar seis meses. Pode até levar um ano. Mas quando isso acontece, é mágico. Uma equipe formada planejará em conjunto, resolverá problemas juntos, enfrentar problemas juntos e fazer as coisas.

Quando isso acontece, é ridículo separá-lo só porque um projeto chega ao fim. É melhor manter a equipe unida e continuar alimentando-a projetos.

O que veio primeiro, a equipe ou o projeto?

Bancos e seguradoras tentaram formar equipes em torno dos projetos. Este é um abordagem tola. As equipes simplesmente não conseguem se unir. Os indivíduos estão apenas no

projeto por um curto período de tempo e apenas por uma porcentagem do seu tempo e, portanto, nunca aprendam a lidar uns com os outros.

Organizações de desenvolvimento profissional alocam projetos para projetos existentes equipes, eles não formam equipes em torno de projetos. Uma equipe formada pode aceitar muitos projetos simultaneamente e dividirão o trabalho de acordo com suas próprias opiniões, habilidades e habilidades. A equipe formada realizará os projetos.

#### MAS COMO VOCÊ GERE ISSO?

As equipes têm velocidades.<sup>1</sup> A velocidade de uma equipe é simplesmente a quantidade de trabalho que ela

pode ser feito em um período fixo de tempo. Algumas equipes medem sua velocidade em pontos por semana, onde os pontos são uma unidade de complexidade. Eles quebram o características de cada projeto em que estão trabalhando e estime-as em pontos. Então eles medem quantos pontos são realizados por semana.

A velocidade é uma medida estatística. Uma equipe pode conseguir 38 pontos em uma semana, 42 feito o próximo e 25 feito o próximo. Com o tempo, isso diminuirá.

A gestão pode definir metas para cada projeto entregue a uma equipe. Por exemplo, se a velocidade média de uma equipe é 50 e eles têm três projetos em que estão trabalhando em diante, a gerência pode pedir à equipe que divida seu esforço em 15, 15 e 20.

Além de ter uma equipe formada trabalhando em seus projetos, a vantagem deste esquema é que, em caso de emergência, a empresa pode dizer: “O Projeto B está em crise; coloque 100% do seu esforço nesse projeto pelas próximas três semanas.”

Realocar prioridades tão rapidamente é praticamente impossível com as equipes que saiu do liquidificador, mas consolidou equipes que estão trabalhando em dois ou três projetos simultaneamente podem custar um centavo.

#### O PROJETO O W N E R D I L E M MA

Uma das objeções à abordagem que defendo é que os proprietários do projeto perder alguma segurança e poder. Proprietários de projetos que possuem uma equipe dedicada a

1. [RCM2003] pp. [COHN2006] Procure no índice muitas referências excelentes à velocidade.

#### CAPÍTULO 13 EQUIPES E PROJETOS



## BIBLIOGRAFIA

seu projeto pode contar com o esforço dessa equipe. Eles sabem disso porque formar e desmembrar uma equipe é uma operação cara, o negócio não afastar a equipe por motivos de curto prazo.

Por outro lado, se os projetos forem entregues a equipes formadas e se essas equipes assumirem em vários projetos ao mesmo tempo, então a empresa é livre para mudar prioridades por capricho. Isso pode deixar o proprietário do projeto inseguro sobre o futuro. Os recursos dos quais o proprietário do projeto depende podem ser subitamente removido dele.

Francamente, prefiro a última situação. A empresa não deve ficar de mãos atadas pela dificuldade artificial de formação e dissolução de equipes. Se o negócio decidir que um projeto tem maior prioridade que outro, deverá ser capaz de realocar recursos rapidamente. É responsabilidade do proprietário do projeto fazer com que caso para seu projeto.

## CONCLUSÃO

Equipes são mais difíceis de construir do que projetos. Portanto, é melhor formar persistente equipes que se movem juntas de um projeto para outro e podem assumir mais mais de um projeto por vez. O objetivo de formar uma equipe é dar a essa equipe tempo suficiente para gelificar e, em seguida, mantê-lo unido como um motor para obter muitos projetos realizados.

## BIBLIOGRAPHY

[RCM2003]: Robert C. Martin, Desenvolvimento Ágil de Software: Princípios, Padrões, e Práticas, Upper Saddle River, NJ: Prentice Hall, 2003.

[COHN2006]: Mike Cohn, Estimativa e Planejamento Ágil, Upper Saddle River, Nova Jersey: Prentice Hall, 2006.

Esta página foi intencionalmente deixada em branco

173

14

MENTORIA,

APRENDIZAGEM, E D

CR AFTS MAN S HIP

Tenho ficado constantemente decepcionado com a qualidade dos graduados em ciências da computação.

Não é

que os formandos não são brilhantes ou talentosos, só que não foram

ensinou o que realmente é programação.

## DEGREES OF FAILURE

Certa vez entrevistei uma jovem que estava fazendo mestrado em ciência da computação para uma grande universidade. Ela estava se candidatando a um estagiário de verão

posição. Pedi a ela para escrever algum código comigo e ela disse: “Eu realmente não escrever código.”

Por favor, leia o parágrafo anterior novamente e pule este para o próximo.

Perguntei a ela quais cursos de programação ela havia feito para obter seu mestrado grau. Ela disse que não tinha tomado nenhum.

Talvez você queira começar no início do capítulo só para ter certeza de que não caiu em algum universo alternativo ou acabou de acordar de um pesadelo.

Neste ponto você deve estar se perguntando como um aluno de um mestrado em Ciência da Computação programa pode evitar um curso de programação. Eu me perguntei a mesma coisa no tempo. Ainda estou me perguntando hoje.

Claro, essa é a mais extrema de uma série de decepções que tive enquanto entrevistando graduados. Nem todos os graduados em ciências da computação são decepcionantes – longe disso!

No entanto, percebi que aqueles que não têm algo em comum: quase todos eles aprenderam a programar sozinhos antes de entrarem na universidade e continuaram a ensinar sozinhos apesar da universidade.

Agora não me interpretem mal. Acredito que é possível obter uma excelente educação em um universidade. Só que também acho que é possível se movimentar através do sistema e sair com um diploma, e nada mais.

E há outro problema. Mesmo os melhores programas de graduação em CS normalmente não preparar o jovem graduado para o que encontrará na indústria. Isto não é um acusação dos programas de graduação, mas é a realidade de quase todas as disciplinas.

O que você aprende na escola e o que você encontra no trabalho são muitas vezes coisas muito diferentes.

## MENTORING

Como aprendemos a programar? Deixe-me contar minha história sobre ser mentorado.

## DIGI-COMP I, MEU PRIMEIRO COMPUTADOR

Em 1964, minha mãe me deu um pequeno computador de plástico no meu aniversário de 12 anos. Isso foi chamado de Digi-Comp I.1. Tinha três chinelos de plástico e seis de plástico e-portões. Você poderia conectar as saídas dos flip-flops às entradas do e-

portões. Você também pode conectar a saída das portas and às entradas do chinelos. Resumindo, isso permitiu criar uma máquina de estados finitos de três bits.

O kit veio com um manual que continha vários programas para executar. Você programou a máquina empurrando pequenos tubos (pequenos segmentos de canudos de refrigerante) em pequenos pinos salientes dos chinelos. O manual lhe disse exatamente onde colocar cada tubo, mas não o que os tubos faziam. Achei isso muito frustrante!

Fiquei olhando para a máquina por horas e determinei como ela funcionava no nível mais baixo nível; mas eu não conseguia, nem de jeito nenhum, descobrir como fazê-lo fazer o que

Eu queria que isso acontecesse. A última página do manual me dizia para enviar um dólar e eles me enviariam um manual dizendo como programar a máquina.<sup>2</sup>

Enviei meu dólar e esperei com a impaciência de uma criança de doze anos. O dia o manual chegou eu o devorei. Foi um tratado simples sobre álgebra booleana cobrindo fatoração básica de equações booleanas, leis associativas e distributivas, e o teorema de DeMorgan. O manual mostrou como expressar um problema em termos de uma sequência de equações booleanas. Também descreveu como reduzir essas equações para caber em 6 portas and.

Concebi meu primeiro programa. Ainda me lembro do nome: Sr. Patterson's Portão Informatizado. Eu escrevi as equações, reduzi-as e mapeei-as para os tubos e pinos da máquina. E funcionou!

Escrever essas três palavras agora causou arrepios na minha espinha. Os mesmos arrepios que percorreu aquela criança de doze anos há quase meio século. Eu estava viciado.

Minha vida nunca mais seria a mesma.

Você se lembra do momento em que seu primeiro programa funcionou? Isso mudou o seu vida ou colocá-lo em um caminho do qual você não poderia se desviar?

1. Existem muitos sites que oferecem simuladores deste pequeno e estimulante computador.
2. Ainda tenho este manual. Ele ocupa um lugar de honra em uma de minhas estantes.

Eu não descobri tudo sozinho. Fui orientado. Alguns muito gentis e muito pessoas adeptas (com quem tenho uma enorme dívida de gratidão) reservaram um tempo para escrever um

tratado sobre álgebra booleana acessível a uma criança de doze anos. Eles conectou a teoria matemática à pragmática do pequeno plástico computador e me capacitou a fazer com que aquele computador fizesse o que eu queria. Acabei de baixar minha cópia daquele manual fatídico. Eu guardo em um saco zip-lock. No entanto, os anos cobraram o seu preço, amarelando as páginas e tornando-as eles quebradiços. Ainda assim, o poder das palavras brilha neles. A elegância de sua descrição da álgebra booleana consumiu três páginas esparsas. Seu passo-a-passo das equações de cada um dos programas originais ainda é convincente. Foi um trabalho de maestria. Foi um trabalho que mudou pelo menos um vida do jovem. No entanto, duvido que nunca saberei os nomes dos autores.

#### O ECP-18 EM HIGH SCHOOL

Aos quinze anos, quando era calouro no ensino médio, eu gostava de ficar no departamento de matemática. (Vai entender!) Um dia eles trouxeram uma máquina do tamanho de um serra de mesa. Era um computador educacional feito para escolas secundárias, chamado ECP-18. Nossa escola estava recebendo uma demonstração de duas semanas. Fiquei ao fundo enquanto os professores e técnicos conversavam. Esta máquina tinha uma palavra de 15 bits (o que é uma palavra?) e uma memória de bateria de 1.024 palavras. (eu sabia o que era a memória de bateria naquela época, mas apenas em conceito.) Quando eles o ligaram, ele emitiu um som que lembrava um avião a jato decolando. Imaginei que fosse o tambor girando. Uma vez atualizado, foi relativamente quieto.

A máquina era adorável. Era essencialmente uma mesa de escritório com um maravilhoso painel de controle projetando-se do topo como a ponte de um navio de guerra. O o painel de controle era adornado com fileiras de luzes que também funcionavam como botões. Sentar naquela mesa era como sentar na cadeira do Capitão Kirk.

Enquanto observava os técnicos apertarem esses botões, notei que eles acenderam quando empurrados e que você pode empurrá-los novamente para desligá-los. Notei também que há havia outros botões que eles apertavam; botões com nomes como depósito e execução.

## MENTORIA

177

Os botões em cada linha foram agrupados em cinco grupos de três. Meu Digi-Comp também tinha três bits, então eu poderia ler um dígito octal quando expresso em binário. Não foi um grande salto perceber que se tratava apenas de cinco dígitos octais. Enquanto os técnicos apertavam os botões, pude ouvi-los murmurar para si mesmos. Eles pressionariam 1, 5, 2, 0, 4 na linha do buffer de memória enquanto diziam para eles próprios, “armazenam em 204”. Eles pressionavam 1, 0, 2, 1, 3 e murmuravam: “carga 213 no acumulador.” Havia uma fileira de botões chamados acumulador!

Dez minutos disso e ficou bem claro para minha mente de quinze anos que o 15 significava armazenamento e 10 significava carga, que o acumulador era o que estava sendo armazenados ou carregados, e que os outros números eram os números de um dos 1024 palavras no tambor. (Então isso é uma palavra!)

Aos poucos (sem trocadilhos), minha mente ansiosa se apegou a mais e mais códigos e conceitos de instrução. Quando os técnicos saíram, eu sabia o noções básicas de como aquela máquina funcionava.

Naquela tarde, durante uma sala de estudos, entrei no laboratório de matemática e comecei mexendo no computador. Eu aprendi há muito tempo que é melhor perguntar perdão do que permissão! Eu alternei um pequeno programa que iria multiplicar o acumulador por dois e adicione um. Coloquei um 5 no acumulador, executei o programa, e vi 138 no acumulador! Tinha funcionado!

Ativei vários outros programas simples como esse e todos funcionaram como planejado. Eu era o dono do universo!

Dias depois, percebi o quão estúpido e sortudo eu tinha sido. encontrei uma instrução folha espalhada no laboratório de matemática. Mostrou todas as diferentes instruções e códigos de operação, incluindo muitos que não aprendi observando os técnicos. eu estava satisfeito por ter interpretado corretamente aqueles que conhecia e emocionado com a outros. Porém, uma das novas instruções foi o HLT. Aconteceu que a instrução de parada era uma palavra só com zeros. E aconteceu que eu tinha coloquei uma palavra com zeros no final de cada um dos meus programas para que eu pudesse carregá-lo no acumulador para limpá-lo. O conceito de parada simplesmente não ocorreu para mim. Achei que o programa iria parar quando terminasse!

Lembro-me de que, a certa altura, estava sentado no laboratório de matemática observando um dos professores

lutar para fazer um programa funcionar. Ele estava tentando digitar dois números decimal no teletipo anexado e depois imprima a soma. Qualquer um que tenha tentei escrever um programa como este em linguagem de máquina em um minicomputador sabe que está longe de ser trivial. Você tem que ler os personagens, convertê-los para dígitos, depois para binário, adicione-os, converta novamente para decimal e codifique novamente em personagens. E, acredite, é muito pior quando você está entrando no programa em binário através do painel frontal!

Observei enquanto ele interrompeu seu programa e o executou até parar.

(Oh! Essa é uma boa ideia!) Esse ponto de interrupção primitivo permitiu que ele examinasse o conteúdo dos registradores para ver o que seu programa havia feito. eu me lembro dele murmurando: “Uau, isso foi rápido!” Rapaz, tenho novidades para ele!

Eu não tinha ideia de qual era o algoritmo dele. Esse tipo de programação ainda era mágico para mim. E ele nunca falou comigo enquanto eu observava por cima do ombro. Na verdade, ninguém falou comigo sobre este computador. Eu acho que eles me consideraram um incômodo ser ignorado, flutuando pelo laboratório de matemática como uma mariposa. Basta dizer que nem o aluno nem os professores desenvolveram um alto grau de habilidade social.

No final, ele conseguiu que seu programa funcionasse. Foi incrível assistir. Ele lentamente digite os dois números porque, apesar de seu protesto anterior, aquele computador não foi rápido (pense em ler palavras consecutivas de um tambor girando em 1967). Quando ele apertou Enter após o segundo número, o computador piscou ferozmente por um tempo e então comecei a imprimir o resultado. Demorou cerca de um segundo por dígito. Imprimiu tudo menos o último dígito, piscou ainda mais ferozmente por cinco segundos e depois imprimiu o dígito final e parou.

Por que essa pausa antes do último dígito? Eu nunca descobri. Mas isso me fez perceber que a abordagem de um problema pode ter um efeito profundo no usuário. Mesmo que o programa produziu a resposta correta, ainda havia algo errado com ele.

Isso foi mentoria. Certamente não foi o tipo de orientação que eu poderia ter esperava. Teria sido bom se um desses professores tivesse me levado para sua ala e trabalhou comigo. Mas isso não importava, porque eu estava observando eles e aprendendo em um ritmo furioso.



## MENTORIA

179

### U N C N V E N T I O N A L M E N T O R I N G

Eu contei essas duas histórias porque elas descrevem dois tipos muito diferentes de mentoria, nenhum dos quais é o tipo que a palavra geralmente implica. No primeiro caso aprendi com os autores de um manual muito bem escrito. No segundo caso, aprendi observando pessoas que tentavam ativamente ignorar eu. Em ambos os casos, o conhecimento adquirido foi profundo e fundamental. Claro, também tive outros tipos de mentores. Lá estava o vizinho gentil que trabalhava na Teletype, que me trouxe para casa uma caixa com 30 relés telefônicos para brincar com. Deixe-me dizer, dê a um rapaz alguns relés e um transformador de trem elétrico e ele pode conquistar o mundo!

Havia um vizinho gentil, operador de radioamador, que me mostrou como fazer use um multímetro (que quebrei imediatamente). Havia a loja de material de escritório proprietário que me permitiu entrar e “brincar” com seu caríssimo calculadora programável. Houve as vendas da Digital Equipment Corporation escritório que me permitiu entrar e “brincar” com seu PDP-8 e PDP-10.

Depois houve o grande Jim Carlin, um programador BAL que me salvou de ser demitido do meu primeiro trabalho de programação, ajudando-me a depurar um programa Cobol isso estava muito além da minha profundidade. Ele me ensinou como ler core dumps e como formatar meu código com linhas em branco apropriadas, linhas de estrelas e comentários. Ele me deu meu primeiro impulso em direção ao artesanato. Me desculpe, eu poderia não retribuiu o favor quando o descontentamento do chefe recaiu sobre ele um ano depois. Mas, francamente, é só isso. Simplesmente não havia muitos programadores seniores em início dos anos setenta. Em todos os outros lugares onde trabalhei, eu era sênior. Não havia ninguém para ajude-me a descobrir o que era a verdadeira programação profissional. Não havia nenhum papel modelo que me ensinou como me comportar ou o que valorizar. Essas coisas que eu tive que aprender por mim mesmo, e não foi nada fácil.

### H A R D K N O C K S

Como eu disse antes, na verdade fui demitido daquele emprego de automação de fábrica em 1976. Embora fosse tecnicamente muito competente, não tinha aprendido a pagar atenção ao negócio ou aos objetivos do negócio. Datas e prazos significavam

nada para mim. Esqueci de uma grande demonstração na manhã de segunda-feira e deixei o sistema quebrado

na sexta-feira, e apareci na segunda-feira com todos me olhando com raiva.

Meu chefe me enviou uma carta avisando que eu precisava fazer alterações imediatamente ou ser demitido. Este foi um alerta significativo para mim. Reavaliei minha vida e carreira e comecei a fazer algumas mudanças significativas em meu comportamento - algumas delas sobre o qual você leu neste livro. Mas era muito pouco, muito tarde.

O impulso estava na direção errada e pequenas coisas que não aconteceriam importaram antes se tornaram significativos. Então, embora eu tenha tentado, eles eventualmente me acompanharam para fora do prédio.

Escusado será dizer que não é divertido levar esse tipo de notícia para uma esposa grávida.

e uma filha de dois anos. Mas eu me recompos e tomei alguns poderosos lições de vida para meu próximo emprego - que ocupei por quinze anos e que formou o verdadeira base da minha carreira atual.

No final, sobrevivi e prosperei. Mas tem que haver uma maneira melhor. Teria teria sido muito melhor para mim se eu tivesse um verdadeiro mentor, alguém para me ensinar os detalhes e

fora. Alguém que eu poderia ter observado enquanto o ajudava em pequenas tarefas e que revisaria e orientaria meu trabalho inicial. Alguém para servir de modelo e ensinar me valores e reflexos apropriados. Um sensei. Um mestre. Um mentor.

#### APRENTICESHIP

O que os médicos fazem? Você acha que os hospitais contratam graduados em medicina e lançam levá-los às salas de cirurgia para fazer uma cirurgia cardíaca no primeiro dia de trabalho? De claro que não.

A profissão médica desenvolveu uma disciplina de intensa orientação

abrigado em ritual e lubrificado com tradição. A profissão médica

supervisiona as universidades e garante que os graduados tenham a melhor educação.

Que a educação envolve quantidades aproximadamente iguais de estudo em sala de aula e atividade em hospitais que trabalham com profissionais.

Após a formatura, e antes de serem licenciados, os médicos recém-formados são obrigado a passar um ano em prática supervisionada e treinamento denominado estágio.

Este é um treinamento intenso no trabalho. O estagiário está rodeado de modelos e professores.

Concluído o estágio, cada uma das especialidades médicas exige mais três a cinco anos de prática e treinamento supervisionados adicionais, conhecidos como residência. O morador ganha confiança ao assumir responsabilidades cada vez maiores enquanto ainda está cercado e supervisionado por médicos experientes.

Muitas especialidades exigem ainda mais um a três anos de bolsa em que o aluno continua o treinamento especializado e a prática supervisionada.

E então eles são elegíveis para fazer os exames e obter a certificação do conselho.

Esta descrição da profissão médica foi um tanto idealizada e provavelmente totalmente impreciso. Mas permanece o facto de que quando as apostas são altas, não mandamos graduados para uma sala, jogamos carne de vez em quando e esperamos coisas boas para sair. Então, por que fazemos isso em software?

É verdade que há relativamente poucas mortes causadas por bugs de software. Mas lá são perdas monetárias significativas. As empresas perdem enormes quantias de dinheiro devido a o treinamento inadequado de seus desenvolvedores de software.

De alguma forma, a indústria de desenvolvimento de software teve a ideia de que a programação mers são programadores e, depois de se formar, você pode codificar. Na verdade, não é Não é nada comum que as empresas contratem crianças logo que saem da escola, transformem-nas em “equipes” e peça-lhes que construam os sistemas mais críticos. É uma loucura!

Pintores não fazem isso. Encanadores não. Eletricistas não. Inferno, eu nem acho que os cozinheiros de fast food se comportam dessa maneira! Parece-me que as empresas que contratar graduados em ciências da computação deveria investir mais em seu treinamento do que o McDonalds

investe em seus servidores.

Não vamos nos enganar pensando que isso não importa. Há muito em jogo. Nosso a civilização funciona com software. É um software que move e manipula o informações que permeiam nosso dia a dia. O software controla nosso automóvel motores, transmissões e freios. Ele mantém nossos saldos bancários, nos envia nossos

contas e aceita nossos pagamentos. O software lava nossas roupas e nos diz o tempo. Ele coloca fotos na TV, envia nossas mensagens de texto, faz nossas ligações, e nos diverte quando estamos entediados. Está em todo lugar.

Dado que confiamos aos desenvolvedores de software todos os aspectos de nossas vidas, desde o minúcia ao importante, sugiro que um período razoável de treinamento e a prática supervisionada não é inadequada.

### S O F T W R E A P P R E N T I C E S H I P

Então, como a profissão de software deveria incluir jovens graduados nas fileiras de profissionalismo? Que passos eles devem seguir? Que desafios eles deveriam conhecer? Que objetivos eles deveriam alcançar? Vamos trabalhar de trás para frente.

#### Mestres

Estes são programadores que assumiram a liderança em mais de um projeto de software. Normalmente eles terão mais de 10 anos de experiência e terão trabalhado em vários tipos diferentes de sistemas, linguagens e sistemas operacionais. Eles sabem liderar e coordenar múltiplas equipes, são designers proficientes e arquitetos, e pode codificar círculos em torno de todos os outros sem quebrar o suor. Foram-lhes oferecidos cargos de gestão, mas ou viraram derrubá-los, fugir depois de aceitá-los ou integrá-los com seu papel principalmente técnico. Eles mantêm esse papel técnico lendo, estudar, praticar, fazer e ensinar. É para um mestre que a empresa irá atribuir responsabilidade técnica para um projeto. Pense, “Scotty”.

#### Viajantes

São programadores treinados, competentes e enérgicos. Durante este período de sua carreira, eles aprenderão a trabalhar bem em equipe e a se tornarem equipes líderes. Eles têm conhecimento sobre a tecnologia atual, mas normalmente não têm experiência com muitos sistemas diversos. Eles tendem a conhecer um idioma, um sistema, uma plataforma; mas eles estão aprendendo mais. Os níveis de experiência variam amplamente entre suas fileiras, mas a média é de cerca de cinco anos. Do outro lado nessa média temos mestres florescentes; no lado mais próximo, temos recentes aprendizes.

## APRENDIZAGEM

183

Os jornalheiros são supervisionados por mestres ou outros jornalheiros mais experientes.

Os jovens jornalheiros raramente têm autonomia. Seu trabalho está intimamente supervisionado. Seu código é examinado. À medida que ganham experiência, autonomia cresce. A supervisão torna-se menos direta e mais matizada. Eventualmente transições para a revisão por pares.

### Aprendizes/Estagiários

Os graduados iniciam suas carreiras como aprendizes. Os aprendizes não têm autonomia. Eles são supervisionados de perto por jornalheiros. No início eles não realizam tarefas em ao todo, eles simplesmente prestam assistência aos jornalheiros. Este deveria ser um momento de programação em pares muito intensa. É quando as disciplinas são aprendidas e reforçado. É quando a base de valores é criada.

Os jornalheiros são os professores. Eles garantem que os aprendizes conheçam o design princípios, padrões de design, disciplinas e rituais. Profissionais ensinam TDD, refatoração, estimativa e assim por diante. Eles atribuem leituras, exercícios e práticas aos aprendizes; eles revisam seu progresso.

O aprendizado deve durar um ano. A essa altura, se os jornalheiros estiverem dispostos aceitar o aprendiz em suas fileiras, eles farão uma recomendação para os mestres. Os mestres devem examinar o aprendiz tanto por entrevista quanto revisando suas realizações. Se os mestres concordarem, então o aprendiz torna-se um viajante.

### A R E A LIDADE

Novamente, tudo isso é idealizado e hipotético. No entanto, se você alterar o nomes e semicerrar os olhos para as palavras, você perceberá que não é tão diferente de a maneira como esperamos que as coisas funcionem agora. Os graduados são supervisionados por jovens equipes

líderes, que são supervisionados por líderes de projeto e assim por diante. O problema é que, em na maioria dos casos, esta supervisão não é técnica! Na maioria das empresas não há supervisão técnica. Programadores recebem aumentos e eventuais promoções porque, bem, isso é exatamente o que você faz com programadores.

A diferença entre o que fazemos hoje e meu programa idealizado de aprendizagem

O estágio é o foco no ensino técnico, treinamento, supervisão e revisão.

A diferença é a própria noção de que os valores profissionais e a perspicácia técnica deve ser ensinado, nutrido, nutrido, mimado e inculcado. O que está faltando da nossa atual abordagem estéril é responsabilidade dos mais velhos ensinar o jovem.

### ARTESANATO MAN S H I P

Portanto, agora estamos em condições de definir esta palavra: artesanato. O que é isso? Para entender, vamos dar uma olhada na palavra artesão. Esta palavra traz à mente habilidade e qualidade. Evoca experiência e competência. Um artesão é alguém que trabalha rapidamente, mas sem pressa, que fornece estimativas razoáveis e cumpre compromissos. Um artesão sabe quando dizer não, mas se esforça para dizer sim. Um artesão é um profissional.

Artesanato é a mentalidade dos artesãos. Artesanato é um meme que contém valores, disciplinas, técnicas, atitudes e respostas.

Mas como os artesãos adotam esse meme? Como eles atingem essa mentalidade?

O meme do artesanato é passado de uma pessoa para outra. É ensinado por mais velhos para os jovens. É trocado entre pares. É observado e reaprendido, como os mais velhos observam os jovens. O artesanato é um contágio, uma espécie de vírus mental. Você percebe isso observando os outros e permitindo que o meme tome conta.

### CO N V I N C I N G PESSOAS

Você não pode convencer as pessoas a serem artesãos. Você não pode convencê-los a aceitar o meme do artesanato. Os argumentos são ineficazes. Os dados são irrelevantes.

Os estudos de caso não significam nada. A aceitação de um meme não é tanto uma questão racional decisão como emocional. Isso é uma coisa muito humana.

Então, como fazer com que as pessoas adotem o meme do artesanato? Lembre-se que um meme é contagioso, mas apenas se puder ser observado. Então você faz o meme observável. Você atua como um modelo. Você se torna um artesão primeiro e deixa seu mostra de artesanato. Depois é só deixar o meme fazer o resto do trabalho.

## CONCLUSÃO

185

## CONCLUSÃO

A escola pode ensinar a teoria da programação de computadores. Mas a escola não, e não pode ensinar a disciplina, a prática e a habilidade de ser um artesão. Aqueles as coisas são adquiridas através de anos de tutela e orientação pessoal. Está na hora para nós da indústria de software enfrentarmos o fato de que orientar o próximo lote de desenvolvedores de software até a maturidade recairá sobre nós, não sobre as universidades. É hora de adotarmos um programa de aprendizagem, estágio e longo prazo orientação.

Esta página foi intencionalmente deixada em branco



Um

## FERRAMENTAS

Em 1978, eu estava trabalhando na Teradyne no sistema de teste telefônico que descrevi anteriormente. O sistema tinha cerca de 80KSLOC do montador M365. Mantivemos a fonte código em fitas.

As fitas eram semelhantes aos cartuchos de fita estéreo de 8 pistas que eram tão popular nos anos 70. A fita era um loop infinito e a unidade de fita podia mover-se apenas em uma direção. Os cartuchos vieram em comprimentos de 10', 25', 50' e 100'. Quanto mais longa a fita, mais tempo demorava para “rebobinar”, já que a unidade de fita precisava simplesmente mover o para frente até encontrar o “ponto de carga”. Uma fita de 100' levou cinco minutos para chegar ao ponto de carregamento, então escolhemos criteriosamente a duração de nossas fitas.<sup>1</sup>

1. Essas fitas só podiam ser movidas em uma direção. Então, quando ocorria um erro de leitura, não havia como o unidade de fita para fazer backup e ler novamente. Você teve que parar o que estava fazendo, mandar a fita de volta para a carga ponto e, em seguida, comece novamente. Isso acontecia duas ou três vezes por dia. Erros de escrita também eram muito comuns, e a unidade não tinha como detectá-los. Então sempre escrevemos as fitas em pares e depois verificamos os pares quando terminamos. Se uma das fitas estivesse ruim, fazíamos imediatamente uma cópia. Se ambos fossem ruins, o que era muito raro, reiniciamos toda a operação. Assim era a vida nos anos 70.

## APÊNDICE A FERRAMENTAS

Logicamente, as fitas foram subdivididas em arquivos. Você poderia ter tantos arquivos em um fita conforme caberia. Para encontrar um arquivo, você carregou a fita e avançou um arquivo por vez até encontrar o que deseja. Mantivemos uma lista dos diretório do código-fonte na parede para que possamos saber quantos arquivos pular antes de chegarmos ao que queríamos.

Havia uma cópia master de 100' da fita do código-fonte em uma prateleira do laboratório. Foi rotulado como Mestre. Quando queríamos editar um arquivo carregamos a fonte Master fita em uma unidade e um espaço em branco de 10' em outra. Nós pularíamos o Master até chegarmos ao arquivo que precisávamos. Em seguida, copiaríamos esse arquivo para a fita adesiva. Depois “rebobinávamos” ambas as fitas e colocávamos a Master de volta na prateleira.

Havia uma lista especial do Mestre em um quadro de avisos no laboratório.

Depois de fazermos as cópias dos arquivos que precisávamos editar, colocamos um colorido fixe no quadro ao lado do nome desse arquivo. Foi assim que verificamos os arquivos!

Editamos as fitas em uma tela. Nosso editor de texto, ED-402, era realmente muito bom. Era muito semelhante ao vi. Líamos uma “página” da fita, editávamos a conteúdo e, em seguida, escreva essa página e leia a próxima. Uma página foi normalmente 50 linhas de código. Você não poderia olhar para frente na fita para ver as páginas que estavam por vir, e você não poderia olhar para trás na fita para ver as páginas que você havia editado. Então usamos listagens.

Na verdade, marcaríamos nossas listagens com todas as alterações que queríamos fazer, e então editaríamos os arquivos de acordo com nossas marcações. Ninguém escreveu ou código modificado no terminal! Isso foi suicídio.

Depois que as alterações fossem feitas em todos os arquivos que precisávamos editar, nós os mesclaríamos

arquivos com o mestre para criar uma fita de trabalho. Esta é a fita que usaríamos para rodar nossas compilações e testes.

Assim que terminarmos os testes e tivermos certeza de que nossas alterações funcionaram, analisariamos o

placa. Se não houvesse novos pinos no quadro, simplesmente renomeariamos nossos trabalhos grave como Mestre e retire nossos pinos do tabuleiro. Se houvesse novos pinos no quadro, removeríamos nossos alfinetes e entregariamos nossa fita de trabalho para a pessoa cujo os pinos ainda estavam no tabuleiro. Eles teriam que fazer a fusão.

## CONTROLE DE CÓDIGO FONTE

Éramos três e cada um tinha sua própria cor de alfinete, então foi fácil para sabermos quem fez check-out de quais arquivos. E como todos nós trabalhamos no mesmo laboratório e conversamos o tempo todo, mantivemos o status do conselho em nossas cabeças. Geralmente o quadro era redundante e muitas vezes não o usávamos.

### MUITO LS

Hoje, os desenvolvedores de software têm uma ampla variedade de ferramentas para escolher. A maioria não vale a pena se envolver, mas há alguns que todo software o desenvolvedor deve estar familiarizado. Este capítulo descreve minha experiência pessoal atual kit de ferramentas. Não fiz um levantamento completo de todas as outras ferramentas disponíveis, então isto não deve ser considerado uma revisão abrangente. Isso é exatamente o que eu uso.

### S O U R C E C O D E C O N T R O L

Quando se trata de controle de código-fonte, as ferramentas de código aberto geralmente são o seu melhor opção. Por que? Porque são escritos por desenvolvedores, para desenvolvedores. O ferramentas de código aberto são o que os desenvolvedores escrevem para si mesmos quando precisam algo que funciona.

Existem alguns controles de versão “empresariais” caros e comerciais sistemas disponíveis. Acho que eles não são vendidos tanto para desenvolvedores quanto para são vendidos para gerentes, executivos e “grupos de ferramentas”. Sua lista de recursos é impressionante e convincente. Infelizmente, muitas vezes eles não têm os recursos que os desenvolvedores realmente precisam. O principal deles é a velocidade.

### UM SISTEMA DE C O N T R O L “E N T E R P R I S E” S O U R C E

Pode ser que sua empresa tenha investido uma pequena fortuna em um “empreendimento” sistema de controle de código-fonte. Se sim, minhas condolências. Provavelmente é politicamente Não é apropriado você sair por aí dizendo a todos: “Tio Bob diz para não usar isso.” No entanto, existe uma solução fácil.

Você pode verificar seu código-fonte no sistema “empresarial” no final de cada iteração (uma vez a cada duas semanas ou mais) e usar um dos sistemas de código aberto

no meio de cada iteração. Isso mantém todos felizes, não viola nenhuma regras corporativas e mantém sua produtividade alta.

#### PESSIMISTA VERSUS PTIMISTIC LOCKING

O bloqueio pessimista parecia uma boa ideia nos anos 80. Afinal, o mais simples

Uma maneira de gerenciar problemas de atualização simultânea é serializá-los. Então, se eu estiver editando um arquivo, é melhor não. Na verdade, o sistema de alfinetes coloridos que usei em o final dos anos 70 foi uma forma de bloqueio pessimista. Se houvesse um alfinete em um arquivo, você não editei esse arquivo.

É claro que o bloqueio pessimista tem os seus problemas. Se eu bloquear um arquivo e continuar férias, todo mundo que quiser editar esse arquivo ficará preso. Na verdade, mesmo que eu manter o arquivo bloqueado por um ou dois dias, posso atrasar outras pessoas que precisam fazer mudanças.

Nossas ferramentas ficaram muito melhores na mesclagem de arquivos de origem que foram editados simultaneamente. Na verdade, é incrível quando você pensa sobre isso. As ferramentas parecem nos dois arquivos diferentes e no ancestral desses dois arquivos, e então eles aplicar múltiplas estratégias para descobrir como integrar as mudanças simultâneas. E eles fazem um trabalho muito bom.

Assim, a era do bloqueio pessimista acabou. Não precisamos bloquear arquivos quando confira mais. Na verdade, não nos preocupamos em verificar arquivos individuais de jeito nenhum. Apenas verificamos todo o sistema e editamos todos os arquivos necessários. Quando estivermos prontos para verificar nossas alterações, realizamos uma operação de “atualização”. Isso nos diz se alguém mais fez check-in do código antes de nós, automaticamente mescla a maioria das mudanças, encontra conflitos e nos ajuda a fazer o restante funde. Em seguida, confirmamos o código mesclado.

Terei muito a dizer sobre o papel que os testes automatizados e contínuos jogo de integração em relação a este processo mais adiante neste capítulo. Para o Por enquanto, digamos apenas que nunca verificamos o código que não passa em todos os testes. Nunca, jamais.

#### APÊNDICE A FERRAMENTAS

### CVS/SVN

O antigo sistema de controle de origem em espera é o CVS. Foi bom para o seu dia, mas tem cresceu um pouco demais para os projetos de hoje. Embora seja muito bom em lidar com arquivos e diretórios individuais, não é muito bom para renomear arquivos ou exclusão de diretórios. E o sótão. . . Bem, quanto menos se falar sobre isso, melhor. O Subversion, por outro lado, funciona muito bem. Ele permite que você confira o todo o sistema em uma única operação. Você pode atualizar, mesclar e confirmar facilmente. Contanto que você não se envolva em ramificações, os sistemas SVN são muito simples de usar. gerenciar.

#### Ramificação

Até 2008, evitei todas as formas de ramificação, exceto as mais simples. Se um desenvolvedor criou uma ramificação, essa ramificação teve que ser trazida de volta para a linha principal antes o final da iteração. Na verdade, fui tão austero em relação à ramificação que foi muito raramente feito nos projetos em que estive envolvido.

Se você estiver usando SVN, ainda acho que é uma boa política. No entanto, existem algumas novas ferramentas que mudam o jogo completamente. Eles são os distribuídos sistemas de controle de origem. git é meu favorito do controle de origem distribuído sistemas. Deixe-me contar a você sobre isso.

idiota

Comecei a usar o git no final de 2008, e desde então ele mudou tudo sobre o maneira como uso o controle do código-fonte. Entendendo por que esta ferramenta é um jogo e tanto changer está além do escopo deste livro. Mas comparando a Figura A-1 com a Figura A-2 deveria valer algumas das palavras que não vou incluir aqui.

A Figura A-1 mostra algumas semanas de desenvolvimento no projeto FitNesse enquanto era controlado pelo SVN. Você pode ver o efeito da minha austera regra de não ramificação. Nós simplesmente não ramificamos. Em vez disso, fizemos muito frequentemente atualiza, mescla e confirma na linha principal.

## APÊNDICE A FERRAMENTAS

Figura A-1 FITNESS sob subversão

A figura A-2 mostra algumas semanas de desenvolvimento no mesmo projeto usando git. Como você pode ver, estamos ramificando e fundindo todo o lugar. Isto não aconteceu porque eu relaxei minha política de proibição de filiais; em vez disso, simplesmente

tornou-se a maneira óbvia e mais conveniente de trabalhar. Desenvolvedores individuais pode fazer ramificações de vida muito curta e depois mesclá-las umas com as outras em um capricho.

Figura A-2 FITNESSSE no git

Observe também que você não consegue ver uma linha principal verdadeira. Isso porque não existe um. Quando você usa o git, não existe um repositório central ou uma linha principal.

Cada desenvolvedor mantém sua própria cópia de todo o histórico do projeto em sua máquina local. Eles fazem check-in e check-out dessa cópia local e, em seguida, mesclam-na com outros conforme necessário.

É verdade que mantenho um repositório especial de ouro no qual coloco todos os lançamentos e construções provisórias. Mas para chamar este repositório faltaria a linha principal o ponto. Na verdade, é apenas um instantâneo conveniente de toda a história que cada desenvolvedor mantém localmente.

Se você não entende isso, tudo bem. git é uma espécie de dobrador de mente em primeiro. Você tem que se acostumar com como isso funciona. Mas vou te dizer uma coisa: git e ferramentas

semelhantes, são como será o futuro do controle do código-fonte.

## IDE / EDITOR

Como desenvolvedores, passamos a maior parte do tempo lendo e editando código. As ferramentas que usamos para esse fim mudaram muito ao longo das décadas. Alguns são imensamente poderosos, e alguns pouco mudaram desde os anos 70.

## VI

Você pensaria que os dias em que se usava o vi como editor de desenvolvimento primário já teria acabado há muito tempo. Existem ferramentas hoje em dia que superam em muito o vi, e outros editores de texto simples gostam. Mas a verdade é que o vi desfrutou de um significativo ressurgimento da popularidade devido à sua simplicidade, facilidade de uso, velocidade e flexibilidade. O Vi pode não ser tão poderoso quanto o Emacs ou o Eclipse, mas ainda é uma solução rápida.

e editor poderoso.

Dito isto, não sou mais um usuário avançado do VI. Houve um dia em que eu estava conhecido como vi “deus”, mas esses dias já se foram. Eu uso o vi de vez em quando se eu precisa fazer uma edição rápida de um arquivo de texto. Eu até usei recentemente para fazer uma rápida muda para um arquivo de origem Java em um ambiente remoto. Mas a quantidade de verdade a codificação que fiz no vi nos últimos 10 anos é extremamente pequena.

## APÊNDICE A FERRAMENTAS



O Emacs ainda é um dos editores mais poderosos que existe e provavelmente irá permanecer assim nas próximas décadas. O modelo lisp interno garante isso. Como um ferramenta de edição de uso geral, nada mais chega perto. Por outro lado,

Eu acho que o Emacs não pode realmente competir com os IDEs de propósito específico que agora domina. A edição de código não é um trabalho de edição de uso geral.

Nos anos 90 eu era um fanático pelo Emacs. Eu não consideraria usar mais nada. O editores de apontar e clicar da época eram brinquedos ridículos que nenhum desenvolvedor poderia levar a sério. Mas no início dos anos 2000 fui apresentado ao IntelliJ, meu IDE atual de escolha, e nunca olhei para trás.

#### EC LI PS E / I NTE LLIJ

Sou um usuário do IntelliJ. Eu amo isso. Eu uso para escrever Java, Ruby, Clojure, Scala, Javascript e muitos outros. Esta ferramenta foi escrita por programadores que entender o que os programadores precisam ao escrever código. Ao longo dos anos, eles raramente me decepcionaram e quase sempre me agradaram.

O Eclipse é semelhante em poder e escopo ao IntelliJ. Os dois são simplesmente saltos e limites acima do Emacs quando se trata de edição de Java. Existem outros IDEs nesta categoria, mas não vou mencioná-los aqui porque não tenho experiência direta com eles.

Os recursos que colocam esses IDEs acima de ferramentas como o Emacs são extremamente maneiras poderosas pelas quais eles ajudam você a manipular o código. No IntelliJ, por exemplo, você pode extrair uma superclasse de uma classe com um único comando. Você pode renomear variáveis, extrair métodos e converter herança em composição, entre muitos outros ótimos recursos.

Com essas ferramentas, a edição de código não se trata mais tanto de linhas e caracteres pois se trata de manipulações complexas. Em vez de pensar nos próximos caracteres e linhas que você precisa digitar, você pensa nas próximas transformações que você precisa fazer. Em suma, o modelo de programação é notavelmente diferente e altamente produtivo.

É claro que esse poder tem um custo. A curva de aprendizado é alta e o projeto o tempo de configuração não é insignificante. Essas ferramentas não são leves. Eles levam muito de recursos de computação para funcionar.

#### TE X TMATE

TextMate é poderoso e leve. Não pode fazer as manipulações maravilhosas que o IntelliJ e o Eclipse podem fazer. Ele não possui o poderoso mecanismo lisp e biblioteca do Emacs. Não tem a velocidade e fluidez do vi. Por outro lado, a curva de aprendizado é pequena e sua operação é intuitiva.

Eu uso o TextMate de vez em quando, especialmente para C++ ocasional. eu faria uso o Emacs para um grande projeto C++, mas estou muito enferrujado para me preocupar com o Emacs para

as pequenas tarefas de C++ que tenho.

#### I S U E TR AC K I N G

No momento estou usando o Pivotal Tracker. É um sistema elegante e simples para usar. Ele se encaixa perfeitamente com a abordagem Ágil/iterativa. Permite que todas as partes interessadas

e desenvolvedores para se comunicarem rapidamente. Estou muito satisfeito com isso.

Para projetos muito pequenos, às vezes uso o Lighthouse. É muito rápido e fácil para configurar e usar. Mas não chega nem perto do poder do Tracker.

Eu também simplesmente usei um wiki. Wikis são adequados para projetos internos. Eles permitem que você

para configurar qualquer esquema que você quiser. Você não é forçado a um determinado processo ou rígido

estrutura. Eles são muito fáceis de entender e usar.

Às vezes, o melhor sistema de rastreamento de problemas é um conjunto de cartões e um boletim placa. O quadro de avisos é dividido em colunas como “A fazer”, “Em andamento”, e “Concluído”. Os desenvolvedores simplesmente movem os cartões de uma coluna para a próxima quando apropriado. Na verdade, este pode ser o sistema de rastreamento de problemas mais comum usado por equipes ágeis hoje.

A recomendação que faço aos clientes é começar com um sistema manual como o quadro de avisos antes de comprar uma ferramenta de rastreamento. Depois de dominar o

#### APÊNDICE A FERRAMENTAS

## CONSTRUÇÃO CONTÍNUA

197

sistema manual, você terá o conhecimento necessário para selecionar o apropriado ferramenta. E, de facto, a escolha apropriada pode ser simplesmente continuar a utilizar o sistema manual.

### CONTADORES DE B U G

As equipes de desenvolvedores certamente precisam de uma lista de problemas para trabalhar. Essas questões incluem

novas tarefas e recursos, bem como bugs. Para qualquer equipe de tamanho razoável (5 a 12 desenvolvedores) o tamanho dessa lista deve estar entre dezenas e centenas. Não milhares.

Se você tem milhares de bugs, algo está errado. Se você tem milhares de recursos e/ou tarefas, algo está errado. Em geral, a lista de questões deve ser relativamente pequeno e, portanto, gerenciável com uma ferramenta leve como um wiki, Farol ou rastreador.

Existem algumas ferramentas comerciais que parecem ser muito boas. eu tenho vi clientes usá-los, mas não tive a oportunidade de trabalhar com eles diretamente. Não me oponho a ferramentas como esta, desde que o número de questões permanece pequeno e administrável. Quando as ferramentas de rastreamento de problemas são forçadas a rastrear

milhares de questões, então a palavra “rastreamento” perde o significado. Eles se tornam “depósitos de lixo” (e muitas vezes cheiram a lixão também).

### C N T I N U O U S B U I L D

Ultimamente tenho usado Jenkins como meu mecanismo de construção contínua. É leve, simples e quase não tem curva de aprendizado. Você baixa, executa, faz alguma configurações rápidas e simples, e você está pronto e funcionando. Muito legal.

Minha filosofia sobre construção contínua é simples: conecte-a à sua fonte sistema de controle de código. Sempre que alguém fizer check-in do código, ele deverá automaticamente construir e depois relatar o status para a equipe.

A equipe deve simplesmente manter a construção funcionando o tempo todo. Se a compilação falhar, deve ser um evento de “pare a imprensa” e a equipe deve se reunir para resolver rapidamente o problema. Sob nenhuma circunstância deve-se permitir que a falha persista por um dia ou mais.

Para o projeto FitNesse, cada desenvolvedor executa o script de construção contínua antes de se comprometerem. A construção leva menos de 5 minutos, portanto não é onerosa. Se houver problemas, os desenvolvedores os resolvem antes do commit. Então a construção automática raramente apresenta problemas. A fonte mais comum de automação falhas de compilação acabam sendo problemas relacionados ao meio ambiente, já que meu ambiente de construção é bem diferente do desenvolvimento do desenvolvedor.

## UNIT FERRAMENTAS DE TESTE

Cada linguagem possui sua própria ferramenta de teste de unidade específica. Meus favoritos são JUnit para Java, rspec para Ruby, NUnit para .Net, Midje para Clojure e CppUTest para C e C++.

Qualquer que seja a ferramenta de teste de unidade que você escolher, existem alguns recursos básicos que ela deveria apoiar.

1. A execução dos testes deve ser rápida e fácil. Quer isso seja feito através

Plug-ins IDE ou ferramentas simples de linha de comando são irrelevantes, desde que os desenvolvedores possam executar esses testes por capricho. O gesto para executar os testes deve ser trivial.

Por exemplo, executo meus testes CppUTest digitando command-M no TextMate.

Eu tenho este comando configurado para executar meu makefile que executa automaticamente o teste e imprime um relatório de uma linha se todos os testes forem aprovados. JUnit e rspec são ambos suportados pelo IntelliJ, então tudo que preciso fazer é apertar um botão. Para NUnit, eu uso o plugin ReSharper para me dar o botão de teste.

2. A ferramenta deve fornecer uma indicação visual clara de aprovação/reprovação. Não importa se esta é uma barra gráfica verde ou uma mensagem do console que diz “Todos os testes foram aprovados”. O

O ponto principal é que você deve ser capaz de dizer que todos os testes passaram de forma rápida e inequívoca.

biguamente. Se você tiver que ler um relatório multilinha, ou pior, compare o conteúdo de dois arquivos para saber se os testes foram aprovados, então você falhou no ponto.

3. A ferramenta deve fornecer uma indicação visual clara do progresso. Não importa se este é um medidor gráfico ou uma sequência de pontos, desde que você possa dizer isso progresso ainda está sendo feito e que os testes não foram interrompidos ou abortados.

## APÊNDICE A FERRAMENTAS

## FERRAMENTAS DE TESTE DE COMPONENTES

199

4. A ferramenta deve desencorajar a comunicação de casos de teste individuais com um ao outro. JUnit faz isso criando uma nova instância da classe de teste para cada método de teste, evitando assim que os testes usem variáveis de instância para comunicar uns com os outros. Outras ferramentas executarão os métodos de teste aleatoriamente ordem para que você não possa depender de um teste precedendo outro. Seja qual for o mecanismo, a ferramenta deve ajudá-lo a manter seus testes independentes de cada outro. Os testes dependentes são uma armadilha profunda na qual você não quer cair.

5. A ferramenta deve facilitar muito a escrita de testes. JUnit faz isso fornecendo um API conveniente para fazer afirmações. Ele também usa reflexão e atributos Java para distinguir funções de teste de funções normais. Isso permite que um bom IDE identifique automaticamente todos os seus testes, eliminando o incômodo de conectar suítes e criação de listas de testes propensas a erros.

### FERRAMENTAS DE TESTE COMPONENTE

Essas ferramentas servem para testar componentes no nível da API. Seu papel é fazer certifique-se de que o comportamento de um componente seja especificado em uma linguagem que o as pessoas de negócios e controle de qualidade podem entender. Na verdade, o caso ideal é quando as empresas

analistas e controle de qualidade podem escrever essa especificação usando a ferramenta.

### A D E F I N I Ç Ã O N O F D O N E

Mais do que qualquer outra ferramenta, as ferramentas de teste de componentes são o meio pelo qual especifique o que significa feito. Quando analistas de negócios e QA colaboram para criar um especificação que define o comportamento de um componente, e quando isso especificação pode ser executada como um conjunto de testes que passam ou falham, então pronto leva em um significado muito inequívoco: “Todos os testes passam”.

### FITN E S S E

Minha ferramenta de teste de componentes favorita é o FitNesse. Escrevi grande parte dele e Eu sou o committer principal. Então é meu bebê.

FitNesse é um sistema baseado em wiki que permite que analistas de negócios e especialistas em controle de qualidade

escrever testes em um formato tabular muito simples. Essas tabelas são semelhantes a Parnas

tabelas tanto na forma quanto na intenção. Os testes podem ser rapidamente montados em suítes, e as suítes podem ser administradas quando você quiser.

FitNesse é escrito em Java, mas pode testar sistemas em qualquer linguagem porque se comunica com um sistema de teste subjacente que pode ser escrito em qualquer linguagem. As linguagens suportadas incluem Java, C#/.NET, C, C++, Python, Ruby, PHP, Delphi e outros.

Existem dois sistemas de teste subjacentes ao FitNesse: Fit e Slim. Ajuste foi escrito de Ward Cunningham e foi a inspiração original para FitNesse e similares.

Slim é um sistema de teste muito mais simples e portátil, preferido por Usuários do FitNesse hoje.

## OUTRAS FERRAMENTAS

Conheço várias outras ferramentas que podem ser classificadas como ferramentas de teste de componentes.

- RobotFX é uma ferramenta desenvolvida pelos engenheiros da Nokia. Ele usa uma tabela semelhante formato para FitNesse, mas não é baseado em wiki. A ferramenta simplesmente roda em arquivos simples preparado com Excel ou similar. A ferramenta é escrita em Python, mas pode testar sistemas em qualquer idioma usando pontes apropriadas.

- Green Pepper é uma ferramenta comercial que possui diversas semelhanças com FitNesse. É baseado no popular wiki do Confluence.

- Cucumber é uma ferramenta de texto simples conduzida por um mecanismo Ruby, mas capaz de testando muitas plataformas diferentes. A linguagem do Pepino é a popular Estilo Dado/Quando/Então.

- JBehave é semelhante ao Cucumber e é o pai lógico do Cucumber. É escrito em Java.

## FERRAMENTAS DE TESTE DE INTEGRAÇÃO

Ferramentas de teste de componentes também podem ser usadas para muitos testes de integração, mas são

menos do que apropriado para testes conduzidos pela IU.

Em geral, não queremos realizar muitos testes por meio da UI porque as UIs são notoriamente volátil. Essa volatilidade torna os testes que passam pela IU muito frágeis.

## APÊNDICE A FERRAMENTAS

Dito isto, existem alguns testes que devem passar pela UI – a maioria mais importante ainda, testes da IU. Além disso, alguns testes ponta a ponta devem passar pelo todo o sistema montado, incluindo a UI.

As ferramentas que mais gosto para testes de UI são Selenium e Watir.

#### UML/MDA

No início dos anos 90, eu tinha muita esperança de que a indústria de ferramentas CASE causaria uma mudança radical na forma como os desenvolvedores de software trabalhavam. Enquanto eu olhava para frente

naqueles dias inebriantes, pensei que agora todos estariam codificando em diagramas em um nível mais alto de abstração e esse código textual seria uma coisa do passado.

Rapaz, eu estava errado. Não só este sonho não foi realizado, mas todas as tentativas de avançar nessa direção encontraram um fracasso abjeto. Não que não existam ferramentas e sistemas existentes que demonstrem o potencial; é que essas ferramentas simplesmente não realizam verdadeiramente o sonho e quase ninguém parece querer usá-los.

O sonho era que os desenvolvedores de software pudessem deixar para trás os detalhes de código textual e sistemas de autoria em uma linguagem de diagramas de nível superior. Na verdade, assim continua o sonho, talvez não precisemos de programadores. Os arquitetos poderiam criar sistemas inteiros a partir de diagramas UML. Motores, vastos e frios e antipático à situação de meros programadores, transformaria aqueles diagramas em código executável. Esse era o grande sonho de Model Driven Arquitetura (MDA).

Infelizmente, esse grande sonho tem uma pequena falha. O MDA assume que o problema é código. Mas o código não é o problema. Nunca foi o problema.

O problema é o detalhe.

#### OS DETALHES

Os programadores são gerentes detalhados. Isso é o que fazemos. Nós especificamos o comportamento dos sistemas nos mínimos detalhes. Acontece que usamos linguagens textuais para isso (código) porque as linguagens textuais são extremamente convenientes (considere inglês, por exemplo).

Que tipos de detalhes gerenciávamos?

Você sabe a diferença entre os dois caracteres `\n` e `\r`? O primeiro, `\n`, é um avanço de linha. O segundo, `\r`, é um retorno de carro. O que é uma carruagem?

Nos anos 60 e início dos anos 70, um dos dispositivos de saída mais comuns para computadores era um teletipo. O modelo ASR332 foi o mais comum.

Este dispositivo consistia em uma cabeça de impressão capaz de imprimir dez caracteres por segundo. A cabeça de impressão era composta por um pequeno cilindro com os caracteres gravados sobre isso. O cilindro giraria e se elevaria para que o caractere correto fosse voltado para o papel, e então um pequeno martelo batia o cilindro contra o papel. Havia uma fita de tinta entre o cilindro e o papel, e a tinta transferida para o papel no formato do personagem.

A cabeça de impressão andava em uma carruagem. Com cada personagem a carruagem se moveria um espaço à direita, levando consigo a cabeça de impressão. Quando a carruagem chegou ao final da linha de 72 caracteres, você tinha que retornar explicitamente o carro enviando os caracteres de retorno de carro (`\r = 0x0D`), caso contrário o cabeçote de impressão continue a imprimir caracteres na 72ª coluna, transformando-a em um preto desagradável retângulo.

Claro, isso não foi suficiente. Devolver a carruagem não levantou o papel para a próxima linha. Se você devolveu o carro e não enviou um avanço de linha caractere (`\n = 0x0A`), então a nova linha seria impressa em cima da linha antiga. Portanto, para um teletipo ASR33 a sequência de fim de linha era `"\r\n"`. Na verdade, você tinha que ter cuidado com isso, pois o transporte pode levar mais de 100ms para retornar. Se você enviou `"\n\r"`, o próximo caractere poderá ser impresso como o carro retornou, criando assim um caractere borrado no meio da linha. Por segurança, frequentemente preenchíamos a sequência de final de linha com um ou dois rubout3 caracteres (`0xFF`).

2. [http://en.wikipedia.org/wiki/ASR-33\\_Teletype](http://en.wikipedia.org/wiki/ASR-33_Teletype)

3. Os caracteres Rubout foram muito úteis para editar fitas de papel. Por convenção, os caracteres de apagamento foram ignorados.

Seu código, `0xFF`, significava que todos os furos daquela linha da fita eram perfurados. Isso significava que qualquer caractere

O personagem pode ser convertido em um golpe com um soco excessivo. Portanto, se você cometeu um erro ao digitar

seu programa, você pode retroceder o soco e clicar em rubout e continuar digitando.

APÊNDICE A FERRAMENTAS



Na década de 70, à medida que os teletipos começaram a desaparecer, sistemas operacionais como o UNIX

encurtou a sequência de final de linha para simplesmente ‘\n’. No entanto, outras operações sistemas, como o DOS, continuaram a usar a convenção ‘\r\n’.

Quando foi a última vez que você teve que lidar com arquivos de texto que usavam a palavra “errada” convenção? Enfrento esse problema pelo menos uma vez por ano. Dois arquivos de origem idênticos não compare e não gere somas de verificação idênticas, porque elas usam extremidades de linha diferentes. Os editores de texto não conseguem quebrar as palavras corretamente ou duplicar o espaço

texto porque os finais de linha estão “errados”. Programas que não esperam linhas em branco trava porque eles interpretam ‘\r\n’ como duas linhas. Alguns programas reconhecem ‘\r\n’ mas não reconhece ‘\n\r’. E assim por diante.

Isso é o que quero dizer com detalhe. Tente codificar a lógica horrível para resolver a linha termina em UML!

### N O ESPERANÇA, N O C H A N G E

A esperança do movimento MDA era que uma grande quantidade de detalhes pudesse ser eliminado usando diagramas em vez de código. Essa esperança até agora provou ser abandonado. Acontece que simplesmente não há muitos detalhes extras incorporados código que pode ser eliminado por imagens. Além do mais, as imagens contêm seus próprios detalhes acidentais. As imagens têm sua própria gramática, sintaxe e regras e restrições. Então, no final das contas, a diferença no detalhe é uma lavagem.

A esperança do MDA era que os diagramas provassem estar em um nível mais alto de abstração do que código, assim como Java está em um nível superior ao assembler. Mas novamente, essa esperança até agora provou ser equivocada. A diferença no nível de a abstração é pequena, na melhor das hipóteses.

E, finalmente, digamos que um dia alguém invente uma ferramenta verdadeiramente útil. linguagem diagramática. Não serão arquitetos desenhando esses diagramas, serão

programadores. Os diagramas simplesmente se tornarão o novo código, e

Serão necessários programadores para desenhar esse código porque, no final, é tudo uma questão de detalhes, e são os programadores que gerenciam esses detalhes.

## CONCLUSÃO

As ferramentas de software tornaram-se muito mais poderosas e abundantes desde que comecei programação. Meu kit de ferramentas atual é um subconjunto simples desse zoológico. eu uso git para controle de código-fonte, Tracker para gerenciamento de problemas, Jenkins para Contínuo Build, IntelliJ como meu IDE, XUnit para teste e FitNesse para componente testando.

Minha máquina é um Macbook Pro, Intel Core i7 de 2,8 GHz, com fosco de 17 polegadas tela, 8 GB de RAM, SSD de 512 GB, com duas telas extras.

## APÊNDICE A FERRAMENTAS

## EM DE X

## Um

## Testes de aceitação

automatizado, 103–105

comunicação e, 103

integração contínua e,  
110-111

definição de, 100

o papel do desenvolvedor em, 106–107

trabalho extra e, 105

GUIs e, 109–111

negociação e, 107-108

agressão passiva e, 107-108

tempo de, 105-106

testes de unidade e, 108–109

escritores de, 105-106

Papéis adversários, 26–29

Estimativa de afinidade, 146-147

Ambiguidade, nos requisitos, 98–100

Desculpas, 12

Aprendizes, 183

Aprendizagem, 180-184

Argumentos, em reuniões, 126-127

Arrogância, 22

Testes de aceitação automatizados,  
103–105

Garantia de qualidade automatizada, 14

Evitação, 131

## B

Becos sem saída, 131-132

Bossavit, Laurent, 89

Jogo de boliche, 89

Ramificação, 191

Contagens de bugs, 197

Metas de negócios, 160

## C

Cafeína, 128

Certeza, 80

## Código

controle, 189-194

propriedade, 163

3h, 59–60

preocupação, 60-61

Codificação Dojo, 89–93

Colaboração, 20, 157–166

Propriedade coletiva, 163-164

## ÍNDICE

206

Compromisso(s), 47–52

controle e, 50

disciplina e, 53-56

estimativa e, 138

expectativas e, 51

identificando, 49-50

implícito, 140-141

importância de, 138

falta de, 48-49

pressão e, 152

Comunicação

testes de aceitação e, 103

pressão e, 154

de requisitos, 95–100

Testes de componentes

na estratégia de teste, 116–117

ferramentas para, 199–200

Conflito, em reuniões, 126–127

Construção contínua, 197-198

Integração contínua, 110–111

Aprendizagem contínua, 19

Controle, comprometimento e, 50

Coragem, 81-82

Artesanato, 184

Entrada criativa, 65–66, 129

Disciplina de crise, 153

Pepino, 200

Cliente, identificação com, 21

CVS, 191

Tempo de ciclo, baseado em testes

desenvolvimento, 78

D

Prazos

entrega falsa e, 73

esperando e, 71

horas extras e, 72

correndo e, 71-72

Depuração, 66–69

Taxa de injeção de defeito, 81

Reuniões de demonstração, 126

Design, desenvolvimento orientado a testes e,

82–83

Padrões de design, 18

Princípios de design, 18

Detalhes, 201–203

Desenvolvimento. veja test drive

desenvolvimento (TDD)

Desentendimentos, em reuniões, 126-127

Disciplina

compromisso e, 53-56

crise, 153

Desengajamento, 70

Documentação, 82

Domínio, conhecimento de, 21

“Concluído”, definindo, 73, 100–103

Abordagem “Não causar danos”, 11–16

funcionar, 11–14

para estruturar, 14-16

Dirigindo, 70

E

Eclipse, 195-196

Emacs, 195

Empregador(es)

identificação com, 21

programadores vs., 159-162

Estimativa

afinidade, 146-147

ansiedade, 98

compromisso e, 138

definição de, 138-139

lei dos grandes números e, 147

nominal, 142

otimista, 141-142

PERT e, 141-144

pessimista, 142

probabilidade e, 139

de tarefas, 144-147

trivariado, 147

## ÍNDICE

207

Expectativas, comprometimento e, 51

Experiência, ampliação, 93

### F

Falha, graus de, 174

Entrega falsa, 73

FitNesse, 199–200

Flexibilidade, 15

Zona de fluxo, 62–64

Dedos voadores, 145

Foco, 127–129

Função, em “não causar danos”

abordagem, 11–14

### G

Gaillot, Emmanuel, 89

Equipe gelificada, 168-170

Git, 191-194

Metas, 26–29, 124

Interfaces gráficas de usuário (GUIs),  
109–111

Pimentão Verde, 200

Grenning, James, 145

GUIs, 109–111

### H

Golpes fortes, 179-180

Ajuda, 73–76

dando, 74

mentoria e, 75-76

pressão e, 154-155

recebendo, 74-75

“Esperança”, 48

Esperança, prazos e, 71

Humildade, 22

eu

IDE/editor, 194

Identificação, com empregador/  
cliente, 21

Compromissos implícitos, 140-141

Entrada, criativo, 65–66, 129

Integração, contínua, 110–111

Testes de integração

na estratégia de teste, 117-118

ferramentas para, 200–201

IntelJ, 195–196

Estagiários, 183

Interrupções, 63-64

Acompanhamento de problemas, 196–197

Reuniões de planejamento de iteração, 125

Reuniões retrospectivas de iteração, 126

J.

JBehave, 200

Jornalistas, 182-183

K

Kata, 90-91

Conhecimento

de domínio, 21

mínimo, 18

ética de trabalho e, 17-19

eu

Atraso, 71-73

Lei dos grandes números, 147

Aprendizagem, ética de trabalho e, 19

“Vamos,” 48

Lindstrom, Lowell, 146

Bloqueio, 190

M

Testes exploratórios manuais, em testes  
estratégia, 118-119

Mestres, 182

MDA, 201–203

Reuniões

agenda em, 124

argumentos e divergências em,  
126–127

diminuindo, 123

## ÍNDICE

208

Reuniões (continuação)

demonstração, 126

gols marcados, 124

planejamento de iteração, 125

retrospectiva da iteração, 126

saindo, 124

em pé, 125

gerenciamento de tempo e, 122-127

Mentoria, 20–21, 75–76, 174–180

Refatoração impiedosa, 15

Bagunça, 132–133, 152

Métodos, 18

Arquitetura Orientada a Modelo (MDA),  
201–203

Foco muscular, 129

Música, 63

## N

“Necessidade”, 48

Negociação, testes de aceitação e,  
107–108

Estimativa nominal, 142

Não profissional, 8

## Ó

Código aberto, 93

Estimativa otimista, 141-142

Bloqueio otimista, 190

Resultados, melhores possíveis, 26–29

Horas extras, 72

Código de propriedade, 163

Propriedade, coletiva, 163-164

## P

Ritmo, 69-70

Emparelhamento, 64, 154–155, 164

Pânico, 153-154

Paixão, 160

Agressão passiva, 34–36, 107–108

Pessoas, programadores vs., 159–164

Questões pessoais, 60-61

PERT (Avaliação de Programas e  
Técnica de Revisão), 141–144

Estimativa pessimista, 142

Bloqueio pessimista, 190

Atividade física, 129

Planejamento de pôquer, 145–146

Pratique

antecedentes, 86-89



ética, 93

experiência e, 93

tempo de resposta e, 88-89

ética de trabalho e, 19–20

Precisão, prematura, em

requisitos, 97–98

Preparação, 58-61

Pressão

evitando, 151-153

limpeza e, 152

compromissos e, 152

comunicação e, 154

manuseio, 153-155

ajuda e, 154-155

bagunça e, 152

pânico e, 153-154

Inversão de prioridade, 131

Probabilidade, 139

Profissionalismo, 8

Programadores

empregadores vs., 159-162

pessoas vs., 159–164

programadores vs., 163

Proposta, projeto, 37–38

P

Garantia de qualidade (QA)

automatizado, 14

como coletores de insetos, 12

como caracterizadores, 114-115

ideal de, como não encontrar problemas,

114–115

## ÍNDICE

209

problemas encontrados por, 12–13

como especificadores, 114

como membro da equipe, 114

## R

Randori, 92-93

Leitura, como contribuição criativa, 65

Recarregando, 128–129

Reputação, 11

Requisitos

comunicação de, 95-100

ansiedade de estimativa e, 98

ambigüidade tardia em, 98-100

precisão prematura em, 97-98

incerteza e, 97-98

Responsabilidade, 8–11

desculpas e, 12

abordagem “não causar danos” e, 11–16

função e, 11–14

estrutura e, 14–16

ética de trabalho e, 16–22

RobôFX, 200

Funções, adversário, 26–29

Correndo, 40–41, 71–72

## S

Santana, Carlos, 89

“Deveria”, 48

Chuveiro, 70

Simplicidade, 40

Sono, 128

Controle de código-fonte, 189–194

Estacas, 29–30

Reuniões stand-up, 125

Estrutura

na abordagem “não causar danos”, 14–16

flexibilidade e, 15

importância de, 14

SVN, 191-194

Testes de sistema, na estratégia de teste, 118

## T

Estimativa de tarefa, 144–147

Equipes e trabalho em equipe, 30-36

gelificado, 168-170

gestão de, 170

agressão passiva e, 34-36

preservando, 169

iniciado pelo projeto, 169–170

dilema do proprietário do projeto com,  
170-171  
tentando e, 32-34  
velocidade de, 170  
Desenvolvimento orientado a testes (TDD)  
benefícios de, 80-83  
certeza e, 80  
coragem e, 81-82  
tempo de ciclo em, 78  
estreia de, 77-78  
taxa de injeção de defeito e, 81  
definição de, 13–14  
design e, 82-83  
documentação e, 82  
interrupções e, 64  
três leis de, 79-80  
o que não é, 83-84  
Teste  
aceitação  
automatizado, 103–105  
comunicação e, 103  
integração contínua e,  
110-111  
definição de, 100  
o papel do desenvolvedor em, 106–107  
trabalho extra e, 105  
GUIs e, 109–111  
negociação e, 107-108  
agressão passiva e, 107-108  
tempo de, 105-106  
testes de unidade e, 108–109  
escritores de, 105-106

## ÍNDICE

210

Teste (continuação)

pirâmide de automação, 115-119

componente

na estratégia de teste, 116–117

ferramentas para, 199–200

importância de, 13–14

integração

na estratégia de teste, 117-118

ferramentas para, 200–201

manual exploratório, 118-119

estrutura e, 15

sistema, 118

unidade

testes de aceitação e, 108–109

na estratégia de teste, 116

ferramentas para, 198–199

TextoMate, 196

Tomás, Dave, 90

Código das 3h, 59–60

Tempo, depuração, 69

Gerenciamento de tempo

evitação e, 131

becos sem saída e, 131-132

exemplos de, 122

foco e, 127-129

reuniões e, 122–127

bagunça e, 132-133

inversão de prioridade e, 131

recarregando e, 128–129

técnica de “tomates” para, 130

Cansaço, 59-60

Gerenciamento de tempo “Tomates”

técnica, 130

Ferramentas, 189

Estimativas trivariadas, 147

Tempo de resposta, prática

e, 88-89

Você

UML, 201

Incerteza, requisitos e, 97-98

Mentoria não convencional, 179.

veja também mentoria

Testes unitários

testes de aceitação e, 108–109

na estratégia de teste, 116

ferramentas para, 198–199

## V

Vi, 194

## W

Indo embora, 70

Wasa, 91-92

Delphi de banda larga, 144–147

“Desejo”, 48

Ética de trabalho, 16–22

colaboração e, 20

aprendizagem contínua e, 19

conhecimento e, 17-19

mentoria e, 20-21

prática e, 19–20

Código de preocupação, 60–61

Bloqueio de escritor, 64-66

## S

“Sim”

custo de, 36–40

aprendendo a dizer, 52-56

Esta página foi intencionalmente deixada em branco

Registre o exame Addison-Wesley  
Cram, Prentice Hall, Que e  
Produtos Sams que você possui para desbloquear  
grandes benefícios.

Para iniciar o processo de registro,  
basta acessar [informit.com/register](http://informit.com/register)  
para entrar ou criar uma conta.

Você será solicitado a inserir  
o ISBN de 10 ou 13 dígitos que aparece  
na contracapa do seu produto.

[informeIT.com](http://informeIT.com)

## A FONTE CONFIÁVEL DE APRENDIZAGEM DE TECNOLOGIA

Addison-Wesley | Imprensa Cisco | Curso de exame

Imprensa IBM

| Que | Salão Prentice

| Sam

## LIVROS SAFARI ON-LINE

Sobre o InformIT — A FONTE CONFIÁVEL DE APRENDIZAGEM DE TECNOLOGIA

INFORMIT É O LAR DA LÍDER EM TECNOLOGIA DE PUBLICAÇÃO DE IMPRESSÕES

Addison-Wesley Professional, Cisco Press, Exam Cram, IBM Press, Prentice Hall

Profissional, Que e Sams. Aqui você terá acesso a conteúdo de qualidade e confiável e

recursos dos autores, criadores, inovadores e líderes de tecnologia. Quer você esteja

procurando um livro sobre uma nova tecnologia, um artigo útil, boletins informativos oportunos ou acesso a

a biblioteca digital Safari Books Online, a InformIT tem uma solução para você.

Registrar seus produtos pode desbloquear

os seguintes benefícios:

- Acesso a conteúdo complementar,  
incluindo capítulos bônus,  
código-fonte ou arquivos de projeto.

- Um cupom para ser usado no seu  
próxima compra.

Os benefícios do registro variam de acordo com o produto.

Os benefícios serão listados em sua conta

página em Produtos Registrados.

[informe.com/register](http://informe.com/register)

ESTE PRODUTO

InformIT é uma marca da Pearson e a presença online para os principais editores de tecnologia do mundo. É a sua fonte por conteúdo e conhecimento confiáveis e qualificados, proporcionando acesso às principais marcas, autores e colaboradores de a comunidade tecnológica.

[informit.com](http://informit.com)

## A FONTE CONFIÁVEL DE APRENDIZAGEM DE TECNOLOGIA

### AprendaIT na InformIT

Procurando um livro, e-book ou vídeo de treinamento sobre uma nova tecnologia? Procure informações e tutoriais oportunos e relevantes? Procurando opinião de especialistas, conselhos e dicas? A InformIT tem a solução.

- 

Saiba mais sobre novos lançamentos e promoções especiais em assinando uma ampla variedade de boletins informativos.

Visite [informit.com/newsletters](http://informit.com/newsletters).

- Acesse podcasts GRATUITOS de especialistas em [informit.com/podcasts](http://informit.com/podcasts).
- Leia os artigos mais recentes dos autores e exemplos de capítulos em [informit.com/articles](http://informit.com/articles).

- 

Acesse milhares de livros e vídeos no Safari Books

Biblioteca digital online em [safari.informit.com](http://safari.informit.com).

- Receba dicas de blogs especializados em [informit.com/blogs](http://informit.com/blogs).

Visite [informit.com/learn](http://informit.com/learn) para descobrir todas as maneiras de acessar o conteúdo de tecnologia mais quente.

## [informit.com](http://informit.com) A FONTE CONFIÁVEL DE APRENDIZAGEM DE TECNOLOGIA

Você faz parte da multidão de TI?

Conecte-se com autores e editores da Pearson via feeds RSS, Facebook, Twitter, YouTube e muito mais! Visite [informit.com/socialconnect](http://informit.com/socialconnect).



Experimente Safari Books Online GRATUITAMENTE

Obtenha acesso online a mais de 5.000 livros e vídeos

Encontre respostas confiáveis rapidamente

Somente o Safari permite pesquisar milhares de livros mais vendidos no topo editores de tecnologia, incluindo Addison-Wesley Professional, Cisco Press, O'Reilly, Prentice Hall, Que e Sams.

Domine as ferramentas e técnicas mais recentes

Além de ter acesso a um incrível estoque de livros técnicos,

A extensa coleção de tutoriais em vídeo do Safari permite que você aprenda com os principais especialistas em treinamento em vídeo.

ESPERE, TEM MAIS!

Mantenha sua vantagem competitiva

Com Rough Cuts, tenha acesso ao manuscrito em desenvolvimento e esteja entre os primeiros para aprender as mais novas tecnologias.

Mantenha-se atualizado com as tecnologias emergentes

Atalhos e folhas de referência rápida são conteúdos curtos, concisos e focados

criado para que você fique atualizado rapidamente sobre tecnologias novas e de ponta.

TESTE GRATUITO - COMECE HOJE!

[www.informit.com/safaritrial](http://www.informit.com/safaritrial)